Zcash Protocol Specification

Version v2026.9.0-13-gde3d90 [Overwinter+Sapling]

Daira-Emma Hopwood[†]
Sean Bowe[‡] — Taylor Hornby[§] — Nathan Wilcox[◊]

November 11, 2026



[1]

Abstract. Zcash is an implementation of the *Decentralized Anonymous Payment scheme* **Zerocash**, with security fixes and improvements to performance and functionality. It bridges the existing transparent payment scheme used by **Bitcoin** with a ***shielded*** payment scheme secured by *zero-knowledge succinct non-interactive arguments of knowledge (zk-SNARKs)*. It attempted to address the problem of mining centralization by use of the *Equihash* memory-hard *proof-of-work* algorithm.

This specification defines the **Zcash** consensus protocol at launch, and after each of the upgrades codenamed **Overwinter** and **Sapling**. It is a work in progress. Protocol differences from **Zerocash** and **Bitcoin** are also explained.

Keywords: anonymity, applications, cryptographic protocols, electronic commerce and payment, financial privacy, proof of work, zero knowledge.

Contents

1

1 Introduction 7

1.1 Caution 7

1.2 High-level Overview 8

[†] Zcash Open Development Lab
[‡] Project Tachyon
[§] Independent consultant
[◊] Shielded Labs

[1] Jubjub bird image credit: Peter Newell 1902; Daira-Emma Hopwood 2018.

2 Notation 9

3 Concepts 12
3.1 Payment Addresses and Keys . . . 12
3.2 Shielded Pools and Notes . . . 13
3.2.1 Note Plaintexts and Memo Fields . . . 14
3.2.2 Note Commitments . . . 15
3.2.3 Nullifiers . . . 15
3.3 The Block Chain . . . 16
3.4 Transactions and Treestates . . . 16
3.5 JoinSplit Transfers and Descriptions . . . 17
3.6 Spend Transfers, Output Transfers, and their Descriptions . . . 18
3.7 Note Commitment Trees . . . 19
3.8 Nullifier Sets . . . 19
3.9 Block Subsidy and Founders' Reward . . . 20
3.10 Coinbase Transactions . . . 20
3.11 Mainnet and Testnet . . . 20

4 Abstract Protocol 21
4.1 Abstract Cryptographic Schemes . . . 22
4.1.1 Hash Functions . . . 22
4.1.2 Pseudo Random Functions . . . 22
4.1.3 Symmetric Encryption . . . 23
4.1.4 Key Agreement . . . 23
4.1.5 Key Derivation . . . 24
4.1.6 Signature . . . 24
4.1.6.1 Signature with Re-Randomizable Keys . . . 26
4.1.6.2 Signature with Signing Key to Validating Key Monomorphism . . . 27
4.1.7 Commitment . . . 27
4.1.8 Represented Group . . . 29
4.1.9 Coordinate Extractor . . . 29
4.1.10 Group Hash . . . 29
4.1.11 Represented Pairing . . . 30
4.1.12 Zero-Knowledge Proving System . . . 30
4.2 Key Components . . . 32
4.2.1 Sprout Key Components . . . 32
4.2.2 Sapling Key Components . . . 32
4.3 JoinSplit Descriptions . . . 34
4.4 Spend Descriptions . . . 35
4.5 Output Descriptions . . . 36
4.6 Sending Notes . . . 37
4.6.1 Sending Notes (Sprout) . . . 37
4.6.2 Sending Notes (Sapling) . . . 37
4.7 Dummy Notes . . . 38
4.7.1 Dummy Notes (Sprout) . . . 38
4.7.2 Dummy Notes (Sapling) . . . 39

4.8 Merkle Path Validity . . . 39
4.9 SIGHASH Transaction Hashing . . . 40
4.10 Non-malleability (Sprout) . . . 41
4.11 Balance (Sprout) . . . 41
4.12 Balance and Binding Signature (Sapling) . . . 42
4.13 Spend Authorization Signature (Sapling) . . . 44
4.14 Computing ρ values and Nullifiers . . . 45
4.15 Chain Value Pool Balances . . . 46
4.16 Zk-SNARK Statements . . . 47
4.16.1 JoinSplit Statement (Sprout) . . . 47
4.16.2 Spend Statement (Sapling) . . . 48
4.16.3 Output Statement (Sapling) . . . 49
4.17 In-band secret distribution (Sprout) . . . 50
4.17.1 Encryption (Sprout) . . . 50
4.17.2 Decryption (Sprout) . . . 51
4.18 In-band secret distribution (Sapling) . . . 51
4.18.1 Encryption (Sapling) . . . 52
4.18.2 Decryption using an Incoming Viewing Key (Sapling) . . . 53
4.18.3 Decryption using an Outgoing Viewing Key (Sapling) . . . 53
4.19 Block Chain Scanning (Sprout) . . . 55
4.20 Block Chain Scanning (Sapling) . . . 55

5 Concrete Protocol 56
5.1 Integers, Bit Sequences, and Endianness . . . 56
5.2 Bit layout diagrams . . . 57
5.3 Constants . . . 57
5.4 Concrete Cryptographic Schemes . . . 58
5.4.1 Hash Functions . . . 58
5.4.1.1 SHA-256, SHA-256d, SHA256Compress, and SHA-512 Hash Functions . . . 58
5.4.1.2 BLAKE2 Hash Functions . . . 59
5.4.1.3 Merkle Tree Hash Function . . . 59
5.4.1.4 $\mathsf{h}_{\mathsf{Sig}}$ Hash Function . . . 60
5.4.1.5 $\mathsf{CRH}^{\mathsf{ivk}}$ Hash Function . . . 60
5.4.1.6 $\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ Hash Function . . . 61
5.4.1.7 Pedersen Hash Function . . . 62
5.4.1.8 Mixing Pedersen Hash Function . . . 63
5.4.1.9 Equihash Generator . . . 64
5.4.2 Pseudo Random Functions . . . 64
5.4.3 Symmetric Encryption . . . 66
5.4.4 Key Agreement And Derivation . . . 66
5.4.4.1 Sprout Key Agreement . . . 66
5.4.4.2 Sprout Key Derivation . . . 66
5.4.4.3 Sapling Key Agreement . . . 67
5.4.4.4 Sapling Key Derivation . . . 67
5.4.5 Ed25519 . . . 67

5.4.6 RedDSA and RedJubjub . . . 69
5.4.6.1 Spend Authorization Signature (Sapling) . . . 71
5.4.6.2 Binding Signature (Sapling) . . . 71
5.4.7 Commitment schemes . . . 72
5.4.7.1 Note Commitments (Sprout) . . . 72
5.4.7.2 Windowed Pedersen commitments . . . 72
5.4.7.3 Homomorphic Pedersen commitments (Sapling) . . . 73
5.4.8 Represented Groups and Pairings . . . 74
5.4.8.1 BN-254 . . . 74
5.4.8.2 BLS12-381 . . . 75
5.4.8.3 Jubjub . . . 77
5.4.8.4 Coordinate Extractor for Jubjub . . . 78
5.4.8.5 Group Hash into Jubjub . . . 79
5.4.9 Zero-Knowledge Proving Systems . . . 79
5.4.9.1 BCTV14 . . . 79
5.4.9.2 Groth16 . . . 81
5.5 Encodings of Note Plaintexts and Memo Fields . . . 81
5.6 Encodings of Addresses and Keys . . . 82
5.6.1 Transparent Encodings . . . 82
5.6.1.1 Transparent Addresses . . . 82
5.6.1.2 Transparent Private Keys . . . 83
5.6.2 Sprout Encodings . . . 83
5.6.2.1 Sprout Payment Addresses . . . 83
5.6.2.2 Sprout Incoming Viewing Keys . . . 84
5.6.2.3 Sprout Spending Keys . . . 84
5.6.3 Sapling Encodings . . . 85
5.6.3.1 Sapling Payment Addresses . . . 85
5.6.3.2 Sapling Incoming Viewing Keys . . . 85
5.6.3.3 Sapling Full Viewing Keys . . . 86
5.6.3.4 Sapling Spending Keys . . . 86
5.7 BCTV14 zk-SNARK Parameters . . . 87
5.8 Groth16 zk-SNARK Parameters . . . 87
5.9 Randomness Beacon . . . 87

6 Network Upgrades 88

7 Consensus Changes from Bitcoin 89
7.1 Transaction Encoding and Consensus . . . 89
7.1.1 Transaction Identifiers . . . 90
7.1.2 Transaction Consensus Rules . . . 90
7.2 JoinSplit Description Encoding and Consensus . . . 93
7.3 Spend Description Encoding and Consensus . . . 93
7.4 Output Description Encoding and Consensus . . . 94
7.5 Block Header Encoding and Consensus . . . 95
7.6 Proof of Work . . . 97
7.6.1 Equihash . . . 97

7.6.2 Difficulty filter . . . 98
7.6.3 Difficulty adjustment . . . 98
7.6.4 nBits conversion . . . 99
7.6.5 Definition of Work . . . 100
7.7 Calculating Block Subsidy and Founders' Reward . . . 100
7.8 Payment of Founders' Reward . . . 101
7.9 Changes to the Script System . . . 103
7.10 Bitcoin Improvement Proposals . . . 103

8 Differences from the Zerocash paper 103
8.1 Transaction Structure . . . 103
8.2 Memo Fields . . . 104
8.3 Unification of Mints and Pours . . . 104
8.4 Faerie Gold attack and fix . . . 104
8.5 Internal hash collision attack and fix . . . 106
8.6 Changes to PRF inputs and truncation . . . 106
8.7 In-band secret distribution . . . 107
8.8 Omission in **Zerocash** security proof . . . 109
8.9 Miscellaneous . . . 109

9 Acknowledgements 110

10 Change History 111

11 References 139

Appendices 149

A Circuit Design 149
A.1 Quadratic Constraint Programs . . . 149
A.2 Elliptic curve background . . . 149
A.3 Circuit Components . . . 150
A.3.1 Operations on individual bits . . . 150
A.3.1.1 Boolean constraints . . . 150
A.3.1.2 Conditional equality . . . 150
A.3.1.3 Selection constraints . . . 151
A.3.1.4 Nonzero constraints . . . 151
A.3.1.5 Exclusive-or constraints . . . 151
A.3.2 Operations on multiple bits . . . 151
A.3.2.1 [Un]packing modulo $r_{\mathbb{S}}$ . . . 151
A.3.2.2 Range check . . . 152
A.3.3 Elliptic curve operations . . . 154
A.3.3.1 Checking that Affine-ctEdwards coordinates are on the curve . . . 154
A.3.3.2 ctEdwards [de]compression and validation . . . 154
A.3.3.3 ctEdwards $\leftrightarrow$ Montgomery conversion . . . 154
A.3.3.4 Affine-Montgomery arithmetic . . . 155
A.3.3.5 Affine-ctEdwards arithmetic . . . 156

A.3.3.6 Affine-ctEdwards nonsmall-order check . . . 157
A.3.3.7 Fixed-base Affine-ctEdwards scalar multiplication . . . 157
A.3.3.8 Variable-base Affine-ctEdwards scalar multiplication . . . 158
A.3.3.9 Pedersen hash . . . 159
A.3.3.10 Mixing Pedersen hash . . . 161
A.3.4 Merkle path check . . . 162
A.3.5 Windowed Pedersen Commitment . . . 162
A.3.6 Homomorphic Pedersen Commitment . . . 162
A.3.7 BLAKE2s hashes . . . 163
A.4 The Sapling Spend circuit . . . 166
A.5 The Sapling Output circuit . . . 168

B Batching Optimizations 169
B.1 RedDSA batch validation . . . 169
B.2 Groth16 batch verification . . . 170

List of Theorems and Lemmata 172

Index 172

1 Introduction

Zcash is an implementation of the *Decentralized Anonymous Payment scheme* **Zerocash** [BCGGMTV2014], with security fixes and improvements to performance and functionality. It bridges the existing transparent payment scheme used by **Bitcoin** [Nakamoto2008] with a ***shielded*** payment scheme secured by *zero-knowledge succinct non-interactive arguments of knowledge (zk-SNARKs)*.

In this document, technical terms for concepts that play an important rôle in **Zcash** are written in *slanted text*, which links to an index entry. ***Italics*** are used for emphasis and for references between sections of the document. The symbol § precedes section numbers in cross-references.

The key words **MUST**, **MUST NOT**, **SHOULD**, **SHOULD NOT**, **RECOMMENDED**, **MAY**, and **OPTIONAL** in this document are to be interpreted as described in [RFC-2119] when they appear in **ALL CAPS**. These words may also appear in this document in lower case as plain English words, absent their normative meanings.

The most significant changes from the original **Zerocash** are explained in § 8 *'Differences from the **Zerocash** paper'* on p. 103.

Changes before the **Sapling** upgrade are not highlighted.

Changes specific to the **Sapling** upgrade following **Overwinter** are highlighted in green.

All of these are also changes from **Zerocash**.

The name "Sprout" refers to the **Zcash** protocol prior to **Sapling** (both before and after **Overwinter**), and in particular its Sprout *shielded protocol*. The name "Sapling" refers to the **Sapling** *network upgrade*, and also the Sapling *shielded protocol* which is used for the *Sapling pool*. .

This specification is structured as follows:

- Notation — definitions of notation used throughout the document;
- Concepts — the principal abstractions needed to understand the protocol;
- Abstract Protocol — a high-level description of the protocol in terms of ideal cryptographic components;
- Concrete Protocol — how the functions and encodings of the abstract protocol are instantiated;
- Network Upgrades — the strategy for upgrading the **Zcash** protocol.
- Consensus Changes from **Bitcoin** — how **Zcash** differs from **Bitcoin** at the consensus layer, including the Proof of Work;
- Differences from the **Zerocash** protocol — a summary of changes from the protocol in [BCGGMTV2014].
- Appendix: Circuit Design — details of how the Sapling circuits are defined as *quadratic constraint programs*.
- Appendix: Batching Optimizations — improvements to the efficiency of validating multiple signatures and verifying multiple proofs.

1.1 Caution

Zcash security depends on consensus. Should a program interacting with the **Zcash** network diverge from consensus, its security will be weakened or destroyed. The cause of the divergence doesn't matter: it could be a bug in your program, it could be an error in this documentation which you implemented as described, or it could be that you do everything right but other software on the network behaves unexpectedly. The specific cause will not matter to the users of your software whose wealth is lost.

Having said that, a specification of ***intended*** behaviour is essential for security analysis, understanding of the protocol, and maintenance of **Zcash** and related software. If you find any mistake in this specification, please file an issue at `https://github.com/zcash/zips/issues`, or if security-relevant please submit a GitHub Security Advisory to the relevant repository (see `https://github.com/zcash/zips/security/policy`).

1.2 High-level Overview

The following overview is intended to give a concise summary of the ideas behind the protocol, for an audience already familiar with *block chain*-based cryptocurrencies such as **Bitcoin**. It is imprecise in some aspects and is not part of the normative protocol specification. This overview applies to both Sprout and Sapling, differences in the cryptographic constructions used notwithstanding.

All value in **Zcash** belongs to some *chain value pool*. The current *chain value pools* are the *transparent pool*, *Sprout pool*, *Sapling pool*. Transfers of *transparent* value work essentially as in **Bitcoin** and have the same privacy properties. Value in a *shielded chain value pool* is carried by *notes*[2], which specify an amount and (indirectly) a *shielded payment address*, which is a destination to which *notes* can be sent. As in **Bitcoin**, this is associated with a *private key* that can be used to spend *notes* sent to the address; in **Zcash** this is called a *spending key*.

To each *note* there is cryptographically associated a *note commitment*. Once the *transaction* creating a *note* has been mined, the *note* is associated with a fixed *note position* in a tree of *note commitments*, and with a *nullifier*[2] unique to that *note*. Computing the *nullifier* requires the associated private *spending key* (or the *nullifier deriving key* for *Sapling-pool notes*). It is infeasible to correlate the *note commitment* or *note position* with the corresponding *nullifier* without knowledge of at least this key. An unspent valid *note*, at a given point on the *block chain*, is one for which the *note commitment* has been publically revealed on the *block chain* prior to that point, but the *nullifier* has not.

A *transaction* can contain *transparent* inputs, outputs, and scripts, which all work as in **Bitcoin** [Bitcoin-Protocol]. It also can include *JoinSplit descriptions*, *Spend descriptions*, and *Output descriptions*. Together these describe *shielded transfers* which take in *shielded input notes*, and/or produce *shielded output notes*. (For Sprout, each *JoinSplit description* handles up to two *shielded inputs* and up to two *shielded outputs*. For Sapling, each *shielded input* or *shielded output* has its own description.) It is also possible for value to be transferred between *chain value pools*, either *transparent* or *shielded*; this always reveals the amount transferred.

In each *shielded transfer*, the *nullifiers* of the input *notes* are revealed (preventing them from being spent again) and the commitments of the output *notes* are revealed (allowing them to be spent in future).

A *transaction* also includes computationally sound *zk-SNARK* proofs and signatures, which prove that all of the following hold except with insignificant probability:

For each *shielded input*,

- [**Sapling** onward] there is a revealed *value commitment* to the same value as the input *note*;
- if the value is nonzero, some revealed *note commitment* exists for this *note*;
- the prover knew the *proof authorizing key* of the *note*;
- the *nullifier* and *note commitment* are computed correctly.

and for each *shielded output*,

- [**Sapling** onward] there is a revealed *value commitment* to the same value as the output *note*;
- the *note commitment* is computed correctly;
- it is infeasible to cause the *nullifier* of the output *note* to collide with the *nullifier* of any other *note*.

For Sprout, the *JoinSplit statement* also includes an explicit balance check. For Sapling, the *value commitments* corresponding to the inputs and outputs are checked to balance (together with any net *transparent* input or output) outside the *zk-SNARK*.

In addition, various measures (differing between Sprout and Sapling) are used to ensure that the *transaction* cannot be modified by a party not authorized to do so.

[2] In **Zerocash** [BCGGMTV2014], *notes* were called "*coins*", and *nullifiers* were called "*serial numbers*".

Outside the *zk-SNARK*, it is checked that the *nullifiers* for the input *notes* had not already been revealed (i.e. they had not already been spent).

A *shielded payment address* includes a *transmission key* for a "*key-private*" asymmetric encryption scheme. *Key-private* means that ciphertexts do not reveal information about which key they were encrypted to, except to a holder of the corresponding *private key*, which in this context is called the *receiving key*. This facility is used to communicate encrypted output *notes* on the *block chain* to their intended recipient, who can use the *receiving key* to scan the *block chain* for *notes* addressed to them and then decrypt those *notes*.

In Sapling, for each *spending key* there is a *full viewing key* that allows recognizing both incoming and outgoing *notes* without having *spending authority*. This is implemented by an additional ciphertext in each *Output description*.

The basis of the privacy properties of **Zcash** is that when a *note* is spent, the spender only proves that some commitment for it had been revealed, without revealing which one. This implies that a spent *note* cannot be linked to the *transaction* in which it was created. That is, from an adversary's point of view the set of possibilities for a given *note* input to a *transaction* —its *note traceability set*— includes ***all*** previous notes that the adversary does not control or know to have been spent.[3] This contrasts with other proposals for private payment systems, such as CoinJoin [Bitcoin-CoinJoin] or CryptoNote [vanSaberh2014], that are based on mixing of a limited number of transactions and that therefore have smaller *note traceability sets*.

The *nullifiers* are necessary to prevent *double-spending*: each *note* on the *block chain* only has one valid *nullifier*, and so attempting to spend a *note* twice would reveal the *nullifier* twice, which would cause the second *transaction* to be rejected.

2 Notation

$\mathbb{B}$ means the type of bit values, i.e. $\{0, 1\}$. $\mathbb{B}^{\mathbb{Y}}$ means the type of byte values, i.e. $\{0 \,..\, 255\}$.

$\mathbb{N}$ means the type of nonnegative integers. $\mathbb{N}^+$ means the type of positive integers. $\mathbb{Z}$ means the type of integers. $\mathbb{Q}$ means the type of rationals.

$x : T$ is used to specify that x has type T. A cartesian product type is denoted by $S \times T$, and a function type by $S \to T$. An argument to a function can determine other argument or result types.

The type of a randomized algorithm is denoted by $S \xrightarrow{\mathrm{R}} T$. The domain of a randomized algorithm may be $()$, indicating that it requires no arguments. Given $f : S \xrightarrow{\mathrm{R}} T$ and $s : S$, sampling a variable $x : T$ from the output of f applied to s is denoted by $x \xleftarrow{\mathrm{R}} f(s)$.

Initial arguments to a function or randomized algorithm may be written as subscripts, e.g. if $x : X$, $y : Y$, and $f : X \times Y \to Z$, then an invocation of $f(x, y)$ can also be written $f_x(y)$.

$\{x : T \mid p_x\}$ means the subset of x from T for which p_x (a boolean expression depending on x) holds.

$T \subseteq U$ indicates that T is an inclusive subset or subtype of U.

$S \cup T$ means the set union of S and T.

$S \cap T$ means the set intersection of S and T, i.e. $\{x : S \mid x \in T\}$.

$S \setminus T$ means the set difference obtained by removing elements in T from S, i.e. $\{x : S \mid x \notin T\}$.

$x : T \mapsto e_x : U$ means the function of type $T \to U$ mapping formal parameter x to e_x (an expression depending on x). The types T and U are always explicit.

$x : T \mapsto_{\notin V} e_x : U$ means $x : T \mapsto e_x : U \cup V$ restricted to the domain $\{x : T \mid e_x \notin V\}$ and range U.

$\mathscr{P}(T)$ means the powerset of T.

[3] We make this claim only for ***fully shielded*** *transactions*. It does not exclude the possibility that an adversary may use data present in the cleartext of a *transaction* such as the number of inputs and outputs, or metadata-based heuristics such as timing, to make probabilistic inferences about *transaction* linkage. For consequences of this in the case of partially shielded *transactions*, see [Peterson2017], [Quesnelle2017], and [KYMM2018].

$\perp$ is a distinguished value used to indicate unavailable information, a failed decryption or validity check, or an exceptional case.

$T^{[\ell]}$, where T is a type and ℓ is an integer, means the type of sequences of length ℓ with elements in T. For example, $\mathbb{B}^{[\ell]}$ means the set of sequences of ℓ bits, and $\mathbb{B}^{\mathbb{Y}[k]}$ means the set of sequences of k bytes.

$\mathbb{B}^{\mathbb{Y}[\mathbb{N}]}$ means the type of *byte sequences* of arbitrary length.

$\mathsf{length}(S)$ means the length of (number of elements in) S.

$\mathsf{truncate}_k(S)$ means the sequence formed from the first k elements of S.

`0x` followed by a string of `monospace` hexadecimal digits means the corresponding integer converted from hexadecimal. $[\texttt{0x00}]^\ell$ means the sequence of ℓ zero bytes.

`"..."` means the given string represented as a sequence of bytes in *US-ASCII*. For example, **`"abc"`** represents the *byte sequence* [`0x61`, `0x62`, `0x63`].

$[0]^\ell$ means the sequence of ℓ zero bits. $[1]^\ell$ means the sequence of ℓ one bits.

$a..b$, used as a subscript, means the sequence of values with indices a through b inclusive. For example, $\mathsf{a}^{\mathsf{new}}_{\mathsf{pk},1..\mathrm{N}^{\mathsf{new}}}$ means the sequence $[\mathsf{a}^{\mathsf{new}}_{\mathsf{pk},1}, \mathsf{a}^{\mathsf{new}}_{\mathsf{pk},2}, \ldots \mathsf{a}^{\mathsf{new}}_{\mathsf{pk},\mathrm{N}^{\mathsf{new}}}]$. (For consistency with the notation in [BCGGMTV2014] and in [BK2016], this specification uses 1-based indexing and inclusive ranges, notwithstanding the compelling arguments to the contrary made in [EWD-831].)

$\{a\,..\,b\}$ means the set or type of integers from a through b inclusive.

[$f(x)$ for x from a up to b] means the sequence formed by evaluating f on each integer from a to b inclusive, in ascending order. Similarly, [$f(x)$ for x from a down to b] means the sequence formed by evaluating f on each integer from a to b inclusive, in descending order.

$a \,||\, b$ means the concatenation of sequences a then b.

$\mathsf{concat}_{\mathbb{B}}(S)$ means the sequence of bits obtained by concatenating the elements of S as *bit sequences*.

$\mathsf{sorted}(S)$ means the sequence formed by sorting the elements of S.

$\mathbb{F}_n$ means the finite field with n elements, and $\mathbb{F}_n^*$ means its group under multiplication (which excludes 0).

Where there is a need to make the distinction, we denote the unique representative of $a : \mathbb{F}_n$ in the range $\{0\,..\,n-1\}$ (or the unique representative of $a : \mathbb{F}_n^*$ in the range $\{1\,..\,n-1\}$) as $a \bmod n$. Conversely, we denote the element of $\mathbb{F}_n$ corresponding to an integer $k : \mathbb{Z}$ as $k \pmod n$. We also use the latter notation in the context of an equality $k = k' \pmod n$ as shorthand for $k \bmod n = k' \bmod n$, and similarly $k \neq k' \pmod n$ as shorthand for $k \bmod n \neq k' \bmod n$. (When referring to constants such as 0 and 1 it is usually not necessary to make the distinction between field elements and their representatives, since the meaning is normally clear from context.)

$\mathbb{F}_n[z]$ means the ring of polynomials over z with coefficients in $\mathbb{F}_n$.

$a + b$ means the sum of a and b. This may refer to addition of integers, rationals, finite field elements, or group elements (see § 4.1.8 ***'Represented Group'*** on p. 29) according to context.

$-a$ means the value of the appropriate integer, rational, finite field, or group type such that $(-a) + a = 0$ (or when a is an element of a group $\mathbb{G}$, $(-a) + a = \mathcal{O}_{\mathbb{G}}$), and $a - b$ means $a + (-b)$.

$a \cdot b$ means the product of multiplying a and b. This may refer to multiplication of integers, rationals, or finite field elements according to context (this notation is not used for group elements).

a/b, also written $\frac{a}{b}$, means the value of the appropriate integer, rational, or finite field type such that $(a/b) \cdot b = a$.

$a \bmod q$, for $a : \mathbb{N}$ and $q : \mathbb{N}^+$, means the remainder on dividing a by q. (This usage does not conflict with the notation above for the unique representative of a field element.)

$a \oplus b$ means the bitwise-exclusive-or of a and b, and $a \,\&\, b$ means the bitwise-and of a and b. These are defined on integers (which include bits and bytes), or elementwise on equal-length sequences of integers, according to context.

$\sum_{i=1}^{\mathrm{N}} a_i$ means the sum of $a_{1..\mathrm{N}}$. $\prod_{i=1}^{\mathrm{N}} a_i$ means the product of $a_{1..\mathrm{N}}$. $\bigoplus_{i=1}^{\mathrm{N}} a_i$ means the bitwise-exclusive-or of $a_{1..\mathrm{N}}$.

When $N = 0$ these yield the appropriate neutral element, i.e. $\sum_{i=1}^{0} a_i = 0$, $\prod_{i=1}^{0} a_i = 1$, and $\bigoplus_{i=1}^{0} a_i = 0$ or the all-zero *bit sequence* of length given by the type of a.

$\sqrt[+]{a}$, where $a : \mathbb{F}_q$, means the positive square root of a in $\mathbb{F}_q$, i.e. in the range $\{0 \,..\, \frac{q-1}{2}\}$. It is only used in cases where the square root must exist.

$\sqrt[?]{a}$, where $a : \mathbb{F}_q$, means an arbitrary square root of a in $\mathbb{F}_q$, or $\perp$ if no such square root exists.

$b \;?\; x : y$ means x when $b = 1$, or y when $b = 0$.

a^b, for a an integer or finite field element and $b : \mathbb{Z}$, means the result of raising a to the exponent b, i.e.

$$a^b := \begin{cases} \prod_{i=1}^{b} a, & \text{if } b \geq 0 \\ \prod_{i=1}^{-b} \frac{1}{a}, & \text{otherwise.} \end{cases}$$

The $[k]\,P$ notation for scalar multiplication in a group is defined in § 4.1.8 ***'Represented Group'*** on p. 29.

The convention of affixing $\star$ to a variable name is used for variables that denote *bit sequence* representations of group elements.

The binary relations $<$, $\leq$, $=$, $\geq$, and $>$ have their conventional meanings on integers and rationals, and are defined lexicographically on sequences of integers.

$\mathsf{floor}(x)$ means the largest integer $\leq x$. $\mathsf{ceiling}(x)$ means the smallest integer $\geq x$.

$\mathsf{bitlength}(x)$, for $x : \mathbb{N}$, means the smallest integer ℓ such that $2^\ell > x$.

The following integer constants will be instantiated in § 5.3 ***'Constants'*** on p. 57:

> $\mathsf{MerkleDepth}^{\mathsf{Sprout}}$, $\mathsf{MerkleDepth}^{\mathsf{Sapling}}$, $\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}$, $\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}$, $\mathrm{N}^{\mathsf{old}}$, $\mathrm{N}^{\mathsf{new}}$, ℓ_{value}, ℓ_{hSig}, $\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}$, $\ell_{\mathsf{PRFexpand}}$, $\ell_{\mathsf{PRFnfSapling}}$, $\ell^{\mathsf{Sprout}}_{\mathsf{rcm}}$, ℓ_{Seed}, $\ell_{\mathsf{a_{sk}}}$, $\ell^{\mathsf{Sprout}}_{\varphi}$, ℓ_{sk}, ℓ_{d}, $\ell^{\mathsf{Sapling}}_{\mathsf{ivk}}$, ℓ_{ovk}, $\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}$, MAX_MONEY, SlowStartInterval, HalvingInterval, MaxBlockSubsidy, NumFounderAddresses, PoWLimit, PoWAveragingWindow, PoWMedianBlockSpan, PoWDampingFactor, and PoWTargetSpacing.

The rational constants FoundersFraction, PoWMaxAdjustDown, and PoWMaxAdjustUp; and the *bit-sequence* constants $\mathsf{Uncommitted}^{\mathsf{Sprout}} : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}]}$ and $\mathsf{Uncommitted}^{\mathsf{Sapling}} : \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]}$ will also be defined in that section.

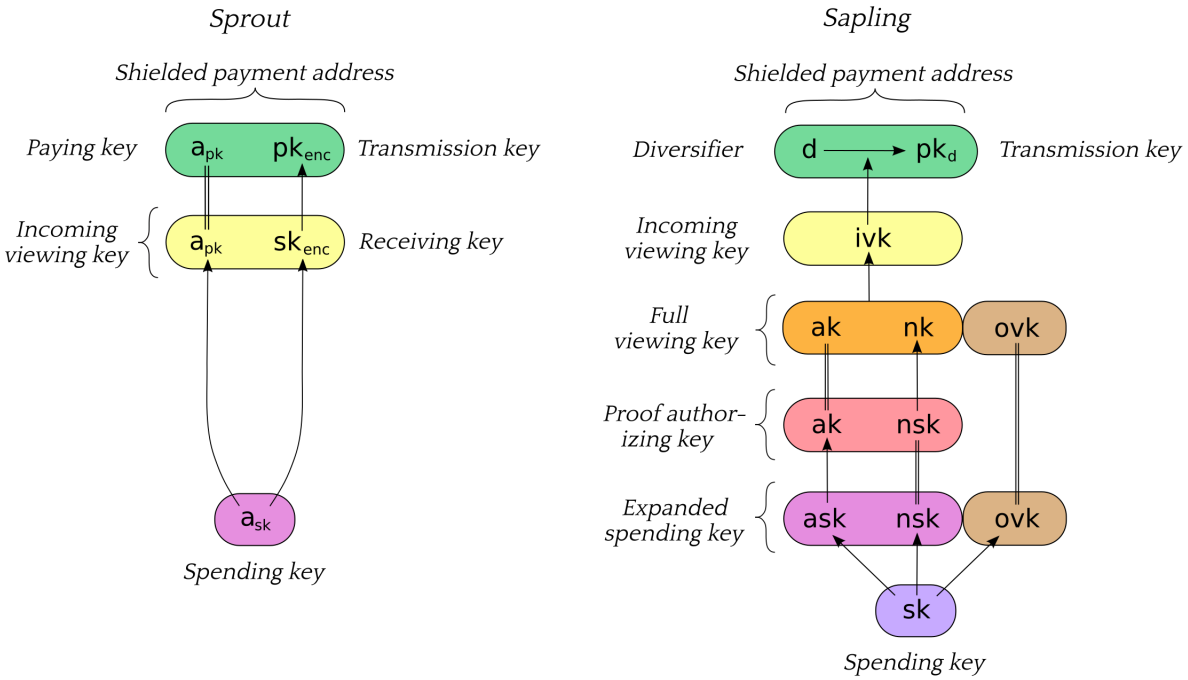
We use the abbreviation "*ctEdwards*" to refer to *complete twisted Edwards elliptic curves* and coordinates (see § 5.4.8.3 ***'Jubjub'*** on p. 77).

3 Concepts

3.1 Payment Addresses and Keys

Users who wish to receive shielded payments in the **Zcash** protocol must have a *shielded payment address*, which is generated from a *spending key*.

The following diagrams depict the relations between key components in Sprout and Sapling. Arrows point from a component to any other component(s) that can be derived from it. Double lines indicate that the same component is used in multiple abstractions.



[Sprout] A Sprout *receiving key* $\mathsf{sk}_{\mathsf{enc}}$, *incoming viewing key* $\mathsf{ivk} = (\mathsf{a}_{\mathsf{pk}}, \mathsf{sk}_{\mathsf{enc}})$, and *shielded payment address* $\mathsf{addr}_{\mathsf{pk}} = (\mathsf{a}_{\mathsf{pk}}, \mathsf{pk}_{\mathsf{enc}})$ are derived from the *spending key* a_{sk}, as described in § 4.2.1 *'Sprout Key Components'* on p. 32.

[**Sapling** onward] A Sapling *expanded spending key* is composed of a *Spend authorizing key* ask, a *nullifier private key* nsk, and an *outgoing viewing key* ovk. From these components we can derive a *proof authorizing key* (ak, nsk), a *full viewing key* (ak, nk, ovk), an *incoming viewing key* ivk, and a set of *diversified payment addresses* $\mathsf{addr}_{\mathsf{d}} = (\mathsf{d}, \mathsf{pk}_{\mathsf{d}})$, as described in § 4.2.2 *'Sapling Key Components'* on p. 32.

The consensus protocol does not depend on how an *expanded spending key* is constructed. Two methods of doing so are defined:

1. Generate a *spending key* sk at random and derive the *expanded spending key* (ask, nsk, ovk) from it, as shown in the diagram above and described in § 4.2.2 *'Sapling Key Components'* on p. 32.
2. Obtain an *extended spending key* as specified in [ZIP-32]; this includes a superset of the components of an *expanded spending key*. This method is used in the context of a *Hierarchical Deterministic Wallet*.

Non-normative note: Most **Zcash** wallets derive Sapling keys and addresses according to [ZIP-32].

The composition of *shielded payment addresses*, *incoming viewing keys*, *full viewing keys*, and *spending keys* is a cryptographic protocol detail that should not normally be exposed to users. However, user-visible operations should be provided to obtain a *shielded payment address*, *incoming viewing key*, or *full viewing key* from a *spending key* or *extended spending key*.

Users can accept payment from multiple parties with a single *shielded payment address* and the fact that these payments are destined to the same payee is not revealed on the *block chain*, even to the paying parties. ***However*** if two parties collude to compare a *shielded payment address* they can trivially determine they are the same. In the case that a payee wishes to prevent this they should create a distinct *shielded payment address* for each payer.

[**Sapling** onward] Sapling provides a mechanism to allow the efficient creation of *diversified payment addresses* with the same *spending authority*. A group of such addresses shares the same *full viewing key* and *incoming viewing key*, and so creating as many unlinkable addresses as needed does not increase the cost of scanning the *block chain* for relevant *transactions*.

Note: It is conventional in cryptography to call the key used to encrypt a message in an asymmetric encryption scheme a "*public key*". However, the *public key* used as the *transmission key* component of an address ($\mathsf{pk}_{\mathsf{enc}}$ or pk_{d}) need not be publically distributed; it has the same distribution as the *shielded payment address* itself. As mentioned above, limiting the distribution of the *shielded payment address* is important for some use cases. This also helps to reduce reliance of the overall protocol on the security of the cryptosystem used for *note* encryption (see § 4.17 ***'In-band secret distribution (Sprout)'*** on p. 50 and § 4.18 ***'In-band secret distribution (Sapling)'*** on p. 51), since an adversary would have to know $\mathsf{pk}_{\mathsf{enc}}$ or some pk_{d} in order to exploit a hypothetical weakness in that cryptosystem.

3.2 Shielded Pools and Notes

A *note* is a representation of value held in a *shielded pool*.

Each *shielded pool* operates according to the rules of a *shielded protocol*, and has its own *treestate* (see § 3.4 ***'Transactions and Treestates'*** on p. 16). The currently deployed *shielded pools* are the *Sprout pool* and *Sapling pool*.

Each *note* (denoted $\mathbf{n}$) belongs to one of these *shielded pools*. It represents that a value v is spendable by the recipient who holds the *spending key* corresponding to a given *shielded payment address*. The form of a *shielded payment address* is defined by a *shielded protocol*; it can be used to send funds represented by new *notes* to the address, in the *shielded pool(s)* supported by that protocol.

Let MAX_MONEY, $\ell_{\mathsf{PRF}}^{\mathsf{Sprout}}$, $\ell_{\mathsf{PRFnfSapling}}$, and ℓ_{d} be as defined in § 5.3 ***'Constants'*** on p. 57.

Let $\mathsf{NoteCommit}^{\mathsf{Sprout}}$ be as defined in § 5.4.7.1 ***'Note Commitments (Sprout)'*** on p. 72.

Let $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ be as defined in § 5.4.7.2 ***'Windowed Pedersen commitments'*** on p. 72.

Let $\mathsf{KA}^{\mathsf{Sapling}}$ be as defined in § 5.4.4.3 ***'Sapling Key Agreement'*** on p. 67.

Let $\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ be as defined in § 5.4.1.6 ***'$\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ Hash Function'*** on p. 61.

A *Sprout-pool note* is a tuple $(\mathsf{a}_{\mathsf{pk}}, \mathsf{v}, \rho, \mathsf{rcm})$, where:

- $\mathsf{a}_{\mathsf{pk}} : \mathbb{B}^{[\ell_{\mathsf{PRF}}^{\mathsf{Sprout}}]}$ is the *paying key* of the recipient's *shielded payment address*;
- $\mathsf{v} : \{0 \,..\, \mathsf{MAX_MONEY}\}$ is an integer representing the value of the *note* in zatoshi (1 **ZEC** = 10^8 zatoshi);
- $\rho : \mathbb{B}^{[\ell_{\mathsf{PRF}}^{\mathsf{Sprout}}]}$ is used as input to $\mathsf{PRF}_{\mathsf{a}_{\mathsf{sk}}}^{\mathsf{nfSprout}}$ to derive the *nullifier* of the *note*;
- $\mathsf{rcm} : \mathsf{NoteCommit}^{\mathsf{Sprout}}.\mathsf{Trapdoor}$ is a random *commitment trapdoor* as defined in § 4.1.7 ***'Commitment'*** on p. 27.

Let $\mathsf{Note}^{\mathsf{Sprout}}$ be the type of a *Sprout-pool note*, i.e.

$$\mathsf{Note}^{\mathsf{Sprout}} := \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}]} \times \{0 \,..\, \mathsf{MAX_MONEY}\} \times \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}]} \times \mathsf{NoteCommit}^{\mathsf{Sprout}}.\mathsf{Trapdoor}.$$

A *Sapling-pool note* is a tuple $(\mathsf{d}, \mathsf{pk_d}, \mathsf{v}, \mathsf{rcm})$, where:

- $\mathsf{d} : \mathbb{B}^{[\ell_{\mathsf{d}}]}$ is the *diversifier* of the recipient's *shielded payment address*;
- $\mathsf{pk_d} : \mathsf{KA}^{\mathsf{Sapling}}.\mathsf{PublicPrimeOrder}$ is the *diversified transmission key* of the recipient's *shielded payment address*;
- $\mathsf{v} : \{0 \,..\, \mathsf{MAX_MONEY}\}$ is an integer representing the value of the *note* in zatoshi;
- $\mathsf{rcm} : \mathsf{NoteCommit}^{\mathsf{Sapling}}.\mathsf{Trapdoor}$ is a random *commitment trapdoor* as defined in § 4.1.7 *'Commitment'* on p. 27.

Let $\mathsf{Note}^{\mathsf{Sapling}}$ be the type of a *Sapling-pool note*, i.e.

$$\mathsf{Note}^{\mathsf{Sapling}} := \mathbb{B}^{[\ell_{\mathsf{d}}]} \times \mathsf{KA}^{\mathsf{Sapling}}.\mathsf{PublicPrimeOrder} \times \{0 \,..\, \mathsf{MAX_MONEY}\} \times \mathsf{NoteCommit}^{\mathsf{Sapling}}.\mathsf{Trapdoor}.$$

Creation of new *notes* is described in § 4.6 *'Sending Notes'* on p. 37.

3.2.1 Note Plaintexts and Memo Fields

Transmitted *notes* are stored on the *block chain* in encrypted form, together with a representation of the *note commitment* cm described in § 3.2.2 *'Note Commitments'* on p. 15.

A *note plaintext* also includes a 512-byte *memo field* associated with this *note*. The usage of the *memo field* is by agreement between the sender and recipient of the *note*. **RECOMMENDED** non-consensus constraints on the *memo field* contents are specified in [ZIP-302].

For Sprout, the *note plaintexts* in each *JoinSplit description* are encrypted to the respective *transmission keys* $\mathsf{pk}^{\mathsf{new}}_{\mathsf{enc},1..N^{\mathsf{new}}}$, as specified in § 4.6.1 *'Sending Notes (Sprout)'* on p. 37.

Each *Sprout-pool note plaintext* (denoted $\mathbf{np}$) consists of

$$(\mathsf{leadByte} : \mathbb{B}^{\mathbb{Y}}, \mathsf{v} : \{0 \,..\, 2^{\ell_{\mathsf{value}}} - 1\}, \rho : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}]}, \mathsf{rcm} : \mathsf{NoteCommit}^{\mathsf{Sprout}}.\mathsf{Trapdoor}, \mathsf{memo} : \mathbb{B}^{\mathbb{Y}^{[512]}}).$$

The field leadByte is always 0x00 for Sprout. The fields v, ρ, and rcm are as defined in § 3.2 *'Shielded Pools and Notes'* on p. 13.

[**Sapling** onward] For Sapling, the *note plaintext* in each *Output description* is encrypted to the *diversified payment address* $(\mathsf{d}, \mathsf{pk_d})$, as specified in § 4.6.2 *'Sending Notes (Sapling)'* on p. 37.

Each *Sapling-pool note plaintext* (denoted $\mathbf{np}$) consists of

$$(\mathsf{leadByte} : \mathbb{B}^{\mathbb{Y}}, \mathsf{d} : \mathbb{B}^{[\ell_{\mathsf{d}}]}, \mathsf{v} : \{0 \,..\, 2^{\ell_{\mathsf{value}}} - 1\}, \underline{\mathsf{rcm}} : \mathbb{B}^{\mathbb{Y}^{[32]}}, \mathsf{memo} : \mathbb{B}^{\mathbb{Y}^{[512]}})$$

The field leadByte indicates the version of the encoding of this *note plaintext*. The allowed *note plaintext lead bytes* differ based on the *shielded pool* and *block height*, as described below.

Let the constants CanopyActivationHeight and ZIP212GracePeriod be as defined in § 5.3 *'Constants'* on p. 57.

Let height be the *block height* of the *block* containing the *transaction* having the encrypted *note plaintext* as an output, and let txVersion be the *transaction version number*.

Let $\mathit{pool} : \{\mathit{SaplingPool}\}$ be the *chain value pool* of the *note*.

Define $\mathsf{allowedLeadBytes}^{\mathit{pool}}(\mathsf{height}, \mathsf{txVersion}) := \{\mathsf{0x01}\}$.

The leadByte of a *Sapling-pool note plaintext* **MUST** satisfy $\mathsf{leadByte} \in \mathsf{allowedLeadBytes}^{\mathit{pool}}(\mathsf{height}, \mathsf{txVersion})$. Senders **SHOULD** choose the highest leadByte allowed under this condition.

Non-normative notes:

- It is intentional that the definition of allowedLeadBytes does not currently depend on *pool* or txVersion. It might do so in future.

The fields d, v, and rcm are as defined in § 3.2 *'Shielded Pools and Notes'* on p. 13.

Encodings are given in § 5.5 *'Encodings of Note Plaintexts and Memo Fields'* on p. 81. The result of encryption forms part of a *transmitted note(s) ciphertext*. For further details, see § 4.17 *'In-band secret distribution (Sprout)'* on p. 50 and § 4.18 *'In-band secret distribution (Sapling)'* on p. 51.

3.2.2 Note Commitments

When a *note* is created as an output of a *transaction*, only a commitment (see § 4.1.7 *'Commitment'* on p. 27) to the *note* contents is disclosed in the associated *JoinSplit description* or *Output description*. If the *transaction* is entered into the *block chain*, each such *note commitment* is appended to the *note commitment tree* of the associated *treestate*. This allows the value and recipient to be kept private, while the commitment is used by the *zk-SNARK proof* when the *note* is spent, to check that it exists on the *block chain*.

Treestates are described in § 3.4 *'Transactions and Treestates'* on p. 16, and *note commitment trees* are described in § 3.7 *'Note Commitment Trees'* on p. 19.

A *Sprout-pool note commitment* on a *note* $\mathbf{n} = (\mathsf{a_{pk}}, \mathsf{v}, \rho, \mathsf{rcm})$ is computed as

$$\mathsf{NoteCommitment}^{\mathsf{Sprout}}(\mathbf{n}) = \mathsf{NoteCommit}^{\mathsf{Sprout}}_{\mathsf{rcm}}(\mathsf{a_{pk}}, \mathsf{v}, \rho),$$

where $\mathsf{NoteCommit}^{\mathsf{Sprout}}$ is instantiated in § 5.4.7.1 *'Note Commitments (Sprout)'* on p. 72.

A *Sapling-pool note commitment* on a *note* $\mathbf{n} = (\mathsf{d}, \mathsf{pk_d}, \mathsf{v}, \mathsf{rcm})$ is computed as

$$\mathsf{g_d} := \mathsf{DiversifyHash}^{\mathsf{Sapling}}(\mathsf{d})$$

$$\mathsf{NoteCommitment}^{\mathsf{Sapling}}(\mathbf{n}) := \begin{cases} \bot, & \text{if } \mathsf{g_d} = \bot \\ \mathsf{NoteCommit}^{\mathsf{Sapling}}_{\mathsf{rcm}}(\mathsf{repr}_{\mathbb{J}}(\mathsf{g_d}), \mathsf{repr}_{\mathbb{J}}(\mathsf{pk_d}), \mathsf{v}), & \text{otherwise.} \end{cases}$$

where $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ is instantiated in § 5.4.7.2 *'Windowed Pedersen commitments'* on p. 72.

Notice that the above definition of a *Sapling-pool note* does not have a ρ component. There is in fact a ρ value associated with each *Sapling-pool note*, but this can only be computed once its position in the *note commitment tree* (see § 3.4 *'Transactions and Treestates'* on p. 16) is known. We refer to the combination of a *note* and its *note position* pos, as a *positioned note*.

For a *positioned note*, we can compute the value ρ as described in § 4.14 *'Computing ρ values and Nullifiers'* on p. 45.

A *Sapling-pool note commitment* is represented in an *Output description* by the u-coordinate of a Jubjub curve point, as specified in § 4.5 *'Output Descriptions'* on p. 36.

3.2.3 Nullifiers

The *nullifier* for a *note*, denoted nf, is a value unique to the *note* that is used to prevent *double-spends*. When a *transaction* that contains one or more *JoinSplit descriptions* or *Spend descriptions* is entered into the *block chain*, all of the *nullifiers* for *notes* spent by that *transaction* are added to the *nullifier set* of the associated *treestate*. A *transaction* is not valid if it would have added a *nullifier* to the *nullifier set* that already exists in the set.

Treestates are described in § 3.4 *'Transactions and Treestates'* on p. 16, and *nullifier sets* are described in § 3.8 *'Nullifier Sets'* on p. 19.

In more detail, when a *note* is spent, the spender creates a *zero-knowledge proof* that it knows $(\rho, \mathsf{a_{sk}})$ or $(\rho, \mathsf{ak}, \mathsf{nsk})$, consistent with the publically disclosed *nullifier* and some previously committed *note commitment*.

Because each *note* can have only a single *nullifier*, and the same *nullifier* value cannot appear more than once in a *valid block chain*, *double-spending* is prevented.

The *nullifier* for a *Sprout-pool note* is derived from the ρ value and the recipient's *spending key* $\mathsf{a_{sk}}$.

The *nullifier* for a *Sapling-pool note* is derived from the ρ value and the recipient's *nullifier deriving key* nk.

The *nullifier* computation uses a *Pseudo Random Function* (see § 4.1.2 ***'Pseudo Random Functions'*** on p. 22), as described in § 4.14 ***'Computing ρ values and Nullifiers'*** on p. 45.

3.3 The Block Chain

At a given point in time, each *full validator* is aware of a set of candidate *blocks*. These form a tree rooted at the *genesis block*, where each node in the tree refers to its parent via the `hashPrevBlock` *block header* field (see § 7.5 ***'Block Header Encoding and Consensus'*** on p. 95).

A path from the root toward the leaves of the tree consisting of a sequence of one or more valid *blocks* consistent with consensus rules, is called a *valid block chain*.

Each *block* in a *block chain* has a *block height*. The *block height* of the *genesis block* is 0, and the *block height* of each subsequent *block* in the *block chain* increments by 1. Implementations **MUST** support *block heights* up to and including $2^{31} - 1$.

In order to choose the *best valid block chain* in its view of the overall *block* tree, a node sums the work, as defined in § 7.6.5 ***'Definition of Work'*** on p. 100, of all *blocks* in each *valid block chain*, and considers the *valid block chain* with greatest total work to be best. To break ties between leaf *blocks*, a node will prefer the *block* that it received first.

The consensus protocol is designed to ensure that for any given *block height*, the vast majority of well-connected nodes should eventually agree on their *best valid block chain* up to that height. A *full validator*[4] **SHOULD** attempt to obtain candidate *blocks* from multiple sources in order to increase the likelihood that it will find a *valid block chain* that reflects a recent consensus state.

A *network upgrade* is *settled* on a given *network* when there is a social consensus that it has activated with a given *activation block* hash. A *full validator* that potentially risks *Mainnet* funds or displays *Mainnet transaction* information to a user **MUST** do so only for a *block chain* that includes the *activation block* of the most recent *settled network upgrade*, with the corresponding *activation block* hash. Currently, there is social consensus that **NU6.3** has activated on the **Zcash** *Mainnet* and *Testnet* with the *activation block* hashes given in § 3.11 ***'Mainnet and Testnet'*** on p. 20.

A *full validator* **MAY** impose a limit on the number of *blocks* it will "roll back" when switching from one *best valid block chain* to another that is not a descendent. For *zcashd* and *zebra* this limit is 100 *blocks*.

3.4 Transactions and Treestates

Each *block* contains one or more *transactions*.

Each *transaction* has a *transaction ID*. *Transaction IDs* are used to refer to *transactions* in `tx_out` fields, in *leaf nodes* of a *block*'s *transaction* tree rooted at `hashMerkleRoot`, and in other parts of the ecosystem; for example they are shown in *block chain* explorers and can be used in higher-level protocols. The computation of *transaction IDs* is described in § 7.1.1 ***'Transaction Identifiers'*** on p. 90.

Transparent inputs to a *transaction* insert value into a *transparent transaction value pool* associated with the *transaction*, and *transparent outputs* remove value from this pool.

The effects of each *shielded protocol* on the *transparent transaction value pool* are described in the following sections:

[4] There is reason to follow the requirements in this section also for non-full validators, but those are outside the scope of this protocol specification.

- *Sprout-pool JoinSplit transfers* in § 4.16.1 ***'JoinSplit Statement (Sprout)'*** on p. 47;
- *Sapling-pool Spend transfers* and *Output transfers* in § 4.12 ***'Balance and Binding Signature (Sapling)'*** on p. 42;

As in **Bitcoin**, the remaining value in the *transparent transaction value pool* of a non-coinbase *transaction* is a *transaction fee*. The sum of those values for non-coinbase *transactions* in each *block* is available to miners: it is treated as an implicit input to the *transparent transaction value pool* balance of the *block*'s *coinbase transaction* (in addition to the implicit input created by issuance).

The remaining value in the *transparent transaction value pool* of *coinbase transactions* is left unclaimed and is not added to any *chain value pool*.

Consensus rule: The remaining value in the *transparent transaction value pool* **MUST** be nonnegative.

To each *transaction* there are associated initial *treestates* for the *Sprout pool* and *Sapling pool*. Each *treestate* consists of:

- a *note commitment tree* (§ 3.7 ***'Note Commitment Trees'*** on p. 19);
- a *nullifier set* (§ 3.8 ***'Nullifier Sets'*** on p. 19).

Validation state associated with *transparent* inputs and outputs, such as the *UTXO (unspent transaction output) set*, is not described in this document; it is used in essentially the same way as in **Bitcoin**.

An *anchor* is a Merkle tree root of one of these *note commitment trees*. It uniquely identifies a *note commitment tree* state given the assumed security properties of the Merkle tree's *hash function*. Since the *nullifier set* is always updated together with the *note commitment tree*, this also identifies a particular state of the associated *nullifier set*.

In a given *block chain*, for each of the *Sprout pool* and *Sapling pool*, *treestates* are chained as follows:

- The input *treestate* of the first *block* is the empty *treestate*.
- The input *treestate* of the first *transaction* of a *block* is the final *treestate* of the immediately preceding *block*.
- The input *treestate* of each subsequent *transaction* in a *block* is the output *treestate* of the immediately preceding *transaction*.
- The final *treestate* of a *block* is the output *treestate* of its last *transaction*.

JoinSplit descriptions also have interstitial input and output *treestates* for the *Sprout pool*, explained in the following section. There is no equivalent of interstitial *treestates* for the *Sapling pool*.

3.5 JoinSplit Transfers and Descriptions

A *JoinSplit description* is data included in a *transaction* that describes a *JoinSplit transfer*, i.e. a *shielded* value transfer. In Sprout, this kind of value transfer was the primary **Zcash**-specific operation performed by *transactions*.

A *JoinSplit transfer* spends $\mathrm{N}^{\mathsf{old}}$ *notes* $\mathbf{n}^{\mathsf{old}}_{1..\mathrm{N}^{\mathsf{old}}}$ and *transparent input* $\mathrm{v}^{\mathsf{old}}_{\mathsf{pub}}$, and creates $\mathrm{N}^{\mathsf{new}}$ *notes* $\mathbf{n}^{\mathsf{new}}_{1..\mathrm{N}^{\mathsf{new}}}$ and *transparent output* $\mathrm{v}^{\mathsf{new}}_{\mathsf{pub}}$. It is associated with a *JoinSplit statement* instance (§ 4.16.1 ***'JoinSplit Statement (Sprout)'*** on p. 47), for which it provides a *zk-SNARK proof*.

Each *transaction* has a sequence of *JoinSplit descriptions*.

The total $\mathrm{v}^{\mathsf{new}}_{\mathsf{pub}}$ value adds to, and the total $\mathrm{v}^{\mathsf{old}}_{\mathsf{pub}}$ value subtracts from the *transparent transaction value pool* of the containing *transaction*.

The *anchor* of each *JoinSplit description* in a *transaction* refers to a *Sprout-pool treestate*.

For each of the $\mathrm{N}^{\mathsf{old}}$ *shielded inputs*, a *nullifier* is revealed. This allows detection of *double-spends* as described in § 3.8 ***'Nullifier Sets'*** on p. 19.

For each *JoinSplit description* in a *transaction*, an interstitial output *treestate* is constructed which adds the *note commitments* and *nullifiers* specified in that *JoinSplit description* to the input *treestate* referred to by its *anchor*. This interstitial output *treestate* is available for use as the *anchor* of subsequent *JoinSplit descriptions* in the same *transaction*. In general, therefore, the set of interstitial *treestates* associated with a *transaction* forms a tree in which the parent of each node is determined by its *anchor*.

Interstitial *treestates* are necessary because when a *transaction* is constructed, it is not known where it will eventually appear in a mined *block*. Therefore the *anchors* that it uses must be independent of its eventual position.

The input and output values of each *JoinSplit transfer* **MUST** balance exactly. This is not a consensus rule since it cannot be checked directly; it is enforced by the **Balance** rule of the *JoinSplit statement*.

Consensus rules:

- For the first *JoinSplit description* of a *transaction*, the *anchor* **MUST** be the output *Sprout-pool treestate* of a previous *block*.
- The *anchor* of each *JoinSplit description* in a *transaction* **MUST** refer to either some earlier *block*'s final *Sprout-pool treestate*, or to the interstitial output *treestate* of any prior *JoinSplit description* in the same *transaction*.

3.6 Spend Transfers, Output Transfers, and their Descriptions

JoinSplit transfers are not used for *Sapling-pool notes*. Instead, there is a separate *Spend transfer* for each *shielded input*, and a separate *Output transfer* for each *shielded output*.

Spend descriptions and *Output descriptions* are data included in a *transaction* that describe *Spend transfers* and *Output transfers*, respectively.

A *Spend transfer* spends a *note* $\mathbf{n}^{\mathsf{old}}$. Its *Spend description* includes a *Pedersen value commitment* to the value of the *note*. It is associated with an instance of a *Spend statement* (§ 4.16.2 ***'Spend Statement (Sapling)'*** on p. 48) for which it provides a *zk-SNARK proof*.

An *Output transfer* creates a *note* $\mathbf{n}^{\mathsf{new}}$. Similarly, its *Output description* includes a *Pedersen value commitment* to the *note* value. It is associated with an instance of an *Output statement* (§ 4.16.3 ***'Output Statement (Sapling)'*** on p. 49) for which it provides a *zk-SNARK proof*.

Each *transaction* has a sequence of *Spend descriptions* and a sequence of *Output descriptions*.

To ensure balance, we use a homomorphic property of *Pedersen commitments* that allows them to be added and subtracted, as elliptic curve points (§ 5.4.7.3 ***'Homomorphic Pedersen commitments (Sapling)'*** on p. 73). The result of adding two *Pedersen value commitments*, committing to values v_1 and v_2, is a new *Pedersen value commitment* that commits to $v_1 + v_2$. Subtraction works similarly.

Therefore, balance can be enforced by adding all of the *value commitments* for *shielded inputs*, subtracting all of the *value commitments* for *shielded outputs*, and proving by use of a *Sapling binding signature* (as described in § 4.12 ***'Balance and Binding Signature (Sapling)'*** on p. 42) that the result commits to a value consistent with the net *transparent* value change. This approach allows all of the *zk-SNARK statements* to be independent of each other, potentially increasing opportunities for precomputation.

A *Spend description* specifies an *anchor*, which refers to the output *Sapling-pool treestate* of a previous *block*. It also reveals a *nullifier*, which allows detection of *double-spends* as described in § 3.2.3 ***'Nullifiers'*** on p. 15.

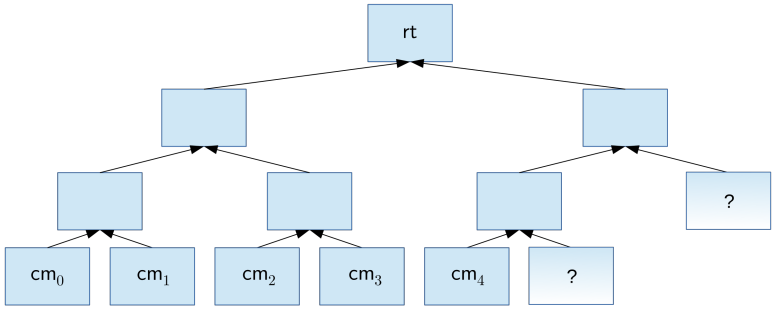
Non-normative note: Interstitial *treestates* are not necessary for Sapling, because a *Spend transfer* in a given *transaction* cannot spend any of the *shielded outputs* of the same *transaction*. This is not an onerous restriction because, unlike Sprout where each *JoinSplit transfer* must balance individually, in Sapling it is only necessary for the whole *transaction* to balance.

Consensus rules:

- The *Spend transfers* and *Action transfers* of a *transaction* **MUST** be consistent with its $v^{SaplingPoolBalance}$ value as specified in § 4.12 ***'Balance and Binding Signature (Sapling)'*** on p. 42.
- The *anchor* of each *Spend description* **MUST** refer to some earlier *block*'s final *Sapling-pool treestate*.

3.7 Note Commitment Trees

Let ℓ^{Sprout}_{Merkle}, $\mathsf{MerkleDepth}^{Sprout}$, $\ell^{Sapling}_{Merkle}$, and $\mathsf{MerkleDepth}^{Sapling}$ be as defined in § 5.3 ***'Constants'*** on p. 57.



A *note commitment tree* is an *incremental Merkle tree*, of fixed depth defined by each *shielded protocol*, used to store *note commitments* that *JoinSplit transfers* or *Spend transfers* produce. Just as the *UTXO (unspent transaction output) set* used in **Bitcoin**, it is used to express the existence of value and the capability to spend it. However, unlike the *UTXO set*, it is ***not*** the job of this tree to protect against *double-spending*, as it is append-only.

A *root* of a *note commitment tree* is associated with the *treestate* (§ 3.4 ***'Transactions and Treestates'*** on p. 16) for each *shielded pool*.

Each *node* in the *incremental Merkle tree* is associated with a *hash value* of size ℓ^{Sprout}_{Merkle} or $\ell^{Sapling}_{Merkle}$ bits. The *layer* numbered h, counting from *layer* 0 at the *root*, has 2^h *nodes* with *indices* 0 to $2^h - 1$ inclusive. The *hash value* associated with the *node* at *index* i in *layer* h is denoted M^h_i.

The *index* of a *note's commitment* at the leafmost layer ($\mathsf{MerkleDepth}^{Sprout}$ or $\mathsf{MerkleDepth}^{Sapling}$) is called its *note position*.

Consensus rules:

- A *block* **MUST NOT** add *Sprout-pool note commitments* that would result in the *Sprout-pool note commitment tree* exceeding its capacity of $2^{\mathsf{MerkleDepth}^{Sprout}}$ *leaf nodes*.
- [**Sapling** onward] A *block* **MUST NOT** add *Sapling-pool note commitments* that would result in the *Sapling-pool note commitment tree* exceeding its capacity of $2^{\mathsf{MerkleDepth}^{Sapling}}$ *leaf nodes*.

3.8 Nullifier Sets

Each *full validator* maintains a *nullifier set* logically associated with the *treestate* for each *shielded pool*. As valid *transactions* containing *JoinSplit transfers* or *Spend transfers* are processed, the *nullifiers* revealed in *JoinSplit descriptions* and *Spend descriptions* are inserted into the *nullifier set* associated with the new *treestate*. *Nullifiers* are enforced to be unique within a *valid block chain*, in order to prevent *double-spends*.

Consensus rule: A *nullifier* **MUST NOT** repeat either within a *transaction*, or across *transactions* in a *valid block chain*. *Sprout-pool* and *Sapling-pool nullifiers* are considered disjoint, even if they have the same bit pattern.

3.9 Block Subsidy and Founders' Reward

Like **Bitcoin**, **Zcash** creates currency when *blocks* are mined. The value created on mining a *block* is called the *block subsidy*.

The *block subsidy* is composed of a *miner subsidy* and a *Founders' Reward*.

As in **Bitcoin**, the miner of a *block* also receives *transaction fees*.

The calculations of the *block subsidy*, *miner subsidy*, and *Founders' Reward* depend on the *block height*, as defined in § 3.3 *'The Block Chain'* on p. 16.

§ 7.7 *'Calculating Block Subsidy and Founders' Reward'* on p. 100 describes these calculations.

3.10 Coinbase Transactions

A *transaction* that has a single *transparent input* with a null `prevout` field, is called a *coinbase transaction*. Every *block* has a single *coinbase transaction* as the first *transaction* in the *block*. The purpose of this *coinbase transaction* is to collect and spend any *miner subsidy*, and the *transaction fees* paid by other *transactions* included in the *block*.

§ 7.8 *'Payment of Founders' Reward'* on p. 101 specifies that the *coinbase transaction* **MUST** also pay the *Founders' Reward*.

3.11 Mainnet and Testnet

The production **Zcash** *network*, which supports the **ZEC** token, is called *Mainnet*. Governance of its protocol is by social consensus on which *full validator* implementations are considered to be faithful implementations of the intended *Mainnet* consensus rules, and on how those implementations should be modified. Currently these implementations are *zebra* maintained by the Zcash Foundation, and *zcashd* maintained by the Zcash Open Development Lab.

Subject to errors and omissions, each version of this document intends to describe some version (or planned version) of the **Zcash** protocol.

All *block hashes* given in this section are in *RPC byte order* (that is, byte-reversed relative to the normal order for a SHA-256 hash).

Mainnet genesis block: `00040fe8ec8471911baa1db1266ea15dd06b4a8a5c453883c000b031973dce08`

Mainnet **NU6.3** *activation block*: `00000000001a8b54bbde4e8996373417a6b33ee2bd984bcf02882e5d8128d761`

Mainnet **NU7** *activation block*: `0000000000246a2b1fecb3f465770894d146f6b7044e46b13ffb65e57e4ccd4a`

There is also a public test *network* called *Testnet*. It supports a **TAZ** token which is intended to have no monetary value. By convention, *Testnet* normally activates *network upgrades* (as described in § 6 *'Network Upgrades'* on p. 88) before *Mainnet*, in order to allow for errors or ambiguities in their specification and implementation to be discovered. The *Testnet block chain* is subject to being rolled back to a prior *block* at any time.

Testnet genesis block: `05a60a92d99d85997cce3b87616c089f6124d7342af37106edc76126334a2c38`

Testnet **NU6.3** *activation block*: `000003943e1a2c60c17b69ed4d448ea9296019a247f1134b507917941957fccd`

Testnet **NU7** *activation block*: `000089ba27100beede16d64b34e3d1b626b428cb7ee9fe6dcfdc217ce24e78af`

We call the smallest units of currency (on either *network*) zatoshi.[5]

On *Mainnet*, 1 **ZEC** = 10^8 zatoshi. On *Testnet*, 1 **TAZ** = 10^8 zatoshi.

Other *networks* using variants of the **Zcash** protocol may exist, but are not described by this specification.

[5] *"tazoshi"* may be used for the smallest units of currency on Testnet, but it is usually more convenient to use a *network*-independent term.

4 Abstract Protocol

> *'We all know that the only mental tool by means of which a very finite piece of reasoning can cover a myriad cases is called "abstraction"; as a result the effective exploitation of [their] powers of abstraction must be regarded as one of the most vital activities of a competent programmer. In this connection it might be worth-while to point out that the purpose of abstracting is* ***not*** *to be vague, but to create a new semantic level in which one can be absolutely precise.'*
>
> — Edsger Dijkstra, "The Humble Programmer" [EWD-340]

Abstraction is an incredibly important idea in the design of any complex system. Without abstraction, we would not be able to design anything as ambitious as a computer, or a cryptographic protocol. Were we to attempt it, the computer would be hopelessly unreliable or the protocol would be insecure, if they could be completed at all.

The aim of abstraction is primarily to limit how much a human working on a piece of a system has to keep in mind at one time, in order to apprehend the connections of that piece to the remainder. The work could be to extend or maintain the system, to understand its security or other properties, or to explain it to others.

In this specification, we make use wherever possible of abstractions developed by the cryptography community to model cryptographic primitives: *Pseudo Random Functions*, *commitment schemes*, *signature schemes*, etc. Each abstract primitive has associated syntax (its interface as used by the rest of the system) and security properties, as documented in this part. Their instantiations are documented in part § 5 ***'Concrete Protocol'*** on p. 56.

In some cases this syntax or these security requirements have been extended to meet the needs of the **Zcash** protocol. For example, some of the PRFs used in **Zcash** need to be *collision-resistant*, which is not part of the usual security requirement for a PRF; some *signature schemes* need to support additional functionality and security properties; and so on. Also, security requirements are sometimes intentionally stronger than what is known to be needed, because the stronger property is simpler or less error-prone to work with, and/or because it has been studied in the cryptographic literature in more depth.

We explicitly ***do not claim***, however, that all of these instantiations satisfying their documented syntax and security requirements would be sufficient for security or correctness of the overall **Zcash** protocol, or that it is always necessary. The claim is only that it helps to understand the protocol; that is, that analysis or extension is simplified by making use of the abstraction. In other words, ***a good way to understand*** the use of that primitive in the protocol is to model it as an instance of the given abstraction. And furthermore, if the instantiated primitive does not in fact satisfy the requirements of the abstraction, then this is an error that should be corrected –whether or not it leads to a vulnerability– since that would compromise the facility to understand its use in terms of the abstraction.

In this respect the abstractions play a similar rôle to that of a type system (which we also use): they add a form of redundancy to the specification that helps to express the intent.

Each property is a claim that may be incorrect (or that may be insufficiently precisely stated to determine whether it is correct). An example of an incorrect security claim occurs in the **Zerocash** protocol [BCGGMTV2014]: the instantiation of the *note commitment* scheme used in **Zerocash** failed to be *binding* at the intended security level (see § 8.5 ***'Internal hash collision attack and fix'*** on p. 106).

Another hazard that we should be aware of is that abstractions can be "leaky": an instantiation may impose conditions on its correct or secure use that are not captured by the abstraction's interface and semantics. Ideally, the abstraction would be changed to explicitly document these conditions, or the protocol changed to rely only on the original abstraction.

An abstraction can also be incomplete (not quite the same thing as being leaky): it intentionally –usually for simplicity– does not model an aspect of behaviour that is important to security or correctness. An example would be resistance to *side-channel* attacks; this specification says little about *side-channel* defence, among many other implementation concerns.

4.1 Abstract Cryptographic Schemes

4.1.1 Hash Functions

Let $\mathsf{MerkleDepth}^{\mathsf{Sprout}}$, $\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}$, $\mathsf{MerkleDepth}^{\mathsf{Sapling}}$, $\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}$, $\ell^{\mathsf{Sapling}}_{\mathsf{ivk}}$, ℓ_{d}, ℓ_{Seed}, $\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}$, ℓ_{hSig}, and $\mathrm{N}^{\mathsf{old}}$ be as defined in § 5.3 *'Constants'* on p. 57.

Let $\mathbb{J}$, $\mathbb{J}^{(r)}$, $\mathbb{J}^{(r)*}$, $r_{\mathbb{J}}$, and $\ell_{\mathbb{J}}$ be as defined in § 5.4.8.3 *'Jubjub'* on p. 77.

The following *hash functions* are used in § 4.8 ***'Merkle Path Validity'*** on p. 39:

$$\mathsf{MerkleCRH}^{\mathsf{Sprout}} : \{0 .. \mathsf{MerkleDepth}^{\mathsf{Sprout}} - 1\} \times \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}]} \times \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}]} \to \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}]}$$
$$\mathsf{MerkleCRH}^{\mathsf{Sapling}} : \{0 .. \mathsf{MerkleDepth}^{\mathsf{Sapling}} - 1\} \times \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]} \times \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]} \to \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]}.$$

$\mathsf{MerkleCRH}^{\mathsf{Sprout}}$ is *collision-resistant* except on its first argument. $\mathsf{MerkleCRH}^{\mathsf{Sapling}}$ is *collision-resistant* on all its arguments.

These functions are instantiated in § 5.4.1.3 ***'Merkle Tree Hash Function'*** on p. 59.

$\mathsf{hSigCRH} : \mathbb{B}^{[\ell_{\mathsf{Seed}}]} \times \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}][\mathrm{N}^{\mathsf{old}}]} \times \mathsf{JoinSplitSig.Public} \to \mathbb{B}^{[\ell_{\mathsf{hSig}}]}$ is a *collision-resistant hash function* used in § 4.3 ***'JoinSplit Descriptions'*** on p. 34. It is instantiated in § 5.4.1.4 ***'$\mathsf{h}_{\mathsf{Sig}}$ Hash Function'*** on p. 60.

$\mathsf{EquihashGen} : (n : \mathbb{N}^+) \times \mathbb{N}^+ \times \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}} \times \mathbb{N}^+ \to \mathbb{B}^{[n]}$ is another *hash function*, used in § 7.6.1 ***'Equihash'*** on p. 97 to generate input to the *Equihash* solver. The first two arguments, representing the *Equihash* parameters n and k, are written subscripted. It is instantiated in § 5.4.1.9 ***'Equihash Generator'*** on p. 64.

$\mathsf{CRH}^{\mathsf{ivk}} : \mathbb{B}^{[\ell_{\mathbb{J}}]} \times \mathbb{B}^{[\ell_{\mathbb{J}}]} \to \{1 .. 2^{\ell^{\mathsf{Sapling}}_{\mathsf{ivk}}} - 1\}$ is a *collision-resistant hash function* used in § 4.2.2 ***'Sapling Key Components'*** on p. 32 to derive an *incoming viewing key* for a Sapling *shielded payment address*. It is also used in the *Spend statement* (§ 4.16.2 ***'Spend Statement (Sapling)'*** on p. 48) to confirm use of the correct keys for the *note* being spent. It is instantiated in § 5.4.1.5 *'$\mathsf{CRH}^{\mathsf{ivk}}$ Hash Function'* on p. 60.

$\mathsf{MixingPedersenHash} : \mathbb{J} \times \{0 .. r_{\mathbb{J}} - 1\} \to \mathbb{J}$ is a *hash function* used in § 4.14 ***'Computing ρ values and Nullifiers'*** on p. 45 to derive the unique ρ value for a *Sapling-pool note*. It is also used in the *Spend statement* to confirm use of the correct ρ value as an input to *nullifier* derivation. It is instantiated in § 5.4.1.8 ***'Mixing Pedersen Hash Function'*** on p. 63.

$\mathsf{DiversifyHash}^{\mathsf{Sapling}} : \mathbb{B}^{[\ell_{\mathsf{d}}]} \to \mathbb{J}^{(r)*} \cup \{\bot\}$ is a *hash function* instantiated in § 5.4.1.6 *'$\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ Hash Function'* on p. 61, satisfying the Unlinkability security property described in that section. It is used to derive a *diversified base* from a *diversifier*, which is specified in § 4.2.2 ***'Sapling Key Components'*** on p. 32.

4.1.2 Pseudo Random Functions

PRF_x denotes a *Pseudo Random Function* keyed by x.

Let $\ell_{\mathsf{a_{sk}}}$, ℓ_{hSig}, $\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}$, $\ell^{\mathsf{Sprout}}_{\varphi}$, ℓ_{sk}, ℓ_{ovk}, $\ell_{\mathsf{PRFexpand}}$, $\ell_{\mathsf{PRFnfSapling}}$, $\mathrm{N}^{\mathsf{old}}$, and $\mathrm{N}^{\mathsf{new}}$ be as defined in § 5.3 ***'Constants'*** on p. 57.

Let Sym be as defined in § 5.4.3 ***'Symmetric Encryption'*** on p. 66.

Let $\ell_{\mathbb{J}}$ and $\mathbb{J}^{(r)}_{\star}$ be as defined in § 5.4.8.3 *'Jubjub'* on p. 77.

For Sprout, four ***independent*** PRF_x are needed:

$$\mathsf{PRF}^{\mathsf{addr}} : \mathbb{B}^{[\ell_{\mathsf{a_{sk}}}]} \times \mathbb{B}^{\mathbb{Y}} \to \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}]}$$
$$\mathsf{PRF}^{\mathsf{pk}} : \mathbb{B}^{[\ell_{\mathsf{a_{sk}}}]} \times \{1..\mathrm{N}^{\mathsf{old}}\} \times \mathbb{B}^{[\ell_{\mathsf{hSig}}]} \to \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}]}$$
$$\mathsf{PRF}^{\rho} : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\varphi}]} \times \{1..\mathrm{N}^{\mathsf{new}}\} \times \mathbb{B}^{[\ell_{\mathsf{hSig}}]} \to \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}]}$$
$$\mathsf{PRF}^{\mathsf{nfSprout}} : \mathbb{B}^{[\ell_{\mathsf{a_{sk}}}]} \times \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}]} \to \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}]}$$

These are used in § 4.16.1 ***'JoinSplit Statement (Sprout)'*** on p. 47; $\mathsf{PRF}^{\mathsf{addr}}$ is also used to derive a *shielded payment address* from a *spending key* in § 4.2.1 ***'Sprout Key Components'*** on p. 32.

For Sapling, three additional PRF_x are needed:

$$\begin{aligned}
\mathsf{PRF}^{\mathsf{expand}} &: \mathbb{B}^{[\ell_{\mathsf{sk}}]} \times \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}} \to \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{PRFexpand}}/8]}} \\
\mathsf{PRF}^{\mathsf{ockSapling}} &: \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{ovk}}/8]}} \times \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathbb{J}}/8]}} \times \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathbb{J}}/8]}} \times \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathbb{J}}/8]}} \to \mathsf{Sym}.\mathbf{K} \\
\mathsf{PRF}^{\mathsf{nfSapling}} &: \mathbb{J}_\star^{(r)} \times \mathbb{B}^{[\ell_{\mathbb{J}}]} \to \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{PRFnfSapling}}/8]}}
\end{aligned}$$

$\mathsf{PRF}^{\mathsf{expand}}$ is used in the following places:

- § 4.2.2 ***'Sapling Key Components'*** on p. 32, with inputs [0x00], [0x01], [0x02], and [0x03, $i : \mathbb{B}^{\mathbb{Y}}$];
- sending (§ 4.6.2 ***'Sending Notes (Sapling)'*** on p. 37) and receiving (§ 4.18 on p. 51) *Sapling-pool notes*, with inputs [0x04] and [0x05];
- in [ZIP-32], with inputs [0x00], [0x01], [0x02] (intentionally matching § 4.2.2 on p. 32), [0x10], [0x13], [0x14], and with first byte in {0x11, 0x12, 0x15, 0x16, 0x17, 0x18, 0x80};
- in [ZIP-316], with first byte 0xD0.

$\mathsf{PRF}^{\mathsf{ockSapling}}$ is used in § 4.18 ***'In-band secret distribution (Sapling)'*** on p. 51.

$\mathsf{PRF}^{\mathsf{nfSapling}}$ is used in § 4.16.2 ***'Spend Statement (Sapling)'*** on p. 48.

All of these *Pseudo Random Functions* are instantiated in § 5.4.2 ***'Pseudo Random Functions'*** on p. 64.

Security requirements:

- Security definitions for *Pseudo Random Functions* are given in [BDJR2000, section 4].
- In addition to being *Pseudo Random Functions*, it is required that $\mathsf{PRF}^{\mathsf{addr}}_x$, PRF^{ρ}_x, $\mathsf{PRF}^{\mathsf{nfSprout}}_x$, and $\mathsf{PRF}^{\mathsf{nfSapling}}_x$ be *collision-resistant* across all x – i.e. finding $(x, y) \neq (x', y')$ such that $\mathsf{PRF}^{\mathsf{addr}}_x(y) = \mathsf{PRF}^{\mathsf{addr}}_{x'}(y')$ should not be feasible, and similarly for PRF^{ρ}, $\mathsf{PRF}^{\mathsf{nfSprout}}$, and $\mathsf{PRF}^{\mathsf{nfSapling}}$.

Non-normative note: $\mathsf{PRF}^{\mathsf{nfSprout}}$ was called $\mathsf{PRF}^{\mathsf{sn}}$ in **Zerocash** [BCGGMTV2014], and just $\mathsf{PRF}^{\mathsf{nf}}$ in some previous versions of this specification.

4.1.3 Symmetric Encryption

Let Sym be an *authenticated one-time symmetric encryption scheme* with keyspace $\mathsf{Sym}.\mathbf{K}$, encrypting plaintexts in $\mathsf{Sym}.\mathbf{P}$ to produce ciphertexts in $\mathsf{Sym}.\mathbf{C}$.

$\mathsf{Sym.Encrypt} : \mathsf{Sym}.\mathbf{K} \times \mathsf{Sym}.\mathbf{P} \to \mathsf{Sym}.\mathbf{C}$ is the encryption algorithm.

$\mathsf{Sym.Decrypt} : \mathsf{Sym}.\mathbf{K} \times \mathsf{Sym}.\mathbf{C} \to \mathsf{Sym}.\mathbf{P} \cup \{\bot\}$ is the decryption algorithm, such that for any $\mathsf{K} \in \mathsf{Sym}.\mathbf{K}$ and $\mathsf{P} \in \mathsf{Sym}.\mathbf{P}$, $\mathsf{Sym.Decrypt}_{\mathsf{K}}(\mathsf{Sym.Encrypt}_{\mathsf{K}}(\mathsf{P})) = \mathsf{P}$. $\bot$ is used to represent the decryption of an invalid ciphertext.

Security requirement: Sym must be *one-time* (INT-CTXT ∧ IND-CPA)-secure [BN2007]. *"One-time"* here means that an honest protocol participant will almost surely encrypt only one message with a given key; however, the adversary may make many adaptive chosen ciphertext queries for a given key.

4.1.4 Key Agreement

A *key agreement scheme* is a cryptographic protocol in which two parties agree a shared secret, each using their *private key* and the other party's *public key*.

A *key agreement scheme* KA defines a type of *public keys* KA.Public, a type of *private keys* KA.Private, and a type of shared secrets KA.SharedSecret. Optionally, it also defines a type $\mathsf{KA.PublicPrimeOrder} \subseteq \mathsf{KA.Public}$.

Optional: Let $\mathsf{KA.FormatPrivate} : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}]} \to \mathsf{KA.Private}$ be a function to convert a bit string of length $\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}$ to a KA *private key*.

Let $\mathsf{KA.DerivePublic} : \mathsf{KA.Private} \times \mathsf{KA.Public} \to \mathsf{KA.Public}$ be a function that derives the KA *public key* corresponding to a given KA *private key* and base point.

Let $\mathsf{KA.Agree} : \mathsf{KA.Private} \times \mathsf{KA.Public} \to \mathsf{KA.SharedSecret}$ be the agreement function.

Optional: Let $\mathsf{KA.Base} : \mathsf{KA.Public}$ be a public base point.

Note: The range of KA.DerivePublic may be a strict subset of KA.Public.

Security requirements:

- KA.FormatPrivate must preserve sufficient entropy from its input to be used as a secure KA *private key*.
- The key agreement and the KDF defined in the next section must together satisfy a suitable adaptive security assumption along the lines of [Bernstein2006, section 3] or [ABR1999, Definition 3].

More precise formalization of these requirements is beyond the scope of this specification.

4.1.5 Key Derivation

A *Key Derivation Function* is defined for a particular *key agreement scheme* and *authenticated one-time symmetric encryption scheme*; it takes the shared secret produced by the key agreement and additional arguments, and derives a key suitable for the encryption scheme.

The inputs to the *Key Derivation Function* differ between the Sprout and Sapling KDFs:

$\mathsf{KDF}^{\mathsf{Sprout}}$ takes as input an output index in $\{1..\mathrm{N}^{\mathsf{new}}\}$, the value $\mathsf{h}_{\mathsf{Sig}}$, the shared Diffie–Hellman secret sharedSecret, the *ephemeral public key* epk, and the recipient's public *transmission key* $\mathsf{pk}_{\mathsf{enc}}$. It is suitable for use with $\mathsf{KA}^{\mathsf{Sprout}}$ and derives keys for Sym.Encrypt.

$$\mathsf{KDF}^{\mathsf{Sprout}} : \{1..\mathrm{N}^{\mathsf{new}}\} \times \mathbb{B}^{[\ell_{\mathsf{hSig}}]} \times \mathsf{KA}^{\mathsf{Sprout}}.\mathsf{SharedSecret} \times \mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Public} \times \mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Public} \to \mathsf{Sym}.\mathbf{K}$$

$\mathsf{KDF}^{\mathsf{Sapling}}$ takes as input the shared Diffie–Hellman secret sharedSecret and the *ephemeral public key* epk. (It does not have inputs taking the place of the output index, $\mathsf{h}_{\mathsf{Sig}}$, or $\mathsf{pk}_{\mathsf{enc}}$.) It is suitable for use with $\mathsf{KA}^{\mathsf{Sapling}}$ and derives keys for Sym.Encrypt.

$$\mathsf{KDF}^{\mathsf{Sapling}} : \mathsf{KA}^{\mathsf{Sapling}}.\mathsf{SharedSecret} \times \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathbb{J}}/8]}} \to \mathsf{Sym}.\mathbf{K}$$

Security requirements:

- The asymmetric encryption scheme in § 4.17 ***'In-band secret distribution (Sprout)'*** on p. 50, constructed from $\mathsf{KA}^{\mathsf{Sprout}}$, $\mathsf{KDF}^{\mathsf{Sprout}}$ and Sym, is required to be IND-CCA2-secure and *key-private*.
- The asymmetric encryption scheme in § 4.18 ***'In-band secret distribution (Sapling)'*** on p. 51, constructed from $\mathsf{KA}^{\mathsf{Sapling}}$, $\mathsf{KDF}^{\mathsf{Sapling}}$ and Sym, is required to be IND-CCA2-secure and *key-private*.

Key privacy is defined in [BBDP2001].

4.1.6 Signature

A *signature scheme* Sig defines:

- a type of *signing keys* Sig.Private;
- a type of *validating keys* Sig.Public;
- a type of messages Sig.Message;

- a type of signatures $\mathsf{Sig.Signature}$;
- a randomized *signing key* generation algorithm $\mathsf{Sig.GenPrivate} : () \xrightarrow{R} \mathsf{Sig.Private}$;
- an injective *validating key* derivation algorithm $\mathsf{Sig.DerivePublic} : \mathsf{Sig.Private} \to \mathsf{Sig.Public}$;
- a randomized signing algorithm $\mathsf{Sig.Sign} : \mathsf{Sig.Private} \times \mathsf{Sig.Message} \xrightarrow{R} \mathsf{Sig.Signature}$;
- a validating algorithm $\mathsf{Sig.Validate} : \mathsf{Sig.Public} \times \mathsf{Sig.Message} \times \mathsf{Sig.Signature} \to \mathbb{B}$;

such that for any *signing key* $\mathsf{sk} \xleftarrow{R} \mathsf{Sig.GenPrivate}()$ and corresponding *validating key* $\mathsf{vk} = \mathsf{Sig.DerivePublic}(\mathsf{sk})$, and any $m : \mathsf{Sig.Message}$ and $s : \mathsf{Sig.Signature} \xleftarrow{R} \mathsf{Sig.Sign}_{\mathsf{sk}}(m)$, $\mathsf{Sig.Validate}_{\mathsf{vk}}(m, s) = 1$.

Zcash uses four *signature schemes*:

- one used for signatures that can be validated by script operations such as `OP_CHECKSIG` and `OP_CHECKMULTISIG` as in **Bitcoin**;
- one called JoinSplitSig which is used to sign *transactions* that contain at least one *JoinSplit description* (instantiated in § 5.4.5 'Ed25519' on p. 67);
- [**Sapling** onward] one called SpendAuthSig which is used to sign authorizations of *Spend transfers* (instantiated in § 5.4.6.1 *'Spend Authorization Signature (Sapling)'* on p. 71);
- [**Sapling** onward] one called BindingSig. A *Sapling binding signature* is used to enforce balance of *Spend transfers* and *Output transfers*, and to prevent their replay across *transactions*. BindingSig is instantiated in § 5.4.6.2 *'Binding Signature (Sapling)'* on p. 71.

The signature scheme used in script operations is instantiated by ECDSA on the *secp256k1 curve*. JoinSplitSig is instantiated by Ed25519. SpendAuthSig and BindingSig are instantiated by RedDSA; on the *Jubjub curve* in Sapling.

The following security property is needed for JoinSplitSig and BindingSig. Security requirements for SpendAuthSig are defined in the next section, § 4.1.6.1 *'Signature with Re-Randomizable Keys'* on p. 26. An additional requirement for BindingSig is defined in § 4.1.6.2 *'Signature with Signing Key to Validating Key Monomorphism'* on p. 27.

Security requirement: JoinSplitSig and BindingSig must be Strongly Unforgeable under (non-adaptive) Chosen Message Attack (SU-CMA), as defined for example in [BDEHR2011, Definition 6].[6] This allows an adversary to obtain signatures on chosen messages, and then requires it to be infeasible for the adversary to forge a previously unseen valid (message, signature) pair without access to the *signing key*.

Non-normative notes:

- We need separate *signing key* generation and *validating key* derivation algorithms, rather than the more conventional combined key pair generation algorithm $\mathsf{Sig.Gen} : () \xrightarrow{R} \mathsf{Sig.Private} \times \mathsf{Sig.Public}$, to support the key derivation in § 4.2.2 *'Sapling Key Components'* on p. 32.

  The definitions of schemes with additional features in § 4.1.6.1 *'Signature with Re-Randomizable Keys'* on p. 26 and in § 4.1.6.2 *'Signature with Signing Key to Validating Key Monomorphism'* on p. 27 also become simpler.
- A fresh signature key pair is generated for each *transaction* containing a *JoinSplit description*. Since each key pair is only used for one signature (see § 4.10 *'Non-malleability (Sprout)'* on p. 41), a *one-time signature scheme* would suffice for JoinSplitSig. This is also the reason why only security against ***non-adaptive*** chosen message attack is needed. In fact the instantiation of JoinSplitSig uses a scheme designed for security under adaptive attack even when multiple signatures are signed under the same key.
- [**Sapling** onward] The same remarks as above apply to BindingSig, except that the key is derived from the randomness of *value commitments*. This results in the same distribution as of freshly generated key pairs, for each *transaction* containing *Spend descriptions* or *Output descriptions*.

[6] The scheme defined in that paper was attacked in [LM2017], but this has no impact on the applicability of the definition.

- SU-CMA security requires it to be infeasible for the adversary, not knowing the *private key*, to forge a distinct signature on a previously seen message. That is, *JoinSplit signatures* and *Sapling binding signatures* are intended to be *nonmalleable* in the sense of [BIP-62].
- The terminology used in this specification is that we "validate" signatures, and "verify" *zk-SNARK proofs*.

4.1.6.1 Signature with Re-Randomizable Keys

A *signature scheme with re-randomizable keys* Sig is a *signature scheme* that additionally defines:

- a type of *randomizers* $\mathsf{Sig.Random}$;
- a *randomizer* generator $\mathsf{Sig.GenRandom} : () \xrightarrow{\mathrm{R}} \mathsf{Sig.Random}$;
- a *signing key* randomization algorithm $\mathsf{Sig.RandomizePrivate} : \mathsf{Sig.Random} \times \mathsf{Sig.Private} \to \mathsf{Sig.Private}$;
- a *validating key* randomization algorithm $\mathsf{Sig.RandomizePublic} : \mathsf{Sig.Random} \times \mathsf{Sig.Public} \to \mathsf{Sig.Public}$;
- a distinguished "identity" *randomizer* $\mathcal{O}_{\mathsf{Sig.Random}} : \mathsf{Sig.Random}$

such that:

- for any $\alpha : \mathsf{Sig.Random}$, $\mathsf{Sig.RandomizePrivate}_\alpha : \mathsf{Sig.Private} \to \mathsf{Sig.Private}$ is injective and easily invertible;
- $\mathsf{Sig.RandomizePrivate}_{\mathcal{O}_{\mathsf{Sig.Random}}}$ is the identity function on $\mathsf{Sig.Private}$.
- for any $\mathsf{sk} : \mathsf{Sig.Private}$,

  $\mathsf{Sig.RandomizePrivate}(\alpha, \mathsf{sk}) : \alpha \xleftarrow{\mathrm{R}} \mathsf{Sig.GenRandom}()$

  is identically distributed to $\mathsf{Sig.GenPrivate}()$.
- for any $\mathsf{sk} : \mathsf{Sig.Private}$ and $\alpha : \mathsf{Sig.Random}$,

  $\mathsf{Sig.RandomizePublic}(\alpha, \mathsf{Sig.DerivePublic}(\mathsf{sk})) = \mathsf{Sig.DerivePublic}(\mathsf{Sig.RandomizePrivate}(\alpha, \mathsf{sk}))$.

The following security requirement for such *signature schemes* is based on that given in [FKMSSS2016, section 3]. Note that we require Strong Unforgeability with *Re-randomized* Keys, not Existential Unforgeability with *Re-randomized* Keys (the latter is called "Unforgeability under *Re-randomized* Keys" in [FKMSSS2016, Definition 8]). Unlike the case for JoinSplitSig, we require security under adaptive chosen message attack with multiple messages signed using a given key. (Although each *note* uses a different *re-randomized* key pair, the same original key pair can be *re-randomized* for multiple *notes*, and also it can happen that multiple *transactions* spending the same *note* are revealed to an adversary.)

Security requirement: Strong Unforgeability with Re-randomized Keys under adaptive Chosen Message Attack (SURK-CMA)

For any $\mathsf{sk} : \mathsf{Sig.Private}$, let

$\mathsf{O}_{\mathsf{sk}} : \mathsf{Sig.Message} \times \mathsf{Sig.Random} \to \mathsf{Sig.Signature}$

be a signing oracle with state $Q : \mathscr{P}(\mathsf{Sig.Message} \times \mathsf{Sig.Signature})$ initialized to $\{\}$ that records queried messages and corresponding signatures.

$\mathsf{O}_{\mathsf{sk}} :=$ let mutable $Q \leftarrow \{\}$ in $(m : \mathsf{Sig.Message}, \alpha : \mathsf{Sig.Random}) \mapsto$
  let $\sigma = \mathsf{Sig.Sign}_{\mathsf{Sig.RandomizePrivate}(\alpha, \mathsf{sk})}(m)$
  set $Q \leftarrow Q \cup \{(m, \sigma)\}$
  return $\sigma : \mathsf{Sig.Signature}$.

For random $\mathsf{sk} \xleftarrow{\mathrm{R}} \mathsf{Sig.GenPrivate}()$ and $\mathsf{vk} = \mathsf{Sig.DerivePublic}(\mathsf{sk})$, it must be infeasible for an adversary given vk and a new instance of O_{sk} to find (m', σ', α') such that $\mathsf{Sig.Validate}_{\mathsf{Sig.RandomizePublic}(\alpha', \mathsf{vk})}(m', \sigma') = 1$ and $(m', \sigma') \notin \mathsf{O}_{\mathsf{sk}}.Q$.

Non-normative notes:

- The *randomizer* and key arguments to Sig.RandomizePrivate and Sig.RandomizePublic are swapped relative to [FKMSSS2016, section 3].
- The requirement for the identity *randomizer* $\mathcal{O}_{\mathsf{Sig.Random}}$ simplifies the definition of SURK-CMA by removing the need for two oracles (because the oracle for original keys, called O_1 in [FKMSSS2016], is a special case of the oracle for randomized keys).
- Since $\mathsf{Sig.RandomizePrivate}(\alpha, \mathsf{sk}) : \alpha \xleftarrow{\mathrm{R}} \mathsf{Sig.Random}$ has an identical distribution to $\mathsf{Sig.GenPrivate}()$, and since Sig.DerivePublic is a deterministic function, the combination of a *re-randomized validating key* and signature(s) under that key do not reveal the key from which it was *re-randomized*.
- Since $\mathsf{Sig.RandomizePrivate}_\alpha$ is injective and easily invertible, knowledge of $\mathsf{Sig.RandomizePrivate}(\alpha, \mathsf{sk})$ ***and*** α implies knowledge of sk.

4.1.6.2 Signature with Signing Key to Validating Key Monomorphism

A *signature scheme with key monomorphism* Sig is a *signature scheme* that additionally defines:

- an abelian group on *signing keys*, with operation $\boxplus : \mathsf{Sig.Private} \times \mathsf{Sig.Private} \to \mathsf{Sig.Private}$ and identity $\mathcal{O}_{\boxplus}$;
- an abelian group on *validating keys*, with operation $\diamondplus : \mathsf{Sig.Public} \times \mathsf{Sig.Public} \to \mathsf{Sig.Public}$ and identity $\mathcal{O}_{\diamondplus}$.

such that for any $\mathsf{sk}_{1..2} : \mathsf{Sig.Private}$, $\mathsf{Sig.DerivePublic}(\mathsf{sk}_1 \boxplus \mathsf{sk}_2) = \mathsf{Sig.DerivePublic}(\mathsf{sk}_1) \diamondplus \mathsf{Sig.DerivePublic}(\mathsf{sk}_2)$.

In other words, Sig.DerivePublic is a *monomorphism* (that is, an injective homomorphism) from the *signing key* group to the *validating key* group.

For $\mathrm{N} : \mathbb{N}^+$,

- $\boxplus_{i=1}^{\mathrm{N}} \mathsf{sk}_i$ means $\mathsf{sk}_1 \boxplus \mathsf{sk}_2 \boxplus \cdots \boxplus \mathsf{sk}_{\mathrm{N}}$;
- $\diamondplus_{i=1}^{\mathrm{N}} \mathsf{vk}_i$ means $\mathsf{vk}_1 \diamondplus \mathsf{vk}_2 \diamondplus \cdots \diamondplus \mathsf{vk}_{\mathrm{N}}$.

When $\mathrm{N} = 0$ these yield the appropriate group identity, i.e. $\boxplus_{i=1}^{0} \mathsf{sk}_i = \mathcal{O}_{\boxplus}$ and $\diamondplus_{i=1}^{0} \mathsf{vk}_i = \mathcal{O}_{\diamondplus}$.

$\boxminus \mathsf{sk}$ means the *signing key* such that $(\boxminus \mathsf{sk}) \boxplus \mathsf{sk} = \mathcal{O}_{\boxplus}$, and $\mathsf{sk}_1 \boxminus \mathsf{sk}_2$ means $\mathsf{sk}_1 \boxplus (\boxminus \mathsf{sk}_2)$.

$\diamondminus \mathsf{vk}$ means the *validating key* such that $(\diamondminus \mathsf{vk}) \diamondplus \mathsf{vk} = \mathcal{O}_{\diamondplus}$, and $\mathsf{vk}_1 \diamondminus \mathsf{vk}_2$ means $\mathsf{vk}_1 \diamondplus (\diamondminus \mathsf{vk}_2)$.

With a change of notation from μ to Sig.DerivePublic, $+$ to $\boxplus$, and $\cdot$ to $\diamondplus$, this is similar to the definition of a *"Signature with Secret Key to Public Key Homomorphism"* in [DS2016, Definition 13], except for an additional requirement for the homomorphism to be injective.

Security requirement: For any $\mathsf{sk}_1 : \mathsf{Sig.Private}$, and an unknown $\mathsf{sk}_2 \xleftarrow{\mathrm{R}} \mathsf{Sig.GenPrivate}()$ chosen independently of sk_1, the distribution of $\mathsf{sk}_1 \boxplus \mathsf{sk}_2$ is computationally indistinguishable from that of $\mathsf{Sig.GenPrivate}()$. (Since $\boxplus$ is an abelian group operation, this implies that for $n : \mathbb{N}^+$, $\boxplus_{i=1}^{n} \mathsf{sk}_i$ is computationally indistinguishable from $\mathsf{Sig.GenPrivate}()$ when at least one of $\mathsf{sk}_{1..n}$ is unknown.)

4.1.7 Commitment

A *commitment scheme* is a function that, given a *commitment trapdoor* generated at random and an input, can be used to commit to the input in such a way that:

- no information is revealed about it without the *trapdoor* (*"hiding"*); and
- given the *trapdoor* and input, the commitment can be verified to *"open"* to that input and no other (*"binding"*).

A *commitment scheme* COMM defines a type of inputs COMM.Input, a type of commitments COMM.Output, a type of *commitment trapdoors* COMM.Trapdoor, and a *trapdoor* generator $\mathsf{COMM.GenTrapdoor} : () \xrightarrow{\mathrm{R}} \mathsf{COMM.Trapdoor}$.

Let $\mathsf{COMM} : \mathsf{COMM.Trapdoor} \times \mathsf{COMM.Input} \to \mathsf{COMM.Output}$ be a function satisfying the following security requirements.

Security requirements:

- **Computational *hiding*:** For all $x, x' : \mathsf{COMM.Input}$, the distributions $\{\, \mathsf{COMM}_r(x) \mid r \xleftarrow{R} \mathsf{COMM.GenTrapdoor}() \,\}$ and $\{\, \mathsf{COMM}_r(x') \mid r \xleftarrow{R} \mathsf{COMM.GenTrapdoor}() \,\}$ are computationally indistinguishable.
- **Computational *binding*:** It is infeasible to find $x, x' : \mathsf{COMM.Input}$ and $r, r' : \mathsf{COMM.Trapdoor}$ such that $x \neq x'$ and $\mathsf{COMM}_r(x) = \mathsf{COMM}_{r'}(x')$.

Notes:

- COMM.GenTrapdoor need not produce the uniform distribution on COMM.Trapdoor. In that case, it is incorrect to choose a *trapdoor* from the latter distribution.
- If it were only feasible to find $x : \mathsf{COMM.Input}$ and $r, r' : \mathsf{COMM.Trapdoor}$ such that $r \neq r'$ and $\mathsf{COMM}_r(x) = \mathsf{COMM}_{r'}(x)$, this would not contradict the computational *binding* security requirement. (In fact, this is feasible for $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ and $\mathsf{ValueCommit}^{\mathsf{Sapling}}$ because *trapdoors* are equivalent modulo $r_{\mathbb{J}}$, and the range of a *trapdoor* for those algorithms is $\{0 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\}$ where $2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} > r_{\mathbb{J}}$.)

Let $\ell^{\mathsf{Sprout}}_{\mathsf{rcm}}$, $\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}$, $\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}$, and ℓ_{value} be as defined in § 5.3 ***'Constants'*** on p. 57.

Define $\mathsf{NoteCommit}^{\mathsf{Sprout}}.\mathsf{Trapdoor} := \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{rcm}}]}$ and $\mathsf{NoteCommit}^{\mathsf{Sprout}}.\mathsf{Output} := \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}]}$.

Sprout uses a *note commitment scheme*

$$\mathsf{NoteCommit}^{\mathsf{Sprout}} : \mathsf{NoteCommit}^{\mathsf{Sprout}}.\mathsf{Trapdoor} \times \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}]} \times \{0 \,..\, 2^{\ell_{\mathsf{value}}} - 1\} \times \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}]} \to \mathsf{NoteCommit}^{\mathsf{Sprout}}.\mathsf{Output},$$

instantiated in § 5.4.7.1 ***'Note Commitments (Sprout)'*** on p. 72.

Let $\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}$ be as defined in § 5.3 ***'Constants'*** on p. 57.

Let $\mathbb{J}^{(r)}$, $\ell_{\mathbb{J}}$, and $r_{\mathbb{J}}$ be as defined in § 5.4.8.3 ***'Jubjub'*** on p. 77.

Define:

$\mathsf{NoteCommit}^{\mathsf{Sapling}}.\mathsf{Trapdoor} := \{0 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\}$ and $\mathsf{NoteCommit}^{\mathsf{Sapling}}.\mathsf{Output} := \mathbb{J}$;

$\mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Trapdoor} := \{0 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\}$ and $\mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Output} := \mathbb{J}$.

Sapling uses two additional *commitment schemes*:

$$\mathsf{NoteCommit}^{\mathsf{Sapling}} : \mathsf{NoteCommit}^{\mathsf{Sapling}}.\mathsf{Trapdoor} \times \mathbb{B}^{[\ell_{\mathbb{J}}]} \times \mathbb{B}^{[\ell_{\mathbb{J}}]} \times \{0 \,..\, 2^{\ell_{\mathsf{value}}} - 1\} \to \mathsf{NoteCommit}^{\mathsf{Sapling}}.\mathsf{Output}$$

$$\mathsf{ValueCommit}^{\mathsf{Sapling}} : \mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Trapdoor} \times \left\{-\tfrac{r_{\mathbb{J}}-1}{2} \,..\, \tfrac{r_{\mathbb{J}}-1}{2}\right\} \to \mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Output}$$

$\mathsf{NoteCommit}^{\mathsf{Sapling}}$ is instantiated in § 5.4.7.2 ***'Windowed Pedersen commitments'*** on p. 72, and $\mathsf{ValueCommit}^{\mathsf{Sapling}}$ is instantiated in § 5.4.7.3 ***'Homomorphic Pedersen commitments (Sapling)'*** on p. 73.

Non-normative note: $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ and $\mathsf{ValueCommit}^{\mathsf{Sapling}}$ always return points in the subgroup $\mathbb{J}^{(r)}$. However, we declare the type of these commitment outputs to be $\mathbb{J}$ because they are not directly checked to be in the subgroup when $\mathsf{ValueCommit}^{\mathsf{Sapling}}$ outputs appear in *Spend descriptions* and *Output descriptions*, or when the `cmu` field derived from a $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ appears in an *Output description*.

4.1.8 Represented Group

A *represented group* $\mathbb{G}$ consists of:

- a subgroup order parameter $r_{\mathbb{G}} : \mathbb{N}^+$, which must be prime;
- a cofactor parameter $h_{\mathbb{G}} : \mathbb{N}^+$;
- a group $\mathbb{G}$ of order $h_{\mathbb{G}} \cdot r_{\mathbb{G}}$, written additively with operation $+ : \mathbb{G} \times \mathbb{G} \to \mathbb{G}$, and additive identity $\mathcal{O}_{\mathbb{G}}$;
- a bit-length parameter $\ell_{\mathbb{G}} : \mathbb{N}$;
- a representation function $\mathsf{repr}_{\mathbb{G}} : \mathbb{G} \to \mathbb{B}^{[\ell_{\mathbb{G}}]}$ and an abstraction function $\mathsf{abst}_{\mathbb{G}} : \mathbb{B}^{[\ell_{\mathbb{G}}]} \to \mathbb{G} \cup \{\bot\}$, such that $\mathsf{abst}_{\mathbb{G}}$ is a left inverse of $\mathsf{repr}_{\mathbb{G}}$, i.e. for all $P \in \mathbb{G}$, $\mathsf{abst}_{\mathbb{G}}\big(\mathsf{repr}_{\mathbb{G}}(P)\big) = P$.

Note: Ideally, we would also have that for all S not in the image of $\mathsf{repr}_{\mathbb{G}}$, $\mathsf{abst}_{\mathbb{G}}(S) = \bot$. This may not be true in all cases, i.e. there can be *non-canonical* encodings $P\star$ such that $\mathsf{repr}_{\mathbb{G}}\big(\mathsf{abst}_{\mathbb{G}}(P\star)\big) \neq P\star$.

Define $\mathbb{G}^{(r)}$ as the order-$r_{\mathbb{G}}$ subgroup of $\mathbb{G}$, which is called a *represented subgroup*. Note that this includes $\mathcal{O}_{\mathbb{G}}$. For the set of points of order $r_{\mathbb{G}}$ (which excludes $\mathcal{O}_{\mathbb{G}}$), we write $\mathbb{G}^{(r)*}$.

Define $\mathbb{G}_\star^{(r)} := \{\mathsf{repr}_{\mathbb{G}}(P) : \mathbb{B}^{[\ell_{\mathbb{G}}]} \mid P \in \mathbb{G}^{(r)}\}$. (This intentionally excludes *non-canonical* encodings if there are any.)

For $G : \mathbb{G}$ we write $-G$ for the negation of G, such that $(-G) + G = \mathcal{O}_{\mathbb{G}}$. We write $G - H$ for $G + (-H)$.

We also extend the $\sum$ notation to addition on group elements.

For $G : \mathbb{G}$ and $k : \mathbb{Z}$ we write $[k]\,G$ for scalar multiplication on the group, i.e.

$$[k]\,G := \begin{cases} \sum_{i=1}^{k} G, & \text{if } k \geq 0 \\ \sum_{i=1}^{-k} (-G), & \text{otherwise.} \end{cases}$$

For $G : \mathbb{G}$ and $a : \mathbb{F}_{r_{\mathbb{G}}}$, we may also write $[a]\,G$ meaning $[a \bmod r_{\mathbb{G}}]\,G$ as defined above. (This variant is not defined for fields other than $\mathbb{F}_{r_{\mathbb{G}}}$.)

4.1.9 Coordinate Extractor

A *coordinate extractor* for a *represented group* $\mathbb{G}$ is a function $\mathsf{Extract}_{\mathbb{G}^{(r)}} : \mathbb{G}^{(r)} \to T$ for some type T.

Note: Unlike the representation function $\mathsf{repr}_{\mathbb{G}}$, $\mathsf{Extract}_{\mathbb{G}^{(r)}}$ need not have an efficiently computable left inverse.

4.1.10 Group Hash

Given a *represented subgroup* $\mathbb{G}^{(r)}$, a *family of group hashes into the subgroup*, denoted $\mathsf{GroupHash}^{\mathbb{G}^{(r)}}$, consists of:

- a type $\mathsf{GroupHash}^{\mathbb{G}^{(r)}}.\mathsf{URSType}$ of *Uniform Random Strings*;
- a type $\mathsf{GroupHash}^{\mathbb{G}^{(r)}}.\mathsf{Input}$ of inputs;
- a function $\mathsf{GroupHash}^{\mathbb{G}^{(r)}} : \mathsf{GroupHash}^{\mathbb{G}^{(r)}}.\mathsf{URSType} \times \mathsf{GroupHash}^{\mathbb{G}^{(r)}}.\mathsf{Input} \to \mathbb{G}^{(r)}$.

In § 5.4.8.5 *'Group Hash into Jubjub'* on p. 79, we instantiate a family of *group hashes* into the *Jubjub curve* defined by § 5.4.8.3 *'Jubjub'* on p. 77.

Security requirement: For a randomly selected $\mathsf{URS} : \mathsf{GroupHash}^{\mathbb{G}^{(r)}}.\mathsf{URSType}$, it must be reasonable to model $\mathsf{GroupHash}^{\mathbb{G}^{(r)}}_{\mathsf{URS}}$ (restricted to inputs for which it does not return $\bot$) as a *random oracle*.

Non-normative notes:

- $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}$ is used to obtain generators of the *Jubjub curve* for various purposes: the bases $\mathcal{G}^{\mathsf{Sapling}}$ and $\mathcal{H}^{\mathsf{Sapling}}$ used in Sapling key generation, the *Pedersen hash* defined in § 5.4.1.7 ***'Pedersen Hash Function'*** on p. 62, and the *commitment schemes* defined in § 5.4.7.2 ***'Windowed Pedersen commitments'*** on p. 72 and in § 5.4.7.3 ***'Homomorphic Pedersen commitments (Sapling)'*** on p. 73.

  The security property needed for these uses can alternatively be defined in the standard model as follows:

  Discrete Logarithm Independence: For a randomly selected member $\mathsf{GroupHash}^{\mathbb{G}^{(r)}}_{\mathsf{URS}}$ of the family, it is infeasible to find a sequence of ***distinct*** inputs $m_{1..n} : \mathsf{GroupHash}^{\mathbb{G}^{(r)}}.\mathsf{Input}^{[n]}$ and a sequence of nonzero $x_{1..n} : {\mathbb{F}^*_{r_{\mathbb{G}}}}^{[n]}$ such that $\sum_{i=1}^{n}\Big([x_i]\,\mathsf{GroupHash}^{\mathbb{G}^{(r)}}_{\mathsf{URS}}(m_i)\Big) = \mathcal{O}_{\mathbb{G}}$.

- Assuming hardness of the *Discrete Logarithm Problem* on $\mathbb{G}^{(r)}$, a *random oracle* almost surely satisfies *Discrete Logarithm Independence*. *Discrete Logarithm Independence* implies *collision resistance*, since a collision (m_1, m_2) for $\mathsf{GroupHash}^{\mathbb{G}^{(r)}}_{\mathsf{URS}}$ trivially gives a *discrete logarithm* relation with $x_1 = 1$ and $x_2 = -1$.

- $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}$ is used in § 5.4.1.6 ***'DiversifyHash^{Sapling} Hash Function'*** on p. 61 to instantiate $\mathsf{DiversifyHash}^{\mathsf{Sapling}}$. We do not know how to prove the Unlinkability property defined in that section in the standard model, but in a model where $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}$ (restricted to inputs for which it does not return $\bot$) is taken as a *random oracle*, it is implied by the *Decisional Diffie–Hellman assumption* on $\mathbb{J}^{(r)}$.

- URS is a *Uniform Random String*; we chose it verifiably at random (see § 5.9 ***'Randomness Beacon'*** on p. 87), after fixing the concrete group hash algorithm to be used. This mitigates the possibility that the group hash algorithm could have been backdoored.

4.1.11 Represented Pairing

A *represented pairing* $\mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}$ consists of:

- a group order parameter $r_{\mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}} : \mathbb{N}^+$ which must be prime;
- two *represented subgroups* $\mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}^{(r)}_{1,2}$, both of order $r_{\mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}}$;
- a group $\mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}^{(r)}_{T}$ of order $r_{\mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}}$, written multiplicatively with operation $\cdot : \mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}^{(r)}_{T} \times \mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}^{(r)}_{T} \to \mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}^{(r)}_{T}$ and group identity $\mathbf{1}_{\mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}}$;
- three generators $\mathcal{P}_{\mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}_{1,2,T}}$ of $\mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}^{(r)}_{1,2,T}$ respectively;
- a pairing function $\hat{e}_{\mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}} : \mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}^{(r)}_{1} \times \mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}^{(r)}_{2} \to \mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}^{(r)}_{T}$ satisfying:
  - (Bilinearity) for all $a, b : \mathbb{F}^*_r$, $P : \mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}^{(r)}_{1}$, and $Q : \mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}^{(r)}_{2}$, $\hat{e}_{\mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}}([a]\,P, [b]\,Q) = \hat{e}_{\mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}}(P, Q)^{a \cdot b}$; and
  - (Nondegeneracy) there does not exist $P : \mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}^{(r)*}_{1}$ such that for all $Q : \mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}^{(r)}_{2}$, $\hat{e}_{\mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}}(P, Q) = \mathbf{1}_{\mathbb{P}\mathbb{A}\mathbb{I}\mathbb{R}}$.

4.1.12 Zero-Knowledge Proving System

A *zero-knowledge proving system* is a cryptographic protocol that allows proving a particular *statement*, dependent on *primary* and *auxiliary inputs*, in zero knowledge — that is, without revealing information about the *auxiliary inputs* other than that implied by the *statement*. The type of *zero-knowledge proving system* needed by **Zcash** is a *preprocessing zk-SNARK* [BCCGLRT2014].

A *preprocessing zk-SNARK* instance ZK defines:

- a type of *zero-knowledge proving keys*, ZK.ProvingKey;
- a type of *zero-knowledge verifying keys*, ZK.VerifyingKey;
- a type of *primary inputs* ZK.PrimaryInput;
- a type of *auxiliary inputs* ZK.AuxiliaryInput;
- a type of *zk-SNARK proofs* ZK.Proof;
- a type ZK.SatisfyingInputs $\subseteq$ ZK.PrimaryInput $\times$ ZK.AuxiliaryInput of inputs satisfying the *statement*;
- a randomized key pair generation algorithm ZK.Gen $: () \xrightarrow{R}$ ZK.ProvingKey $\times$ ZK.VerifyingKey;
- a proving algorithm ZK.`Prove` : ZK.ProvingKey $\times$ ZK.SatisfyingInputs $\to$ ZK.Proof;
- a verifying algorithm ZK.`Verify` : ZK.VerifyingKey $\times$ ZK.PrimaryInput $\times$ ZK.Proof $\to \mathbb{B}$;

The security requirements below are supposed to hold with overwhelming probability for $(\mathsf{pk}, \mathsf{vk}) \xleftarrow{R} \text{ZK.Gen}()$.

Security requirements:

- **Completeness:** An honestly generated proof will convince a verifier: for any $(x, w) \in$ ZK.SatisfyingInputs, if $\text{ZK.}\mathtt{Prove}_{\mathsf{pk}}(x, w)$ outputs π, then $\text{ZK.}\mathtt{Verify}_{\mathsf{vk}}(x, \pi) = 1$.
- **Knowledge Soundness:** For any adversary $\mathcal{A}$ able to find an $x :$ ZK.PrimaryInput and proof $\pi :$ ZK.Proof such that $\text{ZK.}\mathtt{Verify}_{\mathsf{vk}}(x, \pi) = 1$, there is an efficient extractor $\mathcal{E}_{\mathcal{A}}$ such that if $\mathcal{E}_{\mathcal{A}}(\mathsf{vk}, \mathsf{pk})$ returns w, then the probability that $(x, w) \notin$ ZK.SatisfyingInputs is insignificant.
- **Statistical Zero Knowledge:** An honestly generated proof is statistical zero knowledge. That is, there is a feasible stateful simulator $\mathcal{S}$ such that, for all stateful distinguishers $\mathcal{D}$, the following two probabilities are not significantly different:

$$\Pr\left[\begin{array}{c|l} (x,w) \in \text{ZK.SatisfyingInputs} & (\mathsf{pk},\mathsf{vk}) \xleftarrow{R} \text{ZK.Gen}() \\ \mathcal{D}(\pi) = 1 & (x,w) \xleftarrow{R} \mathcal{D}(\mathsf{pk},\mathsf{vk}) \\ & \pi \xleftarrow{R} \text{ZK.}\mathtt{Prove}_{\mathsf{pk}}(x,w) \end{array}\right] \text{ and } \Pr\left[\begin{array}{c|l} (x,w) \in \text{ZK.SatisfyingInputs} & (\mathsf{pk},\mathsf{vk}) \xleftarrow{R} \mathcal{S}() \\ \mathcal{D}(\pi) = 1 & (x,w) \xleftarrow{R} \mathcal{D}(\mathsf{pk},\mathsf{vk}) \\ & \pi \xleftarrow{R} \mathcal{S}(x) \end{array}\right]$$

These definitions are derived from those in [BCTV2014b, Appendix C], adapted to state concrete security for a fixed circuit, rather than asymptotic security for arbitrary circuits. (ZK.`Prove` corresponds to P, ZK.`Verify` corresponds to V, and ZK.SatisfyingInputs corresponds to $\mathcal{R}_C$ in the notation of that appendix.)

The Knowledge Soundness definition is a way to formalize the property that it is infeasible to find a new proof π where $\text{ZK.}\mathtt{Verify}_{\mathsf{vk}}(x, \pi) = 1$ without ***knowing*** an *auxiliary input* w such that $(x, w) \in$ ZK.SatisfyingInputs. Note that Knowledge Soundness implies Soundness — i.e. the property that it is infeasible to find a new proof π where $\text{ZK.}\mathtt{Verify}_{\mathsf{vk}}(x, \pi) = 1$ without ***there existing*** an *auxiliary input* w such that $(x, w) \in$ ZK.SatisfyingInputs.

Non-normative notes:

- The above properties do not include *nonmalleability* [DSDCOPS2001], and the design of the protocol using the *zero-knowledge proving system* must take this into account.
- The terminology used in this specification is that we "validate" signatures, and "verify" *zk-SNARK proofs*.

Zcash uses two *proving systems*:

- BCTV14 (§ 5.4.9.1 *'BCTV14'* on p. 79) is used with the BN-254 pairing (§ 5.4.8.1 *'BN-254'* on p. 74), to prove and verify the Sprout *JoinSplit statement* (§ 4.16.1 ***'JoinSplit Statement (Sprout)'*** on p. 47) before **Sapling** activation.
- Groth16 (§ 5.4.9.2 *'Groth16'* on p. 81) is used with the BLS12-381 pairing (§ 5.4.8.2 *'BLS12-381'* on p. 75), to prove and verify the Sapling *Spend statement* (§ 4.16.2 ***'Spend Statement (Sapling)'*** on p. 48) and *Output statement* (§ 4.16.3 ***'Output Statement (Sapling)'*** on p. 49). It is also used to prove and verify the *JoinSplit statement* after **Sapling** activation.

These specializations are:

- ZKJoinSplit for the Sprout *JoinSplit statement* (with BCTV14 and BN-254, or Groth16 and BLS12-381);
- ZKSpend for the Sapling *Spend statement* and ZKOutput for the Sapling *Output statement*.

We omit key subscripts on ZKJoinSplit.Prove and ZKJoinSplit.Verify, taking them to be either the BCTV14 *proving key* and *verifying key* defined in § 5.7 *'BCTV14 zk-SNARK Parameters'* on p. 87, or the `sprout-groth16.params` Groth16 *proving key* and *verifying key* defined in § 5.8 *'Groth16 zk-SNARK Parameters'* on p. 87, according to whether the proof appears in a *block* before or after **Sapling** activation.

We also omit subscripts on ZKSpend.Prove, ZKSpend.Verify, ZKOutput.Prove, and ZKOutput.Verify, taking them to be the relevant Groth16 *proving keys* and *verifying keys* defined in § 5.8 *'Groth16 zk-SNARK Parameters'* on p. 87.

4.2 Key Components

4.2.1 Sprout Key Components

Let $\ell_{\mathsf{a_{sk}}}$ be as defined in § 5.3 *'Constants'* on p. 57.

Let $\mathsf{PRF}^{\mathsf{addr}}$ be a *Pseudo Random Function*, instantiated in § 5.4.2 *'Pseudo Random Functions'* on p. 64.

Let $\mathsf{KA}^{\mathsf{Sprout}}$ be a *key agreement scheme*, instantiated in § 5.4.4.1 *'Sprout Key Agreement'* on p. 66.

A new Sprout *spending key* $\mathsf{a_{sk}}$ is generated by choosing a *bit sequence* uniformly at random from $\mathbb{B}^{[\ell_{\mathsf{a_{sk}}}]}$.

$\mathsf{a_{pk}}$, $\mathsf{sk_{enc}}$ and $\mathsf{pk_{enc}}$ are derived from $\mathsf{a_{sk}}$ as follows:

$$\begin{aligned}
\mathsf{a_{pk}} &:= \mathsf{PRF}^{\mathsf{addr}}_{\mathsf{a_{sk}}}(0) \\
\mathsf{sk_{enc}} &:= \mathsf{KA}^{\mathsf{Sprout}}.\mathsf{FormatPrivate}(\mathsf{PRF}^{\mathsf{addr}}_{\mathsf{a_{sk}}}(1)) \\
\mathsf{pk_{enc}} &:= \mathsf{KA}^{\mathsf{Sprout}}.\mathsf{DerivePublic}(\mathsf{sk_{enc}}, \mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Base}).
\end{aligned}$$

4.2.2 Sapling Key Components

Let $\ell_{\mathsf{PRFexpand}}$, ℓ_{sk}, $\ell^{\mathsf{Sapling}}_{\mathsf{ivk}}$, ℓ_{ovk}, and ℓ_{d} be as defined in § 5.3 *'Constants'* on p. 57.

Let $\mathbb{J}^{(r)}$, $\mathbb{J}^{(r)*}$, $\mathbb{J}^{(r)}_{\star}$, $\mathsf{repr}_{\mathbb{J}}$, and $r_{\mathbb{J}}$ be as defined in § 5.4.8.3 *'Jubjub'* on p. 77, and let $\mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}$ be as defined in § 5.4.8.5 *'Group Hash into Jubjub'* on p. 79.

Let $\mathsf{PRF}^{\mathsf{expand}}$ and $\mathsf{PRF}^{\mathsf{ockSapling}}$, instantiated in § 5.4.2 *'Pseudo Random Functions'* on p. 64, be *Pseudo Random Functions*.

Let $\mathsf{KA}^{\mathsf{Sapling}}$, instantiated in § 5.4.4.3 *'Sapling Key Agreement'* on p. 67, be a *key agreement scheme*.

Let $\mathsf{CRH}^{\mathsf{ivk}}$, instantiated in § 5.4.1.5 *'$\mathsf{CRH}^{\mathsf{ivk}}$ Hash Function'* on p. 60, be a *hash function*.

Let $\mathsf{DiversifyHash}^{\mathsf{Sapling}}$, instantiated in § 5.4.1.6 *'$\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ Hash Function'* on p. 61, be a *hash function*.

Let $\mathsf{SpendAuthSig}^{\mathsf{Sapling}}$, instantiated in § 5.4.6.1 *'Spend Authorization Signature (Sapling)'* on p. 71, be a *signature scheme with re-randomizable keys*.

Let $\mathsf{LEBS2OSP} : (\ell : \mathbb{N}) \times \mathbb{B}^{[\ell]} \to \mathbb{B}^{\mathbb{Y}^{[\mathsf{ceiling}(\ell/8)]}}$ and $\mathsf{LEOS2IP} : (\ell : \mathbb{N} \mid \ell \bmod 8 = 0) \times \mathbb{B}^{\mathbb{Y}^{[\ell/8]}} \to \{0 \,..\, 2^{\ell}-1\}$ be as defined in § 5.1 *'Integers, Bit Sequences, and Endianness'* on p. 56.

Define $\mathcal{H}^{\mathsf{Sapling}} := \mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}(\texttt{"Zcash_H_"}, \texttt{""})$.

Define $\mathsf{ToScalar}^{\mathsf{Sapling}}(x : \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{PRFexpand}}/8]}}) := \mathsf{LEOS2IP}_{\ell_{\mathsf{PRFexpand}}}(x) \pmod{r_{\mathbb{J}}}$.

A new Sapling *spending key* sk is generated by choosing a *bit sequence* uniformly at random from $\mathbb{B}^{[\ell_{\mathsf{sk}}]}$.

From this *spending key*, the *Spend authorizing key* $\mathsf{ask} : \mathbb{F}^*_{r_{\mathbb{J}}}$, the *proof authorizing key* $\mathsf{nsk} : \mathbb{F}_{r_{\mathbb{J}}}$, and the *outgoing viewing key* $\mathsf{ovk} : \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{ovk}}/8]}}$ are derived as follows:

$$\mathsf{ask} := \mathsf{ToScalar}^{\mathsf{Sapling}}\big(\mathsf{PRF}^{\mathsf{expand}}_{\mathsf{sk}}([0])\big)$$
$$\mathsf{nsk} := \mathsf{ToScalar}^{\mathsf{Sapling}}\big(\mathsf{PRF}^{\mathsf{expand}}_{\mathsf{sk}}([1])\big)$$
$$\mathsf{ovk} := \mathsf{truncate}_{(\ell_{\mathsf{ovk}}/8)}\big(\mathsf{PRF}^{\mathsf{expand}}_{\mathsf{sk}}([2])\big)$$

If $\mathsf{ask} = 0$, discard this key and repeat with a new sk.

$\mathsf{ak} : \mathbb{J}^{(r)*}$, $\mathsf{nk} : \mathbb{J}^{(r)}$, and the *incoming viewing key* $\mathsf{ivk} : \{1 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{ivk}}} - 1\}$ are then derived as:

$$\mathsf{ak} := \mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{DerivePublic}(\mathsf{ask})$$
$$\mathsf{nk} := [\mathsf{nsk}]\,\mathcal{H}^{\mathsf{Sapling}}$$
$$\mathsf{ivk} := \mathsf{CRH}^{\mathsf{ivk}}\big(\mathsf{repr}_{\mathbb{J}}(\mathsf{ak}), \mathsf{repr}_{\mathbb{J}}(\mathsf{nk})\big).$$

If $\mathsf{ivk} = 0$, discard this key and repeat with a new sk.

As explained in § 3.1 ***'Payment Addresses and Keys'*** on p. 12, Sapling allows the efficient creation of many *diversified payment addresses* with the same *spending authority*. A group of such addresses shares the same *full viewing key* and *incoming viewing key*.

To create a new *diversified payment address* given an *incoming viewing key* ivk, repeatedly pick a *diversifier* d uniformly at random from $\mathbb{B}^{[\ell_{\mathsf{d}}]}$ until the *diversified base* $\mathsf{g_d} = \mathsf{DiversifyHash}^{\mathsf{Sapling}}(\mathsf{d})$ is not $\bot$. Then calculate the *diversified transmission key* $\mathsf{pk_d}$:

$$\mathsf{pk_d} := \mathsf{KA}^{\mathsf{Sapling}}.\mathsf{DerivePublic}(\mathsf{ivk}, \mathsf{g_d}).$$

The resulting *diversified payment address* is $(\mathsf{d} : \mathbb{B}^{[\ell_{\mathsf{d}}]}, \mathsf{pk_d} : \mathsf{KA}^{\mathsf{Sapling}}.\mathsf{PublicPrimeOrder})$.

For each *spending key*, there is also a *default diversified payment address* with a "random-looking" *diversifier*. This allows an implementation that does not expose diversified addresses as a user-visible feature, to use a default address that cannot be distinguished (without knowledge of the *spending key*) from one with a random *diversifier* as above. Note however that the *zcashd* wallet picks *diversifiers* as in [ZIP-32], rather than using this procedure.

Let $\mathsf{first} : (\mathbb{B}^{\mathbb{Y}} \to T \cup \{\bot\}) \to T \cup \{\bot\}$ be as defined in § 5.4.8.5 ***'Group Hash into Jubjub'*** on p. 79. Define:

$$\mathsf{CheckDiversifier}(\mathsf{d} : \mathbb{B}^{[\ell_{\mathsf{d}}]}) := \begin{cases} \bot, & \text{if } \mathsf{DiversifyHash}^{\mathsf{Sapling}}(\mathsf{d}) = \bot \\ \mathsf{d}, & \text{otherwise} \end{cases}$$

$$\mathsf{DefaultDiversifier}(\mathsf{sk} : \mathbb{B}^{[\ell_{\mathsf{sk}}]}) := \mathsf{first}\big(i : \mathbb{B}^{\mathbb{Y}} \mapsto \mathsf{CheckDiversifier}(\mathsf{truncate}_{(\ell_{\mathsf{d}}/8)}(\mathsf{PRF}^{\mathsf{expand}}_{\mathsf{sk}}([3, i])))) : \mathbb{J}^{(r)*} \cup \{\bot\}\big).$$

For a random *spending key*, DefaultDiversifier returns $\bot$ with probability approximately 2^{-256}; if this happens, discard the key and repeat with a different sk.

Notes:

- The protocol does not prevent using the *diversifier* d to produce *"vanity"* addresses that start with a meaningful string when encoded in *Bech32* (see § 5.6.3.1 ***'Sapling Payment Addresses'*** on p. 85). Users and writers of software that generates addresses should be aware that this provides weaker privacy properties than a randomly chosen *diversifier*, since a vanity address can obviously be distinguished, and might leak more information than intended as to who created it.
- Similarly, address generators **MAY** encode information in the *diversifier* that can be recovered by the recipient of a payment to determine which *diversified payment address* was used. It is **RECOMMENDED**, instead of directly encoding information in the *diversifier*, to encode it in the *diversifier index* specified in [ZIP-32]. This ensures that the information is only accessible to a holder of the *diversifier key* dk.

Non-normative notes:

- Assume that $\mathsf{PRF}^{\mathsf{expand}}$ is a *PRF* with output range $\mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{PRFexpand}}/8]}}$, where $2^{\ell_{\mathsf{PRFexpand}}}$ is large compared to $r_{\mathbb{J}}$.
  Define $f : \mathbb{B}^{[\ell_{\mathsf{sk}}]} \times \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}} \to \mathbb{F}_{r_{\mathbb{J}}}$ by $f_{\mathsf{sk}}(t) := \mathsf{ToScalar}^{\mathsf{Sapling}}\big(\mathsf{PRF}^{\mathsf{expand}}_{\mathsf{sk}}(t)\big)$.
  f is also a *PRF* since $\mathsf{LEOS2IP}_{\ell_{\mathsf{PRFexpand}}} : \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{PRFexpand}}/8]}} \to \{0 \,..\, 2^{\ell_{\mathsf{PRFexpand}}}-1\}$ is injective; the bias introduced by reduction modulo $r_{\mathbb{J}}$ is small because § 5.3 ***'Constants'*** on p. 57 defines $\ell_{\mathsf{PRFexpand}}$ as 512, while $r_{\mathbb{J}}$ has length 252 bits. It follows that the distribution of ask, i.e. $\mathsf{PRF}^{\mathsf{expand}}_{\mathsf{sk}}([0]) : \mathsf{sk} \xleftarrow{R} \mathbb{B}^{[\ell_{\mathsf{sk}}]}$, is computationally indistinguishable from $\mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{GenPrivate}()$ defined in § 5.4.6.1 ***'Spend Authorization Signature (Sapling)'*** on p. 71.
- The distribution of nsk, i.e. $\mathsf{ToScalar}^{\mathsf{Sapling}}\big(\mathsf{PRF}^{\mathsf{expand}}_{\mathsf{sk}}([1])\big) : \mathsf{sk} \xleftarrow{R} \mathbb{B}^{[\ell_{\mathsf{sk}}]}$, is computationally indistinguishable from the uniform distribution on $\mathbb{F}_{r_{\mathbb{J}}}$. Since $\mathsf{nsk} : \mathbb{F}_{r_{\mathbb{J}}} \mapsto \mathsf{repr}_{\mathbb{J}}\big([\mathsf{nsk}]\,\mathcal{H}^{\mathsf{Sapling}} : \mathbb{J}^{(r)}_{\star}\big)$ is bijective, the distribution of $\mathsf{repr}_{\mathbb{J}}(\mathsf{nk})$ will be computationally indistinguishable from uniform on $\mathbb{J}^{(r)}_{\star}$ (the keyspace of $\mathsf{PRF}^{\mathsf{nfSapling}}$).

4.3 JoinSplit Descriptions

A *JoinSplit transfer*, as specified in § 3.5 ***'JoinSplit Transfers and Descriptions'*** on p. 17, is encoded in *transactions* as a *JoinSplit description*.

Each *transaction* includes a sequence of zero or more *JoinSplit descriptions*. When this sequence is non-empty, the *transaction* also includes encodings of a JoinSplitSig public *validating key* and signature.

Let $\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}$, $\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}$, ℓ_{Seed}, $\mathrm{N}^{\mathsf{old}}$, $\mathrm{N}^{\mathsf{new}}$, and MAX_MONEY be as defined in § 5.3 ***'Constants'*** on p. 57.

Let hSigCRH be as defined in § 4.1.1 ***'Hash Functions'*** on p. 22.

Let $\mathsf{NoteCommit}^{\mathsf{Sprout}}$ be as defined in § 4.1.7 ***'Commitment'*** on p. 27.

Let $\mathsf{KA}^{\mathsf{Sprout}}$ be as defined in § 4.1.4 ***'Key Agreement'*** on p. 23.

Let Sym be as defined in § 4.1.3 ***'Symmetric Encryption'*** on p. 23.

Let ZKJoinSplit be as defined in § 4.1.12 ***'Zero-Knowledge Proving System'*** on p. 30.

A *JoinSplit description* comprises $(\mathsf{v}^{\mathsf{old}}_{\mathsf{pub}}, \mathsf{v}^{\mathsf{new}}_{\mathsf{pub}}, \mathsf{rt}^{\mathsf{Sprout}}, \mathsf{nf}^{\mathsf{old}}_{1..\mathrm{N}^{\mathsf{old}}}, \mathsf{cm}^{\mathsf{new}}_{1..\mathrm{N}^{\mathsf{new}}}, \mathsf{epk}, \mathsf{randomSeed}, \mathsf{h}_{1..\mathrm{N}^{\mathsf{old}}}, \pi_{\mathsf{ZKJoinSplit}}, \mathsf{C}^{\mathsf{enc}}_{1..\mathrm{N}^{\mathsf{new}}})$ where

- $\mathsf{v}^{\mathsf{old}}_{\mathsf{pub}} : \{0 \,..\, \mathsf{MAX_MONEY}\}$ is the value that the *JoinSplit transfer* removes from the *transparent transaction value pool*;
- $\mathsf{v}^{\mathsf{new}}_{\mathsf{pub}} : \{0 \,..\, \mathsf{MAX_MONEY}\}$ is the value that the *JoinSplit transfer* inserts into the *transparent transaction value pool*;
- $\mathsf{rt}^{\mathsf{Sprout}} : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}]}$ is an *anchor*, as defined in § 3.4 ***'Transactions and Treestates'*** on p. 16, for the output *treestate* of either a previous *block*, or a previous *JoinSplit transfer* in this *transaction*.
- $\mathsf{nf}^{\mathsf{old}}_{1..\mathrm{N}^{\mathsf{old}}} : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}][\mathrm{N}^{\mathsf{old}}]}$ is the sequence of *nullifiers* for the input *notes*;
- $\mathsf{cm}^{\mathsf{new}}_{1..\mathrm{N}^{\mathsf{new}}} : \mathsf{NoteCommit}^{\mathsf{Sprout}}.\mathsf{Output}^{[\mathrm{N}^{\mathsf{new}}]}$ is the sequence of *note commitments* for the output *notes*;
- $\mathsf{epk} : \mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Public}$ is a key agreement *public key*, used to derive the key for encryption of the *transmitted notes ciphertext* (§ 4.17 ***'In-band secret distribution (Sprout)'*** on p. 50);
- $\mathsf{randomSeed} : \mathbb{B}^{[\ell_{\mathsf{Seed}}]}$ is a seed that must be chosen independently at random for each *JoinSplit description*;
- $\mathsf{h}_{1..\mathrm{N}^{\mathsf{old}}} : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}][\mathrm{N}^{\mathsf{old}}]}$ is a sequence of tags that bind $\mathsf{h}_{\mathsf{Sig}}$ to each a_{sk} of the input *notes*;
- $\pi_{\mathsf{ZKJoinSplit}} : \mathsf{ZKJoinSplit.Proof}$ is a *zk proof* with *primary input* $(\mathsf{rt}^{\mathsf{Sprout}}, \mathsf{nf}^{\mathsf{old}}_{1..\mathrm{N}^{\mathsf{old}}}, \mathsf{cm}^{\mathsf{new}}_{1..\mathrm{N}^{\mathsf{new}}}, \mathsf{v}^{\mathsf{old}}_{\mathsf{pub}}, \mathsf{v}^{\mathsf{new}}_{\mathsf{pub}}, \mathsf{h}_{\mathsf{Sig}}, \mathsf{h}_{1..\mathrm{N}^{\mathsf{old}}})$ for the *JoinSplit statement* defined in § 4.16.1 ***'JoinSplit Statement (Sprout)'*** on p. 47 (this is a BCTV14 proof before **Sapling** activation, and a Groth16 proof after **Sapling** activation);

- $C^{enc}_{1..N^{new}} : \mathsf{Sym.C}^{[N^{new}]}$ is a sequence of ciphertext components for the encrypted output *notes*.

The `ephemeralKey` and `encCiphertexts` fields together form the *transmitted notes ciphertext*.

The value $\mathsf{h_{Sig}}$ is also computed from randomSeed, $\mathsf{nf}^{old}_{1..N^{old}}$, and the `joinSplitPubKey` of the containing *transaction*:

$\mathsf{h_{Sig}} := \mathsf{hSigCRH}(\mathsf{randomSeed}, \mathsf{nf}^{old}_{1..N^{old}}, \texttt{joinSplitPubKey})$.

Consensus rules:

- Elements of a *JoinSplit description* **MUST** have the types given above (for example: $0 \leq \mathsf{v}^{old}_{pub} \leq$ MAX_MONEY and $0 \leq \mathsf{v}^{new}_{pub} \leq$ MAX_MONEY).
- The proof $\pi_{\mathsf{ZKJoinSplit}}$ **MUST** be valid given a *primary input* formed from the relevant other fields and $\mathsf{h_{Sig}}$ — i.e. $\mathsf{ZKJoinSplit.Verify}\big((\mathsf{rt}^{Sprout}, \mathsf{nf}^{old}_{1..N^{old}}, \mathsf{cm}^{new}_{1..N^{new}}, \mathsf{v}^{old}_{pub}, \mathsf{v}^{new}_{pub}, \mathsf{h_{Sig}}, \mathsf{h}_{1..N^{old}}), \pi_{\mathsf{ZKJoinSplit}}\big) = 1$.
- Either v^{old}_{pub} or v^{new}_{pub} **MUST** be zero.

4.4 Spend Descriptions

A *Spend transfer*, as specified in § 3.6 *'Spend Transfers, Output Transfers, and their Descriptions'* on p. 18, is encoded in *transactions* as a *Spend description*.

Each *transaction* includes a sequence of zero or more *Spend descriptions*.

Each *Spend description* is authorized by a signature, called the *spend authorization signature*.

Let $\ell^{Sapling}_{Merkle}$ and $\ell_{\mathsf{PRFnfSapling}}$ be as defined in § 5.3 *'Constants'* on p. 57.

Let $\mathcal{O}_{\mathbb{J}}$, $\mathsf{abst}_{\mathbb{J}}$, $\mathsf{repr}_{\mathbb{J}}$, and $h_{\mathbb{J}}$ be as defined in § 5.4.8.3 *'Jubjub'* on p. 77.

Let $\mathsf{ValueCommit}^{Sapling}$.Output be as defined in § 4.1.7 *'Commitment'* on p. 27.

Let $\mathsf{SpendAuthSig}^{Sapling}$ be as defined in § 4.13 *'Spend Authorization Signature (Sapling)'* on p. 44.

Let ZKSpend be as defined in § 4.1.12 *'Zero-Knowledge Proving System'* on p. 30.

A *Spend description* comprises $(\mathsf{cv}, \mathsf{rt}^{SaplingPool}, \mathsf{nf}, \mathsf{rk}, \pi_{\mathsf{ZKSpend}}, \mathsf{spendAuthSig})$ where

- $\mathsf{cv} : \mathsf{ValueCommit}^{Sapling}$.Output is the *value commitment* to the value of the input *note*;
- $\mathsf{rt}^{SaplingPool} : \mathbb{B}^{[\ell^{Sapling}_{Merkle}]}$ is an *anchor*, as defined in § 3.4 *'Transactions and Treestates'* on p. 16, for the output *treestate* of a previous *block*;
- $\mathsf{nf} : \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{PRFnfSapling}}/8]}}$ is the *nullifier* for the input *note*;
- $\mathsf{rk} : \mathsf{SpendAuthSig}^{Sapling}$.Public is a randomized *validating key* that should be used to validate spendAuthSig;
- $\pi_{\mathsf{ZKSpend}} : \mathsf{ZKSpend.Proof}$ is a *zk-SNARK proof* with *primary input* $(\mathsf{cv}, \mathsf{rt}^{SaplingPool}, \mathsf{nf}, \mathsf{rk})$ for the Sapling *Spend statement* defined in § 4.16.2 *'Spend Statement (Sapling)'* on p. 48;
- $\mathsf{spendAuthSig} : \mathsf{SpendAuthSig}^{Sapling}$.Signature is a *spend authorization signature*, validated as specified in § 4.13 *'Spend Authorization Signature (Sapling)'* on p. 44.

Consensus rules:

- Elements of a *Spend description* **MUST** be valid encodings of the types given above.
- cv and rk **MUST NOT** be of *small order*, i.e. $[h_{\mathbb{J}}]$ cv **MUST NOT** be $\mathcal{O}_{\mathbb{J}}$ and $[h_{\mathbb{J}}]$ rk **MUST NOT** be $\mathcal{O}_{\mathbb{J}}$.
- The proof π_{ZKSpend} **MUST** be valid given a *primary input* formed from the other fields except spendAuthSig — i.e. $\mathsf{ZKSpend.Verify}\big((\mathsf{cv}, \mathsf{rt}^{SaplingPool}, \mathsf{nf}, \mathsf{rk}), \pi_{\mathsf{ZKSpend}}\big) = 1$.

- Let SigHash be the *SIGHASH transaction hash* of this *transaction*, not associated with an input, as defined in § 4.9 ***'SIGHASH Transaction Hashing'*** on p. 40 using SIGHASH_ALL.

  The *spend authorization signature* **MUST** be a valid $\mathsf{SpendAuthSig}^{\mathsf{Sapling}}$ signature over SigHash using rk as the *validating key*— i.e. $\mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{Validate}_{\mathsf{rk}}(\mathsf{SigHash}, \mathsf{spendAuthSig}) = 1$.

Non-normative notes:

- As stated in § 5.4.7.3 on p. 73, an implementation of $\mathsf{HomomorphicPedersenCommit}^{\mathsf{Sapling}}$ **MAY** resample the *commitment trapdoor* until the resulting commitment is not $\mathcal{O}_{\mathbb{J}}$.
- The rule that cv and rk **MUST** not be *small-order* has the effect of also preventing *non-canonical* encodings of these fields. That is, it is necessarily the case that $\mathsf{repr}_{\mathbb{J}}(\mathsf{abst}_{\mathbb{J}}(\mathsf{cv})) = \mathsf{cv}$ and $\mathsf{repr}_{\mathbb{J}}(\mathsf{abst}_{\mathbb{J}}(\mathsf{rk})) = \mathsf{rk}$.

4.5 Output Descriptions

An *Output transfer*, as specified in § 3.6 ***'Spend Transfers, Output Transfers, and their Descriptions'*** on p. 18, is encoded in *transactions* as an *Output description*.

Each *transaction* includes a sequence of zero or more *Output descriptions*. There are no signatures associated with *Output descriptions*.

Let $\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}$ be as defined in § 5.3 ***'Constants'*** on p. 57.

Let $\mathcal{O}_{\mathbb{J}}$, $\mathsf{abst}_{\mathbb{J}}$, $\mathsf{repr}_{\mathbb{J}}$, and $h_{\mathbb{J}}$ be as defined in § 5.4.8.3 ***'Jubjub'*** on p. 77.

Let $\mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Output}$ be as defined in § 4.1.7 ***'Commitment'*** on p. 27.

Let $\mathsf{KA}^{\mathsf{Sapling}}$ be as defined in § 4.1.4 ***'Key Agreement'*** on p. 23.

Let Sym be as defined in § 4.1.3 ***'Symmetric Encryption'*** on p. 23.

Let ZKOutput be as defined in § 4.1.12 ***'Zero-Knowledge Proving System'*** on p. 30.

An *Output description* comprises $(\mathsf{cv}, \mathsf{cm}_u, \mathsf{epk}, \mathsf{C}^{\mathsf{enc}}, \mathsf{C}^{\mathsf{out}}, \pi_{\mathsf{ZKOutput}})$ where

- $\mathsf{cv} : \mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Output}$ is the *value commitment* to the value of the output *note*;
- $\mathsf{cm}_u : \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]}$ is the result of applying $\mathsf{Extract}_{\mathbb{J}^{(r)}}$ (defined in § 5.4.8.4 ***'Coordinate Extractor for Jubjub'*** on p. 78) to the *note commitment* for the output *note*;
- $\mathsf{epk} : \mathsf{KA}^{\mathsf{Sapling}}.\mathsf{Public}$ is a key agreement *public key*, used to derive the key for encryption of the *transmitted note ciphertext* (§ 4.18 ***'In-band secret distribution (Sapling)'*** on p. 51);
- $\mathsf{C}^{\mathsf{enc}} : \mathsf{Sym}.\mathbf{C}$ is a ciphertext component for the encrypted output *note*;
- $\mathsf{C}^{\mathsf{out}} : \mathsf{Sym}.\mathbf{C}$ is a ciphertext component that allows the holder of the *outgoing cipher key* (which can be derived from a *full viewing key*) to recover the recipient *diversified transmission key* pk_{d} and the *ephemeral private key* esk, hence the entire *note plaintext*;
- $\pi_{\mathsf{ZKOutput}} : \mathsf{ZKOutput}.\mathsf{Proof}$ is a *zk-SNARK proof* with *primary input* $(\mathsf{cv}, \mathsf{cm}_u, \mathsf{epk})$ for the *Output statement* defined in § 4.16.3 ***'Output Statement (Sapling)'*** on p. 49.

Consensus rules:

- Elements of an *Output description* **MUST** be valid encodings of the types given above.
- cv and epk **MUST NOT** be of *small order*, i.e. $[h_{\mathbb{J}}]\,\mathsf{cv}$ **MUST NOT** be $\mathcal{O}_{\mathbb{J}}$ and $[h_{\mathbb{J}}]\,\mathsf{epk}$ **MUST NOT** be $\mathcal{O}_{\mathbb{J}}$.
- The proof π_{ZKOutput} **MUST** be valid given a *primary input* formed from the other fields except $\mathsf{C}^{\mathsf{enc}}$ and $\mathsf{C}^{\mathsf{out}}$ — i.e. $\mathsf{ZKOutput}.\mathsf{Verify}\big((\mathsf{cv}, \mathsf{cm}_u, \mathsf{epk}), \pi_{\mathsf{ZKOutput}}\big) = 1$.

Non-normative notes:

- As stated in § 5.4.7.3 on p. 73, an implementation of $\mathsf{HomomorphicPedersenCommit}^{\mathsf{Sapling}}$ **MAY** resample the *commitment trapdoor* until the resulting commitment is not $\mathcal{O}_{\mathbb{J}}$.
- The rule that cv and epk **MUST** not be *small-order* has the effect of also preventing *non-canonical* encodings of these fields. That is, it is necessarily the case that $\mathsf{repr}_{\mathbb{J}}(\mathsf{abst}_{\mathbb{J}}(\mathsf{cv})) = \mathsf{cv}$ and $\mathsf{repr}_{\mathbb{J}}(\mathsf{abst}_{\mathbb{J}}(\mathsf{epk})) = \mathsf{rk}$.

4.6 Sending Notes

4.6.1 Sending Notes (Sprout)

In order to send *Sprout-pool* funds, the sender constructs a *transaction* containing one or more *JoinSplit descriptions*.

Let JoinSplitSig be as specified in § 4.1.6 *'Signature'* on p. 24.

Let $\mathsf{NoteCommit}^{\mathsf{Sprout}}$ be as specified in § 4.1.7 *'Commitment'* on p. 27.

Let ℓ_{Seed} and $\ell_{\varphi}^{\mathsf{Sprout}}$ be as specified in § 5.3 *'Constants'* on p. 57.

Sending a *transaction* containing *JoinSplit descriptions* involves first generating a new JoinSplitSig key pair:

$\texttt{joinSplitPrivKey} \xleftarrow{R} \mathsf{JoinSplitSig.GenPrivate}()$

$\texttt{joinSplitPubKey} := \mathsf{JoinSplitSig.DerivePublic}(\texttt{joinSplitPrivKey})$.

For each *JoinSplit description*, the sender chooses randomSeed uniformly at random on $\mathbb{B}^{[\ell_{\mathsf{Seed}}]}$, and selects the input *notes*. At this point there is sufficient information to compute $\mathsf{h}_{\mathsf{Sig}}$, as described in the previous section. The sender also chooses φ uniformly at random on $\mathbb{B}^{[\ell_{\varphi}^{\mathsf{Sprout}}]}$. Then it creates each output *note* with index $i : \{1..\mathrm{N}^{\mathsf{new}}\}$:

- Choose uniformly random $\mathsf{rcm}_i \xleftarrow{R} \mathsf{NoteCommit}^{\mathsf{Sprout}}.\mathsf{GenTrapdoor}()$.
- Compute $\rho_i = \mathsf{PRF}_{\varphi}^{\rho}(i, \mathsf{h}_{\mathsf{Sig}})$.
- Compute $\mathsf{cm}_i = \mathsf{NoteCommit}_{\mathsf{rcm}_i}^{\mathsf{Sprout}}(\mathsf{a}_{\mathsf{pk},i}, \mathsf{v}_i, \rho_i)$.
- Let $\mathbf{np}_i = (\mathtt{0x00}, \mathsf{v}_i, \rho_i, \mathsf{rcm}_i, \mathsf{memo}_i)$.

$\mathbf{np}_{1..\mathrm{N}^{\mathsf{new}}}$ are then encrypted to the recipient *transmission keys* $\mathsf{pk}_{\mathsf{enc},1..\mathrm{N}^{\mathsf{new}}}$, giving the *transmitted notes ciphertext* $\left(\mathsf{epk}, \mathsf{C}_{1..\mathrm{N}^{\mathsf{new}}}^{\mathsf{enc}}\right)$, as described in § 4.17 *'In-band secret distribution (Sprout)'* on p. 50.

In order to minimize information leakage, the sender **SHOULD** randomize the order of the input *notes* and of the output *notes*. Other considerations relating to information leakage from the structure of *transactions* are beyond the scope of this specification.

After generating all of the *JoinSplit descriptions*, the sender obtains $\mathsf{dataToBeSigned} : \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}}$ as described in § 4.10 *'Non-malleability (Sprout)'* on p. 41, and signs it with the private *JoinSplit signing key*:

$\texttt{joinSplitSig} \xleftarrow{R} \mathsf{JoinSplitSig.Sign}_{\texttt{joinSplitPrivKey}}(\mathsf{dataToBeSigned})$

Then the encoded *transaction* including joinSplitSig is submitted to the *peer-to-peer network*.

The facility to send to Sprout addresses is **OPTIONAL** for a particular node or wallet implementation.

4.6.2 Sending Notes (Sapling)

In order to send *Sapling-pool shielded* value, the sender constructs a *transaction* with one or more *Output descriptions*.

Let $\mathsf{ValueCommit}^{\mathsf{Sapling}}$ and $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ be as specified in § 4.1.7 *'Commitment'* on p. 27.

Let $\mathsf{KA}^{\mathsf{Sapling}}$ be as specified in § 4.1.4 *'Key Agreement'* on p. 23.

Let $\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ be as specified in § 4.1.1 *'Hash Functions'* on p. 22.

Let $\mathsf{ToScalar}^{\mathsf{Sapling}}$ be as specified in § 4.2.2 *'Sapling Key Components'* on p. 32.

Let $\mathsf{repr}_{\mathbb{J}}$ and $r_{\mathbb{J}}$ be as defined in § 5.4.8.3 *'Jubjub'* on p. 77.

Let ovk be a Sapling *outgoing viewing key* that is intended to be able to decrypt this payment. This may be one of:

- the *outgoing viewing key* for the address (or one of the addresses) from which the payment was sent;
- the *outgoing viewing key* for all payments associated with an *"account"*, to be defined in [ZIP-32];
- $\bot$, if the sender should not be able to decrypt the payment once it has deleted its own copy.

Note: Choosing $\mathsf{ovk} = \bot$ is useful if the sender prefers to obtain forward secrecy of the payment information with respect to compromise of its own secrets.

Let leadByte be the *note plaintext lead byte*, chosen according to § 3.2.1 *'Note Plaintexts and Memo Fields'* on p. 14 with *pool* = *SaplingPool*.

For each *Output description*, the sender selects a value $\mathsf{v} : \{0 .. \mathsf{MAX_MONEY}\}$ and a destination Sapling *shielded payment address* $(\mathsf{d}, \mathsf{pk_d})$, and then performs the following steps:

Check that $\mathsf{pk_d}$ is of type $\mathsf{KA}^{\mathsf{Sapling}}$.PublicPrimeOrder, i.e. it **MUST** be a valid *ctEdwards curve* point on the *Jubjub curve* (as defined in § 5.4.8.3 *'Jubjub'* on p. 77), $[r_{\mathbb{J}}]\,\mathsf{pk_d} = \mathcal{O}_{\mathbb{J}}$, and $\mathsf{pk_d} \neq \mathcal{O}_{\mathbb{J}}$.

Calculate $\mathsf{g_d} = \mathsf{DiversifyHash}^{\mathsf{Sapling}}(\mathsf{d})$ and check that $\mathsf{g_d} \neq \bot$.

Choose a uniformly random *commitment trapdoor* $\mathsf{rcv} \xleftarrow{R} \mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{GenTrapdoor}()$.

Choose a uniformly random *ephemeral private key* $\mathsf{esk} \xleftarrow{R} \mathsf{KA}^{\mathsf{Sapling}}.\mathsf{Private} \setminus \{0\}$.

Choose a uniformly random *commitment trapdoor* $\mathsf{rcm} \xleftarrow{R} \mathsf{NoteCommit.GenTrapdoor}()$.

Set $\underline{\mathsf{rcm}} := \mathsf{I2LEOSP}_{256}(\mathsf{rcm})$.

Let $\mathsf{cv} = \mathsf{ValueCommit}^{\mathsf{Sapling}}_{\mathsf{rcv}}(\mathsf{v})$.

Let $\mathsf{cm} = \mathsf{NoteCommit}^{\mathsf{Sapling}}_{\mathsf{rcm}}(\mathsf{repr}_{\mathbb{J}}(\mathsf{g_d}), \mathsf{repr}_{\mathbb{J}}(\mathsf{pk_d}), \mathsf{v})$.

Let $\mathbf{np} = (\mathsf{leadByte}, \mathsf{d}, \mathsf{v}, \underline{\mathsf{rcm}}, \mathsf{memo})$.

Encrypt $\mathbf{np}$ to the recipient *diversified transmission key* $\mathsf{pk_d}$ with *diversified base* $\mathsf{g_d}$, and to the *outgoing viewing key* ovk, giving the *transmitted note ciphertext* $(\mathsf{epk}, \mathsf{C^{enc}}, \mathsf{C^{out}})$. This procedure is described in § 4.18.1 *'Encryption (Sapling)'* on p. 52; it also uses cv and cmu to derive ock, and takes esk as input.

Generate a proof π_{ZKOutput} for the *Output statement* in § 4.16.3 *'Output Statement (Sapling)'* on p. 49.

Return $(\mathsf{cv}, \mathsf{cm}, \mathsf{epk}, \mathsf{C^{enc}}, \mathsf{C^{out}}, \pi_{\mathsf{ZKOutput}})$.

In order to minimize information leakage, the sender **SHOULD** randomize the order of *Output descriptions* in a *transaction*. Other considerations relating to information leakage from the structure of *transactions* are beyond the scope of this specification. The encoded *transaction* is submitted to the *peer-to-peer network*.

4.7 Dummy Notes

4.7.1 Dummy Notes (Sprout)

The fields in a *JoinSplit description* allow for $\mathrm{N}^{\mathsf{old}}$ input *notes*, and $\mathrm{N}^{\mathsf{new}}$ output *notes*. In practice, we may wish to encode a *JoinSplit transfer* with fewer input or output *notes*. This is achieved using *dummy notes*.

Let $\ell_{\mathsf{a_{sk}}}$ and $\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}$ be as defined in § 5.3 *'Constants'* on p. 57.

Let $\mathsf{PRF}^{\mathsf{nfSprout}}$ be as defined in § 4.1.2 *'Pseudo Random Functions'* on p. 22.

Let $\mathsf{NoteCommit}^{\mathsf{Sprout}}$ be as defined in § 4.1.7 *'Commitment'* on p. 27.

A *dummy Sprout-pool* input *note*, with index i in the *JoinSplit description*, is constructed as follows:

- Generate a new uniformly random *spending key* $\mathsf{a}^{\mathsf{old}}_{\mathsf{sk},i} \xleftarrow{R} \mathbb{B}^{[\ell_{\mathsf{a_{sk}}}]}$ and derive its *paying key* $\mathsf{a}^{\mathsf{old}}_{\mathsf{pk},i}$.
- Set $\mathsf{v}^{\mathsf{old}}_i = 0$.
- Choose uniformly random $\rho^{\mathsf{old}}_i \xleftarrow{R} \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}]}$ and $\mathsf{rcm}^{\mathsf{old}}_i \xleftarrow{R} \mathsf{NoteCommit}^{\mathsf{Sprout}}.\mathsf{GenTrapdoor}()$.
- Compute $\mathsf{nf}^{\mathsf{old}}_i = \mathsf{PRF}^{\mathsf{nfSprout}}_{\mathsf{a}^{\mathsf{old}}_{\mathsf{sk},i}}(\rho^{\mathsf{old}}_i)$.
- Let path_i be a *dummy Merkle path* for the *auxiliary input* to the *JoinSplit statement* (this will not be checked).
- When generating the *JoinSplit proof*, set $\mathsf{enforceMerklePath}_i$ to 0.

A *dummy Sprout-pool* output *note* is constructed as normal but with zero value, and sent to a random *shielded payment address*.

4.7.2 Dummy Notes (Sapling)

In Sapling there is no need to use *dummy notes* simply in order to fill otherwise unused inputs as in the case of a *JoinSplit description*; nevertheless it may be useful for privacy to obscure the number of real *shielded inputs* from *Sapling-pool notes*.

Let ℓ_{sk} be as defined in § 5.3 *'Constants'* on p. 57.

Let $\mathsf{ValueCommit}^{\mathsf{Sapling}}$ and $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ be as defined in § 4.1.7 *'Commitment'* on p. 27.

Let $\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ be as specified in § 4.1.1 *'Hash Functions'* on p. 22.

Let $\mathsf{ToScalar}^{\mathsf{Sapling}}$ be as specified in § 4.2.2 *'Sapling Key Components'* on p. 32.

Let $\mathsf{repr}_{\mathbb{J}}$ and $r_{\mathbb{J}}$ be as defined in § 5.4.8.3 *'Jubjub'* on p. 77.

Let $\mathsf{PRF}^{\mathsf{nfSapling}}$ be as defined in § 4.1.2 *'Pseudo Random Functions'* on p. 22.

Let $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ be as defined in § 4.1.7 *'Commitment'* on p. 27.

A *Spend description* for a *dummy Sapling-pool* input *note* with *note plaintext lead byte* 0x02 is constructed as follows:

- Choose uniformly random $\mathsf{sk} \xleftarrow{R} \mathbb{B}^{[\ell_{\mathsf{sk}}]}$.
- Generate the ak and nk components of a *full viewing key* and a *diversified payment address* $(\mathsf{d}, \mathsf{pk_d})$ for sk, as described in § 4.2.2 *'Sapling Key Components'* on p. 32.
- Let $\mathsf{v} = 0$ and $\mathsf{pos} = 0$.
- Choose uniformly random $\mathsf{rcv} \xleftarrow{R} \mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{GenTrapdoor}()$.
- Choose uniformly random $\mathsf{rseed} \xleftarrow{R} \mathbb{B}^{\mathbb{Y}^{[32]}}$.
- Derive $\mathsf{rcm} = \mathsf{ToScalar}^{\mathsf{Sapling}}\big(\mathsf{PRF}^{\mathsf{expand}}_{\mathsf{rseed}}([4])\big)$.
- Let $\mathsf{cv} = \mathsf{ValueCommit}^{\mathsf{Sapling}}_{\mathsf{rcv}}(\mathsf{v})$.
- Let $\mathsf{cm} = \mathsf{NoteCommit}^{\mathsf{Sapling}}_{\mathsf{rcm}}\big(\mathsf{repr}_{\mathbb{J}}(\mathsf{g_d}), \mathsf{repr}_{\mathbb{J}}(\mathsf{pk_d}), \mathsf{v}\big)$.
- Let $\rho\star = \mathsf{repr}_{\mathbb{J}}\big(\mathsf{MixingPedersenHash}(\mathsf{cm}, \mathsf{pos})\big)$.
- Let $\mathsf{nk}\star = \mathsf{repr}_{\mathbb{J}}(\mathsf{nk})$.
- Let $\mathsf{nf} = \mathsf{PRF}^{\mathsf{nfSapling}}_{\mathsf{nk}\star}(\rho\star)$.
- Construct a *dummy Merkle path* path for use in the *auxiliary input* to the *Spend statement* (this will not be checked, because $\mathsf{v} = 0$).

As in Sprout, a *dummy Sapling-pool* output *note* is constructed as normal but with zero value, and sent to a random *shielded payment address*.

4.8 Merkle Path Validity

Let MerkleDepth be $\mathsf{MerkleDepth}^{\mathsf{Sprout}}$ for the *Sprout-pool note commitment tree*, or $\mathsf{MerkleDepth}^{\mathsf{Sapling}}$ for the *Sapling-pool note commitment tree*. These constants are defined in § 5.3 *'Constants'* on p. 57.

Similarly, let MerkleCRH be $\mathsf{MerkleCRH}^{\mathsf{Sprout}}$ for Sprout, or $\mathsf{MerkleCRH}^{\mathsf{Sapling}}$ for Sapling.

The discussion below applies independently to the *Sprout-pool* and *Sapling-pool note commitment trees*.

Each *node* in the *incremental Merkle tree* is associated with a *hash value*, which is a *bit sequence*.

The *layer* numbered h, counting from *layer* 0 at the *root*, has 2^h *nodes* with *indices* 0 to $2^h - 1$ inclusive.

Let M^h_i be the *hash value* associated with the *node* at *index* i in *layer* h.

The *nodes* at *layer* MerkleDepth are called *leaf nodes*. When a *note commitment* is added to the tree, it occupies the *leaf node hash value* $\mathsf{M}_i^{\mathsf{MerkleDepth}}$ for the next available i.

As-yet unused *leaf nodes* are associated with a distinguished *hash value* $\mathsf{Uncommitted}^{\mathsf{Sprout}}$ or $\mathsf{Uncommitted}^{\mathsf{Sapling}}$. It is assumed to be infeasible to find a preimage *note* $\mathbf{n}$ such that $\mathsf{NoteCommitment}^{\mathsf{Sprout}}(\mathbf{n}) = \mathsf{Uncommitted}^{\mathsf{Sprout}}$. (No similar assumption is needed for Sapling because we use a representation for $\mathsf{Uncommitted}^{\mathsf{Sapling}}$ that cannot occur as an output of $\mathsf{NoteCommitment}^{\mathsf{Sapling}}$.)

The *nodes* at *layers* 0 to $\mathsf{MerkleDepth} - 1$ inclusive are called *internal nodes*, and are associated with MerkleCRH outputs. *Internal nodes* are computed from their children in the next *layer* as follows: for $0 \leq h < \mathsf{MerkleDepth}$ and $0 \leq i < 2^h$,

$$\mathsf{M}_i^h := \mathsf{MerkleCRH}(h, \mathsf{M}_{2i}^{h+1}, \mathsf{M}_{2i+1}^{h+1}).$$

A *Merkle path* from *leaf node* $\mathsf{M}_i^{\mathsf{MerkleDepth}}$ in the *incremental Merkle tree* is the sequence

$$[\ \mathsf{M}_{\mathsf{sibling}(h,i)}^h \text{ for } h \text{ from } \mathsf{MerkleDepth} \text{ down to } 1\ ],$$

where

$$\mathsf{sibling}(h, i) := \mathsf{floor}\left(\frac{i}{2^{\mathsf{MerkleDepth}-h}}\right) \oplus 1$$

Given such a *Merkle path*, it is possible to verify that *leaf node* $\mathsf{M}_i^{\mathsf{MerkleDepth}}$ is in a tree with a given *root* $\mathsf{rt} = \mathsf{M}_0^0$.

Notes:

- For Sapling, Merkle *hash values* are specified to be encoded as *bit sequences*, but the *root* $\mathsf{rt}^{\mathit{SaplingPool}}$ is encoded for the *primary input* of a *Spend proof* as an element of $\mathbb{F}_{q_{\mathbb{J}}}$, as specified in § A.4 ***'The Sapling Spend circuit'*** on p. 166. The *Spend circuit* allows inputs to $\mathsf{MerkleCRH}^{\mathsf{Sapling}}$ at each *node* to be *non-canonically* encoded, as specified in § A.3.4 ***'Merkle path check'*** on p. 162.

4.9 SIGHASH Transaction Hashing

Bitcoin and **Zcash** use signatures and/or non-interactive proofs associated with *transaction* inputs to authorize spending. Because these signatures or proofs could otherwise be replayed in a different *transaction*, it is necessary to "bind" them to the *transaction* for which they are intended. This is done by hashing information about the *transaction* and (where applicable) the specific input, to give a *SIGHASH transaction hash* which is then used for the Spend authorization. The means of authorization differs between *transparent inputs*, inputs to Sprout *JoinSplit transfers*, and Sapling *Spend transfers* but for a given *transaction version* the same *SIGHASH transaction hash* algorithm is used.

In the case of **Zcash**, the BCTV14 and Groth16 proving systems used are ***malleable***, meaning that there is the potential for an adversary who does not know all of the *auxiliary inputs* to a proof, to malleate it in order to create a new proof involving related *auxiliary inputs* [DSDCOPS2001]. This can be understood as similar to a malleability attack on an encryption scheme, in which an adversary can malleate a ciphertext in order to create an encryption of a related plaintext, without knowing the original plaintext. **Zcash** has been designed to mitigate malleability attacks, as described in § 4.10 ***'Non-malleability (Sprout)'*** on p. 41, § 4.12 ***'Balance and Binding Signature (Sapling)'*** on p. 42, and § 4.13 ***'Spend Authorization Signature (Sapling)'*** on p. 44.

To provide additional flexibility when combining spend authorizations from different sources, **Bitcoin** defines several *SIGHASH types* that cover various parts of a transaction [Bitcoin-SigHash]. One of these types is SIGHASH_ALL, which is used for **Zcash**-specific signatures, i.e. *JoinSplit signatures*, *spend authorization signatures*, and *Sapling binding signatures*. In these cases the *SIGHASH transaction hash* is not associated with a *transparent input*, and so the input to hashing excludes ***all*** of the `scriptSig` fields in the non-**Zcash**-specific parts of the *transaction*.

In **Zcash**, all *SIGHASH types* are extended to cover the **Zcash**-specific fields `nJoinSplit`, `vJoinSplit`, and if present `joinSplitPubKey`. These fields are described in § 7.1 ***'Transaction Encoding and Consensus'*** on p. 89. The hash *does not* cover the field `joinSplitSig`. After **Overwinter** activation, all *SIGHASH types* are also extended to cover *transaction* fields introduced in that upgrade, and similarly after **Sapling** activation.

The original *SIGHASH algorithm* defined by **Bitcoin** suffered from some deficiencies as described in [ZIP-143]; in **Zcash** these were addressed by changing this algorithm as part of the **Overwinter** upgrade.

Consensus rules:

- [Pre-**Overwinter**] The *SIGHASH algorithm* used prior to **Overwinter** activation, i.e. for version 1 and 2 *transactions*, will be defined in [ZIP-76] (to be written).
- [**Overwinter** only, pre-**Sapling**] The *SIGHASH algorithm* used after **Overwinter** activation and before **Sapling** activation, i.e. for version 3 *transactions*, is defined in [ZIP-143].
- [**Sapling** onward] The *SIGHASH algorithm* used for version 4 *transactions* is defined in [ZIP-243].
- [**Overwinter** only, pre-**Sapling**] All *transactions* **MUST** use the **Overwinter** *consensus branch ID* `0x5BA81B19` as defined in [ZIP-201].
- [**Sapling** only] All *transactions* **MUST** use the **Sapling** *consensus branch ID* `0x76B809BB` as defined in [ZIP-205].

4.10 Non-malleability (Sprout)

Let dataToBeSigned be the hash of the *transaction*, not associated with an input, using the SIGHASH_ALL *SIGHASH type*.

In order to ensure that a *JoinSplit description* is cryptographically bound to the *transparent* inputs and outputs corresponding to v_{pub}^{new} and v_{pub}^{old}, and to the other *JoinSplit descriptions* in the same *transaction*, an ephemeral JoinSplitSig key pair is generated for each *transaction*, and the dataToBeSigned is signed with the private *signing key* of this key pair. The corresponding public *validating key* is included in the *transaction* encoding as `joinSplitPubKey`.

JoinSplitSig is instantiated in § 5.4.5 **'Ed25519'** on p. 67.

If `nJoinSplit` is zero, the `joinSplitPubKey` and `joinSplitSig` fields are omitted. Otherwise, a *transaction* has a correct *JoinSplit signature* if and only if $\mathsf{JoinSplitSig.Validate}_{\texttt{joinSplitPubKey}}(\mathsf{dataToBeSigned}, \texttt{joinSplitSig}) = 1$.

Let h_{Sig} be computed as specified in § 4.3 ***'JoinSplit Descriptions'*** on p. 34.

Let PRF^{pk} be as defined in § 4.1.2 ***'Pseudo Random Functions'*** on p. 22.

For each $i \in \{1..N^{old}\}$, the creator of a *JoinSplit description* calculates $h_i = PRF^{pk}_{a^{old}_{sk,i}}(i, h_{Sig})$.

The correctness of $h_{1..N^{old}}$ is enforced by the *JoinSplit statement* given in § 4.16.1 ***'JoinSplit Statement (Sprout)'*** on p. 47. This ensures that a holder of all of the $a^{old}_{sk,1..N^{old}}$ for every *JoinSplit description* in the *transaction* has authorized the use of the private *signing key* corresponding to `joinSplitPubKey` to sign this *transaction*.

4.11 Balance (Sprout)

In **Bitcoin**, all inputs to and outputs from a *transaction* are transparent. The total value of *transparent outputs* must not exceed the total value of *transparent inputs*. The net value of *transparent inputs* minus *transparent outputs* is transferred to the miner of the *block* containing the *transaction*; it is added to the *miner subsidy* in the *coinbase transaction* of the *block*.

Zcash Sprout extends this by adding *JoinSplit transfers*. Each *JoinSplit transfer* can be seen, from the perspective of the *transparent transaction value pool*, as an input and an output simultaneously.

v_{pub}^{old} takes value from the *transparent transaction value pool* and v_{pub}^{new} adds value to the *transparent transaction value pool*. As a result, v_{pub}^{old} is treated like an ***output*** value, whereas v_{pub}^{new} is treated like an ***input*** value.

Unlike original **Zerocash** [BCGGMTV2014], **Zcash** does not have a distinction between Mint and Pour operations. The addition of v_{pub}^{old} to a *JoinSplit description* subsumes the functionality of both Mint and Pour.

Also, a difference in the number of real input *notes* does not by itself cause two *JoinSplit descriptions* to be distinguishable.

As stated in § 4.3 ***'JoinSplit Descriptions'*** on p. 34, either v_{pub}^{old} or v_{pub}^{new} **MUST** be zero. No generality is lost because, if a *transaction* in which both v_{pub}^{old} and v_{pub}^{new} were nonzero were allowed, it could be replaced by an equivalent one in which $\mathsf{min}(v_{pub}^{old}, v_{pub}^{new})$ is subtracted from both of these values. This restriction helps to avoid unnecessary distinctions between *transactions* according to client implementation.

4.12 Balance and Binding Signature (Sapling)

Sapling adds *Spend transfers* and *Output transfers* to the transparent and *JoinSplit transfers* present in Sprout. The net value of *Spend transfers* minus *Output transfers* in a *transaction* is called the *Sapling-pool balancing value*, measured in zatoshi as a signed integer $v^{SaplingPoolBalance}$.

$v^{SaplingPoolBalance}$ is encoded in a *transaction* as the field `valueBalanceSapling`. For a v4 *transaction*, $v^{SaplingPoolBalance}$ is always explicitly encoded. Transaction fields are described in § 7.1 ***'Transaction Encoding and Consensus'*** on p. 89.

A positive *Sapling-pool balancing value* takes net value from the *Sapling chain value pool* and adds it to the *transparent transaction value pool*. A negative *Sapling-pool balancing value* does the reverse. As a result, positive $v^{SaplingPoolBalance}$ is treated like an ***input*** to the *transparent transaction value pool*, whereas negative $v^{SaplingPoolBalance}$ is treated like an ***output*** from that pool.

Consistency of $v^{SaplingPoolBalance}$ with the *value commitments* in *Spend descriptions* and *Output descriptions* is enforced by the *Sapling binding signature*. This signature has a dual rôle in the Sapling protocol:

- To prove that the total value spent by *Spend transfers*, minus that produced by *Output transfers*, is consistent with the $v^{SaplingPoolBalance}$ field of the *transaction*;
- To prove that the signer knew the randomness used for the Spend and Output *value commitments*, in order to prevent *Output descriptions* from being replayed by an adversary in a different *transaction*. (A *Spend description* already cannot be replayed due to its *spend authorization signature*.)

Instead of generating a key pair at random, we generate it as a function of the *value commitments* in the *Spend descriptions* and *Output descriptions* of the *transaction*, and the *Sapling-pool balancing value*.

Let $\mathbb{J}^{(r)}$, $\mathbb{J}^{(r)*}$, and $r_{\mathbb{J}}$ be as defined in § 5.4.8.3 ***'Jubjub'*** on p. 77.

§ 5.4.7.3 ***'Homomorphic Pedersen commitments (Sapling)'*** on p. 73 instantiates:

$\mathsf{ValueCommit}^{\mathsf{Sapling}} : \mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Trapdoor} \times \left\{-\frac{r_{\mathbb{J}}-1}{2} .. \frac{r_{\mathbb{J}}-1}{2}\right\} \to \mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Output}$;
$\mathcal{V}^{\mathsf{Sapling}} : \mathbb{J}^{(r)*}$, the value base in $\mathsf{ValueCommit}^{\mathsf{Sapling}}$;
$\mathcal{R}^{\mathsf{Sapling}} : \mathbb{J}^{(r)*}$, the randomness base in $\mathsf{ValueCommit}^{\mathsf{Sapling}}$.

$\mathsf{BindingSig}^{\mathsf{Sapling}}$, $\diamondplus$, and $\boxplus$ are instantiated in § 5.4.6.2 ***'Binding Signature (Sapling)'*** on p. 71.

§ 4.1.6.2 ***'Signature with Signing Key to Validating Key Monomorphism'*** on p. 27 specifies these operations and the derived notation $\diamondminus$, $\bigdiamondplus_{i=1}^{N}$, $\boxminus$, and $\bigboxplus_{i=1}^{N}$, which in this section are to be interpreted as operating on the *prime-order subgroup* of points on the *Jubjub curve* and on its scalar field.

Suppose that the *transaction* has:

- n *Spend descriptions* with *value commitments* $\mathsf{cv}^{\mathsf{old}}_{1..n}$, committing to values $\mathsf{v}^{\mathsf{old}}_{1..n}$ with randomness $\mathsf{rcv}^{\mathsf{old}}_{1..n}$;
- m *Output descriptions* with *value commitments* $\mathsf{cv}^{\mathsf{new}}_{1..m}$, committing to values $\mathsf{v}^{\mathsf{new}}_{1..m}$ with randomness $\mathsf{rcv}^{\mathsf{new}}_{1..m}$;
- *Sapling-pool balancing value* $\mathsf{v}^{\mathit{SaplingPoolBalance}}$.

In a correctly constructed *transaction*, $\mathsf{v}^{\mathit{SaplingPoolBalance}} = \sum_{i=1}^{n} \mathsf{v}^{\mathsf{old}}_i - \sum_{j=1}^{m} \mathsf{v}^{\mathsf{new}}_j$, but validators cannot check this directly because the values are hidden by the commitments.

Instead, validators calculate the *transaction binding validating key* as:

$$\mathsf{bvk}^{\mathsf{Sapling}} := \left(\bigoplus_{i=1}^{n} \mathsf{cv}^{\mathsf{old}}_i\right) \ominus \left(\bigoplus_{j=1}^{m} \mathsf{cv}^{\mathsf{new}}_j\right) \ominus \mathsf{ValueCommit}^{\mathsf{Sapling}}_0\left(\mathsf{v}^{\mathit{SaplingPoolBalance}}\right).$$

(This key is not encoded explicitly in the *transaction* and must be recalculated.)

The signer knows $\mathsf{rcv}^{\mathsf{old}}_{1..n}$ and $\mathsf{rcv}^{\mathsf{new}}_{1..m}$, and so can calculate the corresponding *signing key* as:

$$\mathsf{bsk}^{\mathsf{Sapling}} := \left(\boxplus_{i=1}^{n} \mathsf{rcv}^{\mathsf{old}}_i\right) \boxminus \left(\boxplus_{j=1}^{m} \mathsf{rcv}^{\mathsf{new}}_j\right).$$

In order to check for implementation faults, the signer **SHOULD** also check that

$\mathsf{bvk}^{\mathsf{Sapling}} = \mathsf{BindingSig}^{\mathsf{Sapling}}.\mathsf{DerivePublic}(\mathsf{bsk}^{\mathsf{Sapling}})$.

A *transaction* containing *Action descriptions* is necessarily a version 4 *transaction*. Let SigHash be the *SIGHASH transaction hash* for a version 4 *transaction*, not associated with an input, using the *SIGHASH type* SIGHASH_ALL. This is defined in [ZIP-243].

A validator checks balance by validating that $\mathsf{BindingSig}^{\mathsf{Sapling}}.\mathsf{Validate}_{\mathsf{bvk}^{\mathsf{Sapling}}}(\mathsf{SigHash}, \texttt{bindingSigSapling}) = 1$.

We now explain why this works.

A *Sapling binding signature* proves knowledge of the *discrete logarithm* $\mathsf{bsk}^{\mathsf{Sapling}}$ of $\mathsf{bvk}^{\mathsf{Sapling}}$ with respect to $\mathcal{R}^{\mathsf{Sapling}}$. That is, $\mathsf{bvk}^{\mathsf{Sapling}} = [\mathsf{bsk}^{\mathsf{Sapling}}]\,\mathcal{R}^{\mathsf{Sapling}}$. So the value 0 and randomness $\mathsf{bsk}^{\mathsf{Sapling}}$ is an opening of the *Pedersen commitment* $\mathsf{bvk}^{\mathsf{Sapling}} = \mathsf{ValueCommit}^{\mathsf{Sapling}}_{\mathsf{bsk}^{\mathsf{Sapling}}}(0)$. By the *binding* property of the *Pedersen commitment*, it is infeasible to find another opening of this commitment to a different value.

Similarly, the *binding* property of the *value commitments* in the *Spend descriptions* and *Output descriptions* ensures that an adversary cannot find an opening to more than one value for any of those commitments, i.e. we may assume that $\mathsf{v}^{\mathsf{old}}_{1..n}$ are determined by $\mathsf{cv}^{\mathsf{old}}_{1..n}$, and that $\mathsf{v}^{\mathsf{new}}_{1..m}$ are determined by $\mathsf{cv}^{\mathsf{new}}_{1..m}$. We may also assume, from Knowledge Soundness of Groth16, that the Spend proofs could not have been generated without knowing $\mathsf{rcv}^{\mathsf{old}}_{1..n} \pmod{r_{\mathbb{J}}}$, and the Output proofs could not have been generated without knowing $\mathsf{rcv}^{\mathsf{new}}_{1..m} \pmod{r_{\mathbb{J}}}$.

Using the fact that $\mathsf{ValueCommit}^{\mathsf{Sapling}}_{\mathsf{rcv}}(\mathsf{v}) = [\mathsf{v}]\,\mathcal{V}^{\mathsf{Sapling}} \oplus [\mathsf{rcv}]\,\mathcal{R}^{\mathsf{Sapling}}$, the expression for $\mathsf{bvk}^{\mathsf{Sapling}}$ above is equivalent to:

$$\begin{aligned}
\mathsf{bvk}^{\mathsf{Sapling}} &= \left[\left(\boxplus_{i=1}^{n} \mathsf{v}^{\mathsf{old}}_i\right) \boxminus \left(\boxplus_{j=1}^{m} \mathsf{v}^{\mathsf{new}}_j\right) \boxminus \mathsf{v}^{\mathit{SaplingPoolBalance}}\right] \mathcal{V}^{\mathsf{Sapling}} \oplus \left[\left(\boxplus_{i=1}^{n} \mathsf{rcv}^{\mathsf{old}}_i\right) \boxminus \left(\boxplus_{j=1}^{m} \mathsf{rcv}^{\mathsf{new}}_j\right)\right] \mathcal{R}^{\mathsf{Sapling}} \\
&= \mathsf{ValueCommit}^{\mathsf{Sapling}}_{\mathsf{bsk}^{\mathsf{Sapling}}}\left(\sum_{i=1}^{n} \mathsf{v}^{\mathsf{old}}_i - \sum_{j=1}^{m} \mathsf{v}^{\mathsf{new}}_j - \mathsf{v}^{\mathit{SaplingPoolBalance}}\right).
\end{aligned}$$

Let $\mathsf{v}^* = \sum_{i=1}^{n} \mathsf{v}_i^{\mathsf{old}} - \sum_{j=1}^{m} \mathsf{v}_j^{\mathsf{new}} - \mathsf{v}^{\mathit{SaplingPoolBalance}}$.

Suppose that $\mathsf{v}^* = \mathsf{v}^{\mathsf{bad}} \neq 0 \pmod{r_{\mathbb{J}}}$. Then $\mathsf{bvk}^{\mathsf{Sapling}} = \mathsf{ValueCommit}^{\mathsf{Sapling}}_{\mathsf{bsk}^{\mathsf{Sapling}}}(\mathsf{v}^{\mathsf{bad}})$. If the adversary were able to find the *discrete logarithm* of this $\mathsf{bvk}^{\mathsf{Sapling}}$ with respect to $\mathcal{R}^{\mathsf{Sapling}}$, say bsk' (as needed to create a valid *Sapling binding signature*), then $(\mathsf{v}^{\mathsf{bad}}, \mathsf{bsk}^{\mathsf{Sapling}})$ and $(0, \mathsf{bsk}')$ would be distinct openings of $\mathsf{bvk}^{\mathsf{Sapling}}$ to different values, breaking the *binding* property of the *value commitment scheme*.

The preceding argument shows only that $\mathsf{v}^* = 0 \pmod{r_{\mathbb{J}}}$; in order to show that $\mathsf{v}^* = 0$, we will also demonstrate that it does not overflow $\left\{-\frac{r_{\mathbb{J}}-1}{2} .. \frac{r_{\mathbb{J}}-1}{2}\right\}$.

The *Spend statements* (§ 4.16.2 ***'Spend Statement (Sapling)'*** on p. 48) prove that all of $\mathsf{v}^{\mathsf{old}}_{1..n}$ are in $\{0 .. 2^{\ell_{\mathsf{value}}}-1\}$. Similarly the *Output statements* (§ 4.16.3 ***'Output Statement (Sapling)'*** on p. 49) prove all of $\mathsf{v}^{\mathsf{new}}_{1..m}$ are in $\{0 .. 2^{\ell_{\mathsf{value}}}-1\}$. $\mathsf{v}^{\mathit{SaplingPoolBalance}}$ is encoded in the *transaction* as a signed two's complement 64-bit integer in the range $\{-2^{63} .. 2^{63}-1\}$. ℓ_{value} is defined as 64, so v^* is in the range $\{-m \cdot (2^{64}-1) - 2^{63} + 1 .. n \cdot (2^{64}-1) + 2^{63}\}$. The maximum *transaction* size is 2 MB, and the minimum contributions of a *Spend description* and an *Output description* to *transaction* size are 384 bytes and 948 bytes respectively, limiting n to at most $\mathsf{floor}\left(\frac{2000000}{384}\right) = 5208$ and m to at most $\mathsf{floor}\left(\frac{2000000}{948}\right) = 2109$.

This ensures that $\mathsf{v}^* \in \{-3891340662349029913 1842 .. 9607986650791619958 6728\}$, a subrange of $\left\{-\frac{r_{\mathbb{J}}-1}{2} .. \frac{r_{\mathbb{J}}-1}{2}\right\}$.

Thus checking the *Sapling binding signature* ensures that the *Spend transfers* and *Output transfers* in the *transaction* balance, without their individual values being revealed.

In addition this proves that the signer, knowing the $\boxplus$-sum of the *Sapling-pool value commitment* randomnesses, authorized a *transaction* with the given *SIGHASH transaction hash* by signing SigHash.

Note: The spender **MAY** reveal any strict subset of the *Sapling-pool value commitment* randomnesses to other parties that are cooperating to create the *transaction*. If all of the *value commitment* randomnesses are revealed, that could allow replaying the *Output descriptions* of the *transaction*.

Non-normative note: The technique of checking signatures using a *validating key* derived from a sum of *Pedersen commitments* is also used in the Mimblewimble protocol [Jedusor2016]. The *private key* $\mathsf{bsk}^{\mathsf{Sapling}}$ acts as a *"synthetic blinding factor"*, in the sense that it is synthesized from the other blinding factors (*trapdoors*) $\mathsf{rcv}^{\mathsf{old}}_{1..n}$ and $\mathsf{rcv}^{\mathsf{new}}_{1..m}$; this technique is also used in Bulletproofs [Dalek-notes].

4.13 Spend Authorization Signature (Sapling)

SpendAuthSig is used in Sapling to prove knowledge of the *spending key* authorizing spending of an input *note*. It is instantiated in § 5.4.6.1 ***'Spend Authorization Signature (Sapling)'*** on p. 71.

We use $\mathsf{SpendAuthSig}^{\mathsf{Sapling}}$ to refer to the *spend authorization signature scheme* for Sapling, which is instantiated on the *Jubjub curve*.

Knowledge of the *spending key* could have been proven directly in the *Spend statement*, similar to the check in § 4.16.1 ***'JoinSplit Statement (Sprout)'*** on p. 47 that is part of the *JoinSplit statement*. The motivation for a separate signature is to allow devices that are limited in memory and computational capacity, such as hardware wallets, to authorize a Sapling Spend. Typically such devices cannot create, and may not be able to verify, *zk-SNARK proofs* for a *statement* of the size needed using the Groth16 proving system.

The *validating key* of the signature must be revealed in the *Spend description* so that the signature can be checked by validators. To ensure that the *validating key* cannot be linked to the *shielded payment address* or *spending key* from which the *note* was spent, we use a *signature scheme with re-randomizable keys*. The *Spend statement* proves that this *validating key* is a *re-randomization* of the *Spend validating key* ak with a *randomizer* known to the signer. The *spend authorization signature* is over the *SIGHASH transaction hash*, so that it cannot be replayed in other *transactions*.

Let SigHash be the *SIGHASH transaction hash* as defined in [ZIP-243], not associated with an input, using the *SIGHASH type* SIGHASH_ALL.

Let ask be the *Spend authorizing key*, as defined in § 4.2.2 *'Sapling Key Components'* on p. 32.

For each *Spend description*, the signer chooses a fresh *spend authorization randomizer* α:

1. Choose $\alpha \xleftarrow{R} \mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{GenRandom}()$.
2. Let $\mathsf{rsk} = \mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{RandomizePrivate}(\alpha, \mathsf{ask})$.
3. Let $\mathsf{rk} = \mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{DerivePublic}(\mathsf{rsk})$.
4. Generate a proof π of the *Spend statement* (§ 4.16.2 *'Spend Statement (Sapling)'* on p. 48), with α in the *auxiliary input* and rk in the *primary input*.
5. Let $\mathsf{spendAuthSig} = \mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{Sign}_{\mathsf{rsk}}(\mathsf{SigHash})$.

The resulting spendAuthSig and π are included in the *Spend description*.

Note: If the spender is computationally or memory-limited, step 4 (and only step 4) **MAY** be delegated to a different party that is capable of performing the *zk-SNARK proof*. In this case privacy will be lost to that party since it needs ak and the *proof authorizing key* nsk; this allows also deriving the nk component of the *full viewing key*. Together ak and nk are sufficient to recognize spent *notes* and to recognize and decrypt incoming *notes*. However, the other party will not obtain *spending authority* for other *transactions*, since it is not able to create a *spend authorization signature* by itself.

4.14 Computing ρ values and Nullifiers

In Sprout, each *note* has a ρ component, defined as part of the *note*.

In Sapling, each *positioned note* (as defined in § 3.2.2 *'Note Commitments'* on p. 15) has an associated ρ value, which is computed from its *note commitment* cm and *note position* pos as follows:

$$\rho := \mathsf{MixingPedersenHash}(\mathsf{cm}, \mathsf{pos}).$$

MixingPedersenHash is defined in § 5.4.1.8 *'Mixing Pedersen Hash Function'* on p. 63.

Let $\mathsf{PRF}^{\mathsf{nfSprout}}$ and $\mathsf{PRF}^{\mathsf{nfSapling}}$ be as instantiated in § 5.4.2 *'Pseudo Random Functions'* on p. 64.

For a *Sprout-pool note*, the *nullifier* (see § 3.2.3 *'Nullifiers'* on p. 15) is derived as $\mathsf{PRF}^{\mathsf{nfSprout}}_{\mathsf{a_{sk}}}(\rho)$, where $\mathsf{a_{sk}}$ is the *spending key* associated with the *note*.

For a *Sapling-pool note*, the *nullifier* is derived as $\mathsf{PRF}^{\mathsf{nfSapling}}_{\mathsf{nk}\star}(\rho\star)$, where nk⋆ is a representation of the *nullifier deriving key* associated with the *note* and $\rho\star = \mathsf{repr}_{\mathbb{J}}(\rho)$.

Security requirement: For each *shielded protocol*, the requirements on *nullifier* derivation are as follows:

- The derived *nullifier* must be determined completely by the fields of the *note*, and possibly its position, in a way that can be checked in the corresponding statement that controls spends (i.e. the *JoinSplit statement*, *Spend statement*).
- Under the assumption that ρ values are unique, it must not be possible to generate two *notes* with distinct *note commitments* but the same *nullifier*. (See § 8.4 *'Faerie Gold attack and fix'* on p. 104 for further discussion.)
- Given a set of *nullifiers* of *a priori* unknown *notes*, they must not be linkable to those *notes* with probability greater than expected by chance, even to an adversary with the corresponding *incoming viewing keys* (but not *full viewing keys*), and even if the adversary may have created the *notes*.

4.15 Chain Value Pool Balances

The *transparent chain value pool* balance for a given *block chain* is the sum of the values of all *UTXOs* in the *UTXO (unspent transaction output) set* for that chain. It is denoted by $\mathsf{ChainValuePoolBalance}^{\mathit{TransparentPool}}(\mathsf{height})$.

Consensus rule: If the *transparent chain value pool* balance would become negative in the *block chain* created as a result of accepting a *block*, then all nodes **MUST** reject the block as invalid.

As defined in [ZIP-209], the *Sprout chain value pool* balance for a given *block chain* is the sum of all $v_{\mathsf{pub}}^{\mathsf{old}}$ field values for *transactions* in the *block chain*, minus the sum of all $v_{\mathsf{pub}}^{\mathsf{new}}$ field values for transactions in the *block chain*. It is denoted by $\mathsf{ChainValuePoolBalance}^{\mathit{SproutPool}}(\mathsf{height})$.

Consensus rule: If the *Sprout chain value pool* balance would become negative in the *block chain* created as a result of accepting a *block*, then all nodes **MUST** reject the block as invalid.

As defined in [ZIP-209], the *Sapling chain value pool* balance for a given *block chain* is the negation of the sum of all `valueBalanceSapling` values for *transactions* in the *chain*. It is denoted by $\mathsf{ChainValuePoolBalance}^{\mathit{SaplingPool}}(\mathsf{height})$.

Consensus rule: If the *Sapling chain value pool* balance would become negative in the *block chain* created as a result of accepting a *block*, then all nodes **MUST** reject the block as invalid.

The *total issued supply* of a *block chain* at *block height* height is given by the function:

$$\begin{aligned}\mathsf{IssuedSupply}(\mathsf{height}) \;:=\; & \mathsf{ChainValuePoolBalance}^{\mathit{TransparentPool}}(\mathsf{height}) \\ & + \mathsf{ChainValuePoolBalance}^{\mathit{SproutPool}}(\mathsf{height}) \\ & + \mathsf{ChainValuePoolBalance}^{\mathit{SaplingPool}}(\mathsf{height})\end{aligned}$$

Consensus rule: If the *total issued supply* (computed without possibility of overflow) would become greater than MAX_MONEY zatoshi in the *block chain* created as a result of accepting a *block*, then all nodes **MUST** reject the block as invalid.

4.16 Zk-SNARK Statements

4.16.1 JoinSplit Statement (Sprout)

Let $\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}$, $\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}$, $\mathsf{MerkleDepth}^{\mathsf{Sprout}}$, ℓ_{value}, $\ell_{\mathsf{a_{sk}}}$, $\ell^{\mathsf{Sprout}}_{\varphi}$, ℓ_{hSig}, $\mathrm{N}^{\mathsf{old}}$, $\mathrm{N}^{\mathsf{new}}$ be as defined in § 5.3 *'Constants'* on p. 57.
Let $\mathsf{PRF}^{\mathsf{addr}}$, $\mathsf{PRF}^{\mathsf{nfSprout}}$, $\mathsf{PRF}^{\mathsf{pk}}$, and PRF^{ρ} be as defined in § 4.1.2 *'Pseudo Random Functions'* on p. 22.
Let $\mathsf{NoteCommit}^{\mathsf{Sprout}}$ be as defined in § 4.1.7 *'Commitment'* on p. 27, and let $\mathsf{Note}^{\mathsf{Sprout}}$ and $\mathsf{NoteCommitment}^{\mathsf{Sprout}}$ be as defined in § 3.2 *'Shielded Pools and Notes'* on p. 13.

A valid instance of a *JoinSplit statement*, $\pi_{\mathsf{ZKJoinSplit}}$, assures that given a *primary input*:

$$\begin{aligned}
&\big(\mathsf{rt}^{\mathsf{Sprout}} : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}]},\\
&\ \mathsf{nf}^{\mathsf{old}}_{1..\mathrm{N}^{\mathsf{old}}} : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}][\mathrm{N}^{\mathsf{old}}]},\\
&\ \mathsf{cm}^{\mathsf{new}}_{1..\mathrm{N}^{\mathsf{new}}} : \mathsf{NoteCommit}^{\mathsf{Sprout}}.\mathsf{Output}^{[\mathrm{N}^{\mathsf{new}}]},\\
&\ \mathsf{v}^{\mathsf{old}}_{\mathsf{pub}} : \{0 \mathinner{..} 2^{\ell_{\mathsf{value}}}-1\},\\
&\ \mathsf{v}^{\mathsf{new}}_{\mathsf{pub}} : \{0 \mathinner{..} 2^{\ell_{\mathsf{value}}}-1\},\\
&\ \mathsf{h}_{\mathsf{Sig}} : \mathbb{B}^{[\ell_{\mathsf{hSig}}]},\\
&\ \mathsf{h}_{1..\mathrm{N}^{\mathsf{old}}} : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}][\mathrm{N}^{\mathsf{old}}]}\big),
\end{aligned}$$

the prover knows an *auxiliary input*:

$$\begin{aligned}
&\big(\mathsf{path}_{1..\mathrm{N}^{\mathsf{old}}} : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}][\mathsf{MerkleDepth}^{\mathsf{Sprout}}][\mathrm{N}^{\mathsf{old}}]},\\
&\ \mathsf{pos}_{1..\mathrm{N}^{\mathsf{old}}} : \{0 \mathinner{..} 2^{\mathsf{MerkleDepth}^{\mathsf{Sprout}}}-1\}^{[\mathrm{N}^{\mathsf{old}}]},\\
&\ \mathbf{n}^{\mathsf{old}}_{1..\mathrm{N}^{\mathsf{old}}} : \mathsf{Note}^{\mathsf{Sprout}[\mathrm{N}^{\mathsf{old}}]},\\
&\ \mathsf{a}^{\mathsf{old}}_{\mathsf{sk},1..\mathrm{N}^{\mathsf{old}}} : \mathbb{B}^{[\ell_{\mathsf{a_{sk}}}][\mathrm{N}^{\mathsf{old}}]},\\
&\ \mathbf{n}^{\mathsf{new}}_{1..\mathrm{N}^{\mathsf{new}}} : \mathsf{Note}^{\mathsf{Sprout}[\mathrm{N}^{\mathsf{new}}]},\\
&\ \varphi : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\varphi}]},\\
&\ \mathsf{enforceMerklePath}_{1..\mathrm{N}^{\mathsf{old}}} : \mathbb{B}^{[\mathrm{N}^{\mathsf{old}}]}\big),
\end{aligned}$$

where:

$$\begin{aligned}
&\text{for each } i \in \{1..\mathrm{N}^{\mathsf{old}}\}\text{: } \mathbf{n}^{\mathsf{old}}_i = (\mathsf{a}^{\mathsf{old}}_{\mathsf{pk},i}, \mathsf{v}^{\mathsf{old}}_i, \rho^{\mathsf{old}}_i, \mathsf{rcm}^{\mathsf{old}}_i);\\
&\text{for each } i \in \{1..\mathrm{N}^{\mathsf{new}}\}\text{: } \mathbf{n}^{\mathsf{new}}_i = (\mathsf{a}^{\mathsf{new}}_{\mathsf{pk},i}, \mathsf{v}^{\mathsf{new}}_i, \rho^{\mathsf{new}}_i, \mathsf{rcm}^{\mathsf{new}}_i)
\end{aligned}$$

such that the following conditions hold:

Merkle path validity for each $i \in \{1..\mathrm{N}^{\mathsf{old}}\} \mid \mathsf{enforceMerklePath}_i = 1$: $(\mathsf{path}_i, \mathsf{pos}_i)$ is a valid *Merkle path* (see § 4.8 *'Merkle Path Validity'* on p. 39) of depth $\mathsf{MerkleDepth}^{\mathsf{Sprout}}$ from $\mathsf{NoteCommitment}^{\mathsf{Sprout}}(\mathbf{n}^{\mathsf{old}}_i)$ to the *anchor* $\mathsf{rt}^{\mathsf{Sprout}}$.

Note: Merkle path validity covers conditions 1. (a) and 1. (d) of the NP *statement* in [BCGGMTV2014, section 4.2].

Merkle path enforcement for each $i \in \{1..\mathrm{N}^{\mathsf{old}}\}$, if $\mathsf{v}^{\mathsf{old}}_i \neq 0$ then $\mathsf{enforceMerklePath}_i = 1$.

Balance $\mathsf{v}^{\mathsf{old}}_{\mathsf{pub}} + \sum_{i=1}^{\mathrm{N}^{\mathsf{old}}} \mathsf{v}^{\mathsf{old}}_i = \mathsf{v}^{\mathsf{new}}_{\mathsf{pub}} + \sum_{i=1}^{\mathrm{N}^{\mathsf{new}}} \mathsf{v}^{\mathsf{new}}_i \in \{0 \mathinner{..} 2^{\ell_{\mathsf{value}}}-1\}$.

Nullifier integrity for each $i \in \{1..\mathrm{N}^{\mathsf{old}}\}$: $\mathsf{nf}^{\mathsf{old}}_i = \mathsf{PRF}^{\mathsf{nfSprout}}_{\mathsf{a}^{\mathsf{old}}_{\mathsf{sk},i}}(\rho^{\mathsf{old}}_i)$.

Spend authority for each $i \in \{1..\mathrm{N}^{\mathsf{old}}\}$: $\mathsf{a}^{\mathsf{old}}_{\mathsf{pk},i} = \mathsf{PRF}^{\mathsf{addr}}_{\mathsf{a}^{\mathsf{old}}_{\mathsf{sk},i}}(0)$.

Non-malleability for each $i \in \{1..\mathrm{N}^{\mathsf{old}}\}$: $\mathsf{h}_i = \mathsf{PRF}^{\mathsf{pk}}_{\mathsf{a}^{\mathsf{old}}_{\mathsf{sk},i}}(i, \mathsf{h}_{\mathsf{Sig}})$.

Uniqueness of ρ^{new}_i for each $i \in \{1..\mathrm{N}^{\mathsf{new}}\}$: $\rho^{\mathsf{new}}_i = \mathsf{PRF}^{\rho}_{\varphi}(i, \mathsf{h}_{\mathsf{Sig}})$.

Note commitment integrity for each $i \in \{1..\mathrm{N}^{\mathsf{new}}\}$: $\mathsf{cm}^{\mathsf{new}}_i = \mathsf{NoteCommitment}^{\mathsf{Sprout}}(\mathbf{n}^{\mathsf{new}}_i)$.

For details of the form and encoding of proofs, see § 5.4.9.1 *'BCTV14'* on p. 79.

4.16.2 Spend Statement (Sapling)

Let $\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}$, $\ell_{\mathsf{PRFnfSapling}}$, $\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}$, and $\mathsf{MerkleDepth}^{\mathsf{Sapling}}$ be as defined in § 5.3 *'Constants'* on p. 57.
Let $\mathsf{ValueCommit}^{\mathsf{Sapling}}$ and $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ be as specified in § 4.1.7 *'Commitment'* on p. 27.
Let $\mathsf{SpendAuthSig}^{\mathsf{Sapling}}$ be as defined in § 5.4.6.1 *'Spend Authorization Signature (Sapling)'* on p. 71.
Let $\mathbb{J}$, $\mathbb{J}^{(r)}$, $\mathsf{repr}_{\mathbb{J}}$, $q_{\mathbb{J}}$, $r_{\mathbb{J}}$, and $h_{\mathbb{J}}$ be as defined in § 5.4.8.3 *'Jubjub'* on p. 77.
Let $\mathsf{Extract}_{\mathbb{J}^{(r)}} : \mathbb{J}^{(r)} \to \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]}$ be as defined in § 5.4.8.4 *'Coordinate Extractor for Jubjub'* on p. 78.
Let $\mathcal{H}^{\mathsf{Sapling}}$ be as defined in § 4.2.2 *'Sapling Key Components'* on p. 32.

A valid instance of a *Spend statement*, π_{ZKSpend}, assures that given a *primary input*:

$\big(\mathsf{rt}^{\mathit{SaplingPool}} : \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]},$
$\mathsf{cv}^{\mathsf{old}} : \mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Output},$
$\mathsf{nf}^{\mathsf{old}} : \mathbb{B}^{[\ell_{\mathsf{PRFnfSapling}}/8]},$
$\mathsf{rk} : \mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{Public}\big),$

the prover knows an *auxiliary input*:

$\big(\mathsf{path} : \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}][\mathsf{MerkleDepth}^{\mathsf{Sapling}}]},$
$\mathsf{pos} : \{0 .. 2^{\mathsf{MerkleDepth}^{\mathsf{Sapling}}} - 1\},$
$\mathsf{g_d} : \mathbb{J},$
$\mathsf{pk_d} : \mathbb{J},$
$\mathsf{v}^{\mathsf{old}} : \{0 .. 2^{\ell_{\mathsf{value}}} - 1\},$
$\mathsf{rcv}^{\mathsf{old}} : \{0 .. 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\},$
$\mathsf{cm}^{\mathsf{old}} : \mathbb{J},$
$\mathsf{rcm}^{\mathsf{old}} : \{0 .. 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\},$
$\alpha : \{0 .. 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\},$
$\mathsf{ak} : \mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{Public},$
$\mathsf{nsk} : \{0 .. 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\}\big)$

such that the following conditions hold:

Note commitment integrity $\quad \mathsf{cm}^{\mathsf{old}} = \mathsf{NoteCommit}^{\mathsf{Sapling}}_{\mathsf{rcm}^{\mathsf{old}}}(\mathsf{repr}_{\mathbb{J}}(\mathsf{g_d}), \mathsf{repr}_{\mathbb{J}}(\mathsf{pk_d}), \mathsf{v}^{\mathsf{old}})$.

Merkle path validity $\quad$ Either $\mathsf{v}^{\mathsf{old}} = 0$; or $(\mathsf{path}, \mathsf{pos})$ is a valid *Merkle path* of depth $\mathsf{MerkleDepth}^{\mathsf{Sapling}}$, as defined in § 4.8 *'Merkle Path Validity'* on p. 39, from $\mathsf{cm}_u = \mathsf{Extract}_{\mathbb{J}^{(r)}}(\mathsf{cm}^{\mathsf{old}})$ to the *anchor* $\mathsf{rt}^{\mathit{SaplingPool}}$.

Value commitment integrity $\quad \mathsf{cv}^{\mathsf{old}} = \mathsf{ValueCommit}^{\mathsf{Sapling}}_{\mathsf{rcv}^{\mathsf{old}}}(\mathsf{v}^{\mathsf{old}})$.

Small order checks $\quad \mathsf{g_d}$ and ak are not of *small order*, i.e. $[h_{\mathbb{J}}]\,\mathsf{g_d} \neq \mathcal{O}_{\mathbb{J}}$ and $[h_{\mathbb{J}}]\,\mathsf{ak} \neq \mathcal{O}_{\mathbb{J}}$.

Nullifier integrity $\quad \mathsf{nf}^{\mathsf{old}} = \mathsf{PRF}^{\mathsf{nfSapling}}_{\mathsf{nk}\star}(\rho\star)$ where
$\mathsf{nk}\star = \mathsf{repr}_{\mathbb{J}}\big([\mathsf{nsk}]\,\mathcal{H}^{\mathsf{Sapling}}\big)$
$\rho\star = \mathsf{repr}_{\mathbb{J}}\big(\mathsf{MixingPedersenHash}(\mathsf{cm}^{\mathsf{old}}, \mathsf{pos})\big)$.

Spend authority $\quad \mathsf{rk} = \mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{RandomizePublic}(\alpha, \mathsf{ak})$.

Diversified address integrity $\quad \mathsf{pk_d} = [\mathsf{ivk}]\,\mathsf{g_d}$ where
$\mathsf{ivk} = \mathsf{CRH}^{\mathsf{ivk}}(\mathsf{ak}\star, \mathsf{nk}\star)$
$\mathsf{ak}\star = \mathsf{repr}_{\mathbb{J}}(\mathsf{ak})$.

For details of the form and encoding of *Spend statement* proofs, see § 5.4.9.2 *'Groth16'* on p. 81.

Notes:

- *Primary* and *auxiliary inputs* **MUST** be constrained to have the types specified. In particular, see § A.3.3.2 *'ctEdwards [de]compression and validation'* on p. 154, for required validity checks on compressed representations of *Jubjub curve* points.

  The $\mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Output}$ and $\mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{Public}$ types also represent points, i.e. $\mathbb{J}$.
- In the Merkle path validity check, each *layer* does ***not*** check that its input *bit sequence* is a canonical encoding (in $\{0 \,..\, q_{\mathbb{J}} - 1\}$) of the integer from the previous *layer*.
- It is ***not*** checked in the *Spend statement* that rk is not of *small order*. However, this ***is*** checked outside the *Spend statement*, as specified in § 4.4 ***'Spend Descriptions'*** on p. 35.
- It is ***not*** checked that $\mathsf{rcv}^{\mathsf{old}} < r_{\mathbb{J}}$ or that $\mathsf{rcm}^{\mathsf{old}} < r_{\mathbb{J}}$.
- $\mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{RandomizePublic}(\alpha, \mathsf{ak}) = \mathsf{ak} + [\alpha]\,\mathcal{G}^{\mathsf{Sapling}}$.

  ($\mathcal{G}^{\mathsf{Sapling}}$ is as defined in § 5.4.6.1 ***'Spend Authorization Signature (Sapling)'*** on p. 71.)

4.16.3 Output Statement (Sapling)

Let $\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}$ and $\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}$ be as defined in § 5.3 ***'Constants'*** on p. 57.

Let $\mathsf{ValueCommit}^{\mathsf{Sapling}}$ and $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ be as specified in § 4.1.7 ***'Commitment'*** on p. 27.

Let $\mathbb{J}$, $\mathsf{repr}_{\mathbb{J}}$, and $h_{\mathbb{J}}$ be as defined in § 5.4.8.3 *'Jubjub'* on p. 77.

Let $\mathsf{Extract}_{\mathbb{J}^{(r)}} : \mathbb{J}^{(r)} \to \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]}$ be as defined in § 5.4.8.4 ***'Coordinate Extractor for Jubjub'*** on p. 78.

A valid instance of an *Output statement*, π_{ZKOutput}, assures that given a *primary input*:

$$\begin{aligned}&\big(\mathsf{cv}^{\mathsf{new}} : \mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Output},\\ &\ \mathsf{cm}_u : \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]},\\ &\ \mathsf{epk} : \mathbb{J}\big),\end{aligned}$$

the prover knows an *auxiliary input*:

$$\begin{aligned}&(\mathsf{g_d} : \mathbb{J},\\ &\ \mathsf{pk}\star_{\mathsf{d}} : \mathbb{B}^{[\ell_{\mathbb{J}}]},\\ &\ \mathsf{v}^{\mathsf{new}} : \{0 \,..\, 2^{\ell_{\mathsf{value}}} - 1\},\\ &\ \mathsf{rcv}^{\mathsf{new}} : \{0 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\},\\ &\ \mathsf{rcm}^{\mathsf{new}} : \{0 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\},\\ &\ \mathsf{esk} : \{0 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\})\end{aligned}$$

such that the following conditions hold:

Note commitment integrity $\quad \mathsf{cm}_u = \mathsf{Extract}_{\mathbb{J}^{(r)}}\left(\mathsf{NoteCommit}^{\mathsf{Sapling}}_{\mathsf{rcm}^{\mathsf{new}}}(\mathsf{g}\star_{\mathsf{d}}, \mathsf{pk}\star_{\mathsf{d}}, \mathsf{v}^{\mathsf{new}})\right)$, where $\mathsf{g}\star_{\mathsf{d}} = \mathsf{repr}_{\mathbb{J}}(\mathsf{g_d})$.

Value commitment integrity $\quad \mathsf{cv}^{\mathsf{new}} = \mathsf{ValueCommit}^{\mathsf{Sapling}}_{\mathsf{rcv}^{\mathsf{new}}}(\mathsf{v}^{\mathsf{new}})$.

Small order check $\quad \mathsf{g_d}$ is not of *small order*, i.e. $[h_{\mathbb{J}}]\,\mathsf{g_d} \neq \mathcal{O}_{\mathbb{J}}$.

Ephemeral public key integrity $\quad \mathsf{epk} = [\mathsf{esk}]\,\mathsf{g_d}$.

For details of the form and encoding of *Output statement* proofs, see § 5.4.9.2 *'Groth16'* on p. 81.

Notes:

- *Primary* and *auxiliary inputs* **MUST** be constrained to have the types specified. In particular, see § A.3.3.2 *'ctEdwards [de]compression and validation'* on p. 154, for required validity checks on compressed representations of *Jubjub curve* points. The $\mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Output}$ type also represents points, i.e. $\mathbb{J}$.
- The validity of $\mathsf{pk}\star_{\mathsf{d}}$ is ***not*** checked in this circuit (which is the reason why it is typed as a *bit sequence* rather than as a point).
- It is ***not*** checked that $\mathsf{rcv}^{\mathsf{old}} < r_{\mathbb{J}}$ or that $\mathsf{rcm}^{\mathsf{old}} < r_{\mathbb{J}}$.

4.17 In-band secret distribution (Sprout)

In Sprout, the secrets that need to be transmitted to a recipient of funds in order for them to later spend, are v, ρ, and rcm. A *memo field* (§ 3.2.1 *'Note Plaintexts and Memo Fields'* on p. 14) is also transmitted.

To transmit these secrets securely to a recipient ***without*** requiring an *out-of-band* communication channel, the *transmission key* $\mathsf{pk}_{\mathsf{enc}}$ is used to encrypt them. The recipient's possession of the associated *incoming viewing key* ivk is used to reconstruct the original *note* and *memo field*.

A single *ephemeral public key* is shared between encryptions of the $\mathrm{N}^{\mathsf{new}}$ *shielded outputs* in a *JoinSplit description*. All of the resulting ciphertexts are combined to form a *transmitted notes ciphertext*.

For both encryption and decryption,

- let Sym be the scheme instantiated in § 5.4.3 *'Symmetric Encryption'* on p. 66;
- let $\mathsf{KDF}^{\mathsf{Sprout}}$ be the *Key Derivation Function* instantiated in § 5.4.4.2 *'Sprout Key Derivation'* on p. 66;
- let $\mathsf{KA}^{\mathsf{Sprout}}$ be the *key agreement scheme* instantiated in § 5.4.4.1 *'Sprout Key Agreement'* on p. 66;
- let $\mathsf{h}_{\mathsf{Sig}}$ be the value computed for this *JoinSplit description* in § 4.3 *'JoinSplit Descriptions'* on p. 34.

4.17.1 Encryption (Sprout)

Let $\mathsf{KA}^{\mathsf{Sprout}}$ be the *key agreement scheme* instantiated in § 5.4.4.1 *'Sprout Key Agreement'* on p. 66.

Let $\mathsf{pk}_{\mathsf{enc},1..\mathrm{N}^{\mathsf{new}}}$ be the *transmission keys* for the intended recipient addresses of each new *note*.

Let $\mathbf{np}_{1..\mathrm{N}^{\mathsf{new}}}$ be *Sprout-pool note plaintexts* defined in § 3.2.1 *'Note Plaintexts and Memo Fields'* on p. 14.

Then to encrypt:

- Generate a new $\mathsf{KA}^{\mathsf{Sprout}}$ (public, private) key pair $(\mathsf{epk}, \mathsf{esk})$.
- For $i \in \{1..\mathrm{N}^{\mathsf{new}}\}$,
  - Let $\mathsf{P}_i^{\mathsf{enc}} : \mathbb{B}^{\mathbb{Y}^{[585]}}$ be the *raw encoding* of $\mathbf{np}_i$.
  - Let $\mathsf{sharedSecret}_i = \mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Agree}(\mathsf{esk}, \mathsf{pk}_{\mathsf{enc},i})$.
  - Let $\mathsf{K}_i^{\mathsf{enc}} = \mathsf{KDF}^{\mathsf{Sprout}}(i, \mathsf{h}_{\mathsf{Sig}}, \mathsf{sharedSecret}_i, \mathsf{epk}, \mathsf{pk}_{\mathsf{enc},i})$.
  - Let $\mathsf{C}_i^{\mathsf{enc}} = \mathsf{Sym}.\mathsf{Encrypt}_{\mathsf{K}_i^{\mathsf{enc}}}(\mathsf{P}_i^{\mathsf{enc}})$.

The resulting *transmitted notes ciphertext* is $(\mathsf{epk}, \mathsf{C}_{1..\mathrm{N}^{\mathsf{new}}}^{\mathsf{enc}})$.

Note: It is technically possible to replace $\mathsf{C}_i^{\mathsf{enc}}$ for a given *note* with a random (and undecryptable) dummy ciphertext, relying instead on *out-of-band* transmission of the *note* to the recipient. In this case the ephemeral key **MUST** still be generated as a random *public key* (rather than a random *bit sequence*) to ensure indistinguishability from other *JoinSplit descriptions*. This mode of operation raises further security considerations, for example of how to validate a *Sprout-pool note* received *out-of-band*, which are not addressed in this document.

4.17.2 Decryption (Sprout)

Let $\mathsf{ivk} = (\mathsf{a_{pk}}, \mathsf{sk_{enc}})$ be the recipient's *incoming viewing key*, and let $\mathsf{pk_{enc}}$ be the corresponding *transmission key* derived from $\mathsf{sk_{enc}}$ as specified in § 4.2.1 *'Sprout Key Components'* on p. 32.

Let $\mathsf{cm}_{1..\mathrm{N}^{\mathsf{new}}}$ be the *note commitments* of each output coin.

Then for each $i \in \{1..\mathrm{N}^{\mathsf{new}}\}$, the recipient will attempt to decrypt that ciphertext component $(\mathsf{epk}, \mathsf{C}_i^{\mathsf{enc}})$ as follows:

let $\mathsf{sharedSecret}_i = \mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Agree}(\mathsf{sk_{enc}}, \mathsf{epk})$
let $\mathsf{K}_i^{\mathsf{enc}} = \mathsf{KDF}^{\mathsf{Sprout}}(i, \mathsf{h_{Sig}}, \mathsf{sharedSecret}_i, \mathsf{epk}, \mathsf{pk_{enc}})$
let $\mathsf{P}_i^{\mathsf{enc}} = \mathsf{Sym.Decrypt}_{\mathsf{K}_i^{\mathsf{enc}}}(\mathsf{C}_i^{\mathsf{enc}})$
if $\mathsf{P}_i^{\mathsf{enc}} = \bot$, return $\bot$
return $\mathsf{ExtractNote}^{\mathsf{Sprout}}(\mathsf{P}_i^{\mathsf{enc}}, \mathsf{cm}_i, \mathsf{a_{pk}})$.

$\mathsf{ExtractNote}^{\mathsf{Sprout}}(\mathsf{P}_i^{\mathsf{enc}} : \mathbb{B}^{\mathbb{Y}[585]}, \mathsf{cm}_i : \mathbb{B}^{\mathbb{Y}[32]}, \mathsf{a_{pk}} : \mathbb{B}^{[\ell_{\mathsf{PRF}}^{\mathsf{Sprout}}]})$ is defined as follows:

extract $\mathbf{np}_i = (\mathsf{leadByte}_i : \mathbb{B}^{\mathbb{Y}}, \mathsf{v}_i : \{0 .. 2^{\ell_{\mathsf{value}}}-1\}, \rho_i : \mathbb{B}^{[\ell_{\mathsf{PRF}}^{\mathsf{Sprout}}]}, \mathsf{rcm}_i : \mathsf{NoteCommit}^{\mathsf{Sprout}}.\mathsf{Trapdoor}, \mathsf{memo}_i : \mathbb{B}^{\mathbb{Y}[512]})$ from $\mathsf{P}_i^{\mathsf{enc}}$
let $\mathbf{n}_i = (\mathsf{a_{pk}}, \mathsf{v}_i, \rho_i, \mathsf{rcm}_i)$
if $\mathsf{leadByte}_i \neq$ 0x00 or $\mathsf{NoteCommitment}^{\mathsf{Sprout}}(\mathbf{n}_i) \neq \mathsf{cm}_i$, return $\bot$
return $(\mathbf{n}_i, \mathsf{memo}_i)$.

Notes:

- The decryption algorithm corresponds to step 3 (b) i. and ii. (first bullet point) of the Receive algorithm shown in [BCGGMTV2014, Figure 2].
- To test whether a *note* is unspent in a particular *block chain* also requires the *spending key* $\mathsf{a_{sk}}$; the coin is unspent if and only if $\mathsf{nf} = \mathsf{PRF}^{\mathsf{nfSprout}}_{\mathsf{a_{sk}}}(\rho)$ is not in the *nullifier set* for that *block chain*.
- A *note* can change from being unspent to spent as a node's view of the *best valid block chain* is extended by new *transactions*. Also, *block chain reorganizations* can cause a node to switch to a different *best valid block chain* that does not contain the *transaction* in which a *note* was output.

See § 8.7 *'In-band secret distribution'* on p. 107 for further discussion of the security and engineering rationale behind this encryption scheme.

4.18 In-band secret distribution (Sapling)

In Sapling, the secrets that need to be transmitted to a recipient of a *note* so that they can later spend it, are d, v, and rcm. A *memo field* (§ 3.2.1 *'Note Plaintexts and Memo Fields'* on p. 14) is also transmitted.

To transmit these secrets securely to a recipient ***without*** requiring an *out-of-band* communication channel, the *diversified transmission key* $\mathsf{pk_d}$ is used to encrypt them. The recipient's possession of the associated $\mathsf{KA}^{\mathsf{Sapling}}$ *private key* ivk is used to reconstruct the original *note* and *memo field*.

Unlike in Sprout, each *Sapling-pool* output *note* is encrypted by a fresh *ephemeral public key*.

For both encryption and decryption,

- let ℓ_{ovk} be as defined in § 5.3 *'Constants'* on p. 57;
- let Sym be the encryption scheme instantiated in § 5.4.3 *'Symmetric Encryption'* on p. 66;
- let KA be the *key agreement scheme* $\mathsf{KA}^{\mathsf{Sapling}}$ instantiated in § 5.4.4.3 *'Sapling Key Agreement'* on p. 67;
- let KDF be the *Key Derivation Function* $\mathsf{KDF}^{\mathsf{Sapling}}$ instantiated in § 5.4.4.4 *'Sapling Key Derivation'* on p. 67;
- let $\mathbb{G}$, $\ell_{\mathbb{G}}$, and $\mathsf{repr}_{\mathbb{G}}$ be instantiated as $\mathbb{J}$, $\ell_{\mathbb{J}}$, and $\mathsf{repr}_{\mathbb{J}}$ defined in § 5.4.8.3 *'Jubjub'* on p. 77;

- let $\mathsf{Extract}_{\mathbb{G}^{(r)}}$ be $\mathsf{Extract}_{\mathbb{J}^{(r)}}$ as defined in § 5.4.8.4 *'Coordinate Extractor for Jubjub'* on p. 78;
- let $\mathsf{PRF}^{\mathsf{ock}}$ be $\mathsf{PRF}^{\mathsf{ockSapling}}$ instantiated in § 5.4.2 *'Pseudo Random Functions'* on p. 64;
- let DiversifyHash be $\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ in § 5.4.1.6 *'$\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ Hash Function'* on p. 61;
- let NoteCommitment be $\mathsf{NoteCommitment}^{\mathsf{Sapling}}$ defined in § 3.2.2 *'Note Commitments'* on p. 15;
- let ToScalar be $\mathsf{ToScalar}^{\mathsf{Sapling}}$ defined in § 4.2.2 *'Sapling Key Components'* on p. 32;
- LEBS2OSP, LEOS2IP, I2LEBSP, and I2LEOSP are defined in § 5.1 *'Integers, Bit Sequences, and Endianness'* on p. 56.

4.18.1 Encryption (Sapling)

Let $\mathsf{pk_d} : \mathsf{KA.PublicPrimeOrder}$ be the *diversified transmission key* for the intended recipient address of a new *Sapling-pool note*, and let $\mathsf{g_d} : \mathsf{KA.PublicPrimeOrder}$ be the corresponding *diversified base* computed as $\mathsf{DiversifyHash}(\mathsf{d})$.

Since Sapling *note* encryption is used only in the context of § 4.6.2 *'Sending Notes (Sapling)'* on p. 37, we may assume that $\mathsf{g_d}$ has already been calculated and is not $\bot$. Also, the *ephemeral private key* esk has been chosen.

Let $\mathsf{ovk} : \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{ovk}}/8]}} \cup \{\bot\}$ be as described in § 4.6.2 on p. 37, i.e. the *outgoing viewing key* of the *shielded payment address* from which the *note* is being spent, or an *outgoing viewing key* associated with a [ZIP-32] account, or $\bot$.

Let $\mathbf{np} = (\mathsf{leadByte}, \mathsf{d}, \mathsf{v}, \underline{\mathsf{rcm}}, \mathsf{memo})$ be the Sapling *note plaintext*.

$\mathbf{np}$ is encoded as defined in § 5.5 *'Encodings of Note Plaintexts and Memo Fields'* on p. 81.

Let cv be the *value commitment* for the *Output description*, and let cm be the *note commitment*. These are needed to derive the *outgoing cipher key* ock in order to produce the *outgoing ciphertext* $\mathsf{C^{out}}$.

Then to encrypt:

$\quad \mathsf{let}\ \mathsf{P^{enc}}$ be the *raw encoding* of $\mathbf{np}$

$\quad \mathsf{let}\ \mathsf{epk} = \mathsf{KA.DerivePublic}(\mathsf{esk}, \mathsf{g_d})$

$\quad \mathsf{let}\ \mathtt{ephemeralKey} = \mathsf{LEBS2OSP}_{\ell_{\mathbb{G}}}\big(\mathsf{repr}_{\mathbb{G}}(\mathsf{epk})\big)$

$\quad \mathsf{let}\ \mathsf{sharedSecret} = \mathsf{KA.Agree}(\mathsf{esk}, \mathsf{pk_d})$

$\quad \mathsf{let}\ \mathsf{K^{enc}} = \mathsf{KDF}(\mathsf{sharedSecret}, \mathtt{ephemeralKey})$

$\quad \mathsf{let}\ \mathsf{C^{enc}} = \mathsf{Sym.Encrypt}_{\mathsf{K^{enc}}}(\mathsf{P^{enc}})$

$\quad \mathsf{if}\ \mathsf{ovk} = \bot\mathsf{:}$

$\quad\quad$ choose random ock $\xleftarrow{R}$ Sym.$\mathbf{K}$ and $\mathbf{op} \xleftarrow{R} \mathbb{B}^{\mathbb{Y}^{[(\ell_{\mathbb{G}}+256)/8]}}$

$\quad \mathsf{else}\mathsf{:}$

$\quad\quad \mathsf{let}\ \mathtt{cv} = \mathsf{LEBS2OSP}_{\ell_{\mathbb{G}}}\big(\mathsf{repr}_{\mathbb{G}}(\mathsf{cv})\big)$

$\quad\quad \mathsf{let}\ \mathtt{cm*} = \mathsf{LEBS2OSP}_{256}\big(\mathsf{Extract}_{\mathbb{G}^{(r)}}(\mathsf{cm})\big)$

$\quad\quad \mathsf{let}\ \mathsf{ock} = \mathsf{PRF}^{\mathsf{ock}}_{\mathsf{ovk}}(\mathtt{cv}, \mathtt{cm*}, \mathtt{ephemeralKey})$

$\quad\quad \mathsf{let}\ \mathbf{op} = \mathsf{LEBS2OSP}_{\ell_{\mathbb{G}}+256}\big(\mathsf{repr}_{\mathbb{G}}(\mathsf{pk_d}) \ ||\ \mathsf{I2LEBSP}_{256}(\mathsf{esk})\big)$

$\quad \mathsf{let}\ \mathsf{C^{out}} = \mathsf{Sym.Encrypt}_{\mathsf{ock}}(\mathbf{op})$

The resulting *transmitted note ciphertext* is $(\mathtt{ephemeralKey}, \mathsf{C^{enc}}, \mathsf{C^{out}})$.

Note: It is technically possible to replace $\mathsf{C^{enc}}$ for a given *note* with a random (and undecryptable) dummy ciphertext, relying instead on *out-of-band* transmission of the *note* to the recipient. In this case the ephemeral key **MUST** still be generated as a random *public key* (rather than a random *bit sequence*) to ensure indistinguishability from other *Output descriptions*. This mode of operation raises further security considerations, for example of how to validate a *Sapling-pool note* received *out-of-band*, which are not addressed in this document.

4.18.2 Decryption using an Incoming Viewing Key (Sapling)

Let $\mathsf{ivk} : \{1 \mathinner{..} 2^{\ell^{\mathsf{Sapling}}_{\mathsf{ivk}}} - 1\}$ be the recipient's $\mathsf{KA}^{\mathsf{Sapling}}$ *private key*, as specified in § 4.2.2 *'Sapling Key Components'* on p. 32.

Let $(\mathtt{ephemeralKey}, \mathsf{C}^{\mathsf{enc}}, \mathsf{C}^{\mathsf{out}})$ be the *transmitted note ciphertext* from the *Output description*. Let $\mathtt{cm}*$ be the `cmu` field of the *Output description*. (This encodes the *affine-ctEdwards* u-coordinate of the *note commitment*, i.e. $\mathsf{Extract}_{\mathbb{G}^{(r)}}(\mathsf{cm})$.)

The recipient will attempt to decrypt the `ephemeralKey` and $\mathsf{C}^{\mathsf{enc}}$ components of the *transmitted note ciphertext*:

$\mathsf{let\ epk} = \mathsf{abst}_{\mathbb{G}}(\mathtt{ephemeralKey})$. if $\mathsf{epk} = \bot$, return $\bot$
$\mathsf{let\ sharedSecret} = \mathsf{KA.Agree}(\mathsf{ivk}, \mathsf{epk})$
$\mathsf{let}\ \mathsf{K}^{\mathsf{enc}} = \mathsf{KDF}(\mathsf{sharedSecret}, \mathtt{ephemeralKey})$
$\mathsf{let}\ \mathsf{P}^{\mathsf{enc}} = \mathsf{Sym.Decrypt}_{\mathsf{K}^{\mathsf{enc}}}(\mathsf{C}^{\mathsf{enc}})$. if $\mathsf{P}^{\mathsf{enc}} = \bot$, return $\bot$
$\mathsf{extract}\ \mathbf{np} = (\mathsf{leadByte} : \mathbb{B}^{\mathbb{Y}}, \mathsf{d} : \mathbb{B}^{[\ell_{\mathsf{d}}]}, \mathsf{v} : \{0 \mathinner{..} 2^{\ell_{\mathsf{value}}} - 1\}, \underline{\mathsf{rcm}} : \mathbb{B}^{\mathbb{Y}^{[32]}}, \mathsf{memo} : \mathbb{B}^{\mathbb{Y}^{[512]}})\ \mathsf{from}\ \mathsf{P}^{\mathsf{enc}}$
$\mathsf{if\ leadByte} \notin \mathsf{allowedLeadBytes}^{\mathit{pool}}(\mathsf{height}, \mathsf{txVersion})$, return $\bot$
$\mathsf{let\ rcm} = \mathsf{LEOS2IP}_{256}(\underline{\mathsf{rcm}})$
$\mathsf{if\ rcm} \geq r_{\mathbb{G}}$, return $\bot$
$\mathsf{let}\ \mathsf{g_d} = \mathsf{DiversifyHash}(\mathsf{d})$. if $\mathsf{g_d} = \bot$, return $\bot$
$\mathsf{let}\ \mathsf{pk_d} = \mathsf{KA.DerivePublic}(\mathsf{ivk}, \mathsf{g_d})$
$\mathsf{let}\ \mathbf{n} = (\mathsf{d}, \mathsf{pk_d}, \mathsf{v}, \mathsf{rcm})$
$\mathsf{let}\ \mathsf{cm}'_* = \mathsf{NoteCommitment}(\mathbf{n})$.
$\mathsf{if\ I2LEOSP}_{256}\big(\mathsf{Extract}_{\mathbb{G}^{(r)}}(\mathsf{cm}'_*)\big) \neq \mathtt{cm}*$, return $\bot$
return $(\mathbf{n}, \mathsf{memo})$.

Notes:

- $\mathsf{g_d}$ has already been computed when applying NoteCommitment, and need not be computed again.
- For Sapling, as explained in the note in § 5.4.8.3 *'Jubjub'* on p. 77, $\mathsf{abst}_{\mathbb{J}}$ accepts *non-canonical* compressed encodings of *Jubjub curve* points. Therefore, an implementation **MUST** use the original `ephemeralKey` field as encoded in the *transaction* as input to $\mathsf{KDF}^{\mathsf{Sapling}}$.
- Normally only *transmitted note ciphertexts* of *transactions* in *blocks* need to be decrypted. In that case, any received *Sapling-pool note* is necessarily a *positioned note*, so its ρ value can immediately be calculated per § 4.14 *'Computing ρ values and Nullifiers'* on p. 45. To test whether a *Sapling-pool note* is unspent in a particular *block chain* also requires the *nullifier deriving key* nk; the coin is unspent if and only if the *nullifier* computed as in § 4.14 on p. 45 is not in the *nullifier set* for that *block chain*.
- A *note* can change from being unspent to spent as a node's view of the *best valid block chain* is extended by new *transactions*. Also, *block chain reorganizations* can cause a node to switch to a different *best valid block chain* that does not contain the *transaction* in which a *note* was output.
- A client **MAY** attempt to decrypt a *transmitted note ciphertext* of a *transaction* in the *mempool*. However, in that case it **MUST NOT** assume that the *transaction* will be mined and **MUST** treat the decrypted information as provisional, and private.

4.18.3 Decryption using an Outgoing Viewing Key (Sapling)

Let $\mathsf{ovk} : \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{ovk}}/8]}}$ be the *outgoing viewing key*, as specified in § 4.2.2 on p. 32, that is to be used for decryption. (If $\mathsf{ovk} = \bot$ was used for encryption, the payment is not decryptable by this method.)

Let $(\mathtt{ephemeralKey}, \mathsf{C}^{\mathsf{enc}}, \mathsf{C}^{\mathsf{out}})$ be the *transmitted note ciphertext*.

For a Sapling *transmitted note ciphertext*, let `cv` and $\mathtt{cm}*$ be the `cv` and `cmu` fields of the *Output description*.

The *outgoing viewing key* holder will attempt to decrypt the *transmitted note ciphertext* as follows:

let $\mathsf{ock} = \mathsf{PRF}^{\mathsf{ock}}_{\mathsf{ovk}}(\mathsf{cv}, \mathtt{cm}*, \mathtt{ephemeralKey})$

let $\mathbf{op} = \mathsf{Sym.Decrypt}_{\mathsf{ock}}(\mathsf{C}^{\mathsf{out}})$. if $\mathbf{op} = \bot$, return $\bot$

extract $(\mathsf{pk}\star_{\mathsf{d}} : \mathbb{B}^{[\ell_{\mathbb{G}}]}, \underline{\mathsf{esk}} : \mathbb{B}^{\mathbb{Y}[32]})$ from $\mathbf{op}$

let $\mathsf{esk} = \mathsf{LEOS2IP}_{256}(\underline{\mathsf{esk}})$ and $\mathsf{pk}_{\mathsf{d}} = \mathsf{abst}_{\mathbb{G}}(\mathsf{pk}\star_{\mathsf{d}})$

if $\mathsf{esk} \geq r_{\mathbb{G}}$ or $\mathsf{pk}_{\mathsf{d}} = \bot$, return $\bot$

if $\mathsf{repr}_{\mathbb{P}}(\mathsf{pk}_{\mathsf{d}}) \neq \mathsf{pk}\star_{\mathsf{d}}$, return $\bot$

let $\mathsf{sharedSecret} = \mathsf{KA.Agree}(\mathsf{esk}, \mathsf{pk}_{\mathsf{d}})$

let $\mathsf{K}^{\mathsf{enc}} = \mathsf{KDF}(\mathsf{sharedSecret}, \mathtt{ephemeralKey})$

let $\mathsf{P}^{\mathsf{enc}} = \mathsf{Sym.Decrypt}_{\mathsf{K}^{\mathsf{enc}}}(\mathsf{C}^{\mathsf{enc}})$. if $\mathsf{P}^{\mathsf{enc}} = \bot$, return $\bot$

extract $\mathbf{np} = (\mathsf{leadByte} : \mathbb{B}^{\mathbb{Y}}, \mathsf{d} : \mathbb{B}^{[\ell_{\mathsf{d}}]}, \mathsf{v} : \{0 \,..\, 2^{\ell_{\mathsf{value}}}-1\}, \underline{\mathsf{rcm}} : \mathbb{B}^{\mathbb{Y}[32]}, \mathsf{memo} : \mathbb{B}^{\mathbb{Y}[512]})$ from $\mathsf{P}^{\mathsf{enc}}$

if $\mathsf{leadByte} \notin \mathsf{allowedLeadBytes}^{\mathit{pool}}(\mathsf{height}, \mathsf{txVersion})$, return $\bot$

let $\mathsf{rcm} = \mathsf{LEOS2IP}_{256}(\underline{\mathsf{rcm}})$

if $\mathsf{rcm} \geq r_{\mathbb{G}}$, return $\bot$

let $\mathsf{g}_{\mathsf{d}} = \mathsf{DiversifyHash}(\mathsf{d})$. if $\mathsf{g}_{\mathsf{d}} = \bot$ or $\mathsf{pk}_{\mathsf{d}} \notin \mathbb{J}^{(r)*}$ (see note below), return $\bot$

let $\mathbf{n} = (\mathsf{d}, \mathsf{pk}_{\mathsf{d}}, \mathsf{v}, \mathsf{rcm})$

let $\mathsf{cm}'_* = \mathsf{NoteCommitment}(\mathbf{n})$.

if $\mathsf{I2LEOSP}_{256}\big(\mathsf{Extract}_{\mathbb{G}^{(r)}}(\mathsf{cm}'_*)\big) \neq \mathtt{cm}*$, return $\bot$

if $\mathsf{repr}_{\mathbb{G}}\big(\mathsf{KA.DerivePublic}(\mathsf{esk}, \mathsf{g}_{\mathsf{d}})\big) \neq \mathtt{ephemeralKey}$, return $\bot$

return $(\mathbf{n}, \mathsf{memo})$.

Notes:

- g_{d} has already been computed when applying NoteCommitment, and need not be computed again.
- A previous version of this specification did not have the requirement for the decoded point pk_{d} of a Sapling *note* to be in the set of *prime-order* points $\mathbb{J}^{(r)*}$ (i.e. "if ... $\mathsf{pk}_{\mathsf{d}} \notin \mathbb{J}^{(r)*}$, return $\bot$"). That did not match the implementation in *zcashd*. In fact the history is a little more complicated. The current specification matches the implementation in *librustzcash* as of [librustzcash-109], which has been used in *zcashd* since *zcashd v2.1.2*. However, there was another implementation of Sapling note decryption used in *zcashd* for consensus checks, specifically the check that a *shielded* coinbase output decrypts successfully with the zero ovk. This was corrected to enforce the same restriction on the decrypted pk_{d} in *zcashd v5.5.0*, originally set to activate in a soft fork at *block height* 2121200 on both *Mainnet* and *Testnet* [zcashd-6459]. (On *Testnet* this height was in the past as of the zcashd v5.5.0 release, and so the change would have been immediately enforced on upgrade.) Since the soft fork was observed to be retrospectively valid after that height, the implementation was simplified in [zcashd-6725] to use the *librustzcash* implementation in all cases, which reflects the specification above. *zebra* always used the *librustzcash* implementation.
- As explained in the note in § 5.4.8.3 *'Jubjub'* on p. 77, $\mathsf{abst}_{\mathbb{J}}$ accepts *non-canonical* compressed encodings of *Jubjub curve* points. Therefore, an implementation **MUST** use the original ephemeralKey field as encoded in the *transaction* as input to $\mathsf{PRF}^{\mathsf{ock}}$ and $\mathsf{KDF}^{\mathsf{Sapling}}$, and in the comparison against $\mathsf{repr}_{\mathbb{G}}\big(\mathsf{KA}^{\mathsf{Sapling}}.\mathsf{DerivePublic}(\mathsf{esk}, \mathsf{g}_{\mathsf{d}})\big)$.
- For Sapling *outgoing ciphertexts*, $\mathsf{pk}\star_{\mathsf{d}}$ could also be *non-canonical*. The above algorithm explicitly returns $\bot$ if $\mathsf{repr}_{\mathbb{P}}(\mathsf{pk}_{\mathsf{d}}) \neq \mathsf{pk}\star_{\mathsf{d}}$. However, this is technically redundant with the later check that returns $\bot$ if $\mathsf{pk}_{\mathsf{d}} \notin \mathbb{J}^{(r)*}$, because only *small-order Jubjub curve* points have *non-canonical* encodings. This check is enforced retrospectively for consensus by current *zcashd* and *zebra* versions, and for wallet rescanning by current *zcashd*. Versions of *zcashd* prior to [zcashd-6725] could however have accepted *notes* for which the *outgoing ciphertext* contains either a canonical or a *non-canonical* encoding of $\mathcal{O}_{\mathbb{J}}$ for pk_{d}.
- The comments in § 4.18.2 ***'Decryption using an Incoming Viewing Key (Sapling)'*** on p. 53 concerning calculation of ρ, detection of spent *notes*, and decryption of *transmitted note ciphertexts* for *transactions* in the *mempool* also apply to *notes* decrypted by this procedure.

Non-normative note: Implementors should pay close attention to similarities and differences between this procedure and § 4.18.2 ***'Decryption using an Incoming Viewing Key (Sapling)'*** on p. 53.

4.19 Block Chain Scanning (Sprout)

Let $\ell_{\mathsf{PRF}}^{\mathsf{Sprout}}$ be as defined in § 5.3 *'Constants'* on p. 57.

Let $\mathsf{Note}^{\mathsf{Sprout}}$ be as defined in § 3.2 *'Shielded Pools and Notes'* on p. 13.

Let $\mathsf{KA}^{\mathsf{Sprout}}$ be as defined in § 5.4.4.1 *'Sprout Key Agreement'* on p. 66.

Let $\mathsf{ivk} = (\mathsf{a}_{\mathsf{pk}} : \mathbb{B}^{[\ell_{\mathsf{PRF}}^{\mathsf{Sprout}}]}, \mathsf{sk}_{\mathsf{enc}} : \mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Private})$ be the *incoming viewing key* corresponding to a_{sk}, and let $\mathsf{pk}_{\mathsf{enc}}$ be the associated *transmission key*, as specified in § 4.2.1 *'Sprout Key Components'* on p. 32.

The following algorithm can be used, given the *block chain* and a Sprout *spending key* a_{sk}, to obtain each *note* sent to the corresponding *shielded payment address*, its *memo field*, and its final status (spent or unspent).

let mutable ReceivedSet : $\mathscr{P}\big(\mathsf{Note}^{\mathsf{Sprout}} \times \mathbb{B}^{\mathbb{Y}^{[512]}}\big) \leftarrow \{\}$
let mutable SpentSet : $\mathscr{P}\big(\mathsf{Note}^{\mathsf{Sprout}}\big) \leftarrow \{\}$
let mutable NullifierMap : $\mathbb{B}^{[\ell_{\mathsf{PRF}}^{\mathsf{Sprout}}]} \to \mathsf{Note}^{\mathsf{Sprout}} \leftarrow$ the empty mapping
for each *transaction* tx:
  for each *JoinSplit description* in tx:
    let $(\mathsf{epk}, \mathsf{C}^{\mathsf{enc}}_{1..\mathrm{N}^{\mathsf{new}}})$ be the *transmitted notes ciphertext* of the *JoinSplit description*
    for i in $1..\mathrm{N}^{\mathsf{new}}$:
      Attempt to decrypt the *transmitted notes ciphertext* component $(\mathsf{epk}, \mathsf{C}^{\mathsf{enc}}_i)$ using ivk with the algorithm in § 4.17.2 *'Decryption (Sprout)'* on p. 51. If this succeeds with $(\mathbf{n}, \mathsf{memo})$:
        Add $(\mathbf{n}, \mathsf{memo})$ to ReceivedSet.
        Calculate the *nullifier* nf of $\mathbf{n}$ using a_{sk} as in § 4.14 *'Computing ρ values and Nullifiers'* on p. 45.
        Add the mapping $\mathsf{nf} \to \mathbf{n}$ to NullifierMap.
    let $\mathsf{nf}_{1..\mathrm{N}^{\mathsf{old}}}$ be the *nullifiers* of the *JoinSplit description*
    for i in $1..\mathrm{N}^{\mathsf{old}}$: if nf_i is present in NullifierMap, add NullifierMap(nf_i) to SpentSet
return (ReceivedSet, SpentSet).

4.20 Block Chain Scanning (Sapling)

In Sapling, *block chain* scanning requires only the nk and ivk key components, rather than a *spending key* as in Sprout. Typically, these components are derived from a *full viewing key* (§ 4.2.2 *'Sapling Key Components'* on p. 32).

Let $\ell_{\mathsf{PRFnfSapling}}$ be as defined in § 5.3 *'Constants'* on p. 57.

Let Note be $\mathsf{Note}^{\mathsf{Sapling}}$ as defined in § 3.2 *'Shielded Pools and Notes'* on p. 13.

Let KA be $\mathsf{KA}^{\mathsf{Sapling}}$ as defined in § 5.4.4.3 on p. 67.

Let NullifierType be $\mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{PRFnfSapling}}/8]}}$.

The following algorithm can be used, given the *block chain* and $(\mathsf{nk}, \mathsf{ivk})$, to obtain each *note* sent to the corresponding *shielded payment address*, its *memo field*, and its final status (spent or unspent).

let mutable ReceivedSet : $\mathscr{P}\big(\mathsf{Note} \times \mathbb{B}^{\mathbb{Y}^{[512]}}\big) \leftarrow \{\}$

let mutable SpentSet : $\mathscr{P}(\mathsf{Note}) \leftarrow \{\}$

let mutable NullifierMap : $(\mathsf{NullifierType} \to \mathsf{Note}) \leftarrow$ the empty mapping

for each *transaction* tx:

  for each *Output description* in tx:

    Attempt to decrypt the *transmitted note ciphertext* components epk and $\mathsf{C}^{\mathsf{enc}}$ using ivk with the algorithm § 4.18.2 ***'Decryption using an Incoming Viewing Key (Sapling)'*** on p. 53.

    If this succeeds with $(\mathbf{n}, \mathsf{memo})$:

      Add $(\mathbf{n}, \mathsf{memo})$ to ReceivedSet.

      Calculate the *nullifier* nf of $\mathbf{n}$ using nk as in § 4.14 ***'Computing ρ values and Nullifiers'*** on p. 45. (This also requires pos from the *Output description*.)

      Add the mapping $\mathsf{nf} \to \mathbf{n}$ to NullifierMap.

  for each *nullifier* nf of a *Spend description* in tx:

    if nf is present in NullifierMap, add NullifierMap(nf) to SpentSet

return (ReceivedSet, SpentSet).

Non-normative notes:

- The above algorithm does not use the ovk key component, or the $\mathsf{C}^{\mathsf{out}}$ *transmitted note ciphertext* component. When scanning the whole *block chain*, these are indeed not necessary. The advantage of supporting decryption using ovk as described in § 4.18.3 ***'Decryption using an Outgoing Viewing Key (Sapling)'*** on p. 53, is that it allows recovering information about the *note plaintexts* sent in a *transaction* from that *transaction* alone.
- When scanning only part of a *block chain*, it may be useful to augment the above algorithm with decryption of $\mathsf{C}^{\mathsf{out}}$ components for each *transaction*, in order to obtain information about *notes* that were spent in the scanned period but received outside it.
- The above algorithm does not detect *notes* that were sent "*out-of-band*" or with incorrect *transmitted note ciphertexts*. It is possible to detect whether such *notes* were spent only if their *nullifiers* are known.

5 Concrete Protocol

5.1 Integers, Bit Sequences, and Endianness

All integers in **Zcash**-specific encodings are unsigned, have a fixed bit length, and are encoded in little-endian byte order ***unless otherwise specified***.

The following functions convert between sequences of bits, sequences of bytes, and integers:

- $\mathsf{I2LEBSP} : (\ell : \mathbb{N}) \times \{0 \,..\, 2^\ell - 1\} \to \mathbb{B}^{[\ell]}$, such that $\mathsf{I2LEBSP}_\ell(x)$ is the sequence of ℓ bits representing x in little-endian order;
- $\mathsf{I2LEOSP} : (\ell : \mathbb{N}) \times \{0 \,..\, 2^\ell - 1\} \to \mathbb{B}^{\mathbb{Y}^{[\mathsf{ceiling}(\ell/8)]}}$, such that $\mathsf{I2LEOSP}_\ell(x)$ is the sequence of $\mathsf{ceiling}(\ell/8)$ bytes representing x in little-endian order;
- $\mathsf{I2BEBSP} : (\ell : \mathbb{N}) \times \{0 \,..\, 2^\ell - 1\} \to \mathbb{B}^{[\ell]}$ such that $\mathsf{I2BEBSP}_\ell(x)$ is the sequence of ℓ bits representing x in big-endian order.
- $\mathsf{LEBS2IP} : (\ell : \mathbb{N}) \times \mathbb{B}^{[\ell]} \to \{0 \,..\, 2^\ell - 1\}$ such that $\mathsf{LEBS2IP}_\ell(S)$ is the integer represented in little-endian order by the *bit sequence* S of length ℓ.

- $\mathsf{LEOS2IP} : (\ell : \mathbb{N} \mid \ell \bmod 8 = 0) \times \mathbb{B}^{\mathbb{Y}^{[\ell/8]}} \to \{0 \,..\, 2^\ell - 1\}$ such that $\mathsf{LEOS2IP}_\ell(S)$ is the integer represented in little-endian order by the *byte sequence* S of length $\ell/8$.
- $\mathsf{LEBS2OSP} : (\ell : \mathbb{N}) \times \mathbb{B}^{[\ell]} \to \mathbb{B}^{\mathbb{Y}^{[\mathsf{ceiling}(\ell/8)]}}$ defined as follows: pad the input on the right with $8 \cdot \mathsf{ceiling}(\ell/8) - \ell$ zero bits so that its length is a multiple of 8 bits. Then convert each group of 8 bits to a byte value with the ***least*** significant bit first, and concatenate the resulting bytes in the same order as the groups.
- $\mathsf{LEOS2BSP} : (\ell : \mathbb{N} \mid \ell \bmod 8 = 0) \times \mathbb{B}^{\mathbb{Y}^{[\mathsf{ceiling}(\ell/8)]}} \to \mathbb{B}^{[\ell]}$ defined as follows: convert each byte to a group of 8 bits with the ***least*** significant bit first, and concatenate the resulting groups in the same order as the bytes.

5.2 Bit layout diagrams

We sometimes use bit layout diagrams, in which each box of the diagram represents a sequence of bits. Diagrams are read from left to right, with lines read from top to bottom; the breaking of boxes across lines has no significance. The bit length ℓ is given explicitly in each box, except when it is obvious (e.g. for a single bit, or for the notation $[0]^\ell$ representing the sequence of ℓ zero bits, or for the output of $\mathsf{LEBS2OSP}_\ell$).

The entire diagram represents the sequence of ***bytes*** formed by first concatenating these *bit sequences*, and then treating each subsequence of 8 bits as a byte with the bits ordered from ***most significant*** to ***least significant***. Thus the ***most significant*** bit in each byte is toward the left of a diagram. (This convention is used only in descriptions of the Sprout design; in the Sapling additions, *bit sequence*/*byte sequence* conversions are always specified explicitly.) Where bit fields are used, the text will clarify their position in each case.

5.3 Constants

Define:

$\mathsf{MerkleDepth}^{\mathsf{Sprout}} : \mathbb{N} := 29$

$\mathsf{MerkleDepth}^{\mathsf{Sapling}} : \mathbb{N} := 32$

$\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}} : \mathbb{N} := 256$

$\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}} : \mathbb{N} := 255$

$\mathrm{N}^{\mathsf{old}} : \mathbb{N} := 2$

$\mathrm{N}^{\mathsf{new}} : \mathbb{N} := 2$

$\ell_{\mathsf{value}} : \mathbb{N} := 64$

$\ell_{\mathsf{hSig}} : \mathbb{N} := 256$

$\ell^{\mathsf{Sprout}}_{\mathsf{PRF}} : \mathbb{N} := 256$

$\ell_{\mathsf{PRFexpand}} : \mathbb{N} := 512$

$\ell_{\mathsf{PRFnfSapling}} : \mathbb{N} := 256$

$\ell^{\mathsf{Sprout}}_{\mathsf{rcm}} : \mathbb{N} := 256$

$\ell_{\mathsf{Seed}} : \mathbb{N} := 256$

$\ell_{\mathsf{a_{sk}}} : \mathbb{N} := 252$

$\ell^{\mathsf{Sprout}}_{\mathsf{\varphi}} : \mathbb{N} := 252$

$\ell_{\mathsf{sk}} : \mathbb{N} := 256$

$\ell_{\mathsf{d}} : \mathbb{N} := 88$

$\ell^{\mathsf{Sapling}}_{\mathsf{ivk}} : \mathbb{N} := 251$

$\ell_{\mathsf{ovk}} : \mathbb{N} := 256$

$\ell^{\mathsf{Sapling}}_{\mathsf{scalar}} : \mathbb{N} := 252$

$\mathsf{Uncommitted}^{\mathsf{Sprout}} : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}]} := [0]^{\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}}$

$\mathsf{Uncommitted}^{\mathsf{Sapling}} : \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]} := \mathsf{I2LEBSP}_{\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}}(1)$

$\mathsf{MAX_MONEY} : \mathbb{N} := 2.1 \cdot 10^{15}$ (zatoshi)

$\mathsf{SlowStartInterval} : \mathbb{N} := 20000$

$\mathsf{HalvingInterval} : \mathbb{N} := 840000$

$\mathsf{MaxBlockSubsidy} : \mathbb{N} := 1.25 \cdot 10^{9}$ (zatoshi)

$\mathsf{NumFounderAddresses} : \mathbb{N} := 48$

$\mathsf{FoundersFraction} : \mathbb{Q} := \frac{1}{5}$

$$\mathsf{PoWLimit} : \mathbb{N} := \begin{cases} 2^{243} - 1, & \text{for } \textit{Mainnet} \\ 2^{251} - 1, & \text{for } \textit{Testnet} \end{cases}$$

$\mathsf{PoWAveragingWindow} : \mathbb{N} := 17$

$\mathsf{PoWMedianBlockSpan} : \mathbb{N} := 11$

$\mathsf{PoWMaxAdjustDown} : \mathbb{Q} := \frac{32}{100}$

$\mathsf{PoWMaxAdjustUp} : \mathbb{Q} := \frac{16}{100}$

$\mathsf{PoWDampingFactor} : \mathbb{N} := 4$

$\mathsf{PoWTargetSpacing} : \mathbb{N} := 150$ (seconds).

5.4 Concrete Cryptographic Schemes

5.4.1 Hash Functions

5.4.1.1 SHA-256, SHA-256d, SHA256Compress, and SHA-512 Hash Functions

SHA-256 and SHA-512 are defined by [NIST2015].

Zcash uses the full SHA-256 *hash function* to instantiate $\mathsf{NoteCommitment}^{\mathsf{Sprout}}$.

$\mathsf{SHA\text{-}256} : \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}} \to \mathbb{B}^{\mathbb{Y}^{[32]}}$

[NIST2015] strictly speaking only specifies the application of SHA-256 to messages that are *bit sequences*, producing outputs ("message digests") that are also *bit sequences*. In practice, SHA-256 is universally implemented with a byte-sequence interface for messages and outputs, such that the ***most significant*** bit of each byte corresponds to the first bit of the associated *bit sequence*. (In the NIST specification "first" is conflated with "leftmost".)

SHA-256d, defined as a double application of SHA-256, is used to hash *block headers*:

$\mathsf{SHA\text{-}256d} : \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}} \to \mathbb{B}^{\mathbb{Y}^{[32]}}$

Zcash also uses the SHA-256 compression function, SHA256Compress. This operates on a single 512-bit block and ***excludes*** the padding step specified in [NIST2015, section 5.1].

That is, the input to SHA256Compress is what [NIST2015, section 5.2] refers to as "the message and its padding". The Initial Hash Value is the same as for full SHA-256.

SHA256Compress is used to instantiate several *Pseudo Random Functions* and $\mathsf{MerkleCRH}^{\mathsf{Sprout}}$.

$\mathsf{SHA256Compress} : \mathbb{B}^{[512]} \to \mathbb{B}^{[256]}$

The ordering of bits within words in the interface to SHA256Compress is consistent with [NIST2015, section 3.1], i.e. big-endian.

Ed25519 uses SHA-512:

$\mathsf{SHA\text{-}512} : \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}} \to \mathbb{B}^{\mathbb{Y}^{[64]}}$

The comment above concerning bit vs byte-sequence interfaces also applies to SHA-512.

5.4.1.2 BLAKE2 Hash Functions

BLAKE2 is defined by [ANWW2013]. **Zcash** uses both the BLAKE2b and BLAKE2s variants.

BLAKE2b-$\ell(p, x)$ refers to unkeyed BLAKE2b-ℓ in sequential mode, with an output digest length of $\ell/8$ bytes, 16-byte personalization string p, and input x.

BLAKE2b is used to instantiate hSigCRH, EquihashGen, and $\mathsf{KDF}^{\mathsf{Sprout}}$. From **Overwinter** onward, it is used to compute *SIGHASH transaction hashes* as specified in [ZIP-143], or as in [ZIP-243] after **Sapling** activation. For Sapling, it is also used to instantiate $\mathsf{PRF}^{\mathsf{expand}}$, $\mathsf{PRF}^{\mathsf{ockSapling}}$, $\mathsf{KDF}^{\mathsf{Sapling}}$, and in the RedJubjub *signature scheme* which instantiates $\mathsf{SpendAuthSig}^{\mathsf{Sapling}}$ and $\mathsf{BindingSig}^{\mathsf{Sapling}}$.

$\mathsf{BLAKE2b}\text{-}\ell : \mathbb{B}^{\mathbb{Y}^{[16]}} \times \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}} \to \mathbb{B}^{\mathbb{Y}^{[\ell/8]}}$

Note: BLAKE2b-ℓ is not the same as BLAKE2b-512 truncated to ℓ bits, because the digest length is encoded in the parameter block.

BLAKE2s-$\ell(p, x)$ refers to unkeyed BLAKE2s-ℓ in sequential mode, with an output digest length of $\ell/8$ bytes, 8-byte personalization string p, and input x.

BLAKE2s is used to instantiate $\mathsf{PRF}^{\mathsf{nfSapling}}$, $\mathsf{CRH}^{\mathsf{ivk}}$, and $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}$.

$\mathsf{BLAKE2s}\text{-}\ell : \mathbb{B}^{\mathbb{Y}^{[8]}} \times \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}} \to \mathbb{B}^{\mathbb{Y}^{[\ell/8]}}$

5.4.1.3 Merkle Tree Hash Function

$\mathsf{MerkleCRH}^{\mathsf{Sprout}}$ and $\mathsf{MerkleCRH}^{\mathsf{Sapling}}$ are used to hash *incremental Merkle tree hash values* for Sprout and Sapling respectively.

$\mathsf{MerkleCRH}^{\mathsf{Sprout}}$ Hash Function

$\mathsf{MerkleCRH}^{\mathsf{Sprout}} : \{0 \,..\, \mathsf{MerkleDepth}^{\mathsf{Sprout}} - 1\} \times \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}]} \times \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}]} \to \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{Merkle}}]}$ is defined as follows:

$$\mathsf{MerkleCRH}^{\mathsf{Sprout}}(\mathsf{layer}, \mathsf{left}\star, \mathsf{right}\star) := \mathsf{SHA256Compress}\left(\boxed{256\text{-bit } \mathsf{left}\star \mid 256\text{-bit } \mathsf{right}\star}\right).$$

SHA256Compress is defined in § 5.4.1.1 ‘SHA-256, SHA-256d, SHA256Compress, *and* SHA-512 ***Hash Functions***’ on p. 58.

Security requirement: SHA256Compress must be *collision-resistant*, and it must be infeasible to find a preimage x such that $\mathsf{SHA256Compress}(x) = [0]^{256}$.

Notes:

- The layer argument does not affect the output.
- SHA256Compress is not the same as the SHA-256 function, which hashes arbitrary-length *byte sequences*.

$\mathsf{MerkleCRH}^{\mathsf{Sapling}}$ Hash Function

Let PedersenHash be as specified in § 5.4.1.7 ***'Pedersen Hash Function'*** on p. 62.

$\mathsf{MerkleCRH}^{\mathsf{Sapling}} : \{0 \,..\, \mathsf{MerkleDepth}^{\mathsf{Sapling}} - 1\} \times \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]} \times \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]} \to \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]}$ is defined as follows:

$\mathsf{MerkleCRH}^{\mathsf{Sapling}}(\mathsf{layer}, \mathsf{left}\star, \mathsf{right}\star) := \mathsf{PedersenHash}(\texttt{"Zcash_PH"},\ l\star \,||\, \mathsf{left}\star \,||\, \mathsf{right}\star)$

where $l\star = \mathsf{I2LEBSP}_6\big(\mathsf{MerkleDepth}^{\mathsf{Sapling}} - 1 - \mathsf{layer}\big)$.

Security requirement: PedersenHash must be *collision-resistant*.

Note: The prefix $l\star$ provides domain separation between inputs at different layers of the *note commitment tree*. $\mathsf{NoteCommit}^{\mathsf{Sapling}}$, like PedersenHash, is defined in terms of PedersenHashToPoint, but using a prefix that cannot collide with a layer prefix, as noted in § 5.4.7.2 ***'Windowed Pedersen commitments'*** on p. 72.

5.4.1.4 $\mathsf{h}_{\mathsf{Sig}}$ Hash Function

hSigCRH is used to compute the value $\mathsf{h}_{\mathsf{Sig}}$ in § 4.3 ***'JoinSplit Descriptions'*** on p. 34.

$\mathsf{hSigCRH}(\mathsf{randomSeed}, \mathsf{nf}^{\mathsf{old}}_{1..\mathrm{N}^{\mathsf{old}}}, \texttt{joinSplitPubKey}) := \mathsf{BLAKE2b\text{-}256}(\texttt{"ZcashComputehSig"},\ \mathsf{hSigInput})$

where

$\mathsf{hSigInput} :=$

256-bit randomSeed	256-bit $\mathsf{nf}^{\mathsf{old}}_1$ ...	256-bit $\mathsf{nf}^{\mathsf{old}}_{\mathrm{N}^{\mathsf{old}}}$	256-bit `joinSplitPubKey`

.

BLAKE2b-256(p, x) is defined in § 5.4.1.2 ***'BLAKE2 Hash Functions'*** on p. 59.

Security requirement: BLAKE2b-256$(\texttt{"ZcashComputehSig"}, x)$ must be *collision-resistant* on x.

5.4.1.5 $\mathsf{CRH}^{\mathsf{ivk}}$ Hash Function

$\mathsf{CRH}^{\mathsf{ivk}}$ is used to derive the *incoming viewing key* ivk for a Sapling *shielded payment address*. For its use when generating an address see § 4.2.2 ***'Sapling Key Components'*** on p. 32, and for its use in the *Spend statement* see § 4.16.2 ***'Spend Statement (Sapling)'*** on p. 48.

It is defined as follows:

$\mathsf{CRH}^{\mathsf{ivk}}(\mathsf{ak}\star, \mathsf{nk}\star) := \mathsf{LEOS2IP}_{256}(\mathsf{BLAKE2s\text{-}256}(\texttt{"Zcashivk"},\ \mathsf{crhInput})) \bmod 2^{\ell^{\mathsf{Sapling}}_{\mathsf{ivk}}}$

where

$\mathsf{crhInput} :=$

$\mathsf{LEBS2OSP}_{256}(\mathsf{ak}\star)$	$\mathsf{LEBS2OSP}_{256}(\mathsf{nk}\star)$

BLAKE2s-256(p, x) is defined in § 5.4.1.2 ***'BLAKE2 Hash Functions'*** on p. 59.

Security requirement: $\mathsf{LEOS2IP}_{256}(\mathsf{BLAKE2s\text{-}256}(\texttt{"Zcashivk"}, x)) \bmod 2^{\ell^{\mathsf{Sapling}}_{\mathsf{ivk}}}$ must be *collision-resistant* on a 64-byte input x. Note that this does not follow from *collision resistance* of BLAKE2s-256 (and the best possible concrete security is that of a 251-bit hash rather than a 256-bit hash), but it is a reasonable assumption given the design, structure, and cryptanalysis to date of BLAKE2s.

Non-normative note: BLAKE2s has a variable output digest length feature, but it does not support arbitrary bit lengths, otherwise it would have been used rather than external truncation. However, the protocol-specific personalization string together with truncation achieve essentially the same effect as using that feature.

5.4.1.6 $\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ Hash Function

$\mathsf{DiversifyHash}^{\mathsf{Sapling}} : \mathbb{B}^{[\ell_d]} \to \mathbb{J}^{(r)*} \cup \{\bot\}$ is used to derive a *diversified base* in § 4.2.2 *'Sapling Key Components'* on p. 32.

Let $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}$ and U be as defined in § 5.4.8.5 *'Group Hash into Jubjub'* on p. 79.

Define

$$\mathsf{DiversifyHash}^{\mathsf{Sapling}}(\mathsf{d}) := \mathsf{GroupHash}^{\mathbb{J}^{(r)*}}_{U}\left(\texttt{"Zcash_gd"}, \mathsf{LEBS2OSP}_{\ell_d}(\mathsf{d})\right).$$

Security requirement: **Unlinkability:** Given two randomly selected *shielded payment addresses* from different spend authorities, and a third *shielded payment address* which could be derived from either of those authorities, such that the three addresses use different *diversifiers*, it is not possible to tell which authority the third address was derived from.

Non-normative notes:

- Suppose that $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}$ (restricted to inputs for which it does not return $\bot$) is modelled as a *random oracle* from *diversifiers* to points of order $r_{\mathbb{J}}$ on the *Jubjub curve*. In this model, Unlinkability of $\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ holds under the *Decisional Diffie–Hellman assumption* on the *prime-order subgroup* of points on the *Jubjub curve*.

  To prove this, consider the ElGamal encryption scheme [ElGamal1985] on this *prime-order subgroup*, restricted to encrypting plaintexts encoded as the group identity $\mathcal{O}_{\mathbb{J}}$. (ElGamal was originally defined for $\mathbb{F}_p^*$ but works in any *prime-order group*.) ElGamal *public keys* then have the same form as *diversified payment addresses*. If we make the assumption above on $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}$, then generating a new *diversified payment address* from a given address pk, gives the same distribution of $(\mathsf{g_d}', \mathsf{pk_d}')$ pairs as the distribution of ElGamal ciphertexts obtained by encrypting $\mathcal{O}_{\mathbb{J}}$ under pk. { **TODO: check whether this is justified.** } Then, the definition of *key privacy* (IK-CPA as defined in [BBDP2001, Definition 1]) for ElGamal corresponds to the definition of Unlinkability for $\mathsf{DiversifyHash}^{\mathsf{Sapling}}$. (IK-CCA corresponds to the potentially stronger requirement that $\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ remains Unlinkable when given Diffie–Hellman key agreement oracles for each of the candidate *diversified payment addresses*.) So if ElGamal is *key-private*, then $\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ is Unlinkable under the same conditions. [BBDP2001, Appendix A] gives a security proof for *key privacy* (both IK-CPA and IK-CCA) of ElGamal under the *Decisional Diffie–Hellman assumption* on the relevant group. (In fact the proof needed is the "small modification" described in the last paragraph in which the generator is chosen at random for each key.)
- It is assumed (also for the security of other uses of the group hash, such as Pedersen hashes and commitments) that the discrete logarithm of the output group element with respect to any other generator is unknown. This assumption is justified if the group hash acts as a *random oracle*. Essentially, *diversifiers* act as handles to unknown random numbers. (The group hash inputs used with different personalizations are in different "namespaces".)
- Informally, the random self-reducibility property of DDH implies that an adversary would gain no advantage from being able to query an oracle for additional $(\mathsf{g_d}, \mathsf{pk_d})$ pairs with the same *spending authority* as an existing *shielded payment address*, since they could also create such pairs on their own. This justifies only considering two *shielded payment addresses* in the security definition.

  { **TODO: FIXME This is not correct, because additional pairs don't quite follow the same distribution as an address with a valid diversifier. The security definition may need to be more complex to model this properly.** }
- An 88-bit diversifier cannot be considered cryptographically unguessable at a 128-bit security level; also, randomly chosen diversifiers are likely to suffer birthday collisions when the number of choices approaches 2^{44}.

If most users are choosing diversifiers randomly (as recommended in § 4.2.2 *'Sapling Key Components'* on p. 32), then the fact that they may accidentally choose diversifiers that collide (and therefore reveal the fact that they are not derived from the same *incoming viewing key*) does not appreciably reduce the anonymity set.

In [ZIP-32] an 88-bit *Pseudo Random Permutation*, keyed differently for each node of the derivation tree, is used to select new *diversifiers*. This resolves the potential problem, provided that the input to the *Pseudo Random Permutation* does not repeat for a given node.

- If the holder of an *incoming viewing key* permits an adversary to ask for a new address for that *incoming viewing key* with a given *diversifier*, then it can trivially break Unlinkability for the other *diversified payment addresses* associated with the *incoming viewing key* (this does not compromise other privacy properties). Implementations **SHOULD** avoid providing such a "chosen *diversifier*" oracle.

5.4.1.7 Pedersen Hash Function

PedersenHash is an algebraic *hash function* with *collision resistance* (for fixed input length) derived from assumed hardness of the *Discrete Logarithm Problem* on the *Jubjub curve*. It is based on the work of David Chaum, Ivan Damgård, Jeroen van de Graaf, Jurgen Bos, George Purdy, Eugène van Heijst and Birgit Pfitzmann in [CDvdG1987], [BCP1988] and [CvHP1991], and of Mihir Bellare, Oded Goldreich, and Shafi Goldwasser in [BGG1995], with optimizations for efficient instantiation in *zk-SNARK circuits* by Sean Bowe and Daira-Emma Hopwood.

PedersenHash is used in the definitions of *Pedersen commitments* (§ 5.4.7.2 *'Windowed Pedersen commitments'* on p. 72), and of the *Pedersen hash* for the Sapling *incremental Merkle tree* (§ 5.4.1.3 *'MerkleCRH^{Sapling} Hash Function'* on p. 60).

Let $\mathbb{J}$, $\mathbb{J}^{(r)}$, $\mathcal{O}_{\mathbb{J}}$, $q_{\mathbb{J}}$, $r_{\mathbb{J}}$, $a_{\mathbb{J}}$, and $d_{\mathbb{J}}$ be as defined in § 5.4.8.3 *'Jubjub'* on p. 77.

Let $\mathsf{Extract}_{\mathbb{J}^{(r)}} : \mathbb{J}^{(r)} \to \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]}$ be as defined in § 5.4.8.4 *'Coordinate Extractor for Jubjub'* on p. 78.

Let $\mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}$ be as defined in § 5.4.8.5 *'Group Hash into Jubjub'* on p. 79.

Let $\mathsf{Uncommitted}^{\mathsf{Sapling}}$ be as defined in § 5.3 *'Constants'* on p. 57.

Let c be the largest integer such that $4 \cdot \frac{2^{4 \cdot c} - 1}{15} \leq \frac{r_{\mathbb{J}} - 1}{2}$, i.e. $c := 63$.

Define $\mathcal{I} : \mathbb{B}^{\mathbb{Y}^{[8]}} \times \mathbb{N} \to \mathbb{J}^{(r)*}$ by:

$$\mathcal{I}(D, i) := \mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}\Big(D,\ \boxed{\text{32-bit } i - 1}\Big).$$

Define $\mathsf{PedersenHashToPoint}(D : \mathbb{B}^{\mathbb{Y}^{[8]}}, M : \mathbb{B}^{[\mathbb{N}^+]}) \to \mathbb{J}^{(r)}$ as follows:

Pad M to a multiple of 3 bits by appending zero bits, giving M'.

Let $n = \mathsf{ceiling}\left(\frac{\mathsf{length}(M')}{3 \cdot c}\right)$.

Split M' into n *segments* $M_{1\,..\,n}$ so that $M' = \mathsf{concat}_{\mathbb{B}}(M_{1\,..\,n})$, and each of $M_{1\,..\,n-1}$ is of length $3 \cdot c$ bits. (M_n may be shorter.)

Return $\sum_{i=1}^{n} [\langle M_i \rangle]\, \mathcal{I}(D, i) : \mathbb{J}^{(r)}$.

where $\langle \bullet \rangle : \mathbb{B}^{[3 \cdot \{1\,..\,c\}]} \to \left\{-\frac{r_{\mathbb{J}}-1}{2}\,..\,\frac{r_{\mathbb{J}}-1}{2}\right\} \setminus \{0\}$ is defined as:

Let $k_i = \mathsf{length}(M_i)/3$.

Split M_i into 3-bit *chunks* $m_{1\,..\,k_i}$ so that $M_i = \mathsf{concat}_{\mathbb{B}}(m_{1\,..\,k_i})$.

Write each m_j as $[s^j_0, s^j_1, s^j_2]$, and let $\mathsf{enc}(m_j) = (1 - 2 \cdot s^j_2) \cdot (1 + s^j_0 + 2 \cdot s^j_1) : \mathbb{Z}$.

Let $\langle M_i \rangle = \sum_{j=1}^{k_i} \mathsf{enc}(m_j) \cdot 2^{4 \cdot (j-1)}$.

Finally, define $\mathsf{PedersenHash} : \mathbb{B}^{\mathbb{Y}^{[8]}} \times \mathbb{B}^{[\mathbb{N}^+]} \to \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]}$ by:

$$\mathsf{PedersenHash}(D, M) := \mathsf{Extract}_{\mathbb{J}^{(r)}}\big(\mathsf{PedersenHashToPoint}(D, M)\big).$$

See § A.3.3.9 *'Pedersen hash'* on p. 159 for rationale and efficient circuit implementation of these functions.

Security requirement: PedersenHash and PedersenHashToPoint are required to be *collision-resistant* between inputs of fixed length, for a given personalization input D. No other security properties commonly associated with *hash functions* are needed.

Non-normative note: These *hash functions* are ***not*** *collision-resistant* for variable-length inputs.

Theorem 5.4.1. *The encoding function $\langle \bullet \rangle$ is injective.*

Proof. We first check that the range of $\sum_{j=1}^{k_i} \mathsf{enc}(m_j) \cdot 2^{4 \cdot (j-1)}$ is a subset of the allowable range $\left\{-\frac{r_{\mathbb{J}}-1}{2} \,..\, \frac{r_{\mathbb{J}}-1}{2}\right\} \setminus \{0\}$.

The range of this expression is a subset of $\{-\Delta \,..\, \Delta\} \setminus \{0\}$ where $\Delta = 4 \cdot \sum_{i=1}^{c} 2^{4 \cdot (i-1)} = 4 \cdot \frac{2^{4 \cdot c} - 1}{15}$.

When $c = 63$, we have

$$4 \cdot \frac{2^{4 \cdot c} - 1}{15} = \texttt{0x4444444444444444444444444444444444444444444444444444444444444444}$$

$$\frac{r_{\mathbb{J}} - 1}{2} = \texttt{0x73EDA753299D7D483339D80809A1D8053341049E6640841684B872F6B7B965B}$$

so the required condition is met. This implies that there is no "wrap around" and so $\sum_{j=1}^{k_i} \mathsf{enc}(m_j) \cdot 2^{4 \cdot (j-1)}$ may be treated as an integer expression.

enc is injective. In order to prove that $\langle \bullet \rangle$ is injective, consider $\langle \bullet \rangle^{\Delta} : \mathbb{B}^{[3 \cdot \{1 \,..\, c\}]} \to \{0 \,..\, 2 \cdot \Delta\}$ such that $\langle M_i \rangle^{\Delta} = \langle M_i \rangle + \Delta$. With k_i and m_j defined as above, we have $\langle M_i \rangle^{\Delta} = \sum_{j=1}^{k_i} \mathsf{enc}'(m_j) \cdot 2^{4 \cdot (j-1)}$ where $\mathsf{enc}'(m_j) = \mathsf{enc}(m_j) + 4$ is in $\{0 \,..\, 8\}$ and enc' is injective. Express this sum in hexadecimal; then each m_j affects only one hex digit, and it is easy to see that $\langle \bullet \rangle^{\Delta}$ is injective. Therefore so is $\langle \bullet \rangle$. □

Since the security proof from [BGG1995, Appendix A] depends only on the encoding being injective and its range not including zero, the proof can be adapted straightforwardly to show that PedersenHashToPoint is *collision-resistant* under the same assumptions and security bounds. Because $\mathsf{Extract}_{\mathbb{J}^{(r)}}$ is injective, it follows that PedersenHash is equally *collision-resistant*.

5.4.1.8 Mixing Pedersen Hash Function

A mixing *Pedersen hash* is used to compute ρ from cm and pos in § 4.14 *'Computing ρ values and Nullifiers'* on p. 45. It takes as input a *Pedersen commitment* P, and hashes it with another input x.

Define $\mathcal{J}^{\mathsf{Sapling}} := \mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}(\texttt{"Zcash_J_"}, \texttt{""})$.

We define $\mathsf{MixingPedersenHash} : \mathbb{J} \times \{0 \,..\, r_{\mathbb{J}} - 1\} \to \mathbb{J}$ by:

$$\mathsf{MixingPedersenHash}(P, x) := P + [x]\,\mathcal{J}^{\mathsf{Sapling}}.$$

Security requirement: The function

$$(r, M, x) : \{0 \,..\, r_{\mathbb{J}} - 1\} \times \mathbb{B}^{[\mathbb{N}^+]} \times \{0 \,..\, r_{\mathbb{J}} - 1\} \mapsto \mathsf{MixingPedersenHash}(\mathsf{WindowedPedersenCommit}_r(M), x) : \mathbb{J}$$

must be *collision-resistant* on (r, M, x).

See § A.3.3.10 *'Mixing Pedersen hash'* on p. 161 for efficient circuit implementation of this function.

5.4.1.9 Equihash Generator

$\mathsf{EquihashGen}_{n,k}$ is a specialized *hash function* that maps an input and an index to an output of length n bits. It is used in § 7.6.1 *'Equihash'* on p. 97.

Let powtag := | 64-bit "ZcashPoW" | 32-bit n | 32-bit k |.

Let $\mathsf{powcount}(g)$:= | 32-bit g |.

Let $\mathsf{EquihashGen}_{n,k}(S, i) := T_{h+1\,..\,h+n}$, where

$m = \mathsf{floor}\left(\frac{512}{n}\right)$;

$h = (i - 1 \bmod m) \cdot n$;

T is the *bit sequence* obtained from the *byte sequence* $\mathsf{BLAKE2b}\text{-}(n \cdot m)\big(\mathsf{powtag},\ S \,||\, \mathsf{powcount}\big(\mathsf{floor}\left(\frac{i-1}{m}\right)\big)\big)$ by converting each byte to a group of 8 bits with the ***most*** significant bit first, and concatenating the resulting groups in the same order as the bytes.

Indices of bits in T are 1-based. Note that the conversion to a *bit sequence* is not LEOS2BSP as defined in § 5.1 *'Integers, Bit Sequences, and Endianness'* on p. 56, which takes the ***least*** significant bit of each byte first.

$\mathsf{BLAKE2b}\text{-}\ell(p, x)$ is defined in § 5.4.1.2 *'BLAKE2 Hash Functions'* on p. 59.

Security requirement: $\mathsf{BLAKE2b}\text{-}\ell(\mathsf{powtag}, x)$ must generate output that is sufficiently unpredictable to avoid short-cuts to the *Equihash* solution process. It would suffice to model it as a *random oracle*.

Note: When EquihashGen is evaluated for sequential indices, as in the *Equihash* solving process (§ 7.6.1 *'Equihash'* on p. 97), the number of calls to BLAKE2b can be reduced by a factor of $\mathsf{floor}\left(\frac{512}{n}\right)$ in the best case (which is a factor of 2 for $n = 200$).

5.4.2 Pseudo Random Functions

Let SHA256Compress be as given in § 5.4.1.1 'SHA-256, SHA-256d, SHA256Compress, *and* SHA-512 ***Hash Functions***' on p. 58.

The *Pseudo Random Functions* $\mathsf{PRF}^{\mathsf{addr}}$, $\mathsf{PRF}^{\mathsf{nfSprout}}$, $\mathsf{PRF}^{\mathsf{pk}}$, and PRF^{ρ} from § 4.1.2 *'Pseudo Random Functions'* on p. 22, are all instantiated using SHA256Compress:

$\mathsf{PRF}^{\mathsf{addr}}_{x}(t) := \mathsf{SHA256Compress}\Big($ | 1 | 1 | 0 | 0 | 252-bit x | 8-bit t | $[0]^{248}$ | $\Big)$

$\mathsf{PRF}^{\mathsf{nfSprout}}_{\mathsf{a_{sk}}}(\rho) := \mathsf{SHA256Compress}\Big($ | 1 | 1 | 1 | 0 | 252-bit $\mathsf{a_{sk}}$ | 256-bit ρ | $\Big)$

$\mathsf{PRF}^{\mathsf{pk}}_{\mathsf{a_{sk}}}(i, \mathsf{h_{Sig}}) := \mathsf{SHA256Compress}\Big($ | 0 | i-1 | 0 | 0 | 252-bit $\mathsf{a_{sk}}$ | 256-bit $\mathsf{h_{Sig}}$ | $\Big)$

$\mathsf{PRF}^{\rho}_{\varphi}(i, \mathsf{h_{Sig}}) := \mathsf{SHA256Compress}\Big($ | 0 | i-1 | 1 | 0 | 252-bit φ | 256-bit $\mathsf{h_{Sig}}$ | $\Big)$

Security requirements:

- SHA256Compress must be *collision-resistant*.
- SHA256Compress must be a *PRF* when keyed by the bits corresponding to x, $\mathsf{a_{sk}}$ or φ in the above diagrams, with input in the remaining bits.

Note: The first four bits –i.e. the most significant four bits of the first byte– are used to separate distinct uses of SHA256Compress, ensuring that the functions are independent. As well as the inputs shown here, bits 1011 in this position are used to distinguish uses of the full SHA-256 hash function; see § 5.4.7.1 ***'Note Commitments (Sprout)'*** on p. 72.

(The specific bit patterns chosen here were motivated by the possibility of future extensions that might have increased $\mathrm{N}^{\mathsf{old}}$ and/or $\mathrm{N}^{\mathsf{new}}$ to 3, or added an additional bit to $\mathsf{a_{sk}}$ to encode a new key type, or that would have required an additional *PRF*. In fact since Sapling switches to non-SHA256Compress-based cryptographic primitives, these extensions are unlikely to be necessary.)

$\mathsf{PRF}^{\mathsf{expand}}$ is used in § 4.2.2 ***'Sapling Key Components'*** on p. 32 to derive the *Spend authorizing key* ask and the *proof authorizing key* nsk.

It is instantiated using the BLAKE2b *hash function* defined in § 5.4.1.2 ***'BLAKE2 Hash Functions'*** on p. 59:

$$\mathsf{PRF}^{\mathsf{expand}}_{\mathsf{sk}}(t) := \mathsf{BLAKE2b\text{-}512}(\texttt{"Zcash_ExpandSeed"}, \mathsf{LEBS2OSP}_{256}(\mathsf{sk}) \,||\, t)$$

Security requirement: $\mathsf{BLAKE2b\text{-}512}(\texttt{"Zcash_ExpandSeed"}, \mathsf{LEBS2OSP}_{256}(\mathsf{sk}) \,||\, t)$ must be a *PRF* for output range $\mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{PRFexpand}}/8]}}$ when keyed by the bits corresponding to sk, with input in the bits corresponding to t.

$\mathsf{PRF}^{\mathsf{ockSapling}}$ is used in § 4.18.1 ***'Encryption (Sapling)'*** on p. 52 to derive the *outgoing cipher key* ock used to encrypt an *outgoing ciphertext*.

It is instantiated using the BLAKE2b *hash function* defined in § 5.4.1.2 ***'BLAKE2 Hash Functions'*** on p. 59:

$$\mathsf{PRF}^{\mathsf{ockSapling}}_{\mathsf{ovk}}(\mathsf{cv}, \mathtt{cmu}, \mathtt{ephemeralKey}) := \mathsf{BLAKE2b\text{-}256}(\texttt{"Zcash_Derive_ock"}, \mathsf{ockInput})$$

where ockInput $=$

$\mathsf{LEBS2OSP}_{256}(\mathsf{ovk})$	32-byte cv	32-byte cmu	32-byte ephemeralKey

.

Security requirement: $\mathsf{BLAKE2b\text{-}512}(\texttt{"Zcash_Derive_ock"}, \mathsf{ockInput})$ must be a PRF for output range $\mathsf{Sym}.\mathbf{K}$ (defined in § 5.4.3 ***'Symmetric Encryption'*** on p. 66) when keyed by the bits corresponding to ovk, with input in the bits corresponding to cv, cmu, and ephemeralKey.

$\mathsf{PRF}^{\mathsf{nfSapling}}$ is used to derive the *nullifier* for a *Sapling-pool note*. It is instantiated using the BLAKE2s *hash function* defined in § 5.4.1.2 ***'BLAKE2 Hash Functions'*** on p. 59:

$$\mathsf{PRF}^{\mathsf{nfSapling}}_{\mathsf{nk}\star}(\rho\star) := \mathsf{BLAKE2s\text{-}256}\Big(\texttt{"Zcash_nf"}, \boxed{\mathsf{LEBS2OSP}_{256}(\mathsf{nk}\star) \mid \mathsf{LEBS2OSP}_{256}(\rho\star)}\Big).$$

Security requirement: The function $\mathsf{BLAKE2s\text{-}256}\Big(\texttt{"Zcash_nf"}, \boxed{\mathsf{LEBS2OSP}_{256}(\mathsf{nk}\star) \mid \mathsf{LEBS2OSP}_{256}(\rho\star)}\Big)$ must be a *collision-resistant PRF* for output range $\mathbb{B}^{\mathbb{Y}^{[32]}}$ when keyed by the bits corresponding to $\mathsf{nk}\star$, with input in the bits corresponding to $\rho\star$. Note that $\mathsf{nk}\star : \mathbb{J}^{(r)}_{\star}$ is a representation of a point in the $r_{\mathbb{J}}$-order subgroup of the *Jubjub curve*, and therefore is not uniformly distributed on $\mathbb{B}^{[\ell_{\mathbb{J}}]}$. $\mathbb{J}^{(r)}_{\star}$ is defined in § 5.4.8.3 ***'Jubjub'*** on p. 77.

5.4.3 Symmetric Encryption

Let $\mathsf{Sym}.\mathbf{K} := \mathbb{B}^{[256]}$, $\mathsf{Sym}.\mathbf{P} := \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}}$, and $\mathsf{Sym}.\mathbf{C} := \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}}$.

Let the *authenticated one-time symmetric encryption scheme* $\mathsf{Sym.Encrypt}_{\mathsf{K}}(\mathsf{P})$ be authenticated encryption using AEAD_CHACHA20_POLY1305 [RFC-7539] encryption of plaintext $\mathsf{P} \in \mathsf{Sym}.\mathbf{P}$, with empty "associated data", all-zero nonce $[0]^{96}$, and 256-bit key $\mathsf{K} \in \mathsf{Sym}.\mathbf{K}$.

Similarly, let $\mathsf{Sym.Decrypt}_{\mathsf{K}}(\mathsf{C})$ be AEAD_CHACHA20_POLY1305 decryption of ciphertext $\mathsf{C} \in \mathsf{Sym}.\mathbf{C}$, with empty "associated data", all-zero nonce $[0]^{96}$, and 256-bit key $\mathsf{K} \in \mathsf{Sym}.\mathbf{K}$. The result is either the plaintext *byte sequence*, or $\perp$ indicating failure to decrypt.

Note: The "IETF" definition of AEAD_CHACHA20_POLY1305 from [RFC-7539] is used; this has a 32-bit block count and a 96-bit nonce, rather than a 64-bit block count and 64-bit nonce as in the original definition of ChaCha20.

5.4.4 Key Agreement And Derivation

5.4.4.1 Sprout Key Agreement

$\mathsf{KA}^{\mathsf{Sprout}}$ is a *key agreement scheme* as specified in § 4.1.4 ***'Key Agreement'*** on p. 23.

It is instantiated as Curve25519 key agreement, described in [Bernstein2006], as follows.

Let $\mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Public}$ and $\mathsf{KA}^{\mathsf{Sprout}}.\mathsf{SharedSecret}$ be the type of Curve25519 *public keys* (i.e. $\mathbb{B}^{\mathbb{Y}^{[32]}}$), and let $\mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Private}$ be the type of Curve25519 secret keys.

Let $\mathsf{Curve25519}(\underline{n}, \underline{q})$ be the result of point multiplication of the Curve25519 *public key* represented by the byte sequence $\underline{q}$ by the Curve25519 secret key represented by the *byte sequence* $\underline{n}$, as defined in [Bernstein2006, section 2].

Let $\mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Base} := \underline{9}$ be the public *byte sequence* representing the Curve25519 base point.

Let $\mathsf{clamp}_{\mathsf{Curve25519}}(\underline{x})$ take a 32-byte sequence $\underline{x}$ as input and return a *byte sequence* representing a Curve25519 *private key*, with bits "clamped" as described in [Bernstein2006, section 3]: "clear bits $0, 1, 2$ of the first byte, clear bit 7 of the last byte, and set bit 6 of the last byte." Here the bits of a byte are numbered such that bit b has numeric weight 2^b.

Define $\mathsf{KA}^{\mathsf{Sprout}}.\mathsf{FormatPrivate}(x) := \mathsf{clamp}_{\mathsf{Curve25519}}(x)$.

Define $\mathsf{KA}^{\mathsf{Sprout}}.\mathsf{DerivePublic}(n, q) := \mathsf{Curve25519}(n, q)$.

Define $\mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Agree}(n, q) := \mathsf{Curve25519}(n, q)$.

5.4.4.2 Sprout Key Derivation

$\mathsf{KDF}^{\mathsf{Sprout}}$ is a *Key Derivation Function* as specified in § 4.1.5 ***'Key Derivation'*** on p. 24.

It is instantiated using BLAKE2b-256 as follows:

$$\mathsf{KDF}^{\mathsf{Sprout}}(i, \mathsf{h_{Sig}}, \mathsf{sharedSecret}_i, \mathsf{epk}, \mathsf{pk}^{\mathsf{new}}_{\mathsf{enc},i}) := \mathsf{BLAKE2b\text{-}256}(\mathsf{kdftag}, \mathsf{kdfinput})$$

where:

$\mathsf{kdftag} :=$

64-bit `"ZcashKDF"`	8-bit $i-1$	$[0]^{56}$

$\mathsf{kdfinput} :=$

256-bit $\mathsf{h_{Sig}}$	256-bit $\mathsf{sharedSecret}_i$	256-bit epk	256-bit $\mathsf{pk}^{\mathsf{new}}_{\mathsf{enc},i}$

.

$\mathsf{BLAKE2b\text{-}256}(p, x)$ is defined in § 5.4.1.2 ***'BLAKE2 Hash Functions'*** on p. 59.

5.4.4.3 Sapling Key Agreement

$\mathsf{KA}^{\mathsf{Sapling}}$ is a *key agreement scheme* as specified in § 4.1.4 *'Key Agreement'* on p. 23.

It is instantiated as Diffie–Hellman with cofactor multiplication on Jubjub as follows:

Let $\mathbb{J}$, $\mathbb{J}^{(r)}$, $\mathbb{J}^{(r)*}$, and the cofactor $h_{\mathbb{J}}$ be as defined in § 5.4.8.3 *'Jubjub'* on p. 77.

Define $\mathsf{KA}^{\mathsf{Sapling}}.\mathsf{Public} := \mathbb{J}$.

Define $\mathsf{KA}^{\mathsf{Sapling}}.\mathsf{PublicPrimeOrder} := \mathbb{J}^{(r)*}$.

Define $\mathsf{KA}^{\mathsf{Sapling}}.\mathsf{SharedSecret} := \mathbb{J}^{(r)}$.

Define $\mathsf{KA}^{\mathsf{Sapling}}.\mathsf{Private} := \mathbb{F}_{r_{\mathbb{J}}}$.

Define $\mathsf{KA}^{\mathsf{Sapling}}.\mathsf{DerivePublic}(\mathsf{sk}, B) := [\mathsf{sk}]\, B$.

Define $\mathsf{KA}^{\mathsf{Sapling}}.\mathsf{Agree}(\mathsf{sk}, P) := [h_{\mathbb{J}} \cdot \mathsf{sk}]\, P$.

5.4.4.4 Sapling Key Derivation

$\mathsf{KDF}^{\mathsf{Sapling}}$ is a *Key Derivation Function* as specified in § 4.1.5 *'Key Derivation'* on p. 24.

It is instantiated using BLAKE2b-256 as follows:

$$\mathsf{KDF}^{\mathsf{Sapling}}(\mathsf{sharedSecret}, \mathtt{ephemeralKey}) := \mathsf{BLAKE2b\text{-}256}(\mathtt{"Zcash_SaplingKDF"}, \mathsf{kdfinput}).$$

where:

$$\mathsf{kdfinput} := \boxed{\mathsf{LEBS2OSP}_{256}\big(\mathsf{repr}_{\mathbb{J}}(\mathsf{sharedSecret})\big) \;\big|\; \mathtt{ephemeralKey}}.$$

$\mathsf{BLAKE2b\text{-}256}(p, x)$ is defined in § 5.4.1.2 *'BLAKE2 Hash Functions'* on p. 59.

5.4.5 Ed25519

Ed25519 is a *signature scheme* as specified in § 4.1.6 *'Signature'* on p. 24. It is used to instantiate JoinSplitSig as described in § 4.10 *'Non-malleability (Sprout)'* on p. 41.

Let $\mathsf{PreCanopyExcludedPointEncodings} : \mathscr{P}\big(\mathbb{B}^{\mathbb{Y}^{[32]}}\big) = \{$

[0x00, 0x00],
[0x01, 0x00],
[0x26, 0xe8, 0x95, 0x8f, 0xc2, 0xb2, 0x27, 0xb0, 0x45, 0xc3, 0xf4, 0x89, 0xf2, 0xef, 0x98, 0xf0, 0xd5, 0xdf, 0xac, 0x05, 0xd3, 0xc6, 0x33, 0x39, 0xb1, 0x38, 0x02, 0x88, 0x6d, 0x53, 0xfc, 0x05],
[0xc7, 0x17, 0x6a, 0x70, 0x3d, 0x4d, 0xd8, 0x4f, 0xba, 0x3c, 0x0b, 0x76, 0x0d, 0x10, 0x67, 0x0f, 0x2a, 0x20, 0x53, 0xfa, 0x2c, 0x39, 0xcc, 0xc6, 0x4e, 0xc7, 0xfd, 0x77, 0x92, 0xac, 0x03, 0x7a],
[0x13, 0xe8, 0x95, 0x8f, 0xc2, 0xb2, 0x27, 0xb0, 0x45, 0xc3, 0xf4, 0x89, 0xf2, 0xef, 0x98, 0xf0, 0xd5, 0xdf, 0xac, 0x05, 0xd3, 0xc6, 0x33, 0x39, 0xb1, 0x38, 0x02, 0x88, 0x6d, 0x53, 0xfc, 0x85],
[0xb4, 0x17, 0x6a, 0x70, 0x3d, 0x4d, 0xd8, 0x4f, 0xba, 0x3c, 0x0b, 0x76, 0x0d, 0x10, 0x67, 0x0f, 0x2a, 0x20, 0x53, 0xfa, 0x2c, 0x39, 0xcc, 0xc6, 0x4e, 0xc7, 0xfd, 0x77, 0x92, 0xac, 0x03, 0xfa],
[0xec, 0xff, 0x7f],
[0xed, 0xff, 0x7f],
[0xee, 0xff, 0x7f],
[0xd9, 0xff],
[0xda, 0xff]

$\}$.

Let $p = 2^{255} - 19$.

Let $a = -1$.

Let $d = -121665/121666 \pmod{p}$.

Let $\ell = 2^{252} + 27742317777372353535851937790883648493$ (the order of the Ed25519 curve's *prime-order subgroup*).

Let B be the base point given in [BDLSY2012].

Define the notation $\sqrt[?]{\bullet}$ as in § 2 *'Notation'* on p. 9.

Define I2LEOSP, LEOS2BSP, and LEBS2IP as in § 5.1 *'Integers, Bit Sequences, and Endianness'* on p. 56.

Define $\mathsf{reprBytes}_{\mathsf{Ed25519}} : \mathsf{Ed25519} \to \mathbb{B}^{\mathbb{Y}[32]}$ such that $\mathsf{reprBytes}_{\mathsf{Ed25519}}((x,y)) = \mathsf{I2LEOSP}_{256}\big((y \bmod p) + 2^{255}\cdot\tilde{x}\big)$, where $\tilde{x} = x \bmod 2$.[7]

Define $\mathsf{abstBytes}_{\mathsf{Ed25519}} : \mathbb{B}^{\mathbb{Y}[32]} \to \mathsf{Ed25519} \cup \{\bot\}$ such that $\mathsf{abstBytes}_{\mathsf{Ed25519}}(\underline{P})$ is computed as follows:

let $y\star : \mathbb{B}^{[255]}$ be the first 255 bits of $\mathsf{LEOS2BSP}_{256}(\underline{P})$ and let $\tilde{x} : \mathbb{B}$ be the last bit.
let $y : \mathbb{F}_p = \mathsf{LEBS2IP}_{255}(y\star) \pmod p$.
let $x = \sqrt[?]{\dfrac{1-y^2}{a-d\cdot y^2}}$. (The denominator $a - d\cdot y^2$ cannot be zero, since $\frac{a}{d}$ is not square in $\mathbb{F}_p$.)
if $x = \bot$, return $\bot$.
if $x \bmod 2 = \tilde{x}$ then return (x, y) else return $(p - x, y)$.

Note: This definition of point decoding differs from that of [RFC-8032, section 5.1.3, as corrected by the errata]. In the latter there is an additional step "`If x = 0, and x_0 = 1, decoding fails.`", which rejects the encodings {
[0x01, 0x00, 0x80],
[0xee, 0xff],
[0xec, 0xff]
}.
In this specification, the first two of these are accepted as encodings of $(0, 1)$, and the third is accepted as an encoding of $(0, -1)$.

Ed25519 is defined as in [BDLSY2012], using SHA-512 as the internal *hash function*, with the additional requirements below. A valid Ed25519 *validating key* is defined as a sequence of 32 bytes encoding a point on the Ed25519 curve.

The requirements on a signature $(\underline{R}, \underline{S})$ with *validating key* $\underline{A}$ on a message M are:

- $\underline{S}$ **MUST** represent an integer less than ℓ.
- $\underline{R}$ and $\underline{A}$ **MUST** be encodings of points R and A respectively on the Ed25519 curve;
- $\underline{R}$ **MUST NOT** be in PreCanopyExcludedPointEncodings;
- The validation equation **MUST** be equivalent to $[S]\,B = R + [c]\,A$.

where c is computed as the integer corresponding to $\mathsf{SHA\text{-}512}(\underline{R} \,||\, \underline{A} \,||\, M)$ as specified in [BDLSY2012].

If these requirements are not met or the validation equation does not hold, then the signature is considered invalid.

The encoding of an Ed25519 signature is:

256-bit $\underline{R}$	256-bit $\underline{S}$

where $\underline{R}$ and $\underline{S}$ are as defined in [BDLSY2012].

Notes:

- It is ***not*** required that the integer encoding of the y-coordinate[7] of the points represented by $\underline{R}$ or $\underline{A}$ are less than $2^{255} - 19$.
- It is ***not*** required that $\underline{A} \notin$ PreCanopyExcludedPointEncodings.

[7] Here we use the (x, y) naming of coordinates in [BDLSY2012], which is different from the (u, v) naming used for coordinates of *ctEdwards curves* in § 5.4.8.3 *'Jubjub'* on p. 77 and in § A.2 *'Elliptic curve background'* on p. 149.

Non-normative note: The exclusion of PreCanopyExcludedPointEncodings from R is due to a quirk of version 1.0.15 of the libsodium library [libsodium] which was initially used to implement Ed25519 signature validation in *zcashd*. (The `ED25519_COMPAT` compile-time option was not set.) The intent was to exclude points of order less than ℓ; however, not all such points were covered.

5.4.6 RedDSA and RedJubjub

RedDSA is a Schnorr-based *signature scheme*, optionally supporting key *re-randomization* as described in § 4.1.6.1 ***'Signature with Re-Randomizable Keys'*** on p. 26. It also supports a Secret Key to Public Key Monomorphism as described in § 4.1.6.2 ***'Signature with Signing Key to Validating Key Monomorphism'*** on p. 27. It is based on a scheme from [FKMSSS2016, section 3], with some ideas from EdDSA [BJLSY2015].

RedJubjub is a specialization of RedDSA to the *Jubjub curve* (§ 5.4.8.3 ***'Jubjub'*** on p. 77), using the BLAKE2b-512 hash function.

The *spend authorization signature scheme* $\mathsf{SpendAuthSig}^{\mathsf{Sapling}}$ is instantiated by RedJubjub, using parameters defined in § 5.4.6.1 ***'Spend Authorization Signature (Sapling)'*** on p. 71.

The *binding signature scheme* $\mathsf{BindingSig}^{\mathsf{Sapling}}$ is instantiated by RedJubjub without key *re-randomization*, using parameters defined in § 5.4.6.2 ***'Binding Signature (Sapling)'*** on p. 71.

Let I2LEBSP, I2LEOSP, LEOS2IP, and LEBS2OSP be as defined in § 5.1 ***'Integers, Bit Sequences, and Endianness'*** on p. 56.

We first describe the scheme RedDSA over a general *represented group*. Its parameters are:

- a *represented group* $\mathbb{G}$, which also defines a subgroup $\mathbb{G}^{(r)}$ of order $r_{\mathbb{G}}$, a cofactor $h_{\mathbb{G}}$, a group operation $+$, an additive identity $\mathcal{O}_{\mathbb{G}}$, a bit-length $\ell_{\mathbb{G}}$, a representation function $\mathsf{repr}_{\mathbb{G}}$, and an abstraction function $\mathsf{abst}_{\mathbb{G}}$, as specified in § 4.1.8 ***'Represented Group'*** on p. 29;
- $\mathcal{P}_{\mathbb{G}}$, a generator of $\mathbb{G}^{(r)}$;
- a bit-length $\ell_{\mathsf{H}} : \mathbb{N}$ such that $2^{\ell_{\mathsf{H}}-128} \geq r_{\mathbb{G}}$ and $\ell_{\mathsf{H}} \bmod 8 = 0$;
- a cryptographic *hash function* $\mathsf{H} : \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}} \to \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{H}}/8]}}$.

Its associated types are defined as follows:

$\mathsf{RedDSA.Message} := \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}}$

$\mathsf{RedDSA.Signature} := \mathbb{B}^{\mathbb{Y}^{[\mathsf{ceiling}(\ell_{\mathbb{G}}/8) + \mathsf{ceiling}(\mathsf{bitlength}(r_{\mathbb{G}})/8)]}}$

$\mathsf{RedDSA.Public} := \mathbb{G}$

$\mathsf{RedDSA.Private} := \mathbb{F}_{r_{\mathbb{G}}}$.

$\mathsf{RedDSA.Random} := \mathbb{F}_{r_{\mathbb{G}}}$.

Define $\mathsf{H}^{\circledast} : \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}} \to \mathbb{F}_{r_{\mathbb{G}}}$ by:

$\mathsf{H}^{\circledast}(B) = \mathsf{LEOS2IP}_{\ell_{\mathsf{H}}}\big(\mathsf{H}(B)\big) \pmod{r_{\mathbb{G}}}$

Define $\mathsf{RedDSA.GenPrivate} : () \xrightarrow{\mathrm{R}} \mathsf{RedDSA.Private}$ as:

Return $\mathsf{sk} \xleftarrow{\mathrm{R}} \mathbb{F}_{r_{\mathbb{G}}}$.

Define $\mathsf{RedDSA.DerivePublic} : \mathsf{RedDSA.Private} \to \mathsf{RedDSA.Public}$ by:

$\mathsf{RedDSA.DerivePublic}(\mathsf{sk}) := [\mathsf{sk}]\,\mathcal{P}_{\mathbb{G}}$.

Define RedDSA.GenRandom : $() \xrightarrow{R}$ RedDSA.Random as:

Choose a *byte sequence* T uniformly at random on $\mathbb{B}^{\mathbb{Y}^{[(\ell_{\mathsf{H}}+128)/8]}}$.

Return $\mathsf{H}^{\circledast}(T)$.

Define $\mathcal{O}_{\mathsf{RedDSA.Random}} := 0 \pmod{r_{\mathbb{G}}}$.

Define RedDSA.RandomizePrivate : RedDSA.Random $\times$ RedDSA.Private $\to$ RedDSA.Private by:

$\mathsf{RedDSA.RandomizePrivate}(\alpha, \mathsf{sk}) := \mathsf{sk} + \alpha \pmod{r_{\mathbb{G}}}$.

Define RedDSA.RandomizePublic : RedDSA.Random $\times$ RedDSA.Public $\to$ RedDSA.Public as:

$\mathsf{RedDSA.RandomizePublic}(\alpha, \mathsf{vk}) := \mathsf{vk} + [\alpha]\,\mathcal{P}_{\mathbb{G}}$.

Define RedDSA.Sign : (sk : RedDSA.Private) $\times$ (M : RedDSA.Message) $\xrightarrow{R}$ RedDSA.Signature as:

Choose a *byte sequence* T uniformly at random on $\mathbb{B}^{\mathbb{Y}^{[(\ell_{\mathsf{H}}+128)/8]}}$.

Let $\underline{\mathsf{vk}} = \mathsf{LEBS2OSP}_{\ell_{\mathbb{G}}}\big(\mathsf{repr}_{\mathbb{G}}(\mathsf{RedDSA.DerivePublic(sk)})\big)$.

Let $r = \mathsf{H}^{\circledast}(T \,||\, \underline{\mathsf{vk}} \,||\, M)$.

Let $R = [r]\,\mathcal{P}_{\mathbb{G}}$.

Let $\underline{R} = \mathsf{LEBS2OSP}_{\ell_{\mathbb{G}}}\big(\mathsf{repr}_{\mathbb{G}}(R)\big)$.

Let $S = (r + \mathsf{H}^{\circledast}(\underline{R} \,||\, \underline{\mathsf{vk}} \,||\, M) \cdot \mathsf{sk}) \bmod r_{\mathbb{G}}$.

Let $\underline{S} = \mathsf{I2LEOSP}_{\mathsf{bitlength}(r_{\mathbb{G}})}(S)$.

Return $\underline{R} \,||\, \underline{S}$.

Define RedDSA.Validate : (vk : RedDSA.Public) $\times$ (M : RedDSA.Message) $\times$ (σ : RedDSA.Signature) $\to \mathbb{B}$ as:

Let $\underline{R}$ be the first $\mathsf{ceiling}\big(\ell_{\mathbb{G}}/8\big)$ bytes of σ, and let $\underline{S}$ be the remaining $\mathsf{ceiling}(\mathsf{bitlength}(r_{\mathbb{G}})/8)$ bytes.

Let $R = \mathsf{abst}_{\mathbb{G}}\big(\mathsf{LEOS2BSP}_{\ell_{\mathbb{G}}}(\underline{R})\big)$, and let $S = \mathsf{LEOS2IP}_{8\cdot\mathsf{length}(\underline{S})}(\underline{S})$.

Let $\underline{\mathsf{vk}} = \mathsf{LEBS2OSP}_{\ell_{\mathbb{G}}}\big(\mathsf{repr}_{\mathbb{G}}(\mathsf{vk})\big)$.

Let $c = \mathsf{H}^{\circledast}(\underline{R} \,||\, \underline{\mathsf{vk}} \,||\, M)$.

Return 1 if $R \neq \bot$ and $S < r_{\mathbb{G}}$ and $[h_{\mathbb{G}}]\,\big(-[S]\,\mathcal{P}_{\mathbb{G}} + R + [c]\,\mathsf{vk}\big) = \mathcal{O}_{\mathbb{G}}$, otherwise 0.

Notes:

- The validation algorithm ***does not*** check that R is a point of order at least $r_{\mathbb{G}}$.
- The value $\underline{R}$ used as part of the input to $\mathsf{H}^{\circledast}$ **MUST** be exactly as encoded in the signature.
- Appendix § B.1 'RedDSA *batch validation*' on p. 169 describes an optimization that **MAY** be used to speed up validation of batches of RedDSA signatures.

Non-normative notes:

- The randomization used in RedDSA.RandomizePrivate and RedDSA.RandomizePublic may interact with other uses of additive properties of keys for Schnorr-based signature schemes. In the **Zcash** protocol, such properties are used for *binding signatures* but not at the same time as key randomization. They are also used in [ZIP-32] when deriving child extended keys, but this does not result in any practical security weakness as long as the security recommendations of ZIP 32 are followed. If RedDSA is reused in other protocols making use of these additive properties, careful analysis of potential interactions is required.
- It is **RECOMMENDED** that, for deployments of RedDSA in other protocols than **Zcash**, the requirement for $\underline{R}$ to be canonically encoded is always enforced (which was the original intent of the design).

The two abelian groups specified in § 4.1.6.2 *'Signature with Signing Key to Validating Key Monomorphism'* on p. 27 are instantiated for RedDSA as follows:

- $\mathcal{O}_{\boxplus} := 0 \pmod{r_{\mathbb{G}}}$
- $\mathsf{sk}_1 \boxplus \mathsf{sk}_2 := \mathsf{sk}_1 + \mathsf{sk}_2 \pmod{r_{\mathbb{G}}}$
- $\mathcal{O}_{\oplus} := \mathcal{O}_{\mathbb{G}}$
- $\mathsf{vk}_1 \oplus \mathsf{vk}_2 := \mathsf{vk}_1 + \mathsf{vk}_2$.

As required, RedDSA.DerivePublic is a group monomorphism, since it is injective and:

$$\begin{aligned}\mathsf{RedDSA.DerivePublic}(\mathsf{sk}_1 \boxplus \mathsf{sk}_2) &= [\mathsf{sk}_1 + \mathsf{sk}_2 \pmod{r_{\mathbb{G}}}]\,\mathcal{P}_{\mathbb{G}} \\ &= [\mathsf{sk}_1]\,\mathcal{P}_{\mathbb{G}} + [\mathsf{sk}_2]\,\mathcal{P}_{\mathbb{G}} \text{ (since } \mathcal{P}_{\mathbb{G}} \text{ has order } r_{\mathbb{G}}) \\ &= \mathsf{RedDSA.DerivePublic}(\mathsf{sk}_1) \oplus \mathsf{RedDSA.DerivePublic}(\mathsf{sk}_2).\end{aligned}$$

A RedDSA *validating key* vk can be encoded as a *bit sequence* $\mathsf{repr}_{\mathbb{G}}(\mathsf{vk})$ of length $\ell_{\mathbb{G}}$ bits (or as a corresponding *byte sequence* $\underline{\mathsf{vk}}$ by then applying $\mathsf{LEBS2OSP}_{\ell_{\mathbb{G}}}$).

The scheme RedJubjub specializes RedDSA with:

- $\mathbb{G} := \mathbb{J}$ as defined in § 5.4.8.3 *'Jubjub'* on p. 77;
- $\ell_{\mathsf{H}} := 512$;
- $\mathsf{H}(x) := \mathsf{BLAKE2b\text{-}512}(\texttt{"Zcash_RedJubjubH"}, x)$ as defined in § 5.4.1.2 *'BLAKE2 Hash Functions'* on p. 59.

The generator $\mathcal{P}_{\mathbb{G}} : \mathbb{G}^{(r)}$ is left as an unspecified parameter, different between $\mathsf{BindingSig}^{\mathsf{Sapling}}$ and $\mathsf{SpendAuthSig}^{\mathsf{Sapling}}$.

5.4.6.1 Spend Authorization Signature (Sapling)

Let RedJubjub be as defined in § 5.4.6 'RedDSA *and* RedJubjub' on p. 69.

Define $\mathcal{G}^{\mathsf{Sapling}} := \mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}(\texttt{"Zcash_G_"}, \texttt{""})$.

The *spend authorization signature scheme* $\mathsf{SpendAuthSig}^{\mathsf{Sapling}}$ is instantiated as RedJubjub with key *re-randomization* and with generator $\mathcal{P}_{\mathbb{G}} = \mathcal{G}^{\mathsf{Sapling}}$.

See § 4.13 *'Spend Authorization Signature (Sapling)'* on p. 44 for details on the use of this *signature scheme*.

Security requirement: SpendAuthSig must be a SURK-CMA-secure *signature scheme with re-randomizable keys* as defined in § 4.1.6.1 *'Signature with Re-Randomizable Keys'* on p. 26.

5.4.6.2 Binding Signature (Sapling)

Let RedJubjub be as defined in § 5.4.6 'RedDSA *and* RedJubjub' on p. 69.

The Sapling *binding signature scheme*, $\mathsf{BindingSig}^{\mathsf{Sapling}}$, is instantiated as RedJubjub without key *re-randomization*, using generator $\mathcal{P}_{\mathbb{G}} = \mathcal{R}^{\mathsf{Sapling}}$ defined in § 5.4.7.3 *'Homomorphic Pedersen commitments (Sapling)'* on p. 73. See § 4.12 *'Balance and Binding Signature (Sapling)'* on p. 42 for details on the use of this *signature scheme*.

Security requirement: BindingSig must be a SUF-CMA-secure *signature scheme with key monomorphism* as defined in § 4.1.6.2 *'Signature with Signing Key to Validating Key Monomorphism'* on p. 27. A signature must prove knowledge of the *discrete logarithm* of the *validating key* with respect to the base $\mathcal{R}^{\mathsf{Sapling}}$.

5.4.7 Commitment schemes

5.4.7.1 Note Commitments (Sprout)

The *note commitment scheme* $\mathsf{NoteCommit}^{\mathsf{Sprout}}$ specified in § 4.1.7 *'Commitment'* on p. 27 is instantiated using SHA-256 as follows:

$$\mathsf{NoteCommit}^{\mathsf{Sprout}}_{\mathsf{rcm}}(\mathsf{a_{pk}}, \mathsf{v}, \rho) := \mathsf{SHA\text{-}256}\left(\boxed{1}\boxed{0}\boxed{1}\boxed{1}\boxed{0}\boxed{0}\boxed{0}\boxed{0}\boxed{\text{256-bit } \mathsf{a_{pk}}}\boxed{\text{64-bit } \mathsf{v}}\boxed{\text{256-bit } \rho}\boxed{\text{256-bit } \mathsf{rcm}}\right)$$

$\mathsf{NoteCommit}^{\mathsf{Sprout}}.\mathsf{GenTrapdoor}()$ generates the uniform distribution on $\mathsf{NoteCommit}^{\mathsf{Sprout}}.\mathsf{Trapdoor}$.

Note: The leading byte of the SHA-256 input is `0xB0`.

Security requirements:

- SHA256Compress must be *collision-resistant*.
- SHA256Compress must be a *PRF* when keyed by the bits corresponding to the position of rcm in the second block of SHA-256 input, with input to the *PRF* in the remaining bits of the block and the chaining variable.

5.4.7.2 Windowed Pedersen commitments

§ 5.4.1.7 *'Pedersen Hash Function'* on p. 62 defines a *Pedersen hash* construction. We construct "*windowed*" *Pedersen commitments* by reusing that construction, and adding a randomized point on the *Jubjub curve* (see § 5.4.8.3 *'Jubjub'* on p. 77):

$$\mathsf{WindowedPedersenCommit}_r(s) := \mathsf{PedersenHashToPoint}(\texttt{"Zcash_PH"}, s) + [r]\,\mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}(\texttt{"Zcash_PH"}, \texttt{"r"})$$

See § A.3.5 *'Windowed Pedersen Commitment'* on p. 162 for rationale and efficient circuit implementation of this function.

The *note commitment scheme* $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ specified in § 4.1.7 *'Commitment'* on p. 27 is instantiated as follows using WindowedPedersenCommit:

$$\mathsf{NoteCommit}^{\mathsf{Sapling}}_{\mathsf{rcm}}(\mathsf{g\star_d}, \mathsf{pk\star_d}, \mathsf{v}) := \mathsf{WindowedPedersenCommit}_{\mathsf{rcm}}\left([1]^6 \,||\, \mathsf{I2LEBSP}_{64}(\mathsf{v}) \,||\, \mathsf{g\star_d} \,||\, \mathsf{pk\star_d}\right)$$

$\mathsf{NoteCommit}^{\mathsf{Sapling}}.\mathsf{GenTrapdoor}()$ generates the uniform distribution on $\mathbb{F}_{r_{\mathbb{J}}}$.

Security requirements:

- WindowedPedersenCommit, and hence $\mathsf{NoteCommit}^{\mathsf{Sapling}}$, must be computationally *binding* and at least computationally *hiding commitment schemes*.

(They are in fact unconditionally *hiding commitment schemes*.)

Notes:

- $\mathsf{MerkleCRH}^{\mathsf{Sapling}}$ is also defined in terms of PedersenHashToPoint (see § 5.4.1.3 *'$\mathsf{MerkleCRH}^{\mathsf{Sapling}}$ Hash Function'* on p. 60). The prefix $[1]^6$ distinguishes the use of WindowedPedersenCommit in $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ from the layer prefix used in $\mathsf{MerkleCRH}^{\mathsf{Sapling}}$. That layer prefix is a 6-bit *little-endian* encoding of an integer in the range $\{0 \,..\, \mathsf{MerkleDepth}^{\mathsf{Sapling}} - 1\}$; because $\mathsf{MerkleDepth}^{\mathsf{Sapling}} < 64$, it cannot collide with $[1]^6$.
- The arguments to $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ are in a different order to their encodings in WindowedPedersenCommit. There is no particularly good reason for this.

Theorem 5.4.2. $\mathsf{Uncommitted}^{\mathsf{Sapling}}$ *is not in the range of* $\mathsf{NoteCommit}^{\mathsf{Sapling}}$.

Proof. $\mathsf{Uncommitted}^{\mathsf{Sapling}}$ is defined as $\mathsf{I2LEBSP}_{\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}}(1)$. By injectivity of $\mathsf{I2LEBSP}_{\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}}$ and definitions of $\mathsf{Extract}_{\mathbb{J}^{(r)}}$, $\mathsf{WindowedPedersenCommit}$, and $\mathsf{NoteCommit}^{\mathsf{Sapling}}$, $\mathsf{I2LEBSP}_{\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}}(1)$ can be in the range of $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ only if there exist $\mathsf{rcm} : \mathsf{NoteCommit}^{\mathsf{Sapling}}.\mathsf{Trapdoor}$, $D : \mathbb{B}^{\mathbb{Y}^{[8]}}$, and $M : \mathbb{B}^{[\mathbb{N}^+]}$ such that $\mathcal{U}(\mathsf{WindowedPedersenCommit}_{\mathsf{rcm}}(D, M)) = 1$. The latter can only be the *affine-ctEdwards* u-coordinate of a point in $\mathbb{J}$. We show that there are no points in $\mathbb{J}$ with *affine-ctEdwards* u-coordinate 1. Suppose for a contradiction that $(u, v) \in \mathbb{J}$ for $u = 1$ and some $v : \mathbb{F}_{r_{\mathbb{S}}}$. By writing the curve equation as $v^2 = (1 - a_{\mathbb{J}} \cdot u^2)/(1 - d_{\mathbb{J}} \cdot u^2)$, and noting that $1 - d_{\mathbb{J}} \cdot u^2 \neq 0$ because $d_{\mathbb{J}}$ is nonsquare, we have $v^2 = (1 - a_{\mathbb{J}})/(1 - d_{\mathbb{J}})$. The right-hand-side is a nonsquare in $\mathbb{F}_{r_{\mathbb{S}}}$ (for the *Jubjub curve* parameters), so there are no solutions for v (contradiction). □

5.4.7.3 Homomorphic Pedersen commitments (Sapling)

The windowed Pedersen commitments defined in the preceding section are highly efficient, but they do not support the homomorphic property we need when instantiating ValueCommit.

For more details on the use of this property, see § 4.12 *'Balance and Binding Signature (Sapling)'* on p. 42.

Useful background is given in § 3.6 *'Spend Transfers, Output Transfers, and their Descriptions'* on p. 18.

In order to support this property, we also define *homomorphic Pedersen commitments* for Sapling:

$\mathsf{HomomorphicPedersenCommit}^{\mathsf{Sapling}}_{\mathsf{rcv}}(D, \mathsf{v}) := [\mathsf{v}]\,\mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}(D, \texttt{"v"}) + [\mathsf{rcv}]\,\mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}(D, \texttt{"r"})$

$\mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{GenTrapdoor}()$ generates the uniform distribution on $\mathbb{F}_{r_{\mathbb{J}}}$.

See § A.3.6 *'Homomorphic Pedersen Commitment'* on p. 162 for rationale and efficient circuit implementation of this function.

Define:

$\mathcal{V}^{\mathsf{Sapling}} := \mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}(\texttt{"Zcash_cv"}, \texttt{"v"})$

$\mathcal{R}^{\mathsf{Sapling}} := \mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}(\texttt{"Zcash_cv"}, \texttt{"r"})$

The *commitment scheme* $\mathsf{ValueCommit}^{\mathsf{Sapling}}$ specified in § 4.1.7 *'Commitment'* on p. 27 is instantiated as follows using $\mathsf{HomomorphicPedersenCommit}^{\mathsf{Sapling}}$ on the *Jubjub curve*:

$\mathsf{ValueCommit}^{\mathsf{rcv}}(\mathsf{v}) := \mathsf{HomomorphicPedersenCommit}^{\mathsf{Sapling}}_{\mathsf{rcv}}(\texttt{"Zcash_cv"}, \mathsf{v})$.

which is equivalent to:

$\mathsf{ValueCommit}^{\mathsf{Sapling}}_{\mathsf{rcv}}(\mathsf{v}) := [\mathsf{v}]\,\mathcal{V}^{\mathsf{Sapling}} + [\mathsf{rcv}]\,\mathcal{R}^{\mathsf{Sapling}}$.

Security requirements:

- $\mathsf{HomomorphicPedersenCommit}^{\mathsf{Sapling}}$ must be a computationally *binding* and at least computationally *hiding commitment scheme*, for a given personalization input D.
- $\mathsf{ValueCommit}^{\mathsf{Sapling}}$ must be a computationally *binding* and at least computationally *hiding commitment scheme*.

(They are in fact unconditionally *hiding commitment schemes*.)

Non-normative note: The output of $\mathsf{HomomorphicPedersenCommit}^{\mathsf{Sapling}}$ may (with negligible probability for a randomly chosen *commitment trapdoor*) be the zero point of the curve, $\mathcal{O}_{\mathbb{J}}$. This would be rejected by consensus if it appeared as the cv field of a *Spend description* (§ 4.4 *'Spend Descriptions'* on p. 35) or *Output description* (§ 4.5 *'Output Descriptions'* on p. 36). An implementation of $\mathsf{HomomorphicPedersenCommit}^{\mathsf{Sapling}}$ **MAY** resample the *commitment trapdoor* until the resulting commitment is not $\mathcal{O}_{\mathbb{J}}$.

5.4.8 Represented Groups and Pairings

5.4.8.1 BN-254

The *represented pairing* BN-254 is defined in this section.

Let $q_{\mathbb{G}} := 21888242871839275222246405745257275088696311157297823662689037894645226208583$.

Let $r_{\mathbb{G}} := 21888242871839275222246405745257275088548364400416034343698204186575808495617$.

Let $b_{\mathbb{G}} := 3$.

($q_{\mathbb{G}}$ and $r_{\mathbb{G}}$ are prime.)

Let $\mathbb{G}_1^{(r)}$ be the group (of order $r_{\mathbb{G}}$) of rational points on a Barreto–Naehrig ([BN2005]) curve $E_{\mathbb{G}_1}$ over $\mathbb{F}_{q_{\mathbb{G}}}$ with equation $y^2 = x^3 + b_{\mathbb{G}}$. This curve has embedding degree 12 with respect to $r_{\mathbb{G}}$.

Let $\mathbb{G}_2^{(r)}$ be the subgroup of order $r_{\mathbb{G}}$ in the sextic twist $E_{\mathbb{G}_2}$ of $E_{\mathbb{G}_1}$ over $\mathbb{F}_{q_{\mathbb{G}}{}^2}$ with equation $y^2 = x^3 + \frac{b_{\mathbb{G}}}{\xi}$, where $\xi : \mathbb{F}_{q_{\mathbb{G}}{}^2}$.

We represent elements of $\mathbb{F}_{q_{\mathbb{G}}{}^2}$ as polynomials $a_1 \cdot t + a_0 : \mathbb{F}_{q_{\mathbb{G}}}[t]$, modulo the irreducible polynomial $t^2 + 1$; in this representation, ξ is given by $t + 9$.

Let $\mathbb{G}_T^{(r)}$ be the subgroup of $r_{\mathbb{G}}{}^{\text{th}}$ roots of unity in $\mathbb{F}^*_{q_{\mathbb{G}}{}^{12}}$, with multiplicative identity $\mathbf{1}_{\mathbb{G}}$.

Let $\hat{e}_{\mathbb{G}}$ be the optimal ate pairing (see [Vercauter2009] and [AKLGL2010, section 2]) of type $\mathbb{G}_1^{(r)} \times \mathbb{G}_2^{(r)} \to \mathbb{G}_T^{(r)}$.

For $i : \{1 \,..\, 2\}$, let $\mathcal{O}_{\mathbb{G}_i}$ be the point at infinity (which is the additive identity) in $\mathbb{G}_i^{(r)}$, and let $\mathbb{G}_i^{(r)*} := \mathbb{G}_i^{(r)} \setminus \{\mathcal{O}_{\mathbb{G}_i}\}$.

Let $\mathcal{P}_{\mathbb{G}_1} : \mathbb{G}_1^{(r)*} := (1, 2)$.

Let $\mathcal{P}_{\mathbb{G}_2} : \mathbb{G}_2^{(r)*} := (11559732032986387107991004021392285783925812861821192530917403151452391805634 \cdot t +$
$10857046999023057135944570762232829481370756359578518086990519993285655852781,$
$40823678758634336813322034031454355683168513275934012081057410762141200935 31 \cdot t +$
$84956539231234314176049732474892724384181905872636001487702806493069581019 30)$.

$\mathcal{P}_{\mathbb{G}_1}$ and $\mathcal{P}_{\mathbb{G}_2}$ are generators of $\mathbb{G}_1^{(r)}$ and $\mathbb{G}_2^{(r)}$ respectively.

Define I2BEBSP $: (\ell : \mathbb{N}) \times \{0 \,..\, 2^\ell - 1\} \to \mathbb{B}^{[\ell]}$ as in § 5.1 ***'Integers, Bit Sequences, and Endianness'*** on p. 56.

For a point $P : \mathbb{G}_1^{(r)*} = (x_P, y_P)$:

- The field elements x_P and $y_P : \mathbb{F}_q$ are represented as integers x and $y : \{0 \,..\, q-1\}$.
- Let $\tilde{y} = y \bmod 2$.
- P is encoded as

0	0	0	0	0	0	0	1	1-bit $\tilde{y}$	256-bit I2BEBSP$_{256}(x)$

.

For a point $P : \mathbb{G}_2^{(r)*} = (x_P, y_P)$:

- Define FE2IP $: \mathbb{F}_{q_{\mathbb{G}}}[t]/(t^2 + 1) \to \{0 \,..\, q_{\mathbb{G}}{}^2 - 1\}$ such that FE2IP$(a_{w,1} \cdot t + a_{w,0}) = a_{w,1} \cdot q + a_{w,0}$.
- Let $x =$ FE2IP(x_P), $y =$ FE2IP(y_P), and $y' =$ FE2IP$(-y_P)$.
- Let $\tilde{y} = \begin{cases} 1, & \text{if } y > y' \\ 0, & \text{otherwise.} \end{cases}$
- P is encoded as

0	0	0	0	1	0	1	1-bit $\tilde{y}$	512-bit I2BEBSP$_{512}(x)$	

.

Non-normative notes:

- Only the $r_{\mathbb{G}}$-order subgroups $\mathbb{G}_{2,T}^{(r)}$ are used in the protocol, not their containing groups $\mathbb{G}_{2,T}$. Points in $\mathbb{G}_2^{(r)*}$ are ***always*** checked to be of order $r_{\mathbb{G}}$ when decoding from external representation. (The group of rational points $\mathbb{G}_1$ on $E_{\mathbb{G}_1}/\mathbb{F}_{q_{\mathbb{G}}}$ is of order $r_{\mathbb{G}}$ so no subgroup checks are needed in that case, and elements of $\mathbb{G}_T^{(r)}$ are never represented externally.) The (r) superscripts on $\mathbb{G}_{1,2,T}^{(r)}$ are used for consistency with notation elsewhere in this specification.
- The points at infinity $\mathcal{O}_{\mathbb{G}_{1,2}}$ never occur in proofs and have no defined encodings in this protocol.
- A rational point $P \neq \mathcal{O}_{\mathbb{G}_2}$ on the curve $E_{\mathbb{G}_2}$ can be verified to be of order $r_{\mathbb{G}}$, and therefore in $\mathbb{G}_2^{(r)*}$, by checking that $r_{\mathbb{G}} \cdot P = \mathcal{O}_{\mathbb{G}_2}$.
- The use of *big-endian* order by I2BEBSP is different from the encoding of most other integers in this protocol. The encodings for $\mathbb{G}_{1,2}^{(r)*}$ are consistent with the definition of EC2OSP for compressed curve points in [IEEE2004, section 5.5.6.2]. The LSB compressed form (i.e. EC2OSP-XL) is used for points in $\mathbb{G}_1^{(r)*}$, and the SORT compressed form (i.e. EC2OSP-XS) for points in $\mathbb{G}_2^{(r)*}$.
- Testing $y > y'$ for the compression of $\mathbb{G}_2^{(r)*}$ points is equivalent to testing whether $(a_{y,1}, a_{y,0}) > (a_{-y,1}, a_{-y,0})$ in lexicographic order.
- Algorithms for decompressing points from the above encodings are given in [IEEE2000, Appendix A.12.8] for $\mathbb{G}_1^{(r)*}$, and [IEEE2004, Appendix A.12.11] for $\mathbb{G}_2^{(r)*}$.

When computing square roots in $\mathbb{F}_{q_{\mathbb{G}}}$ or $\mathbb{F}_{q_{\mathbb{G}}{}^2}$ in order to decompress a point encoding, the implementation **MUST NOT** assume that the square root exists, or that the encoding represents a point on the curve.

5.4.8.2 BLS12-381

The *represented pairing* BLS12-381 is defined in this section. Parameters are taken from [Bowe2017].

Let $q_{\mathbb{S}} := 4002409555221667393417789825735904156556882819939007885332058136124031650490837864442687629129015664037894272559787$.

Let $r_{\mathbb{S}} := 52435875175126190479447740508185965837690552500527637822603658699938581184513$.

Let $u_{\mathbb{S}} := -15132376222941642752$.

Let $b_{\mathbb{S}} := 4$.

($q_{\mathbb{S}}$ and $r_{\mathbb{S}}$ are prime.)

Let $\mathbb{S}_1^{(r)}$ be the subgroup of order $r_{\mathbb{S}}$ of the group of rational points on a Barreto–Lynn–Scott ([BLS2002]) curve $E_{\mathbb{S}_1}$ over $\mathbb{F}_{q_{\mathbb{S}}}$ with equation $y^2 = x^3 + b_{\mathbb{S}}$. This curve has embedding degree 12 with respect to $r_{\mathbb{S}}$.

Let $\mathbb{S}_2^{(r)}$ be the subgroup of order $r_{\mathbb{S}}$ in the sextic twist $E_{\mathbb{S}_2}$ of $E_{\mathbb{S}_1}$ over $\mathbb{F}_{q_{\mathbb{S}}{}^2}$ with equation $y^2 = x^3 + 4(i+1)$, where $i : \mathbb{F}_{q_{\mathbb{S}}{}^2}$.

We represent elements of $\mathbb{F}_{q_{\mathbb{S}}{}^2}$ as polynomials $a_1 \cdot t + a_0 : \mathbb{F}_{q_{\mathbb{S}}}[t]$, modulo the irreducible polynomial $t^2 + 1$; in this representation, i is given by t.

Let $\mathbb{S}_T^{(r)}$ be the subgroup of $r_{\mathbb{S}}{}^{\text{th}}$ roots of unity in $\mathbb{F}^*_{q_{\mathbb{S}}{}^{12}}$, with multiplicative identity $\mathbf{1}_{\mathbb{S}}$.

Let $\hat{e}_{\mathbb{S}}$ be the optimal ate pairing of type $\mathbb{S}_1^{(r)} \times \mathbb{S}_2^{(r)} \to \mathbb{S}_T^{(r)}$.

For $i : \{1 \,..\, 2\}$, let $\mathcal{O}_{\mathbb{S}_i}$ be the point at infinity in $\mathbb{S}_i^{(r)}$, and let $\mathbb{S}_i^{(r)*} := \mathbb{S}_i^{(r)} \setminus \{\mathcal{O}_{\mathbb{S}_i}\}$.

Let $\mathcal{P}_{\mathbb{S}_1} : \mathbb{S}_1^{(r)*} :=$

$(36854167537133870167810883151830777579616207957825464098945783786886075923783763188360549476763458215481041854645 07,$
$1339506544944476473020471379941921221584933875938349620426543736416511423956333506472724655353366534992391756441569).$

Let $\mathcal{P}_{\mathbb{S}_2} : \mathbb{S}_2^{(r)*} :=$

$(30591443442442137099712598147537816369864703254766475586593732062916353247689584324335095631043470178378857633657 58 \cdot t +$
$3527010695874666181871391160110601448900299527927752402199086442397937857357150268733476003438651759527619263031 60,$
$9275536654923324557472019657760378807577401934535929700250279787939768770026755649809492897279575655754333442195 82 \cdot t +$
$1985150602287291935568054521177171638300868978215655730859378665066344726373823718423869104263333984641494340347905).$

$\mathcal{P}_{\mathbb{S}_1}$ and $\mathcal{P}_{\mathbb{S}_2}$ are generators of $\mathbb{S}_1^{(r)}$ and $\mathbb{S}_2^{(r)}$ respectively.

Define I2BEBSP $: (\ell : \mathbb{N}) \times \{0 .. 2^\ell - 1\} \to \mathbb{B}^{[\ell]}$ as in § 5.1 ***'Integers, Bit Sequences, and Endianness'*** on p. 56.

For a point $P : \mathbb{S}_1^{(r)*} = (x_P, y_P)$:

- The field elements x_P and $y_P : \mathbb{F}_{q_\mathbb{S}}$ are represented as integers x and $y : \{0 .. q_\mathbb{S} - 1\}$.
- Let $\tilde{y} = \begin{cases} 1, & \text{if } y > q_\mathbb{S} - y \\ 0, & \text{otherwise.} \end{cases}$
- P is encoded as

1	0	1-bit $\tilde{y}$	381-bit I2BEBSP$_{381}(x)$

.

For a point $P : \mathbb{S}_2^{(r)*} = (x_P, y_P)$:

- Define FE2IPP $: \mathbb{F}_{q_\mathbb{S}}[t]/(t^2 + 1) \to \{0 .. q_\mathbb{S} - 1\}^{[2]}$ such that FE2IPP$(a_{w,1} \cdot t + a_{w,0}) = [a_{w,1}, a_{w,0}]$.
- Let $x =$ FE2IPP(x_P), $y =$ FE2IPP(y_P), and $y' =$ FE2IPP$(-y_P)$.
- Let $\tilde{y} = \begin{cases} 1, & \text{if } y > y' \text{ lexicographically} \\ 0, & \text{otherwise.} \end{cases}$
- P is encoded as

1	0	1-bit $\tilde{y}$	381-bit I2BEBSP$_{381}(x_1)$	384-bit I2BEBSP$_{384}(x_2)$

.

Non-normative notes:

- Only the $r_\mathbb{S}$-order subgroups $\mathbb{S}_{1,2,T}^{(r)}$ are used in the protocol, not their containing groups $\mathbb{S}_{1,2,T}$. Points in $\mathbb{S}_{1,2}^{(r)*}$ are ***always*** checked to be of order $r_\mathbb{S}$ when decoding from external representation. (Elements of $\mathbb{S}_T^{(r)}$ are never represented externally.) The (r) superscripts on $\mathbb{S}_{1,2,T}^{(r)}$ are used for consistency with notation elsewhere in this specification.
- The points at infinity $\mathcal{O}_{\mathbb{S}_{1,2}}$ never occur in proofs and have no defined encodings in this protocol.
- In contrast to the corresponding BN-254 curve, $E_{\mathbb{S}_1}$ over $\mathbb{F}_{q_\mathbb{S}}$ is ***not*** a *prime-order curve*.
- A rational point $P \neq \mathcal{O}_{\mathbb{S}_i}$ on the curve $E_{\mathbb{S}_i}$ for $i \in \{1, 2\}$ can be verified to be of order $r_\mathbb{S}$, and therefore in $\mathbb{S}_i^{(r)*}$, by checking that $r_\mathbb{S} \cdot P = \mathcal{O}_{\mathbb{S}_i}$.
- The use of *big-endian* order by I2BEBSP is different from the encoding of most other integers in this protocol.
- The encodings for $\mathbb{S}_{1,2}^{(r)*}$ are specific to **Zcash**.
- Algorithms for decompressing points from the encodings of $\mathbb{S}_{1,2}^{(r)*}$ are defined analogously to those for $\mathbb{G}_{1,2}^{(r)*}$ in § 5.4.8.1 ***'BN-254'*** on p. 74, taking into account that the SORT compressed form (not the LSB compressed form) is used for $\mathbb{S}_1^{(r)*}$.

When computing square roots in $\mathbb{F}_{q_\mathbb{S}}$ or $\mathbb{F}_{q_\mathbb{S}^2}$ in order to decompress a point encoding, the implementation **MUST NOT** assume that the square root exists, or that the encoding represents a point on the curve.

5.4.8.3 Jubjub

"You boil it in sawdust: you salt it in glue:
You condense it with locusts and tape:
Still keeping one principal object in view—
To preserve its symmetrical shape."

— Lewis Carroll, "The Hunting of the Snark" [Carroll1876]

Sapling uses an elliptic curve, Jubjub, designed to be efficiently implementable in *zk-SNARK circuits*. The *represented group* $\mathbb{J}$ of points on this curve is defined in this section.

A *complete twisted Edwards elliptic curve*, as defined in [BL2017, section 4.3.4], is an elliptic curve E over a non-binary field $\mathbb{F}_q$, parameterized by distinct $a, d : \mathbb{F}_q \setminus \{0\}$ such that a is square and d is nonsquare, with equation $E : a \cdot u^2 + v^2 = 1 + d \cdot u^2 \cdot v^2$. We use the abbreviation "*ctEdwards*" to refer to *complete twisted Edwards elliptic curves* and coordinates.

Let $q_{\mathbb{J}} := r_{\mathbb{S}}$, as defined in § 5.4.8.2 *'BLS12-381'* on p. 75.

Let $r_{\mathbb{J}} := 6554484396890773809930967563523245729705921265872317281365359162392183254199$.

($q_{\mathbb{J}}$ and $r_{\mathbb{J}}$ are prime.)

Let $h_{\mathbb{J}} := 8$.

Let $a_{\mathbb{J}} := -1$.

Let $d_{\mathbb{J}} := -10240/10241 \pmod{q_{\mathbb{J}}}$.

Let $\mathbb{J}$ be the group of points (u, v) on a *ctEdwards curve* $E_{\mathbb{J}}$ over $\mathbb{F}_{q_{\mathbb{J}}}$ with equation $a_{\mathbb{J}} \cdot u^2 + v^2 = 1 + d_{\mathbb{J}} \cdot u^2 \cdot v^2$. The zero point with coordinates $(0, 1)$ is denoted $\mathcal{O}_{\mathbb{J}}$. $\mathbb{J}$ has order $h_{\mathbb{J}} \cdot r_{\mathbb{J}}$.

Let $\ell_{\mathbb{J}} := 256$.

Define the notation $\sqrt[?]{\bullet}$ as in § 2 *'Notation'* on p. 9.

Define $\mathsf{I2LEBSP} : (\ell : \mathbb{N}) \times \{0 \,..\, 2^\ell - 1\} \to \mathbb{B}^{[\ell]}$ as in § 5.1 *'Integers, Bit Sequences, and Endianness'* on p. 56, and similarly for $\mathsf{LEBS2IP} : (\ell : \mathbb{N}) \times \mathbb{B}^{[\ell]} \to \{0 \,..\, 2^\ell - 1\}$.

Define $\mathsf{repr}_{\mathbb{J}} : \mathbb{J} \to \mathbb{B}^{[\ell_{\mathbb{J}}]}$ such that $\mathsf{repr}_{\mathbb{J}}\big((u, v)\big) = \mathsf{I2LEBSP}_{256}\big((v \bmod q_{\mathbb{J}}) + 2^{255} \cdot \tilde{u}\big)$, where $\tilde{u} = u \bmod 2$.

Define $\mathsf{abst}_{\mathbb{J}} : \mathbb{B}^{[\ell_{\mathbb{J}}]} \to \mathbb{J} \cup \{\bot\}$ such that $\mathsf{abst}_{\mathbb{J}}(P\star)$ is computed as follows:

- let $v\star : \mathbb{B}^{[255]}$ be the first 255 bits of $P\star$ and let $\tilde{u} : \mathbb{B}$ be the last bit.
- if $\mathsf{LEBS2IP}_{255}(v\star) \geq q_{\mathbb{J}}$ then return $\bot$, otherwise let $v : \mathbb{F}_{q_{\mathbb{J}}} = \mathsf{LEBS2IP}_{255}(v\star) \pmod{q_{\mathbb{J}}}$.
- let $u = \sqrt[?]{\dfrac{1 - v^2}{a_{\mathbb{J}} - d_{\mathbb{J}} \cdot v^2}}$. (The denominator $a_{\mathbb{J}} - d_{\mathbb{J}} \cdot v^2$ cannot be zero, since $\frac{a_{\mathbb{J}}}{d_{\mathbb{J}}}$ is not square in $\mathbb{F}_{q_{\mathbb{J}}}$.)
- if $u = \bot$, return $\bot$.
- if $u \bmod 2 = \tilde{u}$ then return (u, v) else return $(q_{\mathbb{J}} - u, v)$.

Note: In earlier versions of this specification, $\mathsf{abst}_{\mathbb{J}}$ was defined as the left inverse of $\mathsf{repr}_{\mathbb{J}}$ such that if S is not in the range of $\mathsf{repr}_{\mathbb{J}}$, then $\mathsf{abst}_{\mathbb{J}}(S) = \bot$. This differs from the specification above:

- Previously, $\mathsf{abst}_{\mathbb{J}}\Big(\mathsf{I2LEBSP}_{256}\big(2^{255} + 1\big)\Big)$ and $\mathsf{abst}_{\mathbb{J}}\Big(\mathsf{I2LEBSP}_{256}\big(2^{255} + q_{\mathbb{J}} - 1\big)\Big)$ were defined as $\bot$.
- In the current specification, $\mathsf{abst}_{\mathbb{J}}\Big(\mathsf{I2LEBSP}_{256}\big(2^{255} + 1\big)\Big) = \mathsf{abst}_{\mathbb{J}}\big(\mathsf{I2LEBSP}_{256}(1)\big) = (0, 1) = \mathcal{O}_{\mathbb{J}}$, and also $\mathsf{abst}_{\mathbb{J}}\Big(\mathsf{I2LEBSP}_{256}\big(2^{255} + q_{\mathbb{J}} - 1\big)\Big) = \mathsf{abst}_{\mathbb{J}}\big(\mathsf{I2LEBSP}_{256}\big(q_{\mathbb{J}} - 1\big)\big) = (0, -1)$.

Define $\mathbb{J}^{(r)}$ as the order-$r_{\mathbb{J}}$ subgroup of $\mathbb{J}$. Note that this includes $\mathcal{O}_{\mathbb{J}}$. For the set of points of order $r_{\mathbb{J}}$ (which excludes $\mathcal{O}_{\mathbb{J}}$), we write $\mathbb{J}^{(r)*}$.

Define $\mathbb{J}^{(r)}_{\star} := \left\{ \mathsf{repr}_{\mathbb{J}}(P) : \mathbb{B}^{[\ell_{\mathbb{J}}]} \mid P \in \mathbb{J}^{(r)} \right\}$.

Non-normative notes:

- The *ctEdwards compressed encoding* used here is consistent with that used in EdDSA [BJLSY2015] for *validating keys* and the R element of a signature.
- [BJLSY2015, "Encoding and parsing curve points"] gives algorithms for decompressing points from the encoding of $\mathbb{J}$.
- [BJLSY2015, "Encoding and parsing integers"] describes several possibilities for parsing of integers; the specification of $\mathsf{abst}_{\mathbb{J}}$ above requires "strict" parsing.

When computing square roots in $\mathbb{F}_{q_{\mathbb{J}}}$ in order to decompress a point encoding, the implementation **MUST NOT** assume that the square root exists, or that the encoding represents a point on the curve.

Note that algorithms elsewhere in this specification that use Jubjub may impose other conditions on points, for example that they have order at least $r_{\mathbb{J}}$.

5.4.8.4 Coordinate Extractor for Jubjub

Let $\mathcal{U}\big((u, v)\big) = u$ and let $\mathcal{V}\big((u, v)\big) = v$.

Define $\mathsf{Extract}_{\mathbb{J}^{(r)}} : \mathbb{J}^{(r)} \to \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]}$ by

$$\mathsf{Extract}_{\mathbb{J}^{(r)}}(P) := \mathsf{I2LEBSP}_{\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}}\big(\mathcal{U}(P)\big).$$

Facts: The point $(0, 1) = \mathcal{O}_{\mathbb{J}}$, and the point $(0, -1)$ has order 2 in $\mathbb{J}$. $\mathbb{J}^{(r)}$ is of odd-prime order.

Lemma 5.4.3. *Let $P = (u, v) \in \mathbb{J}^{(r)}$. Then $(u, -v) \notin \mathbb{J}^{(r)}$.*

Proof. If $P = \mathcal{O}_{\mathbb{J}}$ then $(u, -v) = (0, -1) \notin \mathbb{J}^{(r)}$. Else, P is of odd-prime order. Note that $v \neq 0$. (If $v = 0$ then $a \cdot u^2 = 1$, and so applying the doubling formula gives $[2]\,P = (0, -1)$, then $[4]\,P = (0, 1) = \mathcal{O}_{\mathbb{J}}$; contradiction since then P would not be of odd-prime order.) Therefore, $-v \neq v$. Now suppose $(u, -v) = Q$ is a point in $\mathbb{J}^{(r)}$. Then by applying the doubling formula we have $[2]\,Q = -[2]\,P$. But also $[2]\,(-P) = -[2]\,P$. Therefore either $Q = -P$ (then $\mathcal{V}(Q) = \mathcal{V}(-P)$; contradiction since $-v \neq v$), or doubling is not injective on $\mathbb{J}^{(r)}$ (contradiction since $\mathbb{J}^{(r)}$ is of odd order [KvE2013]). □

Theorem 5.4.4. *$\mathcal{U}$ is injective on $\mathbb{J}^{(r)}$.*

Proof. By writing the curve equation as $v^2 = (1 - a \cdot u^2)/(1 - d \cdot u^2)$, and noting that the potentially exceptional case $1 - d \cdot u^2 = 0$ does not occur for a *ctEdwards curve*, we see that for a given u there can be at most two possible solutions for v, and that if there are two solutions they can be written as v and $-v$. In that case by the Lemma, at most one of (u, v) and $(u, -v)$ is in $\mathbb{J}^{(r)}$. Therefore, $\mathcal{U}$ is injective on points in $\mathbb{J}^{(r)}$. □

Since $\mathsf{I2LEBSP}_{\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}}$ is injective, it follows that $\mathsf{Extract}_{\mathbb{J}^{(r)}}$ is injective on $\mathbb{J}^{(r)}$.

5.4.8.5 Group Hash into Jubjub

Let URS be the MPC randomness beacon defined in § 5.9 *'Randomness Beacon'* on p. 87.

Let BLAKE2s-256 be as defined in § 5.4.1.2 *'BLAKE2 Hash Functions'* on p. 59.

Let LEOS2IP be as defined in § 5.1 *'Integers, Bit Sequences, and Endianness'* on p. 56.

Let $\mathbb{J}^{(r)}$, $\mathbb{J}^{(r)*}$, and $\mathsf{abst}_{\mathbb{J}}$ be as defined in § 5.4.8.3 *'Jubjub'* on p. 77.

Let $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}.\mathsf{Input} := \mathbb{B}^{\mathbb{Y}^{[8]}} \times \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}}$, and let $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}.\mathsf{URSType} := \mathbb{B}^{\mathbb{Y}^{[64]}}$.

(The input element with type $\mathbb{B}^{\mathbb{Y}^{[8]}}$ is intended to act as a "personalization" parameter to distinguish uses of the *group hash* for different purposes.)

Let $D : \mathbb{B}^{\mathbb{Y}^{[8]}}$ be an 8-byte domain separator, and let $M : \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}}$ be the hash input.

The hash $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}_{\mathsf{URS}}(D, M) : \mathbb{J}^{(r)*} \cup \{\bot\}$ is calculated as follows:

let $\underline{H} = \mathsf{BLAKE2s\text{-}256}(D, \mathsf{URS} \mid\mid M)$
let $P = \mathsf{abst}_{\mathbb{J}}\big(\mathsf{LEOS2BSP}_{256}(\underline{H})\big)$
if $P = \bot$ then return $\bot$
let $Q = [h_{\mathbb{J}}]\, P$
if $Q = \mathcal{O}_{\mathbb{J}}$ then return $\bot$, else return Q.

Notes:

- The use of $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}_{\mathsf{URS}}$ for $\mathsf{DiversifyHash}^{\mathsf{Sapling}}$ and to generate independent bases needs a *random oracle* (for inputs on which $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}_{\mathsf{URS}}$ does not return $\bot$); here we show that it is sufficient to employ a simpler *random oracle* instantiated by BLAKE2s-256 in the security analysis.

  $\underline{H} : \mathbb{B}^{\mathbb{Y}^{[32]}} \mapsto_{\notin \{\bot, \mathcal{O}_{\mathbb{J}}, (0,-1)\}} \mathsf{abst}_{\mathbb{J}}\big(\mathsf{LEOS2BSP}_{256}(\underline{H})\big) : \mathbb{J}$ is injective, and both it and its inverse are efficiently computable.

  $P : \mathbb{J} \mapsto_{\notin \{\mathcal{O}_{\mathbb{J}}\}} [h_{\mathbb{J}}]\, P : \mathbb{J}^{(r)*}$ is exactly $h_{\mathbb{J}}$-to-1, and both it and its inverse relation are efficiently computable.

  It follows that when $\big(D : \mathbb{B}^{\mathbb{Y}^{[8]}}, M : \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}}\big) \mapsto \mathsf{BLAKE2s\text{-}256}(D, \mathsf{URS} \mid\mid M) : \mathbb{B}^{\mathbb{Y}^{[32]}}$ is modelled as a *random oracle*, $\big(D : \mathbb{B}^{\mathbb{Y}^{[8]}}, M : \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}}\big) \mapsto_{\notin \{\bot\}} \mathsf{GroupHash}^{\mathbb{J}^{(r)*}}_{\mathsf{URS}}(D, M) : \mathbb{J}^{(r)*}$ also acts as a *random oracle*.
- The BLAKE2s-256 chaining variable after processing URS may be precomputed.

Define $\mathsf{first} : (\mathbb{B}^{\mathbb{Y}} \to T \cup \{\bot\}) \to T \cup \{\bot\}$ so that $\mathsf{first}(f) = f(i)$ where i is the least integer in $\mathbb{B}^{\mathbb{Y}}$ such that $f(i) \neq \bot$, or $\bot$ if no such i exists.

Define $\mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}(D, M) := \mathsf{first}(i : \mathbb{B}^{\mathbb{Y}} \mapsto \mathsf{GroupHash}^{\mathbb{J}^{(r)*}}_{\mathsf{URS}}(D, M \mid\mid [i]) : \mathbb{J}^{(r)*} \cup \{\bot\})$.

Note: For random input, $\mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}$ returns $\bot$ with probability approximately 2^{-256}. In the **Zcash** protocol, most uses of $\mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}$ are for constants and do not return $\bot$; the only use that could potentially return $\bot$ is in the computation of a *default diversified payment address* in § 4.2.2 *'Sapling Key Components'* on p. 32.

5.4.9 Zero-Knowledge Proving Systems

5.4.9.1 BCTV14

Before **Sapling** activation, **Zcash** uses *zk-SNARKs* generated by a fork of *libsnark* [Zcash-libsnark] with the BCTV14 *proving system* described in [BCTV2014a], which is a modification of the systems in [PHGR2013] and [BCGTV2013].

A BCTV14 proof comprises $(\pi_A : \mathbb{G}_1^{(r)*}, \pi'_A : \mathbb{G}_1^{(r)*}, \pi_B : \mathbb{G}_2^{(r)*}, \pi'_B : \mathbb{G}_1^{(r)*}, \pi_C : \mathbb{G}_1^{(r)*}, \pi'_C : \mathbb{G}_1^{(r)*}, \pi_K : \mathbb{G}_1^{(r)*}, \pi_H : \mathbb{G}_1^{(r)*})$. It is computed as described in [BCTV2014a, Appendix B], using the pairing parameters specified in § 5.4.8.1 *'BN-254'* on p. 74.

Note: Many details of the *proving system* are beyond the scope of this protocol document. For example, the *quadratic constraint program* verifying the *JoinSplit statement*, or its translation to a *Quadratic Arithmetic Program* [BCTV2014a, section 2.3], are not specified in this document. In 2015, Bryan Parno found a bug in this translation, which is corrected by the *libsnark* implementation[8] [WCBTV2015] [Parno2015] [BCTV2014a, Remark 2.5]. In practice it will be necessary to use the specific proving and verifying keys that were generated for the **Zcash** production *block chain*, given in § 5.7 *'BCTV14 zk-SNARK Parameters'* on p. 87, together with a *proving system* implementation that is interoperable with the **Zcash** fork of *libsnark*, to ensure compatibility.

Vulnerability disclosure: BCTV14 is subject to a security vulnerability, separate from [Parno2015], that could allow violation of Knowledge Soundness (and Soundness) [CVE-2019-7167] [SWB2019] [Gabizon2019]. The consequence for **Zcash** is that balance violation could have occurred before activation of the **Sapling** *network upgrade*, although there is no evidence of this having happened. Use of the vulnerability to produce false proofs is believed to have been fully mitigated by activation of **Sapling**. The use of BCTV14 in **Zcash** is now limited to verifying proofs that were made prior to the **Sapling** upgrade.

Due to this issue, new forks of **Zcash MUST NOT** use BCTV14, and any other users of the **Zcash** protocol **SHOULD** discontinue use of BCTV14 as soon as possible.

The vulnerability does not affect the Zero Knowledge property of the scheme (as described in any version of [BCTV2014a] or as implemented in any version of *libsnark* that has been used in **Zcash**), even under subversion of the parameter generation [BGG2017, Theorem 4.10].

[**Sapling** onward] An implementation of **Zcash** that checkpoints on a *block* after **Sapling MAY** choose to skip verification of BCTV14 proofs. Note that in § 3.3 *'The Block Chain'* on p. 16, there is a requirement that a *full validator* that potentially risks *Mainnet* funds or displays *Mainnet transaction* information to a user **MUST** do so only for a *block chain* that includes the *activation block* of the most recent *settled network upgrade*, with its known *block hash* as specified in § 3.11 *'Mainnet and Testnet'* on p. 20. Since the most recent *settled network upgrade* is after the **Sapling** upgrade, this mitigates the potential risks due to skipping BCTV14 proof verification.

Encoding of BCTV14 Proofs

A BCTV14 proof is encoded by concatenating the encodings of its elements; for the BN-254 pairing this is:

264-bit π_A	264-bit π'_A	520-bit π_B	264-bit π'_B	264-bit π_C	264-bit π'_C	264-bit π_K	264-bit π_H

The resulting proof size is 296 bytes.

In addition to the steps to verify a proof given in [BCTV2014a, Appendix B], the verifier **MUST** check, for the encoding of each element, that:

- the lead byte is of the required form;
- the remaining bytes encode a *big-endian* representation of an integer in $\{0 \,..\, q_{\mathbb{S}} - 1\}$ or (for π_B) $\{0 \,..\, {q_{\mathbb{S}}}^2 - 1\}$;
- the encoding represents a point in $\mathbb{G}_1^{(r)*}$ or (for π_B) $\mathbb{G}_2^{(r)*}$, including checking that it is of order $r_{\mathbb{G}}$ in the latter case.

[8] Confusingly, the bug found by Bryan Parno was fixed in *libsnark* in 2015, but that fix was incompletely described in the May 2015 update [BCTV2014a-old, Theorem 2.4]. It is described completely in [BCTV2014a, Theorem 2.4] and in [Gabizon2019].

5.4.9.2 Groth16

After **Sapling** activation, **Zcash** uses *zk-SNARKs* with the Groth16 *proving system* described in [BGM2017], which is a modification of the system in [Groth2016]. An independent security proof of this system and its setup is given in [Maller2018].

Groth16 *zk-SNARK proofs* are used in *transaction version* 4 and later (§ 7.1 ***'Transaction Encoding and Consensus'*** on p. 89), both in Sprout *JoinSplit descriptions* and in Sapling *Spend descriptions* and *Output descriptions*. They are generated by the *bellman* library [Bowe-bellman].

A Groth16 proof comprises $(\pi_A : \mathbb{S}_1^{(r)*},\ \pi_B : \mathbb{S}_2^{(r)*},\ \pi_C : \mathbb{S}_1^{(r)*})$. It is computed as described in [Groth2016, section 3.2], using the pairing parameters specified in § 5.4.8.2 ***'BLS12-381'*** on p. 75. The proof elements are in a different order to the presentation in [Groth2016].

Note: The *quadratic constraint programs* verifying the *Spend statement* and *Output statement* are described in Appendix § A ***'Circuit Design'*** on p. 149. However, many other details of the *proving system* are beyond the scope of this protocol document. For example, certain details of the translations of the *Spend statement* and *Output statement* to *Quadratic Arithmetic Programs* are not specified in this document. In practice it will be necessary to use the specific proving and verifying keys generated for the **Zcash** production *block chain* (see § 5.8 ***'Groth16 zk-SNARK Parameters'*** on p. 87), and a *proving system* implementation that is interoperable with the *bellman* library used by **Zcash**, to ensure compatibility.

Encoding of Groth16 Proofs

A Groth16 proof is encoded by concatenating the encodings of its elements; for the BLS12-381 pairing this is:

384-bit π_A	768-bit π_B	384-bit π_C

The resulting proof size is 192 bytes.

In addition to the steps to verify a proof given in [Groth2016], the verifier **MUST** check, for the encoding of each element, that:

- the leading bitfield is of the required form;
- the remaining bits encode a *big-endian* representation of an integer in $\{0 \,..\, q_{\mathbb{S}} - 1\}$ or (in the case of π_B) two integers in that range;
- the encoding represents a point in $\mathbb{S}_1^{(r)*}$ or (in the case of π_B) $\mathbb{S}_2^{(r)*}$, including checking that it is of order $r_{\mathbb{S}}$ in each case.

5.5 Encodings of Note Plaintexts and Memo Fields

As explained in § 3.2.1 ***'Note Plaintexts and Memo Fields'*** on p. 14, transmitted *notes* are stored on the *block chain* in encrypted form. The components and usage of *note plaintexts*, and which keys they are encrypted to, are defined in that section.

The encoding of a Sprout *note plaintext* consists of:

8-bit leadByte	64-bit v	256-bit ρ	256-bit rcm	memo (512 bytes)

- A byte, 0x00, indicating this version of the encoding of a Sprout *note plaintext*.
- 8 bytes specifying v.

- 32 bytes specifying ρ.
- 32 bytes specifying rcm.
- 512 bytes specifying memo.

The encoding of a Sapling *note plaintext* consists of:

8-bit leadByte	88-bit d	64-bit v	256-bit rcm	memo (512 bytes)

- A byte, 0x01 as specified in § 3.2.1 ***'Note Plaintexts and Memo Fields'*** on p. 14, indicating this version of the encoding of a Sapling *note plaintext*.
- 11 bytes specifying d.
- 8 bytes specifying v.
- 32 bytes specifying rcm.
- 512 bytes specifying memo.

5.6 Encodings of Addresses and Keys

This section describes how **Zcash** encodes *shielded payment addresses*, *incoming viewing keys*, and *spending keys*.

Addresses and keys can be encoded as a *byte sequence*; this is called the *raw encoding*. For Sprout *shielded payment addresses*, this *byte sequence* can then be further encoded using *Base58Check*. The *Base58Check* layer is the same as for upstream **Bitcoin** addresses [Bitcoin-Base58].

For Sapling-specific key and address formats, *Bech32* [ZIP-173] is used instead of *Base58Check*.

Non-normative note: ZIP 173 is similar to **Bitcoin**'s BIP 173, except for dropping the limit of 90 characters on an encoded *Bech32* string (which does not hold for Sapling viewing keys, for example), and requirements specific to Bitcoin's Segwit addresses.

Payment addresses **MAY** be encoded as QR codes; in this case, the **RECOMMENDED** format for a Sapling *payment address* is the *Bech32* form converted to uppercase, using the Alphanumeric mode [ISO2015, sections 7.3.4 and 7.4.4].

5.6.1 Transparent Encodings

5.6.1.1 Transparent Addresses

Transparent addresses are either *P2SH* (Pay to Script Hash) addresses [BIP-13] or *P2PKH* (Pay to Public Key Hash) addresses [Bitcoin-P2PKH].

The *raw encoding* of a *P2SH address* consists of:

8-bit 0x1C	8-bit 0xBD	160-bit script hash

- Two bytes [0x1C, 0xBD], indicating this version of the *raw encoding* of a *P2SH address* on *Mainnet*. (Addresses on *Testnet* use [0x1C, 0xBA] instead.)
- 20 bytes specifying a script hash [Bitcoin-P2SH].

The *raw encoding* of a *P2PKH address* consists of:

8-bit `0x1C`	8-bit `0xB8`	160-bit *validating key* hash

- Two bytes [`0x1C`, `0xB8`], indicating this version of the *raw encoding* of a *P2PKH address* on *Mainnet*. (Addresses on *Testnet* use [`0x1D`, `0x25`] instead.)
- 20 bytes specifying a *validating key* hash, which is a RIPEMD-160 hash [RIPEMD160] of a SHA-256 hash [NIST2015] of a compressed ECDSA key encoding.

Notes:

- In **Bitcoin** a single byte is used for the version field identifying the address type. In **Zcash** two bytes are used. For addresses on *Mainnet*, this and the encoded length cause the first two characters of the *Base58Check* encoding to be fixed as **"`t3`"** for *P2SH addresses*, and as **"`t1`"** for *P2PKH addresses*. (This does ***not*** imply that a *transparent* **Zcash** address can be parsed identically to a **Bitcoin** address just by removing the **"`t`"**.)
- **Zcash** does not yet support Hierarchical Deterministic Wallet addresses [BIP-32].

5.6.1.2 Transparent Private Keys

These are encoded in the same way as in **Bitcoin** [Bitcoin-Base58], for both *Mainnet* and *Testnet*.

5.6.2 Sprout Encodings

5.6.2.1 Sprout Payment Addresses

Let $\mathsf{KA}^{\mathsf{Sprout}}$ be as defined in § 5.4.4.1 *'Sprout Key Agreement'* on p. 66.

A Sprout *shielded payment address* consists of $\mathsf{a_{pk}} : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}]}$ and $\mathsf{pk_{enc}} : \mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Public}$.

$\mathsf{a_{pk}}$ is a SHA256Compress output. $\mathsf{pk_{enc}}$ is a $\mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Public}$ key, for use with the encryption scheme defined in § 4.17 *'In-band secret distribution (Sprout)'* on p. 50. These components are derived from a *spending key* as described in § 4.2.1 *'Sprout Key Components'* on p. 32.

The *raw encoding* of a Sprout *shielded payment address* consists of:

8-bit `0x16`	8-bit `0x9A`	256-bit $\mathsf{a_{pk}}$	256-bit $\mathsf{pk_{enc}}$

- Two bytes [`0x16`, `0x9A`], indicating this version of the *raw encoding* of a Sprout *shielded payment address* on *Mainnet*. (Addresses on *Testnet* use [`0x16`, `0xB6`] instead.)
- 32 bytes specifying $\mathsf{a_{pk}}$.
- 32 bytes specifying $\mathsf{pk_{enc}}$, using the normal encoding of a Curve25519 *public key* [Bernstein2006].

Note: For addresses on *Mainnet*, the lead bytes and encoded length cause the first two characters of the *Base58Check* encoding to be fixed as **"`zc`"**. For *Testnet*, the first two characters are fixed as **"`zt`"**.

5.6.2.2 Sprout **Incoming Viewing Keys**

Let $\mathsf{KA}^{\mathsf{Sprout}}$ be as defined in § 5.4.4.1 *'Sprout Key Agreement'* on p. 66.

A Sprout *incoming viewing key* consists of $\mathsf{a_{pk}} : \mathbb{B}^{[\ell^{\mathsf{Sprout}}_{\mathsf{PRF}}]}$ and $\mathsf{sk_{enc}} : \mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Private}$.

$\mathsf{a_{pk}}$ is a SHA256Compress output. $\mathsf{sk_{enc}}$ is a $\mathsf{KA}^{\mathsf{Sprout}}.\mathsf{Private}$ key, for use with the encryption scheme defined in § 4.17 *'In-band secret distribution (Sprout)'* on p. 50. These components are derived from a *spending key* as described in § 4.2.1 *'Sprout Key Components'* on p. 32.

The *raw encoding* of a Sprout *incoming viewing key* consists of:

8-bit `0xA8`	8-bit `0xAB`	8-bit `0xD3`	256-bit $\mathsf{a_{pk}}$	256-bit $\mathsf{sk_{enc}}$

- Three bytes [`0xA8`, `0xAB`, `0xD3`], indicating this version of the *raw encoding* of a **Zcash** *incoming viewing key* on *Mainnet*. (Addresses on *Testnet* use [`0xA8`, `0xAC`, `0x0C`] instead.)
- 32 bytes specifying $\mathsf{a_{pk}}$.
- 32 bytes specifying $\mathsf{sk_{enc}}$, using the normal encoding of a Curve25519 *private key* [Bernstein2006].

$\mathsf{sk_{enc}}$ **MUST** be "clamped" using $\mathsf{KA}^{\mathsf{Sprout}}.\mathsf{FormatPrivate}$ as specified in § 4.2.1 *'Sprout Key Components'* on p. 32. That is, a decoded *incoming viewing key* **MUST** be considered invalid if $\mathsf{sk_{enc}} \neq \mathsf{KA}^{\mathsf{Sprout}}.\mathsf{FormatPrivate}(\mathsf{sk_{enc}})$.

$\mathsf{KA}^{\mathsf{Sprout}}.\mathsf{FormatPrivate}$ is defined in § 5.4.4.1 *'Sprout Key Agreement'* on p. 66.

Note: For addresses on *Mainnet*, the lead bytes and encoded length cause the first four characters of the *Base58Check* encoding to be fixed as **"`ZiVK`"**. For *Testnet*, the first four characters are fixed as **"`ZiVt`"**.

5.6.2.3 Sprout **Spending Keys**

A Sprout *spending key* consists of $\mathsf{a_{sk}}$, which is a sequence of 252 bits (see § 4.2.1 *'Sprout Key Components'* on p. 32).

The *raw encoding* of a Sprout *spending key* consists of:

8-bit `0xAB`	8-bit `0x36`	$[0]^4$	252-bit $\mathsf{a_{sk}}$

- Two bytes [`0xAB`, `0x36`], indicating this version of the *raw encoding* of a **Zcash** *spending key* on *Mainnet*. (Addresses on *Testnet* use [`0xAC`, `0x08`] instead.)
- 32 bytes: 4 zero padding bits and 252 bits specifying $\mathsf{a_{sk}}$.

The zero padding occupies the most significant 4 bits of the third byte.

Notes:

- If an implementation represents $\mathsf{a_{sk}}$ internally as a sequence of 32 bytes with the 4 bits of zero padding intact, it will be in the correct form for use as an input to $\mathsf{PRF}^{\mathsf{addr}}$, $\mathsf{PRF}^{\mathsf{nfSprout}}$, and $\mathsf{PRF}^{\mathsf{pk}}$ without need for bit-shifting.
- For addresses on *Mainnet*, the lead bytes and encoded length cause the first two characters of the *Base58Check* encoding to be fixed as **"`SK`"**. For *Testnet*, the first two characters are fixed as **"`ST`"**.

5.6.3 Sapling **Encodings**

5.6.3.1 Sapling **Payment Addresses**

Let $\mathsf{KA}^{\mathsf{Sapling}}$ be as defined in § 5.4.4.3 ***'Sapling Key Agreement'*** on p. 67.

Let ℓ_{d} be as defined in § 5.3 ***'Constants'*** on p. 57.

Let $\mathbb{J}^{(r)}$, $\mathsf{abst}_{\mathbb{J}}$, and $\mathsf{repr}_{\mathbb{J}}$ be as defined in § 5.4.8.3 *'Jubjub'* on p. 77.

Let $\mathsf{LEBS2OSP} : (\ell : \mathbb{N}) \times \mathbb{B}^{[\ell]} \to \mathbb{B}^{\mathbb{Y}^{[\mathsf{ceiling}(\ell/8)]}}$ be as defined in § 5.1 ***'Integers, Bit Sequences, and Endianness'*** on p. 56.

A Sapling *shielded payment address* consists of $\mathsf{d} : \mathbb{B}^{[\ell_{\mathsf{d}}]}$ and $\mathsf{pk_d} : \mathsf{KA}^{\mathsf{Sapling}}.\mathsf{PublicPrimeOrder}$.

$\mathsf{pk_d}$ is an encoding of a $\mathsf{KA}^{\mathsf{Sapling}}$ *public key* of type $\mathsf{KA}^{\mathsf{Sapling}}.\mathsf{PublicPrimeOrder}$, for use with the encryption scheme defined in § 4.18 ***'In-band secret distribution (Sapling)'*** on p. 51. d is a *diversifier*. These components are derived as described in § 4.2.2 ***'Sapling Key Components'*** on p. 32.

The *raw encoding* of a Sapling *shielded payment address* consists of:

$\mathsf{LEBS2OSP}_{88}(\mathsf{d})$	$\mathsf{LEBS2OSP}_{256}(\mathsf{repr}_{\mathbb{J}}(\mathsf{pk_d}))$

- 11 bytes specifying d.
- 32 bytes specifying the *ctEdwards compressed encoding* of $\mathsf{pk_d}$ (see § 5.4.8.3 *'Jubjub'* on p. 77).

When decoding the representation of $\mathsf{pk_d}$, the address **MUST** be considered invalid if $\mathsf{abst}_{\mathbb{J}}$ returns $\bot$, or the encoding of $\mathsf{pk_d}$ is a *non-canonical* encoding as defined in § 4.1.8 ***'Represented Group'*** on p. 29, or the resulting $\mathsf{pk_d}$ is not in $\mathbb{J}^{(r)*}$.

Non-normative notes:

- There are no *non-canonical* encodings of *Jubjub curve* points in $\mathbb{J}^{(r)*}$.
- The restriction on $\mathsf{pk_d}$ reflects its current type $\mathsf{KA}^{\mathsf{Sapling}}.\mathsf{PublicPrimeOrder} = \mathbb{J}^{(r)*}$. In versions of this specification prior to v2025.6.0, $\mathsf{pk_d}$ had type $\mathsf{KA}^{\mathsf{Sapling}}.\mathsf{PublicPrimeSubgroup} = \mathbb{J}^{(r)}$, i.e. including $\mathcal{O}_{\mathbb{J}}$. Implementations of consumers for this encoding may need to be updated to exclude $\mathcal{O}_{\mathbb{J}}$, and should be checked for consistency with the current version of [ZIP-216].

For addresses on *Mainnet*, the *Human-Readable Part* (as defined in [ZIP-173]) is **"`zs`"**. For addresses on *Testnet*, the *Human-Readable Part* is **"`ztestsapling`"**.

5.6.3.2 Sapling **Incoming Viewing Keys**

Let $\mathsf{KA}^{\mathsf{Sapling}}$ be as defined in § 5.4.4.3 ***'Sapling Key Agreement'*** on p. 67.

Let $\ell_{\mathsf{ivk}}^{\mathsf{Sapling}}$ be as defined in § 5.3 ***'Constants'*** on p. 57.

A Sapling *incoming viewing key* consists of $\mathsf{ivk} : \{1 \,..\, 2^{\ell_{\mathsf{ivk}}^{\mathsf{Sapling}}} - 1\}$.

ivk is a $\mathsf{KA}^{\mathsf{Sapling}}.\mathsf{Private}$ key (restricted to $\ell_{\mathsf{ivk}}^{\mathsf{Sapling}}$ bits), derived as described in § 4.2.2 ***'Sapling Key Components'*** on p. 32. It is used with the encryption scheme defined in § 4.18 ***'In-band secret distribution (Sapling)'*** on p. 51.

The *raw encoding* of a Sapling *incoming viewing key* consists of:

256-bit ivk

- 32 bytes (*little-endian*) specifying ivk, padded with zeros in the most significant bits.

ivk **MUST** be in the range $\{1 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{ivk}}} - 1\}$ as specified in § 4.2.2 *'Sapling Key Components'* on p. 32. That is, a decoded *incoming viewing key* **MUST** be considered invalid if ivk is not in this range.

For *incoming viewing keys* on *Mainnet*, the *Human-Readable Part* is **"zivks"**. For *incoming viewing keys* on *Testnet*, the *Human-Readable Part* is **"zivktestsapling"**.

Non-normative notes:

- The *diversifier key* is not present in this encoding, so it does not provide sufficient information to decrypt the *diversifier* to obtain a *diversifier index*. This encoding is therefore deprecated: the preferred way to encode a Sapling *incoming viewing key* is as a component of a *unified incoming viewing key* using the *IVK Encoding* specified in [ZIP-316], which does include the *diversifier key*.
- In versions of this specification prior to v2025.6.0, the range of ivk was defined as $\{0 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{ivk}}} - 1\}$, i.e. including 0. Implementations of consumers for this encoding may need to be updated to exclude 0.

5.6.3.3 Sapling **Full Viewing Keys**

Let $\mathsf{KA}^{\mathsf{Sapling}}$ be as defined in § 5.4.4.3 *'Sapling Key Agreement'* on p. 67.

A Sapling *full viewing key* consists of $\mathsf{ak} : \mathbb{J}^{(r)*}$, $\mathsf{nk} : \mathbb{J}^{(r)}$, and $\mathsf{ovk} : \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{ovk}}/8]}}$.

ak and nk are points on the *Jubjub curve* (see § 5.4.8.3 *'Jubjub'* on p. 77). They are derived as described in § 4.2.2 *'Sapling Key Components'* on p. 32.

The *raw encoding* of a Sapling *full viewing key* consists of:

$\mathsf{LEBS2OSP}_{256}(\mathsf{repr}_{\mathbb{J}}(\mathsf{ak}))$	$\mathsf{LEBS2OSP}_{256}(\mathsf{repr}_{\mathbb{J}}(\mathsf{nk}))$	32-byte ovk

- 32 bytes specifying the *ctEdwards compressed encoding* of ak (see § 5.4.8.3 *'Jubjub'* on p. 77).
- 32 bytes specifying the *ctEdwards compressed encoding* of nk.
- 32 bytes specifying the *outgoing viewing key* ovk.

When decoding this representation, the key **MUST** be considered invalid if $\mathsf{abst}_{\mathbb{J}}$ returns $\bot$ for either ak or nk, or if $\mathsf{ak} \notin \mathbb{J}^{(r)*}$, or if $\mathsf{nk} \notin \mathbb{J}^{(r)}$.

For *full viewing keys* on *Mainnet*, the *Human-Readable Part* is **"zviews"**. For *full viewing keys* on *Testnet*, the *Human-Readable Part* is **"zviewtestsapling"**.

5.6.3.4 Sapling **Spending Keys**

A Sapling *spending key* consists of $\mathsf{sk} : \mathbb{B}^{[\ell_{\mathsf{sk}}]}$ (see § 4.2.2 *'Sapling Key Components'* on p. 32).

The *raw encoding* of a Sapling *spending key* consists of:

$\mathsf{LEBS2OSP}_{256}(\mathsf{sk})$

- 32 bytes specifying sk.

For *spending keys* on *Mainnet*, the *Human-Readable Part* is **"secret-spending-key-main"**. For *spending keys* on *Testnet*, the *Human-Readable Part* is **"secret-spending-key-test"**.

5.7 BCTV14 zk-SNARK Parameters

The SHA-256 hashes of the *proving key* and *verifying key* for the Sprout *JoinSplit circuit*, encoded in *libsnark* format, are:

```
8bc20a7f013b2b58970cddd2e7ea028975c88ae7ceb9259a5344a16bc2c0eef7 sprout-proving.key
4bd498dae0aacfd8e98dc306338d017d9c08dd0918ead18172bd0aec2fc5df82 sprout-verifying.key
```

These parameters were obtained by a *multi-party computation* described in [BGG-mpc] and [BGG2017]. They are used only before **Sapling** activation. Due to the security vulnerability described in § 5.4.9.1 *'BCTV14'* on p. 79, it is not recommended to use these parameters in new protocols, and it is recommended to stop using them in protocols other than **Zcash** where they are currently used.

5.8 Groth16 zk-SNARK Parameters

bellman [Bowe-bellman] encodes the *proving key* and *verifying key* for a *zk-SNARK circuit* in a single parameters file. The BLAKE2b-512 hashes of this file for the Sapling *Spend circuit* and *Output circuit*, and for the implementation of the Sprout *JoinSplit circuit* used after **Sapling** activation, are respectively:

```
8270785a1a0d0bc77196f000ee6d221c9c9894f55307bd9357c3f0105d31ca63
991ab91324160d8f53e2bbd3c2633a6eb8bdf5205d822e7f3f73edac51b2b70c sapling-spend.params
657e3d38dbb5cb5e7dd2970e8b03d69b4787dd907285b5a7f0790dcc8072f60b
f593b32cc2d1c030e00ff5ae64bf84c5c3beb84ddc841d48264b4a171744d028 sapling-output.params
e9b238411bd6c0ec4791e9d04245ec350c9c5744f5610dfcce4365d5ca49dfef
d5054e371842b3f88fa1b9d7e8e075249b3ebabd167fa8b0f3161292d36c180a sprout-groth16.params
```

These parameters were obtained by a *multi-party computation* described in [BGM2017].

5.9 Randomness Beacon

Let URS := `"096b36a5804bfacef1691e173c366a47ff5ba84a44f26ddd7e8d9f79d5b42df0"`.

This value is used in the definition of $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}$ in § 5.4.8.5 *'Group Hash into Jubjub'* on p. 79, and in the *multi-party computation* to obtain the Sapling parameters given in § 5.8 *'Groth16 zk-SNARK Parameters'* on p. 87.

It is derived as described in [Bowe2018]:

- Take the hash of the **Bitcoin** *block* at height 514200 in *RPC byte order*, i.e. the *big-endian* 32-byte representation of `0x00000000000000000034b33e842ac1c50456abe5fa92b60f6b3dfc5d247f7b58`.
- Apply SHA-256 2^{42} times.
- Convert to a *US-ASCII* lowercase hexadecimal string.

Note: URS is a 64-byte *US-ASCII* string, i.e. the first byte is `0x30`, not `0x09`.

6 Network Upgrades

Zcash launched with a protocol revision that we call Sprout.

A first upgrade, called **Overwinter**, activated on *Mainnet* on 26 June, 2018 at *block height* 347500 [Swihart2018]. Its specifications are described in this document, [ZIP-201], [ZIP-202], [ZIP-203], and [ZIP-143].

A second upgrade, called **Sapling**, activated on *Mainnet* on 28 October, 2018 at *block height* 419200 [Hamdon2018]. Its specifications are described in this document, [ZIP-205], and [ZIP-243].

This section summarizes the strategy for upgrading from the Sprout epoch to subsequent epochs of the protocol (**Overwinter**, **Sapling**, **Blossom**, **Heartwood**, **Canopy**, **NU5**, **NU6**, **NU6.1**, **NU6.2**, **NU6.3**, and **NU7**), and for future upgrades.

The *network upgrade* mechanism is described in [ZIP-200].

Each *network upgrade* is introduced as a "*bilateral consensus rule change*". In this kind of upgrade,

- there is an *activation block height* at which the *consensus rule change* takes effect;
- *blocks* and *transactions* that are valid according to the post-upgrade rules are not valid before the upgrade *block height*;
- *blocks* and *transactions* that are valid according to the pre-upgrade rules are no longer valid at or after the *activation block height*.

Full support for each *network upgrade* is indicated by a minimum version of the *peer-to-peer protocol*. At the planned *activation block height*, nodes that support a given upgrade will disconnect from (and will not reconnect to) nodes with a protocol version lower than this minimum. See [ZIP-201] for how this applies to the **Overwinter** upgrade, for example.

This ensures that upgrade-supporting nodes transition cleanly from the old protocol to the new protocol. Nodes that do not support the upgrade will find themselves on a network that uses the old protocol and is fully partitioned from the upgrade-supporting network. This allows us to specify arbitrary protocol changes that take effect at a given *block height*.

Note, however, that a *block chain reorganization* across the upgrade *activation block height* is possible. In the case of such a reorganization, *blocks* at a height before the *activation block height* will still be created and validated according to the pre-upgrade rules, and upgrade-supporting nodes **MUST** allow for this.

7 Consensus Changes from Bitcoin

7.1 Transaction Encoding and Consensus

The **Zcash** *transaction* format up to and including *transaction version* 4 is as follows (this should be read in the context of consensus rules later in the section):

Version*	Bytes	Name	Data Type	Description
1 .. 4	4	`header`	`uint32`	Contains: · `fOverwintered` flag (bit 31) · `version` (bits 30 .. 0) – *transaction version*.
3 .. 4	4	`nVersionGroupId`	`uint32`	Version group ID (nonzero).
1 .. 4	*Varies*	`tx_in_count`	`compactSize`	Number of *transparent inputs*.
1 .. 4	*Varies*	`tx_in`	`tx_in`	*Transparent* inputs, encoded as in **Bitcoin**.
1 .. 4	*Varies*	`tx_out_count`	`compactSize`	Number of *transparent outputs*.
1 .. 4	*Varies*	`tx_out`	`tx_out`	*Transparent* outputs, encoded as in **Bitcoin**.
1 .. 4	4	`lock_time`	`uint32`	Unix-epoch UTC time or *block height*, encoded as in **Bitcoin**.
3 .. 4	4	`nExpiryHeight`	`uint32`	A *block height* after which the *transaction* will expire, or 0 to disable expiry. [ZIP-203]
4	8	`valueBalanceSapling`	`int64`	The net value of *Sapling-pool* spends minus outputs.
4	*Varies*	`nSpendsSapling`	`compactSize`	The number of *Spend descriptions* in `vSpendsSapling`.
4	384· `nSpendsSapling`	`vSpendsSapling`	`SpendDescriptionV4 [nSpendsSapling]`	A sequence of *Spend descriptions*, encoded per § 7.3 ***'Spend Description Encoding and Consensus'*** on p. 93.
4	*Varies*	`nOutputsSapling`	`compactSize`	The number of *Output descriptions* in `vOutputsSapling`.
4	948· `nOutputsSapling`	`vOutputsSapling`	`OutputDescriptionV4 [nOutputsSapling]`	A sequence of *Output descriptions*, encoded per § 7.4 ***'Output Description Encoding and Consensus'*** on p. 94.
2 .. 4	*Varies*	`nJoinSplit`	`compactSize`	The number of *JoinSplit descriptions* in `vJoinSplit`.
2 .. 3	1802· `nJoinSplit`	`vJoinSplit`	`JSDescriptionBCTV14 [nJoinSplit]`	A sequence of *JoinSplit descriptions* using BCTV14 proofs, encoded per § 7.2 ***'JoinSplit Description Encoding and Consensus'*** on p. 93.
4	1698· `nJoinSplit`	`vJoinSplit`	`JSDescriptionGroth16 [nJoinSplit]`	A sequence of *JoinSplit descriptions* using Groth16 proofs, encoded per § 7.2 ***'JoinSplit Description Encoding and Consensus'*** on p. 93.
2 .. 4 †	32	`joinSplitPubKey`	`byte[32]`	An encoding of a JoinSplitSig public *validating key*.
2 .. 4 †	64	`joinSplitSig`	`byte[64]`	A signature on a prefix of the *transaction* encoding, validated using `joinSplitPubKey` as specified in § 4.10 ***'Non-malleability (Sprout)'*** on p. 41.
4 ‡	64	`bindingSigSapling`	`byte[64]`	A *Sapling binding signature* on the *SIGHASH transaction hash*, validated as specified in § 5.4.6.2 ***'Binding Signature (Sapling)'*** on p. 71.

* Version constraints apply to the effectiveVersion, which is equal to $\mathsf{min}(2, \texttt{version})$ when $\texttt{fOverwintered} = 0$ and to `version` otherwise. The consensus rules later in this section specify constraints on `nVersionGroupId` depending on effectiveVersion.

† The `joinSplitPubKey` and `joinSplitSig` fields are present if and only if effectiveVersion ≥ 2 and $\texttt{nJoinSplit} > 0$.

‡ `bindingSigSapling` is present if and only if effectiveVersion $= 4$ and $\texttt{nSpendsSapling} + \texttt{nOutputsSapling} > 0$.

Note that the `valueBalanceSapling` field is always present for these *transaction versions*.

Several Sapling fields have been renamed from previous versions of this specification:
`valueBalance` → `valueBalanceSapling`; `nShieldedSpend` → `nSpendsSapling`; `vShieldedSpend` → `vSpendsSapling`; `nShieldedOutput` → `nOutputsSapling`; `vShieldedOutput` → `vOutputsSapling`; `bindingSig` → `bindingSigSapling`.

7.1.1 Transaction Identifiers

The *transaction ID* of a version 4 or earlier *transaction* is the SHA-256d hash of the *transaction* encoding in the format described above.

7.1.2 Transaction Consensus Rules

Consensus rules:

- The *transaction version number* **MUST** be greater than or equal to 1.
- [Pre-**Overwinter**] The `fOverwintered` flag **MUST NOT** be set.
- [**Overwinter** onward] The `fOverwintered` flag **MUST** be set.
- [**Overwinter** onward] The *version group ID* **MUST** be recognized.
- [**Overwinter** only, pre-**Sapling**] The *transaction version number* **MUST** be 3, and the *version group ID* **MUST** be `0x03C48270`.
- [**Sapling** onward] The *transaction version number* **MUST** be 4.
- [**Sapling** onward] If the *transaction version number* is 4 then the *version group ID* **MUST** be `0x892F2085`.
- [Pre-**Sapling**] The encoded size of the *transaction* **MUST** be less than or equal to 100000 bytes.
- [Pre-**Sapling**] If effectiveVersion $= 1$ or `nJoinSplit` $= 0$, then both `tx_in_count` and `tx_out_count` **MUST** be nonzero.
- [**Sapling** onward] If effectiveVersion < 5, then at least one of `tx_in_count`, `nSpendsSapling`, and `nJoinSplit` **MUST** be nonzero.
- [**Sapling** onward] If effectiveVersion < 5, then at least one of `tx_out_count`, `nOutputsSapling`, and `nJoinSplit` **MUST** be nonzero.
- A *transaction* with one or more *transparent inputs* from *coinbase transactions* **MUST** have no *transparent outputs* (i.e. `tx_out_count` **MUST** be 0). Inputs from *coinbase transactions* include *Founders' Reward* outputs.
- If effectiveVersion ≥ 2 and `nJoinSplit` > 0, then:
  - `joinSplitPubKey` **MUST** be a valid encoding (see § 5.4.5 **'Ed25519'** on p. 67) of an Ed25519 *validating key*.
  - `joinSplitSig` **MUST** represent a valid signature under `joinSplitPubKey` of dataToBeSigned, as defined in § 4.10 ***'Non-malleability (Sprout)'*** on p. 41.
- [**Sapling** onward] If effectiveVersion ≥ 4 and `nSpendsSapling` + `nOutputsSapling` > 0, then:
  - let $\mathsf{bvk}^{\mathsf{Sapling}}$ and SigHash be as defined in § 4.12 ***'Balance and Binding Signature (Sapling)'*** on p. 42;
  - `bindingSigSapling` **MUST** represent a valid signature under the *transaction binding validating key* $\mathsf{bvk}^{\mathsf{Sapling}}$ of SigHash — i.e. $\mathsf{BindingSig}^{\mathsf{Sapling}}.\mathsf{Validate}_{\mathsf{bvk}^{\mathsf{Sapling}}}(\mathsf{SigHash}, \texttt{bindingSigSapling}) = 1$.
- [**Sapling** onward] If effectiveVersion $= 4$ and there are no *Spend descriptions* or *Output descriptions*, then `valueBalanceSapling` **MUST** be 0.[9]
- For the *block* at *block height* height:
  - define the *total output value* of its *coinbase transaction* to be the total value in zatoshi of its *transparent outputs*;
  - define the *total input value* of its *coinbase transaction* to be the value in zatoshi of the *block subsidy*, plus the *transaction fees* paid by *transactions* in the *block*.

[9] Prior to ***zcashd v6.12.1***, ***zcashd*** misimplemented the rule that requires `valueBalanceSapling` $= 0$ for a v4 *transaction* with no *Spend descriptions* and no *Output descriptions*. It incorrectly normalized the value to 0 when no *Spend descriptions* or *Output descriptions* were present, with the consequence that the failure case for this rule was never reached. For clarification, `valueBalanceSapling` refers to the value as encoded in the *transaction*, which is not normalized in this way; it does not refer to $v^{\mathit{SaplingPoolBalance}}$.

The *total output value* of a *coinbase transaction* **MUST NOT** be greater than its *total input value*.

- A *coinbase transaction* **MUST NOT** have any *JoinSplit descriptions*.
- A *coinbase transaction* **MUST NOT** have any *Spend descriptions*.
- A *coinbase transaction* **MUST NOT** have any *Output descriptions*.
- A *coinbase transaction* for a *block* at *block height* greater than 0 **MUST** have a script that, as its first item, encodes the *block height* height as follows. For height in the range $\{1 .. 16\}$, the encoding is a single byte of value 0x50 + height. Otherwise, let heightBytes be the signed *little-endian* representation of height, using the minimum nonzero number of bytes such that the most significant byte is $<$ 0x80. The length of heightBytes **MUST** be in the range $\{1 .. 5\}$. Then the encoding is the length of heightBytes encoded as one byte, followed by heightBytes itself. This matches the encoding used by **Bitcoin** in the implementation of [BIP-34] (but the description here is to be considered normative).
- A *coinbase transaction* script **MUST** have length in $\{2 .. 100\}$ bytes.
- A *transparent input* in a non-coinbase *transaction* **MUST NOT** have a null *prevout*.
- Every non-null *prevout* **MUST** point to a unique *UTXO* in either a preceding *block*, or a ***previous*** *transaction* in the same *block*.
- A *transaction* **MUST NOT** spend a *transparent output* of a *coinbase transaction* from a *block* less than 100 *blocks* prior to the spend. Note that *transparent outputs* of *coinbase transactions* include *Founders' Reward* outputs.
- A *transaction* **MUST NOT** spend an output of the *genesis block coinbase transaction*. (There is one such zero-valued output, on each of *Testnet* and *Mainnet*.)
- [**Overwinter** onward] `nExpiryHeight` **MUST** be less than or equal to 499999999.
- [**Overwinter** onward] If a *transaction* is not a *coinbase transaction* and its `nExpiryHeight` field is nonzero, then it **MUST NOT** be mined at a *block height* greater than its `nExpiryHeight`.
- [**Sapling** onward] `valueBalanceSapling`, if it exists, **MUST** be in the range $\{-\text{MAX_MONEY} .. \text{MAX_MONEY}\}$.
- { **TODO: Other rules inherited from Bitcoin.** }

The types specified in § 7.1 ***'Transaction Encoding and Consensus'*** on p. 89 are part of the consensus rules.

Consensus rules associated with each *JoinSplit description* (§ 7.2 ***'JoinSplit Description Encoding and Consensus'*** on p. 93), each *Spend description* (§ 7.3 ***'Spend Description Encoding and Consensus'*** on p. 93), and each *Output description* (§ 7.4 ***'Output Description Encoding and Consensus'*** on p. 94) **MUST** also be followed.

Notes:

- Previous versions of this specification defined what is now the `header` field as a signed `int32` field which was required to be positive. The consensus rule that the `fOverwintered` flag **MUST NOT** be set before **Overwinter** has activated, has the same effect.
- The semantics of *transactions* with *version number* not any of $\{1, 2, 3, 4\}$ is not currently defined.
- The exclusion of *transactions* with *transaction version number* ***greater than*** 2 is not a consensus rule before **Overwinter** activation. Such *transactions* may exist in the *block chain* and **MUST** be treated identically to version 2 *transactions*.
- [**Overwinter** onward] Once **Overwinter** has activated, limits on the maximum *transaction version number* are consensus rules.
- The *transaction version number* 0x7FFFFFFF, and the *version group ID* 0xFFFFFFFF, are reserved for use in experimental extensions to *transaction* format or semantics on private testnets. They **MUST NOT** be used on the **Zcash** *Mainnet* or *Testnet*.
- Note that a future upgrade might use ***any*** *transaction version number* or *version group ID*. It is likely that an upgrade that changes the *transaction version number* or *version group ID* will also change the *transaction* format, and software that parses *transactions* **SHOULD** take this into account.

- [**Overwinter** onward] The purpose of *version group ID* is to allow unambiguous parsing of *"loose" transactions*, independent of the context of a *block chain*. Code that parses *transactions* is likely to be reused between *consensus branches* as defined in [ZIP-200], and in that case the `fOverwintered` and `version` fields alone may be insufficient to determine the format to be used for parsing.
- A *transaction version number* of 2 does not have the same meaning as in **Bitcoin**, where it is associated with support for `OP_CHECKSEQUENCEVERIFY` as specified in [BIP-68]. **Zcash** was forked from *BitcoinCore* v0.11.2 and does not currently support BIP 68.
- [**Sapling** onward] Because *coinbase transactions* have no *Spend descriptions* or *Output descriptions*, the `valueBalanceSapling` field of a *coinbase transaction* must have a zero value.

The changes relative to **Bitcoin** version 1 *transactions* as described in [Bitcoin-Format] are:

- *Transaction version* 0 is not supported.
- A version 1 *transaction* is equivalent to a version 2 *transaction* with `nJoinSplit` $= 0$.
- The fields `nJoinSplit`, `vJoinSplit`, `joinSplitPubKey`, and `joinSplitSig` have been added.
- [**Overwinter** onward] The field `nVersionGroupId` has been added.
- [**Sapling** onward] The following fields have been added: `nSpendsSapling`, `vSpendsSapling`, `nOutputsSapling`, `vOutputsSapling`, and `bindingSigSapling`.
- In **Zcash** it is permitted for a *transaction* to have no *transparent inputs*, provided that there could be *shielded inputs*.
- A consensus rule limiting *transaction* size has been added. In **Bitcoin** there is a corresponding standard rule but no consensus rule.

7.2 JoinSplit Description Encoding and Consensus

An abstract *JoinSplit description*, as described in § 3.5 *'JoinSplit Transfers and Descriptions'* on p. 17, is encoded in a *transaction* as an instance of a `JoinSplitDescription` type:

Bytes	Name	Data Type	Description
8	`vpub_old`	`uint64`	A value v^{old}_{pub} that the *JoinSplit transfer* removes from the *transparent transaction value pool*.
8	`vpub_new`	`uint64`	A value v^{new}_{pub} that the *JoinSplit transfer* inserts into the *transparent transaction value pool*.
32	`anchor`	`byte[32]`	A *root* $rt^{SproutPool}$ of the *Sprout-pool note commitment tree* at some *block height* in the past, or the *root* produced by a previous *JoinSplit transfer* in this *transaction*.
64	`nullifiers`	`byte[32]` [N^{old}]	A sequence of *nullifiers* of the input *notes* $nf^{old}_{1..N^{old}}$.
64	`commitments`	`byte[32]` [N^{new}]	A sequence of *note commitments* for the output *notes* $cm^{new}_{1..N^{new}}$.
32	`ephemeralKey`	`byte[32]`	A Curve25519 *public key* epk.
32	`randomSeed`	`byte[32]`	A 256-bit seed that must be chosen independently at random for each *JoinSplit description*.
64	`vmacs`	`byte[32]` [N^{old}]	A sequence of message authentication tags $h_{1..N^{old}}$ binding h_{Sig} to each a_{sk} of the *JoinSplit description*, computed as described in § 4.10 *'Non-malleability (Sprout)'* on p. 41.
296 †	`zkproof`	`byte[296]`	An encoding of the *zk-SNARK proof* $\pi_{ZKJoinSplit}$ (see § 5.4.9.1 *'BCTV14'* on p. 79).
192 ‡	`zkproof`	`byte[192]`	An encoding of the *zk-SNARK proof* $\pi_{ZKJoinSplit}$ (see § 5.4.9.2 *'Groth16'* on p. 81).
1202	`encCiphertexts`	`byte[601]` [N^{new}]	A sequence of ciphertext components for the encrypted output *notes*, $C^{enc}_{1..N^{new}}$.

† BCTV14 proofs are used when the *transaction version* is 2 or 3, i.e. before **Sapling** activation.

‡ Groth16 proofs are used when the *transaction version* is ≥ 4, i.e. after **Sapling** activation.

The `ephemeralKey` and `encCiphertexts` fields together form the *transmitted notes ciphertext*, which is computed as described in § 4.17 *'In-band secret distribution (Sprout)'* on p. 50.

Consensus rules applying to a *JoinSplit description* are given in § 4.3 *'JoinSplit Descriptions'* on p. 34.

7.3 Spend Description Encoding and Consensus

Let LEBS2OSP be as defined in § 5.1 *'Integers, Bit Sequences, and Endianness'* on p. 56.

Let $repr_{\mathbb{J}}$ and $q_{\mathbb{J}}$ be as defined in § 5.4.8.3 *'Jubjub'* on p. 77.

Let spendAuthSig be the *spend authorization signature* for this *Spend transfer*, and let $\pi_{ZKSpend}$ be the *zk-SNARK proof* of the corresponding *Spend statement*. In a version 4 *transaction* these are encoded in the `spendAuthSig` field and `zkproof` field respectively of the *Spend description*.

An abstract *Spend description*, as described in § 3.6 ***'Spend Transfers, Output Transfers, and their Descriptions'*** on p. 18, is encoded in a *transaction* as an instance of a `SpendDescriptionV4` type:

Bytes	Name	Data Type	Description
32	`cv`	`byte[32]`	A *value commitment* to the value of the input *note*, $\mathsf{LEBS2OSP}_{256}(\mathsf{repr}_{\mathbb{J}}(\mathsf{cv}))$.
32	`anchor`	`byte[32]`	A *root* of the *Sapling-pool note commitment tree* at some *block height* in the past, $\mathsf{LEBS2OSP}_{256}(\mathsf{rt}^{\mathit{SaplingPool}})$.
32	`nullifier`	`byte[32]`	The *nullifier* of the input *note*, nf.
32	`rk`	`byte[32]`	The randomized *validating key* for $\mathsf{spendAuthSig}$, $\mathsf{LEBS2OSP}_{256}(\mathsf{repr}_{\mathbb{J}}(\mathsf{rk}))$.
192	`zkproof`	`byte[192]`	An encoding of the *zk-SNARK proof* π_{ZKSpend} (see § 5.4.9.2 *'Groth16'* on p. 81).
64	`spendAuthSig`	`byte[64]`	A signature authorizing this Spend.

Consensus rule: $\mathsf{LEOS2IP}_{256}(\texttt{anchorSapling})$ **MUST** be less than $q_{\mathbb{J}}$.

Other consensus rules applying to a *Spend description* are given in § 4.4 ***'Spend Descriptions'*** on p. 35.

7.4 Output Description Encoding and Consensus

Let LEBS2OSP be as defined in § 5.1 ***'Integers, Bit Sequences, and Endianness'*** on p. 56.

Let $\mathsf{repr}_{\mathbb{J}}$ and $q_{\mathbb{J}}$ be as in § 5.4.8.3 ***'Jubjub'*** on p. 77, and $\mathsf{Extract}_{\mathbb{J}^{(r)}}$ as in § 5.4.8.4 ***'Coordinate Extractor for Jubjub'*** on p. 78.

Let π_{ZKOutput} be the *zk-SNARK proof* of the *Output statement* for this *Output transfer*. In a version 4 *transaction* this is encoded in the `zkproof` field of the *Spend description*.

An abstract *Output description*, described in § 3.6 ***'Spend Transfers, Output Transfers, and their Descriptions'*** on p. 18, is encoded in a *transaction* as an instance of an `OutputDescriptionV4` type:

Bytes	Name	Data Type	Description
32	`cv`	`byte[32]`	A *value commitment* to the value of the output *note*, $\mathsf{LEBS2OSP}_{256}(\mathsf{repr}_{\mathbb{J}}(\mathsf{cv}))$.
32	`cmu`	`byte[32]`	The u-coordinate of the *note commitment* for the output *note*, $\mathsf{LEBS2OSP}_{256}(\mathsf{cm}_u)$ where $\mathsf{cm}_u = \mathsf{Extract}_{\mathbb{J}^{(r)}}(\mathsf{cm})$.
32	`ephemeralKey`	`byte[32]`	An encoding of an ephemeral Jubjub *public key*, $\mathsf{LEBS2OSP}_{256}(\mathsf{repr}_{\mathbb{J}}(\mathsf{epk}))$.
580	`encCiphertext`	`byte[580]`	A ciphertext component for the encrypted output *note*, $\mathsf{C}^{\mathsf{enc}}$.
80	`outCiphertext`	`byte[80]`	A ciphertext component that allows the holder of the *outgoing cipher key* to recover the *diversified transmission key* $\mathsf{pk_d}$ and *ephemeral private key* esk, hence the entire *note plaintext*.
192	`zkproof`	`byte[192]`	An encoding of the *zk-SNARK proof* π_{ZKOutput} (see § 5.4.9.2 *'Groth16'* on p. 81).

The `ephemeralKey`, `encCiphertext`, and `outCiphertext` fields together form the *transmitted note ciphertext*, which is computed as described in § 4.18 ***'In-band secret distribution (Sapling)'*** on p. 51.

Consensus rule: $\mathsf{LEOS2IP}_{256}(\texttt{cmu})$ **MUST** be less than $q_{\mathbb{J}}$.

Other consensus rules applying to an *Output description* are given in § 4.5 ***'Output Descriptions'*** on p. 36.

7.5 Block Header Encoding and Consensus

The **Zcash** *block header* format is as follows (this should be read in the context of consensus rules later in the section):

Bytes	Name	Data Type	Description
4	`nVersion`	`int32`	The *block version number* indicates which set of *block* validation rules to follow. The current and only defined *block version number* for **Zcash** is 4.
32	`hashPrevBlock`	`byte[32]`	A SHA-256d hash in internal byte order of the previous *block*'s *header*. This ensures no previous *block* can be changed without also changing this *block*'s *header*.
32	`hashMerkleRoot`	`byte[32]`	A SHA-256d hash in internal byte order. The merkle root is derived from the hashes of all *transactions* included in this *block*, ensuring that none of those *transactions* can be modified without modifying the *header*.
32	`hashReserved` / `hashFinalSaplingRoot`	`byte[32]`	[Pre-**Sapling**] A reserved field, to be ignored. [**Sapling**] The *root* $\mathsf{LEBS2OSP}_{256}(\mathsf{rt}^{SaplingPool})$ of the *Sapling pool note commitment tree* corresponding to the final *Sapling-pool treestate* of this *block*.
4	`nTime`	`uint32`	The *block timestamp* is a Unix epoch time (UTC) when the miner started hashing the *header* (according to the miner).
4	`nBits`	`uint32`	An encoded version of the *target threshold* this *block*'s *header* hash must be less than or equal to, in the same nBits format used by **Bitcoin**. [Bitcoin-nBits]
32	`nNonce`	`byte[32]`	An arbitrary field that miners can change to modify the *header* hash in order to produce a hash less than or equal to the *target threshold*.
3	`solutionSize`	`compactSize`	The size of an *Equihash* solution in bytes (always 1344).
1344	`solution`	`byte[1344]`	The *Equihash* solution.

A *block* consists of a *block header* and a sequence of *transactions*. How transactions are encoded in a *block* is part of the Zcash *peer-to-peer protocol* but not part of the consensus protocol.

Let ThresholdBits be as defined in § 7.6.3 ***'Difficulty adjustment'*** on p. 98, and let PoWMedianBlockSpan be the constant defined in § 5.3 ***'Constants'*** on p. 57.

Define the *median-time-past* of a *block* to be the median (as defined in § 7.6.3 ***'Difficulty adjustment'*** on p. 98) of the `nTime` fields of the ***preceding*** PoWMedianBlockSpan *blocks* (or all preceding *blocks* if there are fewer than PoWMedianBlockSpan). The *median-time-past* of a *genesis block* is not defined.

Consensus rules:

- The *block version number* **MUST** be greater than or equal to 4.
- For a *block* at *block height* height, `nBits` **MUST** be equal to ThresholdBits(height).
- The *block* **MUST** pass the difficulty filter defined in § 7.6.2 ***'Difficulty filter'*** on p. 98.
- `solution` **MUST** represent a *valid Equihash solution* as defined in § 7.6.1 ***'Equihash'*** on p. 97.
- For each *block* other than the *genesis block*, `nTime` **MUST** be strictly greater than the *median-time-past* of that *block*.
- For each *block* at *block height* 2 or greater on *Mainnet*, or *block height* 653606 or greater on *Testnet*, `nTime` **MUST** be less than or equal to the *median-time-past* of that *block* plus $90 \cdot 60$ seconds.
- The size of a *block* **MUST** be less than or equal to 2000000 bytes.
- [**Sapling** onward] `hashFinalSaplingRoot` **MUST** be $\mathsf{LEBS2OSP}_{256}\left(\mathsf{rt}^{\mathit{SaplingPool}}\right)$ where $\mathsf{rt}^{\mathit{SaplingPool}}$ is the *root* of the *Sapling-pool note commitment tree* for the final *Sapling-pool treestate* of the *block*.
- A *block* **MUST** have at least one *transaction*.
- The first *transaction* in a *block* **MUST** be a *coinbase transaction*, and subsequent *transactions* **MUST NOT** be *coinbase transactions*.
- { TODO: Other rules inherited from **Bitcoin**. }

In addition, a *full validator* **MUST NOT** accept *blocks* with `nTime` more than two hours in the future according to its clock. This is not strictly a consensus rule because it is nondeterministic, and clock time varies between nodes. Also note that a *block* that is rejected by this rule at a given point in time may later be accepted.

Notes:

- The semantics of blocks with *block version number* not equal to 4 is not currently defined. Miners **MUST NOT** create such *blocks*.
- The exclusion of *blocks* with *block version number* ***greater than*** 4 is not a consensus rule; such *blocks* may exist in the *block chain* and **MUST** be treated identically to version 4 *blocks* by *full validators*. Note that a future upgrade might use *block version number* either greater than or less than 4. It is likely that such an upgrade will change the *block* header and/or *transaction* format, and software that parses *blocks* **SHOULD** take this into account.
- The `nVersion` field is a signed integer. (It was specified as unsigned in a previous version of this specification.) A future upgrade might use negative values for this field, or otherwise change its interpretation.
- There is no relation between the values of the `version` field of a *transaction*, and the `nVersion` field of a *block header*.
- Like other serialized fields of type `compactSize`, the `solutionSize` field **MUST** be encoded with the minimum number of bytes (3 in this case), and other encodings **MUST** be rejected. This is necessary to avoid a potential attack in which a miner could test several distinct encodings of each *Equihash* solution against the difficulty filter, rather than only the single intended encoding.
- As in **Bitcoin**, the `nTime` field **MUST** represent a time ***strictly greater than*** the median of the timestamps of the past PoWMedianBlockSpan *blocks*. The Bitcoin Developer Reference [Bitcoin-Block] was previously in error on this point, but has now been corrected.
- The rule limiting `nTime` to be no later than $90 \cdot 60$ seconds after the *median-time-past* is a retrospective consensus change, applied as a soft fork in *zcashd v2.1.1-1*. It had not been violated by any *block* from the given *block heights* in the consensus *block chains* of either *Mainnet* or *Testnet*.
- There are no changes to the *block version number* or format for **Overwinter**.
- Although the *block version number* does not change for **Sapling**, the previously reserved (and ignored) field `hashReserved` has been repurposed for `hashFinalSaplingRoot`. There are no other format changes.

The changes relative to **Bitcoin** version 4 blocks as described in [Bitcoin-Block] are:

- *Block versions* less than 4 are not supported.
- The `hashReserved` / `hashFinalSaplingRoot`, `solutionSize`, and `solution` fields have been added.
- The type of the `nNonce` field has changed from `uint32` to `byte[32]`.
- The maximum *block* size has been doubled to 2000000 bytes.

7.6 Proof of Work

Zcash uses *Equihash* [BK2016] as its *proof-of-work*. The original motivations for changing the *proof-of-work* from SHA-256d used by **Bitcoin** were described in [WG2016].

A *block* satisfies the *proof-of-work* if and only if:

- The `solution` field encodes a *valid Equihash solution* according to § 7.6.1 ***'Equihash'*** on p. 97.
- The *block header* satisfies the difficulty check according to § 7.6.2 ***'Difficulty filter'*** on p. 98.

7.6.1 Equihash

An instance of the *Equihash* algorithm is parameterized by positive integers n and k, such that n is a multiple of $k + 1$. We assume $k \geq 3$, $n \leq 512$, and $(n \cdot \mathsf{floor}(\frac{512}{n})) \bmod 8 = 0$.

The *Equihash* parameters for *Mainnet* and *Testnet* are $n = 200, k = 9$.

Equihash is based on a variation of the Generalized Birthday Problem [AR2017]: given a sequence $X_{1\,..\,\mathrm{N}}$ of n-bit strings, find 2^k distinct X_{i_j} such that $\bigoplus_{j=1}^{2^k} X_{i_j} = 0$.

In *Equihash*, $\mathrm{N} = 2^{\frac{n}{k+1}+1}$, and the sequence $X_{1\,..\,\mathrm{N}}$ is derived from the *block header* and a nonce.

Let powheader :=

32-bit `nVersion`	256-bit `hashPrevBlock`		256-bit `hashMerkleRoot`
256-bit `hashReserved`		32-bit `nTime`	32-bit `nBits`	256-bit `nNonce`

For $i \in \{1\,..\,N\}$, let $X_i = \mathsf{EquihashGen}_{n,k}(\mathsf{powheader}, i)$.

EquihashGen is instantiated in § 5.4.1.9 ***'Equihash Generator'*** on p. 64.

Define I2BEBSP $: (\ell : \mathbb{N}) \times \{0\,..\,2^\ell - 1\} \to \mathbb{B}^{[\ell]}$ as in § 5.1 ***'Integers, Bit Sequences, and Endianness'*** on p. 56.

A *valid Equihash solution* is then a sequence $i : \{1\,..\,N\}^{2^k}$ of ***distinct*** indices that satisfies the following conditions:

Generalized Birthday condition $\bigoplus_{j=1}^{2^k} X_{i_j} = 0$.

Algorithm Binding conditions

- For all $r \in \{1\,..\,k-1\}$, for all $w \in \{0\,..\,2^{k-r}-1\}$: $\bigoplus_{j=1}^{2^r} X_{i_{w\cdot 2^r + j}}$ has $\frac{n\cdot r}{k+1}$ leading zeros; and
- For all $r \in \{1\,..\,k\}$, for all $w \in \{0\,..\,2^{k-r}-1\}$: $i_{w\cdot 2^r + 1\,..\,w\cdot 2^r + 2^{r-1}} < i_{w\cdot 2^r + 2^{r-1} + 1\,..\,w\cdot 2^r + 2^r}$ lexicographically.

Notes:

- This does not include a difficulty condition, because here we are defining validity of an *Equihash* solution independent of difficulty.
- Previous versions of this specification incorrectly specified the range of r to be $\{1\,..\,k-1\}$ for both parts of the algorithm binding condition. The implementation in *zcashd* was as intended.

An *Equihash* solution with $n = 200$ and $k = 9$ is encoded in the `solution` field of a *block header* as follows:

$\mathsf{I2BEBSP}_{21}(i_1 - 1)$	$\mathsf{I2BEBSP}_{21}(i_2 - 1)$	$\cdots$	$\mathsf{I2BEBSP}_{21}(i_{512} - 1)$

Recall from § 5.2 ***'Bit layout diagrams'*** on p. 57 that the bits in the above diagram are ordered from most to least significant in each byte. For example, if the first 3 elements of i are $[69, 42, 2^{21}]$, then the corresponding bit array is:

$\mathsf{I2BEBSP}_{21}(68)$			$\mathsf{I2BEBSP}_{21}(41)$		$\mathsf{I2BEBSP}_{21}(2^{21} - 1)$	
0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0			0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 1		1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1	
8-bit 0	8-bit 2	8-bit 32	8-bit 0	8-bit 10	8-bit 127	8-bit 255	$\cdots$

and so the first 7 bytes of `solution` would be $[0, 2, 32, 0, 10, 127, 255]$.

Note: I2BEBSP is *big-endian*, while integer field encodings in powheader and in the instantiation of EquihashGen are *little-endian*. The rationale for this is that *little-endian* serialization of *block headers* is consistent with **Bitcoin**, but *little-endian* ordering of bits in the solution encoding would require bit-reversal (as opposed to only shifting).

7.6.2 Difficulty filter

Let ToTarget be as defined in § 7.6.4 ***'nBits conversion'*** on p. 99.

Difficulty is defined in terms of a *target threshold*, which is adjusted for each *block* according to the algorithm defined in § 7.6.3 ***'Difficulty adjustment'*** on p. 98.

The difficulty filter is unchanged from **Bitcoin**, and is calculated using SHA-256d on the whole *block header* (including `solutionSize` and `solution`). The result is interpreted as a 256-bit integer represented in *little-endian* byte order, which **MUST** be less than or equal to the *target threshold* given by ToTarget(`nBits`).

7.6.3 Difficulty adjustment

The desired time between *blocks* is called the *block target spacing*. **Zcash** uses a difficulty adjustment algorithm based on DigiShield v3/v4 [DigiByte-PoW], with simplifications and altered parameters, to adjust difficulty to target the desired *block target spacing*. Unlike **Bitcoin**, the difficulty adjustment occurs after every *block*.

PoWLimit, HalvingInterval, PoWAveragingWindow, PoWMaxAdjustDown, PoWMaxAdjustUp, PoWDampingFactor, and PoWTargetSpacing are specified in section § 5.3 ***'Constants'*** on p. 57.

Let ToCompact and ToTarget be as defined in § 7.6.4 ***'nBits conversion'*** on p. 99.

Let nTime(height) be the value of the `nTime` field in the *header* of the *block* at *block height* height.

Let nBits(height) be the value of the `nBits` field in the *header* of the *block* at *block height* height.

Block header fields are specified in § 7.5 ***'Block Header Encoding and Consensus'*** on p. 95.

Define:

$$\mathsf{mean}(S) := \frac{\sum_{i=1}^{\mathsf{length}(S)} S_i}{\mathsf{length}(S)}$$

$$\mathsf{median}(S) := \mathsf{sorted}(S)_{\mathsf{ceiling}((\mathsf{length}(S)+1)/2)}$$

$$\mathsf{bound}_{\mathsf{lower}}^{\mathsf{upper}}(x) := \mathsf{max}(\mathsf{lower}, \mathsf{min}(\mathsf{upper}, x)))$$

$$\mathsf{trunc}(x) := \begin{cases} \mathsf{floor}(x)\,, & \text{if } x \geq 0 \\ -\mathsf{floor}(-x)\,, & \text{otherwise} \end{cases}$$

$$\mathsf{AveragingWindowTimespan} := \mathsf{PoWAveragingWindow} \cdot \mathsf{PoWTargetSpacing}$$

$$\mathsf{MinActualTimespan} := \mathsf{floor}(\mathsf{AveragingWindowTimespan} \cdot (1 - \mathsf{PoWMaxAdjustUp}))$$

$$\mathsf{MaxActualTimespan} := \mathsf{floor}(\mathsf{AveragingWindowTimespan} \cdot (1 + \mathsf{PoWMaxAdjustDown}))$$

$$\mathsf{MedianTime}(\mathsf{height} : \mathbb{N}) := \mathsf{median}([\![\, \mathsf{nTime}(i) \text{ for } i \text{ from } \mathsf{max}(0, \mathsf{height} - \mathsf{PoWMedianBlockSpan}) \text{ up to } \mathsf{height} - 1 \,]\!])$$

$$\mathsf{ActualTimespan}(\mathsf{height} : \mathbb{N}) := \mathsf{MedianTime}(\mathsf{height}) - \mathsf{MedianTime}(\mathsf{height} - \mathsf{PoWAveragingWindow})$$

$$\mathsf{ActualTimespanDamped}(\mathsf{height} : \mathbb{N}) :=$$
$$\mathsf{AveragingWindowTimespan} + \mathsf{trunc}\left(\frac{\mathsf{ActualTimespan}(\mathsf{height}) - \mathsf{AveragingWindowTimespan}}{\mathsf{PoWDampingFactor}}\right)$$

$$\mathsf{ActualTimespanBounded}(\mathsf{height} : \mathbb{N}) := \mathsf{bound}^{\mathsf{MaxActualTimespan}}_{\mathsf{MinActualTimespan}}(\mathsf{ActualTimespanDamped}(\mathsf{height}))$$

$$\mathsf{MeanTarget}(\mathsf{height} : \mathbb{N}) :=$$
$$\begin{cases} \mathsf{PoWLimit}, & \text{if } \mathsf{height} \leq \mathsf{PoWAveragingWindow} \\ \mathsf{mean}([\![\, \mathsf{ToTarget}(\mathsf{nBits}(i)) \text{ for } i \text{ from } \mathsf{height} - \mathsf{PoWAveragingWindow} \text{ up to } \mathsf{height} - 1 \,]\!]), & \\ & \text{otherwise.} \end{cases}$$

The *target threshold* for a given *block height* height is then calculated as:

$$\mathsf{Threshold}(\mathsf{height} : \mathbb{N}) :=$$
$$\begin{cases} \mathsf{PoWLimit}, & \text{if } \mathsf{height} = 0 \\ \mathsf{min}(\mathsf{PoWLimit}, \mathsf{floor}\left(\frac{\mathsf{MeanTarget}(\mathsf{height})}{\mathsf{AveragingWindowTimespan}}\right) \cdot \mathsf{ActualTimespanBounded}(\mathsf{height})), & \\ & \text{otherwise} \end{cases}$$

$$\mathsf{ThresholdBits}(\mathsf{height} : \mathbb{N}) := \mathsf{ToCompact}(\mathsf{Threshold}(\mathsf{height})).$$

Notes:

- The convention used for the height parameters to the functions MedianTime, MeanTarget, ActualTimespan, ActualTimespanDamped, ActualTimespanBounded, Threshold, and ThresholdBits is that these functions use only information from *blocks **preceding*** the given *block height*.
- When the median function is applied to a sequence of even length (which only happens in the definition of MedianTime during the first PoWAveragingWindow $- 1$ *blocks* of the *block chain*), the element that begins the second half of the sequence is taken. This corresponds to the ***zcashd*** implementation, but was not specified correctly in versions of this specification prior to v2019.0.0.

On *Testnet* from *block height* 299188 onward, the difficulty adjustment algorithm is changed to allow minimum-difficulty *blocks*, as described in [ZIP-205]. This change does not apply to *Mainnet*.

7.6.4 nBits conversion

Deterministic conversions between a *target threshold* and a "compact" nBits value are not fully defined in the Bitcoin documentation [Bitcoin-nBits], and so we define them here:

$$\mathsf{size}(x) := \mathsf{ceiling}\left(\frac{\mathsf{bitlength}(x)}{8}\right)$$

$$\mathsf{mantissa}(x) := \mathsf{floor}\left(x \cdot 256^{3 - \mathsf{size}(x)}\right)$$

$$\mathsf{ToCompact}(x) := \begin{cases} \mathsf{mantissa}(x) + 2^{24} \cdot \mathsf{size}(x), & \text{if } \mathsf{mantissa}(x) < 2^{23} \\ \mathsf{floor}\left(\frac{\mathsf{mantissa}(x)}{256}\right) + 2^{24} \cdot (\mathsf{size}(x) + 1), & \text{otherwise} \end{cases}$$

$$\mathsf{ToTarget}(x) := \begin{cases} 0, & \text{if } x \mathbin{\&} 2^{23} = 2^{23} \\ (x \mathbin{\&} (2^{23} - 1)) \cdot 256^{\mathsf{floor}(x/2^{24}) - 3}, & \text{otherwise.} \end{cases}$$

7.6.5 Definition of Work

As explained in § 3.3 *'The Block Chain'* on p. 16, a node chooses the "best" *block chain* visible to it by finding the chain of valid *blocks* with the greatest total work.

Let ToTarget be as defined in § 7.6.4 *'nBits conversion'* on p. 99.

The work of a *block* with value nBits for the `nBits` field in its *block header* is defined as $\mathsf{floor}\left(\frac{2^{256}}{\mathsf{ToTarget}(\mathsf{nBits})+1}\right)$.

7.7 Calculating Block Subsidy and Founders' Reward

In § 3.9 *'Block Subsidy and Founders' Reward'* on p. 20 the *block subsidy*, *miner subsidy*, and *Founders' Reward* are defined. Their amounts in zatoshi are calculated from the *block height* using the formulae below.

Let SlowStartInterval, HalvingInterval, MaxBlockSubsidy, and FoundersFraction be as defined in § 5.3 *'Constants'* on p. 57.

$$\mathsf{SlowStartShift} : \mathbb{N} := \frac{\mathsf{SlowStartInterval}}{2}$$

$$\mathsf{SlowStartRate} : \mathbb{N} := \frac{\mathsf{MaxBlockSubsidy}}{\mathsf{SlowStartInterval}}$$

$$\mathsf{Halving}(\mathsf{height} : \mathbb{N}) :=$$

$$\begin{cases} 0, & \text{if } \mathsf{height} < \mathsf{SlowStartShift} \\ \mathsf{floor}\left(\frac{\mathsf{height} - \mathsf{SlowStartShift}}{\mathsf{HalvingInterval}}\right), & \text{otherwise} \end{cases}$$

$$\mathsf{BlockSubsidy}(\mathsf{height} : \mathbb{N}) := \begin{cases} \mathsf{SlowStartRate} \cdot \mathsf{height}, & \text{if } \mathsf{height} < \mathsf{SlowStartShift} \\ \mathsf{SlowStartRate} \cdot (\mathsf{height} + 1), & \text{if } \mathsf{SlowStartShift} \leq \mathsf{height} \\ & \quad \text{and } \mathsf{height} < \mathsf{SlowStartInterval} \\ \mathsf{floor}\left(\frac{\mathsf{MaxBlockSubsidy}}{2^{\mathsf{Halving}(\mathsf{height})}}\right), & \text{otherwise} \end{cases}$$

$$\mathsf{FoundersReward}(\mathsf{height} : \mathbb{N}) := \begin{cases} \mathsf{BlockSubsidy}(\mathsf{height}) \cdot \mathsf{FoundersFraction}, & \text{if } \mathsf{Halving}(\mathsf{height}) < 1 \\ 0, & \text{otherwise} \end{cases}$$

$$\mathsf{MinerSubsidy}(\mathsf{height}) := \mathsf{BlockSubsidy}(\mathsf{height}) - \mathsf{FoundersReward}(\mathsf{height}).$$

7.8 Payment of Founders' Reward

The *Founders' Reward* is paid by a *transparent output* in the *coinbase transaction*, to one of NumFounderAddresses *transparent addresses*, depending on the *block height*.

For *Mainnet*, $\mathsf{FounderAddressList}_{1..\mathsf{NumFounderAddresses}}$ is:

["t3Vz22vK5z2LcKEdg16Yv4FFneEL1zg9ojd", "t3cL9AucCajm3HXDhb5jBnJK2vapVoXsop3",
"t3fqvkzrrNaMcamkQMwAyHRjfDdM2xQvDTR", "t3TgZ9ZT2CTSK44AnUPi6qeNaHa2eC7pUyF",
"t3SpkcPQPfuRYHsP5vz3Pv86PgKo5m9KVmx", "t3Xt4oQMRPagwbpQqkgAViQgtST4VoSWR6S",
"t3ayBkZ4w6kKXynwoHZFUSSgXRKtogTXNgb", "t3adJBQuaa21u7NxbR8YMzp3km3TbSZ4MGB",
"t3K4aLYagSSBySdrfAGGeUd5H9z5Qvz88t2", "t3RYnsc5nhEvKiva3ZPhfRSk7eyh1CrA6Rk",
"t3Ut4KUq2ZSMTPNE67pBU5LqYCi2q36KpXQ", "t3ZnCNAvgu6CSyHm1vWtrx3aiN98dSAGpnD",
"t3fB9cB3eSYim64BS9xfwAHQUKLgQQroBDG", "t3cwZfKNNj2vXMAHBQeewm6pXhKFdhk18kD",
"t3YcoujXfspWy7rbNUsGKxFEWZqNstGpeG4", "t3bLvCLigc6rbNrUTS5NwkgyVrZcZumTRa4",
"t3VvHWa7r3oy67YtU4LZKGCWa2J6eGHvShi", "t3eF9X6X2dSo7MCvTjfZEzwWrVzquxRLNeY",
"t3esCNwwmcyc8i9qQfyTbYhTqmYXZ9AwK3X", "t3M4jN7hYE2e27yLsuQPPjuVek81WV3VbBj",
"t3gGWxdC67CYNoBbPjNvrrWLAWxPqZLxrVY", "t3LTWeoxeWPbmdkUD3NWBquk4WkazhFBmvU",
"t3P5KKX97gXYFSaSjJPiruQEX84yF5z3Tjq", "t3f3T3nCWsEpzmD35VK62JgQfFig74dV8C9",
"t3Rqonuzz7afkF7156ZA4vi4iimRSEn41hj", "t3fJZ5jYsyxDtvNrWBeoMbvJaQCj4JJgbgX",
"t3Pnbg7XjP7FGPBUuz75H65aczphHgkpoJW", "t3WeKQDxCijL5X7rwFem1MTL9ZwVJkUFhpF",
"t3Y9FNi26J7UtAUC4moaETLbMo8KS1Be6ME", "t3aNRLLsL2y8xcjPheZZwFy3Pcv7CsTwBec",
"t3gQDEavk5VzAAHK8TrQu2BWDLxEiF1unBm", "t3Rbykhx1TUFrgXrmBYrAJe2STxRKFL7G9r",
"t3aaW4aTdP7a8d1VTE1Bod2yhbeggHgMajR", "t3YEiAa6uEjXwFL2v5ztU1fn3yKgzMQqNyo",
"t3g1yUUwt2PbmDvMDevTCPWUcbDatL2iQGP", "t3dPWnep6YqGPuY1CecgbeZrY9iUwH8Yd4z",
"t3QRZXHDPh2hwU46iQs2776kRuuWfwFp4dV", "t3enhACRxi1ZD7e8ePomVGKn7wp7N9fFJ3r",
"t3PkLgT71TnF112nSwBToXsD77yNbx2gJJY", "t3LQtHUDoe7ZhhvddRv4vnaoNAhCr2f4oFN",
"t3fNcdBUbycvbCtsD2n9q3LuxG7jVPvFB8L", "t3dKojUU2EMjs28nHV84TvkVEUDu1M1FaEx",
"t3aKH6NiWN1ofGd8c19rZiqgYpkJ3n679ME", "t3MEXDF9Wsi63KwpPuQdD6by32Mw2bNTbEa",
"t3WDhPfik343yNmPTqtkZAoQZeqA83K7Y3f", "t3PSn5TbMMAEw7Eu36DYctFezRzpX1hzf3M",
"t3R3Y5vnBLrEn8L6wFjPjBLnxSUQsKnmFpv", "t3Pcm737EsVkGTbhsu2NekKtJeG92mvYyoN"]

For *Testnet*, $\mathsf{FounderAddressList}_{1..\mathsf{NumFounderAddresses}}$ is:

["t2UNzUUx8mWBCRYPRezvA363EYXyEpHokyi", "t2N9PH9Wk9xjqYg9iin1Ua3aekJqfAtE543",
"t2NGQjYMQhFndDHguvUw4wZdNdsssA6K7x2", "t2ENg7hHVqqs9JwU5cgjvSbxnT2a9USNfhy",
"t2BkYdVCHzvTJJUTx4yZB8qeegD8QsPx8bo", "t2J8q1xH1EuigJ52MfExyyjYtN3VgvshKDf",
"t2Crq9mydTm37kZokC68HzT6yez3t2FBnFj", "t2EaMPUiQ1kthqcP5UEkF42CAFKJqXCkXC9",
"t2F9dtQc63JDDyrhnfpzvVYTJcr57MkqA12", "t2LPirmnfYSZc481GgZBa6xUGcoovfytBnC",
"t26xfxoSw2UV9Pe5o3C8V4YybQD4SESfxtp", "t2D3k4fNdErd66YxtvXEdft9xuLoKD7CcVo",
"t2DWYBkxKNivdmsMiivNJzutaQGqmoRjRnL", "t2C3kFF9iQRxfc4B9zgbWo4dQLLqzqjpuGQ",
"t2MnT5tzu9HSKcppRyUNwoTp8MUueuSGNaB", "t2AREsWdoW1F8EQYsScsjkgqobmgrkKeUkK",
"t2Vf4wKcJ3ZFtLj4jezUUKkwYR92BLHn5UT", "t2K3fdViH6R5tRuXLphKyoYXyZhyWGghDNY",
"t2VEn3KiKyHSGyzd3nDw6ESWtaCQHwuv9WC", "t2F8XouqdNMq6zzEvxQXHV1TjwZRHwRg8gC",
"t2BS7Mrbaef3fA4xrmkvDisFVXVrRBnZ6Qj", "t2FuSwoLCdBVPwdZuYoHrEzxAb9qy4qjbnL",
"t2SX3U8NtrT6gz5Db1AtQCSGjrpptr8JC6h", "t2V51gZNSoJ5kRL74bf9YTtbZuv8Fcqx2FH",
"t2FyTsLjjdm4jeVwir4xzj7FAkUidbr1b4R", "t2EYbGLekmpqHyn8UBF6kqpahrYm7D6N1Le",
"t2NQTrStZHtJECNFT3dUBLYA9AErxPCmkka", "t2GSWZZJzoesYxfPTWXkFn5UaxjiYxGBU2a",
"t2RpffkzyLRevGM3w9aWdqMX6bd8uuAK3vn", "t2JzjoQqnuXtTGSN7k7yk5keURBGvYofh1d",
"t2AEefc72ieTnsXKmgK2bZNckiwvZe3oPNL", "t2NNs3ZGZFsNj2wvmVd8BSwSfvETgiLrD8J",
"t2ECCQPVcxUCSSQopdNquguEPE14HsVfcUn", "t2JabDUkG8TaqVKYfqDJ3rqkVdHKp6hwXvG",
"t2FGzW5Zdc8Cy98ZKmRygsVGi6oKcmYir9n", "t2DUD8a21FtEFn42oVLp5NGbogY13uyjy9t",
"t2UjVSd3zheHPgAkuX8WQW2CiC9xHQ8EvWp", "t2TBUAhELyHUn8i6SXYsXz5Lmy7kDzA1uT5",
"t2Tz3uCyhP6eizUWDc3bGH7XUC9GQsEyQNc", "t2NysJSZtLwMLWEJ6MH3BsxRh6h27mNcsSy",
"t2KXJVVyyrjVxxSeazbY9ksGyft4qsXUNm9", "t2J9YYtH31cveiLZzjaE4AcuwVho6qjTNzp",
"t2QgvW4sP9zaGpPMH1GRzy7cpydmuRfB4AZ", "t2NDTJP9MosKpyFPHJmfjc5pGCvAU58XGa4",
"t29pHDBWq7qN4EjwSEHg8wEqYe9pkmVrtRP", "t2Ez9KM8VJLuArcxuEkNRAkhNvidKkzXcjJ",
"t2D5y7J5fpXajLbGrMBQkFg2mFN8fo3n8cX", "t2UV2wr1PTaUiybpkV3FdSdGxUJeZdZztyt"]

Note: For *Testnet* only, the addresses from index 4 onward have been changed from what was implemented at launch. This reflects an upgrade on *Testnet*, starting from *block height* 53127. [Zcash-Issue2113]

Each address representation in FounderAddressList denotes a *transparent P2SH multisig address*.

Let SlowStartShift and Halving be defined as in the previous section.

Define:

$$\mathsf{FounderAddressChangeInterval} := \mathsf{ceiling}\left(\frac{\mathsf{SlowStartShift} + \mathsf{HalvingInterval}}{\mathsf{NumFounderAddresses}}\right)$$

$$\mathsf{FounderAddressIndex}(\mathsf{height} : \mathbb{N}) := 1 + \mathsf{floor}\left(\frac{\mathsf{height}}{\mathsf{FounderAddressChangeInterval}}\right)$$

$$\mathsf{FoundersRewardLastBlockHeight} := \mathsf{SlowStartShift} + \mathsf{HalvingInterval} - 1\,.$$

Let $\mathsf{FounderRedeemScriptHash}(\mathsf{height} : \mathbb{N})$ be the *standard redeem script* hash, as specified in [Bitcoin-Multisig], for the *P2SH multisig address* with *Base58Check* form given by $\mathsf{FounderAddressList}_{\mathsf{FounderAddressIndex}(\mathsf{height})}$.

Consensus rule: A *coinbase transaction* at $\mathsf{height} \in \{1\,..\,\mathsf{FoundersRewardLastBlockHeight}\}$ **MUST** include at least one output that pays exactly $\mathsf{FoundersReward}(\mathsf{height})$ zatoshi with a *standard P2SH script* of the form `OP_HASH160` $\mathsf{FounderRedeemScriptHash}(\mathsf{height})$ `OP_EQUAL` as its `scriptPubKey`.

Notes:

- No *Founders' Reward* is required to be paid for $\mathsf{height} > \mathsf{FoundersRewardLastBlockHeight}$ (i.e. after the first *halving*), or for $\mathsf{height} = 0$ (i.e. the *genesis block*).
- The *Founders' Reward* addresses are not treated specially in any other way, and there can be other outputs to them, in *coinbase transactions* or otherwise. In particular, it is valid for a *coinbase transaction* with $\mathsf{height} \in \{1\,..\,\mathsf{FoundersRewardLastBlockHeight}\}$ to have other outputs, possibly to the same address, that do not meet the criterion in the above consensus rule, as long as at least one output meets it.

- The assertion $\mathsf{FounderAddressIndex}(\mathsf{FoundersRewardLastBlockHeight}) \leq \mathsf{NumFounderAddresses}$ holds, ensuring that the *Founders' Reward* address index remains in range for the whole period in which the *Founders' Reward* is paid.

7.9 Changes to the Script System

The `OP_CODESEPARATOR` opcode has been disabled. This opcode also no longer affects the calculation of *SIGHASH transaction hashes*.

7.10 Bitcoin Improvement Proposals

In general, Bitcoin Improvement Proposals (BIPs) do not apply to **Zcash** unless otherwise specified in this section.

All of the BIPs referenced below should be interpreted by replacing "BTC", or "bitcoin" used as a currency unit, with "ZEC"; and "satoshi" with "zatoshi".

The following BIPs apply, otherwise unchanged, to **Zcash**: [BIP-11], [BIP-14], [BIP-31], [BIP-35], [BIP-37], [BIP-61].

The following BIPs apply starting from the **Zcash** *genesis block*, i.e. any activation rules or exceptions for particular *blocks* in the **Bitcoin** *block chain* are to be ignored: [BIP-16], [BIP-30], [BIP-65], [BIP-66].

The effect of [BIP-34] has been incorporated into the consensus rules (§ 7.1.2 *'Transaction Consensus Rules'* on p. 90). This excludes the *Mainnet* and *Testnet genesis blocks*, for which the "height in coinbase" was inadvertently omitted.

[BIP-13] applies with the changes to address version bytes described in § 5.6.1.1 *'Transparent Addresses'* on p. 82.

[BIP-111] applies from *peer-to-peer protocol* version 170004 onward; that is:

- references to protocol version 70002 are to be replaced by 170003;
- references to protocol version 70011 are to be replaced by 170004;
- the reference to protocol version 70000 is to be ignored (**Zcash** nodes have supported Bloom-filtered connections since launch).

8 Differences from the Zerocash paper

8.1 Transaction Structure

Zerocash introduces two new operations, which are described in the paper as new transaction types, in addition to the original transaction type of the cryptocurrency on which it is based (e.g. **Bitcoin**).

In **Zcash**, there is only the original **Bitcoin** transaction type, which is extended to contain a sequence of zero or more **Zcash**-specific operations.

This allows for the possibility of chaining transfers of *shielded* value in a single **Zcash** *transaction*, e.g. to spend a *shielded note* that has just been created. (In **Zcash**, we refer to value stored in *UTXOs* as *transparent*, and value stored in output *notes* of *JoinSplit transfers* or *Output transfers*) as *shielded*.) This was not possible in the **Zerocash** design without using multiple transactions. It also allows *transparent* and *shielded* transfers to happen atomically — possibly under the control of nontrivial script conditions, at some cost in distinguishability.

Computation of *SIGHASH transaction hashes*, as described in § 4.9 *'SIGHASH Transaction Hashing'* on p. 40, was changed to clean up handling of an error case for SIGHASH_SINGLE, to remove the special treatment of `OP_CODESEPARATOR`, and to include **Zcash**-specific fields in the hash [ZIP-76].

8.2 Memo Fields

Zcash adds a *memo field* sent from the creator of a *JoinSplit description* to the recipient of each output *note*. This feature is described in more detail in § 3.2.1 *'Note Plaintexts and Memo Fields'* on p. 14.

8.3 Unification of Mints and Pours

In the original **Zerocash** protocol, there were two kinds of transaction relating to *shielded notes*:

- a "Mint" transaction takes value from *UTXOs (unspent transaction outputs)* as input and produces a new *shielded note* as output.
- a "Pour" transaction takes up to N^{old} *shielded notes* as input, and produces up to N^{new} *shielded notes* and a *UTXO* as output.

Only "Pour" transactions included a *zk-SNARK* proof.

[Pre-**Sapling**] In **Zcash**, the sequence of operations added to a *transaction* (see § 8.1 *'Transaction Structure'* on p. 103) consists only of *JoinSplit transfers*. A *JoinSplit transfer* is a Pour operation generalized to take a *UTXO* as input, allowing *JoinSplit transfers* to subsume the functionality of Mints. An advantage of this is that a **Zcash** *transaction* that takes input from a *UTXO* can produce up to N^{new} output *notes*, improving the indistinguishability properties of the protocol. A related change conceals the input arity of the *JoinSplit transfer*: an unused (zero-valued) input is indistinguishable from an input that takes value from a *note*.

This unification also simplifies the fix to the Faerie Gold attack described below, since no special case is needed for Mints.

[**Sapling** onward] In Sapling, there are still no "Mint" transactions. Instead of *JoinSplit transfers*, there are *Spend transfers* and *Output transfers*. These make use of *Pedersen value commitments* to represent the shielded values that are transferred. Because these commitments are additively homomorphic, it is possible to check that all *Spend transfers* and *Output transfers* balance; see § 4.12 *'Balance and Binding Signature (Sapling)'* on p. 42 for detail. This reduces the granularity of the circuit, allowing a substantial performance improvement (orthogonal to other Sapling circuit improvements) when the numbers of *shielded* inputs and outputs are significantly different. This comes at the cost of revealing the exact number of *shielded* inputs and outputs, but *dummy* (zero-valued) outputs are still possible.

8.4 Faerie Gold attack and fix

When a *shielded note* is created in **Zerocash**, the creator is supposed to choose a new ρ value at random. The *nullifier* of the *note* is derived from its *spending key* (a_{sk}) and ρ. The *note commitment* is derived from the recipient address component a_{pk}, the value v, and the *commitment trapdoor* rcm, as well as ρ. However nothing prevents creating multiple *notes* with different v and rcm (hence different *note commitments*) but the same ρ.

An adversary can use this to mislead a *note* recipient, by sending two *notes* both of which are verified as valid by Receive (as defined in [BCGGMTV2014, Figure 2]), but only one of which can be spent.

We call this a "Faerie Gold" attack — referring to various Celtic legends in which faeries pay mortals in what appears to be gold, but which soon after reveals itself to be leaves, gorse blossoms, gingerbread cakes, or other less valuable things [LG2004].

This attack does not violate the security definitions given in [BCGGMTV2014]. The issue could be framed as a problem either with the definition of Completeness, or the definition of Balance:

- The Completeness property asserts that a validly received *note* can be spent provided that its *nullifier* does not appear on the ledger. This does not take into account the possibility that distinct *notes*, which are validly received, could have the same *nullifier*. That is, the security definition depends on a protocol detail –*nullifiers*– that is not part of the intended abstract security property, and that could be implemented incorrectly.

- The Balance property only asserts that an adversary cannot obtain *more* funds than they have minted or received via payments. It does not prevent an adversary from causing others' funds to decrease. In a Faerie Gold attack, an adversary can cause spending of a *note* to reduce (to zero) the effective value of another *note* for which the adversary does not know the *spending key*, which violates an intuitive conception of global balance.

These problems with the security definitions need to be repaired; how to do so is discussed in [Hopwood2022], but that is outside the scope of this specification. Here we only describe how **Zcash** addresses the immediate attack.

It would be possible to address the attack by requiring that a recipient remember all of the ρ values for all *notes* they have ever received, and reject duplicates (as proposed in [GGM2016]). However, this requirement would interfere with the intended **Zcash** feature that a holder of a *spending key* can recover access to (and be sure that they are able to spend) all of their funds, even if they have forgotten everything but the *spending key*.

[Sprout] Instead, **Zcash** enforces that an adversary must choose distinct values for each ρ, by making use of the fact that all of the *nullifiers* in *JoinSplit descriptions* that appear in a *valid block chain* must be distinct. This is true regardless of whether the *nullifiers* corresponded to real or *dummy notes* (see § 4.7.1 ***'Dummy Notes (Sprout)'*** on p. 38). The *nullifiers* are used as input to hSigCRH to derive a public value $\mathsf{h}_{\mathsf{Sig}}$ which uniquely identifies the transaction, as described in § 4.3 ***'JoinSplit Descriptions'*** on p. 34. ($\mathsf{h}_{\mathsf{Sig}}$ was already used in **Zerocash** in a way that requires it to be unique in order to maintain indistinguishability of *JoinSplit descriptions*; adding the *nullifiers* to the input of the hash used to calculate it has the effect of making this uniqueness property robust even if the *transaction* creator is an adversary.)

[Sprout] The ρ value for each output *note* is then derived from a random private seed φ and $\mathsf{h}_{\mathsf{Sig}}$ using $\mathsf{PRF}^{\rho}_{\varphi}$. The correct construction of ρ for each output *note* is enforced by § 4.16.1 ***'JoinSplit Statement (Sprout)'*** on p. 47 in the *JoinSplit statement*.

[Sprout] Now even if the creator of a *JoinSplit description* does not choose φ randomly, uniqueness of *nullifiers* and *collision resistance* of both hSigCRH and PRF^{ρ} will ensure that the derived ρ values are unique, at least for any two *JoinSplit descriptions* that get into a *valid block chain*. This is sufficient to prevent the Faerie Gold attack.

A variation on the attack attempts to cause the *nullifier* of a sent *note* to be repeated, without repeating ρ. However, since the *nullifier* is computed as $\mathsf{PRF}^{\mathsf{nfSprout}}_{\mathsf{a_{sk}}}(\rho)$ or $\mathsf{PRF}^{\mathsf{nfSapling}}_{\mathsf{nk}}(\rho\star)$; this is only possible if the adversary finds a collision across both inputs on $\mathsf{PRF}^{\mathsf{nfSprout}}$ or $\mathsf{PRF}^{\mathsf{nfSapling}}$, which is assumed to be infeasible — see § 4.1.2 ***'Pseudo Random Functions'*** on p. 22.

[Sprout] Crucially, "*nullifier* integrity" is enforced whether or not the $\mathsf{enforceMerklePath}_i$ flag is set for an input *note* (§ 4.16.1 ***'JoinSplit Statement (Sprout)'*** on p. 47). If this were not the case then an adversary could perform the attack by creating a zero-valued *note* with a repeated *nullifier*, since the *nullifier* would not depend on the value.

[Sprout] *Nullifier* integrity also prevents a "roadblock attack" in which the adversary sees a victim's *transaction*, and is able to publish another *transaction* that is mined first and blocks the victim's *transaction*. This attack would be possible if the public value(s) used to enforce uniqueness of ρ could be chosen arbitrarily by the *transaction* creator: the victim's *transaction*, rather than the adversary's, would be considered to be repeating these values. In the chosen solution that uses *nullifiers* for these public values, they are enforced to be dependent on *spending keys* controlled by the original *transaction* creator (whether or not each input *note* is a *dummy*), and so a roadblock attack cannot be performed by another party who does not know these keys.

[**Sapling** onward] In Sapling, uniqueness of ρ is ensured by making it dependent on the position of the *note commitment* in the *Sapling pool note commitment tree*. Specifically, $\rho = \mathsf{cm} + [\mathsf{pos}]\,\mathcal{J}^{\mathsf{Sapling}}$, where $\mathcal{J}^{\mathsf{Sapling}}$ is a generator independent of the generators used in $\mathsf{NoteCommit}^{\mathsf{Sapling}}$. Therefore, ρ commits uniquely to the *note* and its position, and this commitment is *collision-resistant* by the same argument used to prove *collision resistance* of *Pedersen hashes*. Note that it is possible for two distinct *Sapling-pool positioned notes* (having different ρ values and *nullifiers*, but different *note positions*) to have the same *note commitment*, but this causes no security problem. Roadblock attacks are not possible because a given *note position* does not repeat for outputs of different *transactions* in the same *block chain*. Note that this depends on the fact that the value is bound by the *note commitment*: it could be the case that the adversary uses a *dummy note* that is not required to have a *note commitment* in the *note commitment tree* when it is spent. If this happens and the victim's *note* is not a *dummy*, the *note commitments*

will differ and so will the *nullifiers*. If both *notes* are dummies, the adversary cannot know the inputs to the *note commitment* since they are generated at random for the victim's spend, regardless of the adversary's potential knowledge of viewing keys.

8.5 Internal hash collision attack and fix

The **Zerocash** security proof requires that the composition of $\mathsf{COMM}_{\mathsf{rcm}}$ and $\mathsf{COMM}_{\mathsf{s}}$ is a computationally *binding* commitment to its inputs a_{pk}, v, and ρ. However, the instantiation of $\mathsf{COMM}_{\mathsf{rcm}}$ and $\mathsf{COMM}_{\mathsf{s}}$ in section 5.1 of the paper did not meet the definition of a *binding* commitment at a 128-bit security level. Specifically, the internal hash of a_{pk} and ρ is truncated to 128 bits (motivated by providing statistical *hiding* security). This allows an attacker, with a work factor on the order of 2^{64}, to find distinct pairs $(\mathsf{a}_{\mathsf{pk}}, \rho)$ and $(\mathsf{a}_{\mathsf{pk}}', \rho')$ with colliding outputs of the truncated hash, and therefore the same *note commitment*. This would have allowed such an attacker to break the Balance property by *double-spending notes*, potentially creating arbitrary amounts of currency for themself [HW2016].

Zcash uses a simpler construction with a single hash evaluation for the commitment: SHA-256 for *Sprout-pool notes*, and PedersenHashToPoint for *Sapling-pool notes*. The motivation for the nested construction in **Zerocash** [BCGGMTV2014, section 1.3, under step 3] was to allow Mint transactions to be publically verified without requiring *zk-SNARK proofs*. Since **Zcash** combines "Mint" and "Pour" transactions into generalized *JoinSplit transfers* (for Sprout), or *Spend transfers* and *Output transfers* (for Sapling), and each transfer always uses a *zk-SNARK proof*, **Zcash** does not require the nesting. A side benefit is that this reduces the cost of computing the *note commitments*: for Sprout it reduces the number of SHA256Compress evaluations needed to compute each *note commitment* from three to two, saving a total of four SHA256Compress evaluations in the *JoinSplit statement*.

[Sprout] **Note:** The full SHA-256 algorithm is used for $\mathsf{NoteCommit}^{\mathsf{Sprout}}$, with randomness appended after the commitment input. The commitment input can be split into two blocks, call them x of length 64 bytes, and y of the remaining length (9 bytes). Let $\mathsf{COMM}'_r(z : \mathbb{B}^{\mathbb{Y}^{[41]}})$ be the *commitment scheme* that applies SHA256Compress with the first 32 bytes of z in the IV, and the rest of z (9 bytes), the randomness r (32 bytes), and padding up to 64 bytes in the SHA256Compress input block. Then we have $\mathsf{NoteCommit}^{\mathsf{Sprout}}_r(x \,||\, y) = \mathsf{COMM}'_r(\mathsf{SHA256Compress}(x) \,||\, y)$. Suppose we make the reasonable assumption that COMM' is a computationally *binding* and *hiding commitment scheme*. If SHA256Compress is *collision-resistant* with the standard IV[10], then $\mathsf{NoteCommit}^{\mathsf{Sprout}}$ is as secure for *binding* as COMM'. Also $\mathsf{NoteCommit}^{\mathsf{Sprout}}$ is as secure for *hiding* as COMM' (without any assumption on SHA256Compress). This effectively rules out potential concerns about the Merkle–Damgård structure [Damgård1989] of SHA-256 causing any security problem for $\mathsf{NoteCommit}^{\mathsf{Sprout}}$.

[Sprout] **Note:** *Sprout-pool note commitments* are not statistically *hiding*, so for *Sprout-pool notes*, **Zcash** does not support the "everlasting anonymity" property described in [BCGGMTV2014, section 8.1], even when used as described in that section. While it is possible to define a statistically *hiding*, computationally *binding commitment scheme* for this use at a 128-bit security level, the overhead of doing so within the *JoinSplit statement* was not considered to justify the benefits.

[**Sapling** onward] In Sapling, *Pedersen* commitments are used instead of SHA256Compress. These commitments are statistically *hiding*, and so "everlasting anonymity" is supported for *Sapling-pool notes* under the same conditions as in **Zerocash** (by the protocol, not necessarily by *zcashd*). Note that *diversified payment addresses* can be linked if the *Decisional Diffie–Hellman Problem* on the *Jubjub curve* can be broken.

8.6 Changes to PRF inputs and truncation

The format of inputs to the *PRFs* instantiated in § 5.4.2 ***'Pseudo Random Functions'*** on p. 64 has changed relative to **Zerocash**. There is also a requirement for another *PRF*, PRF^{ρ}, which must be domain-separated from the others.

In the **Zerocash** protocol, ρ_i^{old} is truncated from 256 to 254 bits in the input to $\mathsf{PRF}^{\mathsf{sn}}$ (which corresponds to $\mathsf{PRF}^{\mathsf{nfSprout}}$ in **Zcash**). Also, $\mathsf{h}_{\mathsf{Sig}}$ is truncated from 256 to 253 bits in the input to $\mathsf{PRF}^{\mathsf{pk}}$. These truncations are not taken into account in the security proofs.

[10] If SHA256Compress is not *collision-resistant* with the standard IV, then SHA-256 is not *collision-resistant* for a 2-block input.

Both truncations affect the validity of the proof sketch for Lemma D.2 in the proof of Ledger Indistinguishability in [BCGGMTV2014, Appendix D].

In more detail:

- In the argument relating $\mathbf{H}$ and Θ_2, it is stated that in Θ_2, "for each $i \in \{1, 2\}$, $\mathsf{sn}_i := \mathsf{PRF}^{\mathsf{sn}}_{\mathsf{a_{sk}}}(\rho)$ for a random (and not previously used) ρ". It is also argued that "the calls to $\mathsf{PRF}^{\mathsf{sn}}_{\mathsf{a_{sk}}}$ are each by definition unique". The latter assertion depends on the fact that ρ is "not previously used". However, the argument is incorrect because the truncated input to $\mathsf{PRF}^{\mathsf{sn}}_{\mathsf{a_{sk}}}$, i.e. $[\rho]_{254}$, may repeat even if ρ does not.
- In the same argument, it is stated that "with overwhelming probability, $\mathsf{h_{Sig}}$ is unique". In fact what is required to be unique is the truncated input to $\mathsf{PRF}^{\mathsf{pk}}$, i.e. $[\mathsf{h_{Sig}}]_{253} = [\mathsf{CRH}(\mathsf{pk_{sig}})]_{253}$. In practice this value will be unique under a plausible assumption on CRH provided that $\mathsf{pk_{sig}}$ is chosen randomly, but no formal argument for this is presented.

Note that ρ is truncated in the input to $\mathsf{PRF}^{\mathsf{sn}}$ but not in the input to $\mathsf{COMM}_{\mathsf{rcm}}$, which further complicates the analysis.

As further evidence that it is essential for the proofs to explicitly take any such truncations into account, consider a slightly modified protocol in which ρ is truncated in the input to $\mathsf{COMM}_{\mathsf{rcm}}$ but not in the input to $\mathsf{PRF}^{\mathsf{sn}}$. In that case, it would be possible to violate balance by creating two *notes* for which ρ differs only in the truncated bits. These *notes* would have the same *note commitment* but different *nullifiers*, so it would be possible to spend the same value twice.

[Sprout] For resistance to Faerie Gold attacks as described in § 8.4 ***'Faerie Gold attack and fix'*** on p. 104, **Zcash** depends on *collision resistance* of hSigCRH and PRF^{ρ} (instantiated using BLAKE2b-256 and SHA256Compress respectively). *Collision resistance* of a truncated hash does not follow from *collision resistance* of the original hash, even if the truncation is only by one bit. This motivated avoiding truncation along any path from the inputs to the computation of $\mathsf{h_{Sig}}$ to the uses of ρ.

[Sprout] Since the *PRFs* are instantiated using SHA256Compress which has an input block size of 512 bits (of which 256 bits are used for the *PRF* input and 4 bits are used for domain separation), it was necessary to reduce the size of the PRF key to 252 bits. The key is set to $\mathsf{a_{sk}}$ in the case of $\mathsf{PRF}^{\mathsf{addr}}$, $\mathsf{PRF}^{\mathsf{nfSprout}}$, and $\mathsf{PRF}^{\mathsf{pk}}$, and to φ (which does not exist in **Zerocash**) for PRF^{ρ}, and so those values have been reduced to 252 bits. This is preferable to requiring reasoning about truncation, and 252 bits is quite sufficient for security of these cryptovalues.

Sapling uses *Pedersen hashes* and BLAKE2s where Sprout used SHA256Compress. *Pedersen hashes* can be efficiently instantiated for arbitrary input lengths. BLAKE2s has an input block size of 512 bits, and uses a finalization flag rather than padding of the last input block; it also supports domain separation via a personalization parameter distinct from the input. Therefore, there is no need for truncation in the inputs to any of these hashes. Note however that the ***output*** of $\mathsf{CRH}^{\mathsf{ivk}}$ is truncated, requiring a security assumption on BLAKE2s truncated to 251 bits (see § 5.4.1.5 '$\mathsf{CRH}^{\mathsf{ivk}}$ ***Hash Function***' on p. 60).

8.7 In-band secret distribution

Zerocash specified ECIES (referencing Certicom's SEC 1 standard) as the encryption scheme used for the *in-band* secret distribution. This has been changed to a *key agreement scheme* based on Curve25519 (for Sprout) or Jubjub (for Sapling) and the authenticated encryption algorithm AEAD_CHACHA20_POLY1305. This scheme is still loosely based on ECIES, and on the crypto_box_seal scheme defined in libsodium [libsodium-Seal].

The motivations for this change were as follows:

- The **Zerocash** paper did not specify the curve to be used. We believe that Curve25519 has significant *side-channel* resistance, performance, implementation complexity, and robustness advantages over most other available curve choices, as explained in [Bernstein2006]. For Sapling, the *Jubjub curve* was designed according to a similar design process following the "Safe curves" criteria [BL-SafeCurves] [Hopwood2018]. This retains Curve25519's advantages while keeping *shielded payment address* sizes short, because the same *public key* material supports both encryption and spend authentication.

- ECIES permits many options, which were not specified. There are at least –counting conservatively– 576 possible combinations of options and algorithms over the four standards (ANSI X9.63, IEEE Std 1363a-2004, ISO/IEC 18033-2, and SEC 1) that define ECIES variants [MÁEÁ2010].
- Although the **Zerocash** paper states that ECIES satisfies *key privacy* (as defined in [BBDP2001]), it is not clear that this holds for all curve parameters and key distributions. For example, if a group of non-prime order is used, the distribution of ciphertexts could be distinguishable depending on the order of the points representing the ephemeral and recipient *public keys*. Public key validity is also a concern. Curve25519 (and Jubjub) key agreement is defined in a way that avoids these concerns due to the curve structure and the "clamping" of *private keys* (or explicit cofactor multiplication and point validation for Sapling).
- Unlike the DHAES/DHIES proposal on which it is based [ABR1999], ECIES does not require a representation of the sender's *ephemeral public key* to be included in the input to the KDF, which may impair the security properties of the scheme. (The Std 1363a-2004 version of ECIES [IEEE2004] has a "DHAES mode" that allows this, but the representation of the key input is underspecified, leading to incompatible implementations.) The scheme we use for Sprout has both the ephemeral and recipient *public key* encodings –which are unambiguous for Curve25519– and also $\mathsf{h_{Sig}}$ and a nonce as described below, as input to the KDF. For Sapling, it is only possible to include the ephemeral public key encoding, but this is sufficient to retain the original security properties of DHAES. Note that being able to break the Elliptic Curve Diffie–Hellman Problem on Curve25519 or Jubjub (without breaking AEAD_CHACHA20_POLY1305 as an authenticated encryption scheme or BLAKE2b-256 as a KDF) would not help to decrypt the *transmitted note(s) ciphertext* unless $\mathsf{pk_{enc}}$ or $\mathsf{pk_d}$ is known or guessed.
- [Sprout] The KDF also takes a public seed $\mathsf{h_{Sig}}$ as input. This can be modeled as using a different "randomness extractor" for each *JoinSplit transfer*, which limits degradation of security with the number of *JoinSplit transfers*. This facilitates security analysis as explained in [DGKM2011] — see section 7 of that paper for a security proof that can be applied to this construction under the assumption that single-block BLAKE2b-256 is a "*weak PRF*". Note that $\mathsf{h_{Sig}}$ is authenticated, by the *zk-SNARK proof*, as having been chosen with knowledge of $\mathsf{a}^{\mathsf{old}}_{\mathsf{sk},1..N^{\mathsf{old}}}$, so an adversary cannot modify it in a ciphertext from someone else's transaction for use in a chosen-ciphertext attack without detection. (In Sapling, there is no equivalent to $\mathsf{h_{Sig}}$, but the *binding signature* and *spend authorization signatures* prevent such modifications.)
- [Sprout] The scheme used by Sprout includes an optimization that reuses the same ephemeral key (with different nonces) for the two ciphertexts encrypted in each *JoinSplit description*.

The security proofs of [ABR1999] can be adapted straightforwardly to the resulting scheme. Although DHAES as defined in that paper does not pass the recipient *public key* or a public seed to the *hash function* H, this does not impair the proof because we can consider H to be the specialization of our KDF to a given recipient key and seed. (Passing the recipient *public key* to the KDF could in principle compromise *key privacy*, but not confidentiality of encryption.) [Sprout] It is necessary to adapt the "HDH independence" assumptions and the proof slightly to take into account that the ephemeral key is reused for two encryptions.

Note that the 256-bit key for AEAD_CHACHA20_POLY1305 maintains a high concrete security level even under attacks using parallel hardware [Bernstein2005] in the multi-user setting [Zaverucha2012]. This is especially necessary because the privacy of **Zcash** transactions may need to be maintained far into the future, and upgrading the encryption algorithm would not prevent a future adversary from attempting to decrypt ciphertexts encrypted before the upgrade. Other cryptovalues that could be attacked to break the privacy of transactions are also sufficiently long to resist parallel brute force in the multi-user setting: for Sprout, $\mathsf{a_{sk}}$ is 252 bits, and $\mathsf{sk_{enc}}$ is no shorter than $\mathsf{a_{sk}}$.

In Sapling, ivk is an output of $\mathsf{CRH}^{\mathsf{ivk}}$, which is a 251-bit value. This degree of divergence from a uniform distribution on the scalar field is not expected to cause any weakness in *note* encryption.

For all *shielded protocols*, the checking of *note commitments* makes *partitioning oracle attacks* [LGR2021] against the *transmitted note ciphertext* infeasible, at least in the absence of *side-channel* attacks. The following argument applies to Sapling, but can be adapted to Sprout by replacing ivk with $\mathsf{sk_{enc}}$, $\mathsf{pk_d}$ with $\mathsf{pk_{enc}}$, and using a fixed base. The decryption procedure for *transmitted note ciphertexts* in Sapling is specified in § 4.18.2 on p. 53; it ensures that a successful decryption cannot occur unless the decrypted *note plaintext* encodes a *note* consistent with the *note commitment* (encoded as the cm_u field of the *Output description*). Suppose that it were feasible to find a pair of *transmitted note ciphertext* and *note commitment* that decrypts successfully for two different *incoming viewing*

keys ivk_1 and ivk_2. Assuming that the *note commitment scheme* is *binding* and that *note commitment* opens to a *note* with $\mathsf{pk_d}$ and $\mathsf{g_d}$, we must have $\mathsf{pk_d} = \mathsf{KA.Agree}(\mathsf{ivk}_1, \mathsf{g_d}) = \mathsf{KA.Agree}(\mathsf{ivk}_2, \mathsf{g_d})$. But this is impossible given that $\mathsf{g_d}$ has order greater than the maximum value of ivk that can be an output of $\mathsf{CRH}^{\mathsf{ivk}}$.

There is also a decryption procedure making use of *outgoing ciphertexts* in Sapling, as specified in § 4.18.3 on p. 53. It checks (via KA.DerivePublic) that the decrypted esk value is consistent with the *transmitted note ciphertext*, which is protected from *partitioning oracle attacks* as described above. It also checks that the $\mathsf{pk_d}$ value is consistent with the *note commitment*. Since these are the only fields in an *outgoing ciphertext*, even if a *partitioning oracle attack* occurred against an *outgoing ciphertext*, it could not result in any equivocation of the decrypted data. Because ovk and ock are each 256 bits, *partitioning oracle attacks* that speed up a search for these keys (analogous to the attacks against Password-based AEAD in [LGR2021]) are infeasible, even given knowledge of ivk.

8.8 Omission in Zerocash security proof

The abstract **Zerocash** protocol requires $\mathsf{PRF}^{\mathsf{addr}}$ only to be a *PRF*; it is not specified to be *collision-resistant*. This reveals a flaw in the proof of the Balance property.

Suppose that an adversary finds a collision on $\mathsf{PRF}^{\mathsf{addr}}$ such that $\mathsf{a}_{\mathsf{sk}}^1$ and $\mathsf{a}_{\mathsf{sk}}^2$ are distinct *spending keys* for the same $\mathsf{a_{pk}}$. Because the *note commitment* is to $\mathsf{a_{pk}}$, but the *nullifier* is computed from $\mathsf{a_{sk}}$ (and ρ), the adversary is able to *double-spend* the note, once with each $\mathsf{a_{sk}}$. This is not detected because each Spend reveals a different *nullifier*. The *JoinSplit statements* are still valid because they can only check that the $\mathsf{a_{sk}}$ in the witness is ***some*** preimage of the $\mathsf{a_{pk}}$ used in the *note commitment*.

The error is in the proof of Balance in [BCGGMTV2014, Appendix D.3]. For the "$\mathcal{A}$ violates Condition I" case, the proof says:

> "(i) If $\mathsf{cm}_1^{\mathsf{old}} = \mathsf{cm}_2^{\mathsf{old}}$, then the fact that $\mathsf{sn}_1^{\mathsf{old}} \neq \mathsf{sn}_2^{\mathsf{old}}$ implies that the witness a contains two distinct openings of $\mathsf{cm}_1^{\mathsf{old}}$ (the first opening contains $(\mathsf{a}_{\mathsf{sk},1}^{\mathsf{old}}, \rho_1^{\mathsf{old}})$, while the second opening contains $(\mathsf{a}_{\mathsf{sk},2}^{\mathsf{old}}, \rho_2^{\mathsf{old}})$). This violates the *binding* property of the *commitment scheme* COMM."

In fact the openings do not contain $\mathsf{a}_{\mathsf{sk},i}^{\mathsf{old}}$; they contain $\mathsf{a}_{\mathsf{pk},i}^{\mathsf{old}}$. (In Sprout $\mathsf{cm}_i^{\mathsf{old}}$ opens directly to $(\mathsf{a}_{\mathsf{pk},i}^{\mathsf{old}}, \mathsf{v}_i^{\mathsf{old}}, \rho_i^{\mathsf{old}})$, and in **Zerocash** it opens to $(\mathsf{v}_i^{\mathsf{old}}, \mathsf{COMM_s}(\mathsf{a}_{\mathsf{pk},i}^{\mathsf{old}}, \rho_i^{\mathsf{old}}))$.)

A similar error occurs in the argument for the "$\mathcal{A}$ violates Condition II" case.

The flaw is not exploitable for the actual instantiations of $\mathsf{PRF}^{\mathsf{addr}}$ in **Zerocash** and Sprout, which ***are*** *collision-resistant* assuming that SHA256Compress is.

The proof can be straightforwardly repaired. The intuition is that we can rely on *collision resistance* of $\mathsf{PRF}^{\mathsf{addr}}$ (on both its arguments) to argue that distinctness of $\mathsf{a}_{\mathsf{sk},1}^{\mathsf{old}}$ and $\mathsf{a}_{\mathsf{sk},2}^{\mathsf{old}}$, together with constraint 1(b) of the *JoinSplit statement* (see § 4.16.1 ***'JoinSplit Statement (Sprout)'*** on p. 47), implies distinctness of $\mathsf{a}_{\mathsf{pk},1}^{\mathsf{old}}$ and $\mathsf{a}_{\mathsf{pk},2}^{\mathsf{old}}$, therefore distinct openings of the *note commitment* when Condition I or II is violated.

8.9 Miscellaneous

- The paper defines a *note* as $((\mathsf{a_{pk}}, \mathsf{pk_{enc}}), \mathsf{v}, \rho, \mathsf{rcm}, \mathsf{s}, \mathsf{cm})$, whereas this specification defines a *Sprout-pool note* as $(\mathsf{a_{pk}}, \mathsf{v}, \rho, \mathsf{rcm})$. The instantiation of $\mathsf{COMM_s}$ in section 5.1 of the paper did not actually use s, and neither does the new instantiation of $\mathsf{NoteCommit}^{\mathsf{Sprout}}$ in Sprout. $\mathsf{pk_{enc}}$ is also not needed as part of a *note*: it is not an input to $\mathsf{NoteCommit}^{\mathsf{Sprout}}$ nor is it constrained by the **Zerocash** `POUR` *statement* or the **Zcash** *JoinSplit statement*. cm can be computed from the other fields. (The definition of *notes* for Sapling is different again.)
- The length of proof encodings given in the paper is 288 bytes. [Sprout] This differs from the 296 bytes specified in § 5.4.9.1 ***'BCTV14'*** on p. 79, because both the x-coordinate and compressed y-coordinate of each point need to be represented. Although it is possible to encode a proof in 288 bytes by making use of the fact that elements of $\mathbb{F}_q$ can be represented in 254 bits, we prefer to use the standard formats for points defined in

[IEEE2004]. The fork of *libsnark* used by **Zcash** uses this standard encoding rather than the less efficient (uncompressed) one used by upstream *libsnark*. In Sapling, a customized encoding is used for BLS12-381 points in Groth16 proofs to minimize length.

- The range of monetary values differs. In **Zcash** this range is $\{0 \,..\, \text{MAX_MONEY}\}$, while in **Zerocash** it is $\{0 \,..\, 2^{\ell_{\text{value}}} - 1\}$. (The *JoinSplit statement* still only directly enforces that the sum of amounts in a given *JoinSplit transfer* is in the latter range; this enforcement is technically redundant given that the Balance property holds.)

9 Acknowledgements

The inventors of **Zerocash** are Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza.

The designers of the **Zcash** protocol are the **Zerocash** inventors and also Daira-Emma Hopwood, Sean Bowe, Jack Grigg, Simon Liu, Taylor Hornby, Nathan Wilcox, Zooko Wilcox, Jay Graber, Eirik Ogilvie-Wigley, Ariel Gabizon, George Tankersley, Ying Tong Lai, Kris Nuttycombe, Jack Gavigan, Steven Smith, and Greg Pfeil. The *Equihash proof-of-work* algorithm was designed by Alex Biryukov and Dmitry Khovratovich.

The authors would like to thank everyone with whom they have discussed the **Zerocash** and **Zcash** protocol designs; in addition to the preceding, this includes Mike Perry, isis agora lovecruft, Leif Ryge, Andrew Miller, Ben Blaxill, Samantha Hulsey, Alex Balducci, Jake Tarren, Solar Designer, Ling Ren, John Tromp, Paige Peterson, jl777, Alison Stevenson, Maureen Walsh, Filippo Valsorda, Zaki Manian, Kexin Hu, Brian Warner, Mary Maller, Michael Dixon, Andrew Poelstra, Benjamin Winston, Josh Cincinnati, Kobi Gurkan, Weikeng Chen, Henry de Valence, Deirdre Connolly, Chelsea Komlo, Zancas Wilcox, Jane Lusby, teor, Izaak Meckler, Zac Williamson, Vitalik Buterin, Jakub Zalewski, Oana Ciobotaru, Andre Serrano, Brad Miller, Charlie O'Keefe, David Campbell, Elena Giralt, Francisco Gindre, Joseph Van Geffen, Josh Swihart, Kevin Gorham, Larry Ruane, Marshall Gaucher, Ryan Taylor, Sasha Meyer, Conrado Gouvêa, Aditya Bharadwaj, Andrew Arnott, Arya, Andrea Kobrlova, Lukáš Korba, Honza Rychnovský, Schell Scivally, @Peacemonger, Nicolas Gailly, Danny Willems, Jason McGee, Mark Henderson, Tomek Piotrowski, Mariusz Pilarek, Paul Dann, Dev Ojha, Evan Forbes, Anna Rose, Jens Groth, Judah Caruso, Pablo Kogan, Vivek Arte, Daniel Benarroch, Aurelien Nicolas, Alexey Koren, Antoine Rondelet, Constance Beguier, Georgios Konstantopoulos, Matt Luongo, Alex Bornstein, @dontbeevil, @GGuy, @mistfpga, Gordon Mohr, Dimitris Apostolou, Gregory Maxwell, Pieter Wuille, Julian Abraham, and no doubt others. We would also like to thank the designers and developers of **Bitcoin** and *BitcoinCore*.

Zcash has benefited from security audits performed by NCC Group, Coinspect, Least Authority, Mary Maller, Kudelski Security, QEDIT, and Trail of Bits. We also thank Mary Maller for her work on reviewing the security proofs for Halo 2 (any remaining errors are ours).

The Faerie Gold attack was found by Zooko Wilcox (who also came up with the name) and Brian Warner. The fix for this attack in Sprout was proposed by Daira-Emma Hopwood; subsequent analysis of variations on the attack was performed by Daira-Emma Hopwood and Sean Bowe.

The internal hash collision attack was found by Taylor Hornby.

The error in the **Zerocash** proof of Balance relating to *collision resistance* of $\mathsf{PRF}^{\mathsf{addr}}$ was found by Daira-Emma Hopwood.

The errors in the proof of Ledger Indistinguishability mentioned in § 8.6 ***'Changes to PRF inputs and truncation'*** on p. 106 were also found by Daira-Emma Hopwood.

The 2015 Soundness vulnerability in BCTV14 [Parno2015] was found by Bryan Parno. An additional condition needed to resist this attack was documented by Ariel Gabizon [Gabizon2019, section 3]. The 2019 Soundness vulnerability in BCTV14 [Gabizon2019] was found by Ariel Gabizon.

The 2026 Orchard Action circuit soundness vulnerability was found by Taylor Hornby.

The design of Sapling is primarily due to Matthew Green, Ian Miers, Daira-Emma Hopwood, Sean Bowe, Jack Grigg, and Jack Gavigan. A potential attack linking *diversified payment addresses*, avoided in the adopted design, was found by Brian Warner.

The design of Orchard is primarily due to Daira-Emma Hopwood, Sean Bowe, Jack Grigg, Kris Nuttycombe, Ying Tong Lai, and Steven Smith.

The observation in § 5.4.1.6 *'DiversifyHash^{Sapling} Hash Function'* on p. 61 that *diversified payment address* unlinkability can be proven in the same way as *key privacy* for ElGamal, is due to Mary Maller.

We thank Ariel Gabizon for teaching us the techniques of [BFIJSV2010] used in § B.2 *'Groth16 batch verification'* on p. 170, by applying them to BCTV14.

The arithmetization used by Halo 2 is based on that used by PLONK [GWC2019], which was designed by Ariel Gabizon, Zachary Williamson, and Oana Ciobotaru.

Numerous people have contributed to the science of *zero-knowledge proving systems*, but we would particularly like to acknowledge the work of Shafi Goldwasser, Silvio Micali, Oded Goldreich, Mihir Bellare, Charles Rackoff, Joe Kilian, Yael Tauman Kalai, Guy Rothblum, Rosario Gennaro, Bryan Parno, Jon Howell, Craig Gentry, Mariana Raykova, Jens Groth, Rafail Ostrovsky, and Amit Sahai.

We thank the organizers of the ZKProof standardization effort and workshops; and also Anna Rose, Fredrik Harrysson, Tarun Chitra, James Prestwich, Josh Cincinnati, Tanya Karsou, Henrik Jose, Chris Ward, and others for their work on the Zero Knowledge Podcast, ZK Summits, and ZK Study Club. These efforts have enriched the zero knowledge community immeasurably.

Many of the ideas used in **Zcash** —including the use of *zero-knowledge proofs* to resolve the tension between privacy and auditability, Merkle trees over note commitments (using Pedersen hashes as in Sapling), and the use of "serial numbers" or *nullifiers* to detect or prevent *double-spends*— were first applied to privacy-preserving digital currencies by Tomas Sander and Amnon Ta-Shma. To a large extent **Zcash** is a refinement of their "Auditable, Anonymous Electronic Cash" proposal in [ST1999].

We thank Alexandra Elbakyan for her tireless work in dismantling barriers to scientific research.

This document is set in the beautiful Quattrocento font designed by Pablo Impallari. *Shielded pool* names are set in Raleway Italic, by Matt McInerney, Pablo Impallari, and Rodrigo Fuenzalida. The New Century Schoolbook font by URW Type Foundry, based on Century Schoolbook designed by Morris Fuller Benton, is used for *italics*.

Finally, we would like to thank the Internet Archive for their scan of Peter Newell's illustration of the Jubjub bird, from [Carroll1902].

10 Change History

2026.9.0 2026-11-06

- Add an acknowledgement to Taylor Hornby for finding the 2026 Orchard Action circuit soundness vulnerability.
- Add acknowledgements to @Peacemonger, Nicolas Gailly, Danny Willems, Jason McGee, Mark Henderson, Tomek Piotrowski, Mariusz Pilarek, Paul Dann, Dev Ojha, Evan Forbes, Anna Rose, Jens Groth, Judah Caruso, Pablo Kogan, Vivek Arte, Daniel Benarroch, Aurelien Nicolas, Alexey Koren, Antoine Rondelet, Constance Beguier, Georgios Konstantopoulos, Matt Luongo, Alex Bornstein, @dontbeevil, @GGuy, @mistfpga, Gordon Mohr, Dimitris Apostolou, Gregory Maxwell, and Pieter Wuille for discussions on the Zcash protocol.
- Correct the names of Tarun Chitra and Morris Fuller Benton in the Acknowledgements.

2026.8.0 2026-11-06

- Consistently use the terms "*Spend authorizing key*" for ask and "*Spend validating key*" for ak, removing the alternative terms "spend authorization private key" and "spend authorization address key".
- Correct a typo: "for this *Output statement*" should be "for this *Output transfer*" in § 7.4 on p. 94.
- Correct the note on shielded recipients in **§?** on p. **??**: before its Revision 2, [ZIP-207] does not permit Orchard *shielded payment addresses* as recipient identifiers.
- Correct the definition of a *valid Equihash solution* in § 7.6.1 ***'Equihash'*** on p. 97 to require that its indices are distinct.
- Clarify the bit ordering of the BLAKE2b output T in § 5.4.1.9 ***'Equihash Generator'*** on p. 64, and define T as the resulting *bit sequence*.
- State the restrictions on n in § 7.6.1 ***'Equihash'*** on p. 97 that § 5.4.1.9 ***'Equihash Generator'*** on p. 64 needs in order to be defined.
- Fix a typo in the definition of I2LEOSP in § 5.1 ***'Integers, Bit Sequences, and Endianness'*** on p. 56.
- Add an acknowledgement to Julian Abraham for discussions on the Zcash protocol.

2026.7.0 2026-07-05

- No changes before **NU5**.

2026.6.4 2026-07-05

- Apply the retrospective changes described in [ZIP-256] in response to vulnerabilities found after **NU6.1**:
  - Add a footnote in § 7.1.2 ***'Transaction Consensus Rules'*** on p. 90 clarifying that `valueBalanceSapling` for a v4 *transaction* refers to the value as encoded, not the normalized $v^{SaplingPoolBalance}$.
  - In § 4.15 ***'Chain Value Pool Balances'*** on p. 46, add consensus rules that the *transparent chain value pool* balance **MUST NOT** become negative and that the *total issued supply* **MUST NOT** exceed MAX_MONEY zatoshi.
- Consistently use *discrete logarithm* rather than "discrete log".
- Expand § 3.2 ***'Shielded Pools and Notes'*** on p. 13, retitled "Shielded Pools and Notes", to describe the relation between *shielded payment addresses*, *shielded pools*, and *notes*.
- Distinguish *network upgrades*, *shielded protocols*, and *shielded pools* typographically, like this: **Upgrade**, Protocol, *Pool*.
- Drop the blue highlighting for **Overwinter** and the brown for TODOs, so that those colours can be reused.
- Update author affiliations.
- Update how to report security-relevant issues, suggesting to use a GitHub Security Advisory.

2025.6.3 2025-12-02

- Update the description in § 3.11 ***'Mainnet and Testnet'*** on p. 20 of protocol governance.

2025.6.2 2025-11-11

- Rename § 4.18.3 on p. 53 from "Decryption using a Full Viewing Key (Sapling)" to "Decryption using an Outgoing Viewing Key (Sapling)".
- Define allowedLeadBytes in § 3.2.1 *'Note Plaintexts and Memo Fields'* on p. 14, and use it to refactor the constraints on *note plaintext lead bytes*.
- Correct the accents on Lukáš Korba's name.
- Add acknowledgements to Joe Kilian, Yael Tauman Kalai, and Guy Rothblum for contributions to the science of *zero-knowledge proving systems*.

2025.6.1 2025-10-08

- No changes before **NU6.1**.

2025.6.0 2025-09-17

- Adjust the recommendation in § 4.2.2 *'Sapling Key Components'* on p. 32 not to encode information in the *diversifier*, to recommend instead using a [ZIP-32] *diversifier index*.
- Deprecate the encoding of a Sapling ivk in § 5.6.3.2 *'Sapling Incoming Viewing Keys'* on p. 85, in favour of using a *unified incoming viewing key* with a Sapling component that includes the *diversifier key* dk.
- Tighten the type of ivk in Sapling to $\{1 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{ivk}}} - 1\}$, and the type of $\mathsf{pk_d}$ in Sapling to $\mathsf{KA}^{\mathsf{Sapling}}$.PublicPrimeOrder, in order to make the exclusion of the zero point for $\mathsf{pk_d}$ more obvious [Zips-Issue664]. This also has the effect that a Sapling *incoming viewing key* § 5.6.3.2 *'Sapling Incoming Viewing Keys'* on p. 85 or a Sapling *IVK Encoding* in a *unified incoming viewing key* [ZIP-316] that encodes the zero ivk **MUST** be considered invalid when imported. Notes have been added in § 5.6.3.1 *'Sapling Payment Addresses'* on p. 85 and § 5.6.3.2 *'Sapling Incoming Viewing Keys'* on p. 85 calling out these changes.
- In § 4.18.3 *'Decryption using an Outgoing Viewing Key (Sapling)'* on p. 53, retrospectively enforce the check for canonicity of $\mathsf{pk_d}$. This has no effect on consensus relative to the previous version, because only small-order *Jubjub curve* points have *non-canonical* encodings, and so the check that returns $\bot$ if $\mathsf{pk_d} \notin \mathbb{J}^{(r)*}$ would catch all such cases.
- Change incorrect uses of "block reward" to *block subsidy*.
- Added dark mode rendering (`https://zips.z.cash/protocol/protocol-dark.pdf`).
- Add an acknowledgement to Schell Scivally for discussions on the Zcash protocol.

2024.5.1 2024-09-26

- Collect the definitions of balances for each *chain value pool* into § 4.15 *'Chain Value Pool Balances'* on p. 46.
- Add a definition of *total issued supply*.
- Add acknowledgements to Aditya Bharadwaj, Andrew Arnott, Arya, Andrea Kobrlova, Lukáš Korba, and Honza Rychnovský for discussions on the Zcash protocol.

2024.5.0 2024-08-28

- Add the hyphen in Daira-Emma Hopwood.
- Correct some author lists in the References.
- Prevent incorrect line-breaking on hyphens.
- Add an acknowledgement to Conrado Gouvêa for discussions on the Zcash protocol.
- Add boilerplate support for **NU6**.

2023.4.0 2023-12-19

- The return type of $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}$ in § 5.4.8.5 ***'Group Hash into Jubjub'*** on p. 79 was incorrectly given as $\mathbb{J}^{(r)*}$, rather than the correct $\mathbb{J}^{(r)*} \cup \{\bot\}$.
- In the discussion of partitioning oracle attacks on *note* encryption in § 8.7 ***'In-band secret distribution'*** on p. 107, we now use the fact that $\mathsf{g_d}$ has order greater than the maximum value of ivk, rather than assuming that $\mathsf{g_d}$ is a nonzero point in the *prime-order subgroup*. (In the case of Sapling, the circuits only enforce that $\mathsf{g_d}$ is not a *small-order* point, not that it is in the *prime-order subgroup*. It is true that honestly generated addresses have *prime-order* $\mathsf{g_d}$ which would have been sufficient for the security argument against this class of attacks, but the chosen fix is more direct.)
- Delete a confusing claim in § 4.4 ***'Spend Descriptions'*** on p. 35 that "The check that rk is not of *small order* is technically redundant with a check in the *Spend circuit*...". The *small-order* check excludes the zero point $\mathcal{O}_{\mathbb{J}}$, which the **Spend authority** check that this claim was intending to reference does not.
- An implementation of $\mathsf{HomomorphicPedersenCommit}^{\mathsf{Sapling}}$ **MAY** resample the *commitment trapdoor* until the resulting commitment is not $\mathcal{O}_{\mathbb{J}}$, in order to avoid it being rejected as the cv field of a *Spend description* or *Output description*. Add notes in § 4.4 ***'Spend Descriptions'*** on p. 35, § 4.5 ***'Output Descriptions'*** on p. 36, and § 5.4.7.3 ***'Homomorphic Pedersen commitments (Sapling)'*** on p. 73 to that effect.
- Rename the section "Note Commitments and Nullifiers" to § 4.14 ***'Computing ρ values and Nullifiers'*** on p. 45, to more accurately reflect its contents.
- Split some of the content of the section "Notes" into subsections § 3.2.2 ***'Note Commitments'*** on p. 15 and § 3.2.3 ***'Nullifiers'*** on p. 15. Make the descriptions of how *note commitments* and *nullifiers* are used more precise and explicit, and add forward references where helpful.
- Remove redundancy in the definition of *note plaintexts* between § 3.2.1 ***'Note Plaintexts and Memo Fields'*** on p. 14 and § 5.5 ***'Encodings of Note Plaintexts and Memo Fields'*** on p. 81.
- Acknowledge Greg Pfeil as a co-designer of the **Zcash** protocol.
- Acknowledge Daira-Emma Hopwood for the fix to the Faerie Gold attack in Sprout, and add a reference to hir *Explaining the Security of Zcash* talk at Zcon3 [Hopwood2022] for repairs to the **Zerocash** security definitions.
- Acknowledge the font designers Pablo Impallari and Morris Fuller.
- Change Daira-Emma Hopwood's name.

2022.3.8 2022-09-15

- Correct Jurgen Bos' name.

2022.3.7 2022-09-10

- Specify in § 3.3 ***'The Block Chain'*** on p. 16 that **NU5** is the most recent settled *network upgrade*.

2022.3.6 2022-09-01

- Correct Kexin Hu's name.
- Correct cross-references for the definition of an *anchor*.
- Replace ResearchGate links for [CDvdG1987] with alternatives that do not cause false-positive link checker errors.
- In `protocol/README.rst`: update the build dependency documentation for Debian Bullseye, mention the "`make linkcheck`" target, and correct the description of "`make all`".
- Update the `Makefile` to build correctly with newer versions of `latexmk`.

2022.3.5 2022-08-02

- § 5.4.1.5 *'CRH^{ivk} Hash Function'* on p. 60 incorrectly cross-referenced BLAKE2b-256 rather than BLAKE2s-256. The actual specification was correct.

2022.3.4 2022-06-22

- Update references for [ECCZF2019] and [ZIP-302].

2022.3.3 2022-06-21

- Rename ExcludedPointEncodings to PreCanopyExcludedPointEncodings.
- In § 5.6.2.3 *'Sprout Spending Keys'* on p. 84, remove the statement that future key representations might use the padding bits of Sprout spending keys.
- Give a full-text URL for [Nakamoto2008].

2022.3.2 2022-06-06

- Make § 1.2 *'High-level Overview'* on p. 8 more precise about *chain value pools*.

2022.3.1 2022-04-28

- Correct "block chain branch" to "*consensus branch*" to match [ZIP-200].
- Add an acknowledgement to Mary Maller for reviewing the Halo 2 security proofs.
- Add an acknowledgement to Josh Cincinnati for discussions on the Zcash protocol.
- Add acknowledgements to more people associated with the ZK Podcast.

2022.3.0 2022-03-18

- In § 3.3 *'The Block Chain'* on p. 16, define what a *settled network upgrade* is, specify requirements for checkpointing, and allow nodes to impose a limitation on rollback depth.
- In § 5.4.9.1 *'BCTV14'* on p. 79, note that the above checkpointing requirement mitigates the risks of not performing BCTV14 *zk proof* verification.
- Document the consensus rule that coinbase script length **MUST** be $\{2 \,..\, 100\}$ bytes.
- § 3.10 *'Coinbase Transactions'* on p. 20 effectively defined a *coinbase transaction* as the first *transaction* in a *block*. This wording was copied from the Bitcoin Developer Reference [Bitcoin-CbInput], but it does not match the implementation in *zcashd* that was inherited from *BitcoinCore*. Instead, a *coinbase transaction* should be, and now is, defined as a *transaction* with a single null *prevout*. The specifications of consensus rules have been clarified and adjusted (without any actual consensus change) to take this into account, as follows:
  - a *block* **MUST** have at least one *transaction*;
  - the first *transaction* in a *block* **MUST** be a *coinbase transaction*, and subsequent *transactions* **MUST NOT** be *coinbase transactions*;
  - a *transparent input* in a non-coinbase *transaction* **MUST NOT** have a null *prevout*;
  - every non-null *prevout* **MUST** point to a unique *UTXO* in either a preceding *block*, or a ***previous*** *transaction* in the same *block* (this rule was previously not given explicitly because it was assumed to be inherited from **Bitcoin**);
  - the rule that "A *coinbase transaction* **MUST NOT** have any *transparent inputs* with non-null `prevout` fields" is removed as an explicit consensus rule because it is implied by the corrected definition of *coinbase transaction*.

2022.2.19 2022-01-19

- In § 3.5 *'JoinSplit Transfers and Descriptions'* on p. 17, clarify that balance for *JoinSplit transfers* is enforced by the *JoinSplit statement*, and that there is no consensus rule to check it directly.
- In § 8.5 *'Internal hash collision attack and fix'* on p. 106, add a security argument that, under reasonable assumptions about SHA256Compress, the SHA-256-based *commitment scheme* $\mathsf{NoteCommit}^{\mathsf{Sprout}}$ is *binding* and *hiding*.

2022.2.18 2022-01-03

- Refine the security argument about *partitioning oracle attacks* in § 8.7 *'In-band secret distribution'* on p. 107:
  - The argument for decryption with an *incoming viewing key* does not need to depend on the *Decisional Diffie–Hellman Problem*, since $\mathsf{g_d}$ is committed to by the *note commitment* as well as $\mathsf{pk_d}$.
  - It is necessary to say that the *note commitment* is always checked for a successful decryption.
  - Pedantically, it was not correct to conclude from the given security argument that *partitioning oracle attacks* against an *outgoing ciphertext* are necessarily prevented, according to the definition in [LGR2021]. Instead, the correct conclusions are that such attacks could not feasibly result in any equivocation of the decrypted data, or in recovery of ovk or ock.
- Correct the note about domain separators for $\mathsf{PRF}^{\mathsf{expand}}$ in § 4.1.2 *'Pseudo Random Functions'* on p. 22, and ensure that new domain separators for deriving internal keys from [ZIP-32] and [ZIP-316] are included.

2021.2.17 2021-12-01

- Add notes in § B.1 'RedDSA *batch validation'* on p. 169 and § B.2 *'Groth16 batch verification'* on p. 170 that z_j may be sampled from $\{0 \,..\, 2^{128} - 1\}$ instead of $\{1 \,..\, 2^{128} - 1\}$.
- Add note in § 8.7 *'In-band secret distribution'* on p. 107 about resistance of *note* encryption to *partitioning oracle attacks* [LGR2021].
- Add acknowledgement to Mihir Bellare for contributions to the science of *zero-knowledge proofs*.
- Add acknowledgement to Sasha Meyer.

2021.2.16 2021-09-30

- Correct the consensus rule about the maximum value of outputs in a *coinbase transaction*: it should reference the *block subsidy* rather than the *miner subsidy*.

2021.2.15 2021-09-01

- Fix a reference to nonexistent version 2019.0-beta-40 of this specification (in § 7.6.3 *'Difficulty adjustment'* on p. 98) that should be v2019.0.0.
- Fix URL links to [BBDP2001] and [BDJR2000].
- Improve `protocol/links_and_dests.py` to eliminate false positives when checking DOI links.

2021.2.14 2021-08-12

- Reword the reference to a Sapling *full viewing key* in § 4.7.2 *'Dummy Notes (Sapling)'* on p. 39 (the *full viewing key* would include ovk, although it is not used in that section).

2021.2.13 2021-07-29

- Add consensus rules in § 3.7 *'Note Commitment Trees'* on p. 19 that, for each *note commitment tree*, a *block* **MUST NOT** add *note commitments* that exceed the capacity of that tree.

2021.2.12 2021-07-29

- No changes before **NU5**.

2021.2.11 2021-07-20

- No changes before **NU5**.

2021.2.10 2021-07-13

- Remove a spurious reference to rseed in § 4.17 *'In-band secret distribution (Sprout)'* on p. 50. There were no changes for Sprout in [ZIP-212].

2021.2.9 2021-07-01

- Correct l to $l\star$ in two places in § 5.4.1.3 'MerkleCRH^{Sapling} *Hash Function*' on p. 60.
- Correct an erroneous statement in § 3.4 *'Transactions and Treestates'* on p. 16 that claimed *transaction IDs* are not part of the consensus protocol.

2021.2.8 2021-06-29

- Describe *transaction IDs* in § 3.4 *'Transactions and Treestates'* on p. 16.
- Add a section § 7.1.1 *'Transaction Identifiers'* on p. 90 on how to compute *transaction IDs*.
- Split the *transaction*-related consensus rules into their own subsection § 7.1.2 *'Transaction Consensus Rules'* on p. 90, for more precise cross-referencing.

2021.2.7 2021-06-28

- No changes before **NU5**.

2021.2.6 2021-06-26

- Give cross-references to § 2 *'Notation'* on p. 9 where $\sqrt[?]{\cdot}$ and $\sqrt[+]{\cdot}$ are used.

2021.2.5 2021-06-19

- No changes before **NU5**.

2021.2.4 2021-06-08

- No changes before **NU5**.

2021.2.3 2021-06-06

- Move the section on abstraction (previously section 5.1) to § 4 *'Abstract Protocol'* on p. 21. Section 5.2 has been split into two (§ 5.1 *'Integers, Bit Sequences, and Endianness'* on p. 56 and § 5.2 *'Bit layout diagrams'* on p. 57) to avoid renumbering later subsections.
- Correct an error in the encoding of height-in-coinbase for *blocks* at heights 1 .. 16.
- Clarify, in § 3.3 *'The Block Chain'* on p. 16, requirements on the range of *block heights* that should be supported.
- Delete the sentence "All conversions between Ed25519 points, *byte sequences*, and integers used in this section are as specified in [BDLSY2012]." from § 5.4.5 'Ed25519' on p. 67. This sentence was misleading given that the conversions in [BDLSY2012] are not sufficiently well-specified for a consensus protocol; it should have been deleted earlier when explicit definitions for $\mathsf{reprBytes}_{\mathsf{Ed25519}}$ and $\mathsf{abstBytes}_{\mathsf{Ed25519}}$ were added.
- Make the **NU5** specification the default.

2021.2.2 2021-05-20

- No changes before **NU5**.

2021.2.1 2021-05-20

- Add a note to § 4.8 *'Merkle Path Validity'* on p. 39 clarifying the encoding of $\mathsf{rt}^{SaplingPool}$ as a *primary input* to the Sapling *Spend circuit*, and that *non-canonical* encodings are allowed as input to $\mathsf{MerkleCRH}^{\mathsf{Sapling}}$.
- Change the notation $\mathcal{I}_i^D$ for a Sapling Pedersen generator to $\mathcal{I}(D, i)$.

2021.2.0 2021-05-07

- Clarify notation by changing ℓ_{rcm} to $\ell_{\mathsf{rcm}}^{\mathsf{Sprout}}$.

2021.1.24 2021-04-23

- Explicitly say that *coinbase transactions* **MUST NOT** have *transparent inputs* (this is a consensus rule inherited from **Bitcoin** which has been present since launch).

2021.1.23 2021-04-19

- Fix some URLs in references.

2021.1.22 2021-04-05

- Correct ZKSpend.Verify to ZKOutput.Verify in § 4.5 *'Output Descriptions'* on p. 36.
- Make sure that Change History entries are URL destinations.

2021.1.21 2021-04-01

- Fix type error in kdfinput for $\mathsf{KDF}^{\mathsf{Sapling}}$ (`ephemeralKey` is already a *byte sequence*).
- Make a note in § 8.7 *'In-band secret distribution'* on p. 107 of the divergence of ivk for Sapling from a uniform scalar.
- Correct the set of inputs to $\mathsf{PRF}^{\mathsf{expand}}$ used for [ZIP-32] in § 4.1.2 *'Pseudo Random Functions'* on p. 22.
- Write the caution about linkage between the abstract and concrete protocols in § 4 *'Abstract Protocol'* on p. 21.
- Update the Sprout key component diagram in § 3.1 *'Payment Addresses and Keys'* on p. 12 to remove magenta highlighting.

2021.1.20 2021-03-25

- Credit Eirik Ogilvie-Wigley as a designer of the Zcash protocol. Add Andre Serrano, Brad Miller, Charlie O'Keefe, David Campbell, Elena Giralt, Francisco Gindre, Joseph Van Geffen, Josh Swihart, Kevin Gorham, Larry Ruane, Marshall Gaucher, and Ryan Taylor to the acknowledgements.
- Describe the recommended way to encode a Sapling *payment address* as a QR code.
- Move the definition of $\bot$ to before its first use.
- Delete a confusing part of the definition of $\mathsf{concat}_{\mathbb{B}}$ that we don't rely on.
- Add a definition for the § symbol in § 1 *'Introduction'* on p. 7, before its first use.
- Remove specification of *memo field* contents, which will be in [ZIP-302].
- Remove support for building the Sprout-only specification (`sprout.pdf`).
- Remove magenta highlighting of differences from **Zerocash**.

2021.1.19 2021-03-17

- No changes before **NU5**.

2021.1.18 2021-03-17

- The subgroup check added to § 4.18.3 *'Decryption using an Outgoing Viewing Key (Sapling)'* on p. 53 for Sapling in v2021.1.17 was applied to the wrong variable ($\mathsf{g_d}$, when it should have been $\mathsf{pk_d}$), despite being described correctly in the Change History entry below.

2021.1.17 2021-03-15

- The definition of an abstraction function in § 4.1.8 *'Represented Group'* on p. 29 incorrectly required canonicity, i.e. that $\mathsf{abst}_{\mathbb{G}}$ does not accept inputs outside the range of $\mathsf{repr}_{\mathbb{G}}$. While this was originally intended, it is not true of $\mathsf{abst}_{\mathbb{J}}$. (It is also not true of $\mathsf{abstBytes}_{\mathsf{Ed25519}}$, but Ed25519 is not strictly defined as a *represented group* in this specification.)
- Correct Theorem 5.4.2 on p. 73, which was proving the wrong thing. It needs to prove that $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ does not return $\mathsf{Uncommitted}^{\mathsf{Sapling}}$, but was previously proving that PedersenHash does not return that value.
- The note about *non-canonical* encodings in § 5.4.8.3 *'Jubjub'* on p. 77 gave incorrect values for the encodings of the point of order 2, by omitting a $q_{\mathbb{J}}$ term.
- The specification of decryption in § 4.18.3 on p. 53 differed from the *zcashd* implementation in two respects:
  - The specification had a type error in that it failed to check whether $\mathsf{abst}_{\mathbb{J}}(\mathsf{pk}\star_{\mathsf{d}}) = \bot$, which is needed in order for its use as input to $\mathsf{KA}^{\mathsf{Sapling}}$.Agree to be well-typed.
  - The specification did not require $\mathsf{pk_d}$ to be in the subgroup $\mathbb{J}^{(r)}$, while the implementation in *zcashd* did. This check is not needed for security; however, since Jubjub public keys are normally of type $\mathsf{KA}^{\mathsf{Sapling}}$.PublicPrimeOrder, we change the specification to match *zcashd*.
- Correct the procedure for generating *dummy Sapling-pool notes* in § 4.7.2 *'Dummy Notes (Sapling)'* on p. 39.
- Add a note in § 5.4.9.1 *'BCTV14'* on p. 79 describing conditions under which an implementation that checkpoints on **Sapling** can omit verifying BCTV14 proofs.
- Rename "hash extractor" to *coordinate extractor*. This is a more accurate name since it is also used on commitments.
- Rename `char` to `byte` in field type declarations.

2021.1.16 2021-01-11

- Add macros and `Makefile` support for building the **NU5** draft specification.
- Clarify the encoding of *block heights* for the "height in coinbase" rule. The description of this rule has also moved from § 7.5 on p. 95 to § 7.1.2 *'Transaction Consensus Rules'* on p. 90.
- Include the activation dates of **Heartwood** and **Canopy** in § 6 *'Network Upgrades'* on p. 88.
- Section links in the **Heartwood** and **Canopy** versions of the specification now go to the correct document URL.
- Attempt to improve search and cut-and-paste behaviour for ligatures in some PDF readers.

2020.1.15 2020-11-06

- Add a missing consensus rule that has always been implemented in *zcashd*: there must be at least one *transparent output*, *Output description*, or *JoinSplit description* in a *transaction*.
- Add a consensus rule that the (zero-valued) *coinbase transaction* output of the *genesis block* cannot be spent.
- Define ChainValuePoolBalance^{SproutPool} and ChainValuePoolBalance^{SaplingPool}, and include consensus rules from [ZIP-209].
- Correct the *Sapling-pool note* decryption algorithms:
  - `ephemeralKey` is kept as a *byte sequence* rather than immediately converted to a curve point; this matters because of *non-canonical* encoding.
  - The representation of $\mathsf{pk_d}$ in a *note plaintext* may also be *non-canonical* and need not be in the *prime-order subgroup*.
  - Move checking of cm_u in decryption with ivk to the end of the algorithm, to more closely match the implementation.
  - The note about decryption of outputs in *mempool transactions* should have been normative.
- Reserve *transaction version number* `0x7FFFFFFF` and *version group ID* `0xFFFFFFFF` for experimental use.
- Remove a statement that the language consisting of key and address encoding possibilities is prefix-free. (The human-readable forms are prefix-free but the raw encodings are not; for example, the *raw encoding* of a Sapling *spending key* can be a prefix of several of the other encodings.)
- Use "let mutable" to introduce mutable variables in algorithms.
- Include a reference to [BFIJSV2010] for batch pairing verification techniques.
- Acknowledge Jack Gavigan as a co-designer of Sapling and of the **Zcash** protocol.
- Acknowledge Izaak Meckler, Zac Williamson, Vitalik Buterin, and Jakub Zalewski.
- Acknowledge Alexandra Elbakyan.

2020.1.14 2020-08-19

- The consensus rule that a *coinbase transaction* must not spend more than is available from the *block subsidy* and *transaction fees*, was not explicitly stated. (This rule was correctly implemented in *zcashd*.)
- Fix a type error in the output of PRF^{nfSapling}; a *Sapling-pool nullifier* is a sequence of 32 bytes, not a *bit sequence*.
- Correct an off-by-one in an expression used in the definition of c in § 5.4.1.7 ***'Pedersen Hash Function'*** on p. 62 (this does not change the value of c).

2020.1.13 2020-08-11

- Rename the type of Sapling *transmission keys* from $\mathsf{KA}^{\mathsf{Sapling}}$.PublicPrimeOrder to $\mathsf{KA}^{\mathsf{Sapling}}$.PublicPrimeSubgroup. This type is defined as $\mathbb{J}^{(r)}$, which reflects the implementation in *zcashd* (subject to the next point below); it was never enforced that a *transmission key* ($\mathsf{pk_d}$) cannot be $\mathcal{O}_{\mathbb{J}}$.
- Add a non-normative note saying that *zcashd* does not fully conform to the requirement to treat *transmission keys* not in $\mathsf{KA}^{\mathsf{Sapling}}$.PublicPrimeSubgroup as invalid when importing *shielded payment addresses*.
- **Retrospective note:** Changing $\mathsf{KA}^{\mathsf{Sapling}}$.PublicPrimeOrder to $\mathsf{KA}^{\mathsf{Sapling}}$.PublicPrimeSubgroup was a mistake and has since been reverted in specification version v2025.6.0. As discussed in notes added in version v2023.4.0 at § 4.18.3 on p. 53, *librustzcash* changed in [librustzcash-109] to enforce that $\mathsf{pk_d}$ is not $\mathcal{O}_{\mathbb{J}}$. *zcashd* also used a different implementation for a consensus check on *shielded* coinbase outputs. The missing check on $\mathsf{pk_d}$ for the latter was corrected in [zcashd-6459], and simplified when it was observed to be retrospectively valid in [zcashd-6725]. However [ZIP-216] was only corrected later [Zips-Issue664], at the same time as the publication of v2025.6.0.

- Make $\mathsf{Halving}(\mathsf{height})$ return 0 (rather than -1) for $\mathsf{height} < \mathsf{SlowStartShift}$. This has no effect on consensus since the $\mathsf{Halving}$ function is not used in that case, but it makes the definition match the intuitive meaning of the function.
- Rename sections under § 7 *'Consensus Changes from **Bitcoin**'* on p. 89 to clarify that these sections do not only concern encoding, but also consensus rules.
- Make the **Canopy** specification the default.

2020.1.12 2020-08-03

- Include SHA-512 in § 5.4.1.1 *'SHA-256, SHA-256d, SHA256Compress, and SHA-512 Hash Functions'* on p. 58.
- Add a reference to [BCCGLRT2014] in § 4.1.12 *'Zero-Knowledge Proving System'* on p. 30.

2020.1.11 2020-07-13

- Change instances of "the production network" to "*Mainnet*", and "the test network" to *Testnet*. This follows the terminology used in ZIPs.
- Update stale references to **Bitcoin** documentation.

2020.1.10 2020-07-05

- Corrections to a note in § 5.4.5 *'Ed25519'* on p. 67.

2020.1.9 2020-07-05

- Add § 3.11 *'Mainnet and Testnet'* on p. 20.
- Acknowledge Jane Lusby and teor.
- Precisely specify the encoding and decoding of Ed25519 points.
- Correct an error introduced in v2020.1.8; "$-\mathcal{O}_{\mathbb{J}}$" was incorrectly used when the point $(0, -1)$ on Jubjub was meant.
- Precisely specify the conversion from a *bit sequence* in $\mathsf{abst}_{\mathbb{J}}$.

2020.1.8 2020-07-04

- Add Ying Tong Lai and Kris Nuttycombe as **Zcash** protocol designers.
- Change the specification of $\mathsf{abst}_{\mathbb{J}}$ in § 5.4.8.3 *'Jubjub'* on p. 77 to match the implementation.
- Repair the argument for $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}_{\mathsf{URS}}$ being usable as a *random oracle*, which previously depended on $\mathsf{abst}_{\mathbb{J}}$ being injective.
- In RedDSA verification, clarify that $\underline{R}$ used as part of the input to $\mathsf{H}^{\circledR}$ **MUST** be exactly as encoded in the signature.

2020.1.7 2020-06-26

- Delete some 'new' superscripts that only added notational clutter.
- Add an explicit lead byte field to *Sprout-pool note plaintexts*, and clearly specify the error handling when it is invalid.
- Define a *Sapling-pool note plaintext lead byte* as having type $\mathbb{B}^{\mathbb{Y}}$ (so that decoding to a *note plaintext* always succeeds, and error handling is more explicit).
- Fix a sign error in the fixed-base term of the batch validation equation in § B.1 *'RedDSA batch validation'* on p. 169.

2020.1.6 2020-06-17

- Correct an error in the specification of Ed25519 *validating keys*: they should not have been specified to be checked against PreCanopyExcludedPointEncodings, since libsodium v1.0.15 does not do so.
- Consistently use "validating" for signatures and "verifying" for proofs.
- Use the symbol $\sqrt[+]{\bullet}$ for positive square root.

2020.1.5 2020-06-02

- Reference [ZIP-173] instead of BIP 173.
- Mark more index entries as definitions.

2020.1.4 2020-05-27

- Reference [BIP-32] and [ZIP-32] when describing keys and their encodings.
- Network Upgrade 4 has been given the name **Canopy**.
- Improve LaTeX portability of this specification.

2020.1.3 2020-04-22

- Correct a wording error transposing *transparent inputs* and *transparent outputs* in § 4.11 *'Balance (Sprout)'* on p. 41.

2020.1.2 2020-03-20

- The implementation of Sprout Ed25519 signature validation in *zcashd* differed from what was specified in § 5.4.5 'Ed25519' on p. 67. The specification has been changed to match the implementation.
- Remove "pvc" `Makefile` targets.
- Make the **Heartwood** specification the default.
- Add macros and `Makefile` support for building the **Canopy** specification.

2020.1.1 2020-02-13

- Resolve conflicts in the specification of *memo fields* by deferring to [ZIP-302].

2020.1.0 2020-02-06

- Specify a retrospective soft fork implemented in *zcashd v2.1.1-1* that limits the `nTime` field of a *block* relative to its *median-time-past*.
- Correct the definition of *median-time-past* for the first PoWMedianBlockSpan *blocks* in a *block chain*.
- Add acknowledgements to Henry de Valence, Deirdre Connolly, Chelsea Komlo, and Zancas Wilcox.
- Add an acknowledgement to Trail of Bits for their security audit.
- Change indices in the *incremental Merkle tree* diagram to be zero-based.
- Use the term "*monomorphism*" for an injective homomorphism, in the context of a *signature scheme with key monomorphism*.

2019.0.9 2019-12-27

- No changes to Sprout or Sapling.
- `Makefile` updates for **Heartwood**.

2019.0.8 2019-09-24

- Fix a typo in the generator $\mathcal{P}_{\mathbb{S}_1}$ in § 5.4.8.2 *'BLS12-381'* on p. 75 found by magrady.
- Clarify the type of $\mathsf{v}^{\mathsf{new}}$ in § 4.6.2 *'Sending Notes (Sapling)'* on p. 37.

2019.0.7 2019-09-24

- Fix a discrepancy in the number of constraints for BLAKE2s found by QED-it.
- Fix an error in the expression for Δ in § A.3.3.9 *'Pedersen hash'* on p. 159, and add acknowledgement to Kobi Gurkan.
- Fix a typo in § 4.8 *'Merkle Path Validity'* on p. 39 and add acknowledgement to Weikeng Chen.
- Update references to ZIPs and to the Electric Coin Company blog.
- `Makefile` improvements to suppress unneeded output.

2019.0.6 2019-08-23

- No changes to Sprout or Sapling.

2019.0.5 2019-08-23

- Correct the packing of $\mathsf{nf}^{\mathsf{old}}$ into input elements in § A.4 *'The Sapling Spend circuit'* on p. 166.
- Add an epigraph from [Carroll1876] to the start of § 5.4.8.3 *'Jubjub'* on p. 77.
- Clarify how the constant c in § 5.4.1.7 *'Pedersen Hash Function'* on p. 62 is obtained.
- Add a footnote that *zcashd* uses [ZIP-32] *extended spending keys* instead of the derivation from sk in § 3.1 *'Payment Addresses and Keys'* on p. 12.
- Remove "optimized" `Makefile` targets (which actually produced a larger PDF, with TeXLive 2019).
- Remove "html" `Makefile` targets.
- Make the **Blossom** spec the default.

2019.0.4 2019-07-23

- Clicking on a section heading now shows section labels.
- Add a **List of Theorems and Lemmata**.
- Changes needed to support TeXLive 2019.

2019.0.3 2019-07-08

- Experimental support for building using LuaTeX and XeTeX.
- Add an **Index**.

2019.0.2 2019-06-18

- Correct a misstatement in the security argument in § 4.12 *'Balance and Binding Signature (Sapling)'* on p. 42: *binding* for a *commitment scheme* does not imply that the commitment determines its randomness. The rest of the security argument did not depend on this; it is simpler to rely on knowledge soundness of the Spend and Output proofs.
- Give a definition for *complete twisted Edwards elliptic curves* in § 5.4.8.3 *'Jubjub'* on p. 77.
- Clarify that Theorem 5.4.2 on p. 73 depends on the parameters of the *Jubjub curve*.
- Ensure that this document builds correctly and without missing characters on recent versions of TeXLive.
- Update the `Makefile` to use Ghostscript for PDF optimization.
- Ensure that hyperlinks are preserved, and available as "Destination names" in URL fragments and links from other PDF documents.

2019.0.1 2019-05-20

- No changes to Sprout or Sapling.

2019.0.0 2019-05-01

- Fix a specification error in the *Founders' Reward* calculation during the slow start period.
- Correct an inconsistency in difficulty adjustment between the spec and *zcashd* implementation for the first PoWAveragingWindow $- 1$ *blocks* of the *block chain*. This inconsistency was pointed out by NCC Group in their **Blossom** specification audit.

2019.0-beta-39 2019-04-18

- Change author affiliations from "Zerocoin Electric Coin Company" to "Electric Coin Company".
- Add acknowledgement to Mary Maller for the observation that *diversified payment address* unlinkability can be proven in the same way as *key privacy* for ElGamal.

2019.0-beta-38 2019-04-18

- Correct the generators $\mathcal{P}_{\mathbb{S}_1}$ and $\mathcal{P}_{\mathbb{S}_2}$ for BLS12-381.
- Update `README.rst` to include `Makefile` targets for **Blossom**.
- `Makefile` updates:
  - Fix a typo for the `pvcblossom` target.
  - Update the pinned `git` hashes for `sam2p` and `pdfsizeopt`.

2019.0-beta-37 2019-02-22

- The rule that miners **SHOULD NOT** mine *blocks* that chain to other *blocks* with a *block version number* greater than 4, has been removed. This is because such *blocks* (mined nonconformantly) exist in the current *Mainnet* consensus *block chain*.
- Clarify that *Equihash* is based on a ***variation*** of the Generalized Birthday Problem, and cite [AR2017].
- Update reference [BGG2017] (previously [BGG2016]).
- Clarify which transaction fields are added by **Overwinter** and **Sapling**.
- Correct the rule about when a *transaction* is permitted to have no *transparent* inputs.
- Explain the differences between the system in [Groth2016] and what we refer to as Groth16.

- Reference Mary Maller's security proof for Groth16 [Maller2018].
- Correct [BGM2018] to [BGM2017].
- Fix a typo in § B.2 *'Groth16 batch verification'* on p. 170 and clarify the costs of Groth16 batch verification.
- Add macros and `Makefile` support for building the **Blossom** specification.

2019.0-beta-36 2019-02-09

- Correct isis agora lovecruft's name.

2019.0-beta-35 2019-02-08

- Cite [Gabizon2019] and acknowledge Ariel Gabizon.
- Correct [SBB2019] to [SWB2019].
- The [Gabizon2019] vulnerability affected Soundness of BCTV14 as well as Knowledge Soundness.
- Clarify the history of the [Parno2015] vulnerability and acknowledge Bryan Parno.
- Specify the difficulty adjustment change that occurred on *Testnet* at *block height* 299188.
- Add Eirik Ogilvie-Wigley and Benjamin Winston to acknowledgements.
- Rename *zk-SNARK* Parameters sections to be named according to the *proving system* (BCTV14 or Groth16), not the *shielded protocol* construction (Sprout or Sapling).
- In § 6 *'Network Upgrades'* on p. 88, say when the **Sapling** *network upgrade* activated.

2019.0-beta-34 2019-02-05

- Disclose a security vulnerability in BCTV14 that affected Sprout before activation of the **Sapling** *network upgrade* (see § 5.4.9.1 *'BCTV14'* on p. 79).
- Rename PHGR13 to BCTV2014.
- Rename reference [BCTV2015] to [BCTV2014a], and [BCTV2014] to [BCTV2014b].

2018.0-beta-33 2018-11-14

- Complete § A.4 *'The Sapling Spend circuit'* on p. 166.
- Add § A.5 *'The Sapling Output circuit'* on p. 168.
- Change the description of window lookup in § A.3.3.7 *'Fixed-base Affine-ctEdwards scalar multiplication'* on p. 157 to match sapling-crypto.
- Describe 2-bit window lookup with conditional negation in § A.3.3.9 *'Pedersen hash'* on p. 159.
- Fix or complete various calculations of constraint costs.
- Adjust the notation used for scalar multiplication in Appendix A to allow *bit sequences* as scalars.

2018.0-beta-32 2018-10-24

- Correct the input to $\mathsf{H}^{\circledast}$ used to derive the nonce r in RedDSA.Sign, from $T \,||\, M$ to $T \,||\, \underline{\mathsf{vk}} \,||\, M$. This matches the sapling-crypto implementation; the specification of this input was unintentionally changed in v2018.0-beta-20.
- Clarify the description of the Merkle path check in § A.3.4 *'Merkle path check'* on p. 162.

2018.0-beta-31 2018-09-30

- Correct some uses of $r_{\mathbb{J}}$ that should have been $r_{\mathbb{S}}$ or q.
- Correct uses of $\mathsf{LEOS2IP}_\ell$ in RedDSA.Validate and RedDSA.BatchValidate to ensure that ℓ is a multiple of 8 as required.
- Minor changes to avoid clashing notation for Edwards curves $E_{\mathsf{Edwards}(a,d)}$, *Montgomery curves* $E_{\mathsf{Mont}(A,B)}$, and extractors $\mathcal{E}_{\mathcal{A}}$.
- Correct a use of $\mathbb{J}$ that should have been $\mathbb{M}$ in the proof of Theorem A.3.4 on p. 155, and make a minor tweak to the theorem statement ($k_2 \neq \pm k_1$ instead of $k_1 \neq \pm k_2$) to make the contradiction derived by the proof clearer.
- Clarify notation in the proof of Theorem A.3.3 on p. 155.
- Address some of the findings of the QED-it report:
  - Improved cross-referencing in § 5.4.1.7 ***'Pedersen Hash Function'*** on p. 62.
  - Clarify the notes concerning domain separation of prefixes in § 5.4.1.3 **'MerkleCRH^{Sapling} *Hash Function*'** on p. 60 and § 5.4.7.2 ***'Windowed Pedersen commitments'*** on p. 72.
  - Correct the statement and proof of Theorem A.3.2 on p. 155.
- Add the QED-it report to the acknowledgements.

2018.0-beta-30 2018-09-02

- Give an informal security argument for Unlinkability of *diversified payment addresses* based on reduction to *key privacy* of ElGamal encryption, for which a security proof is given in [BBDP2001]. (This argument has gaps which will be addressed in a future version.)
- Add a reference to [BGM2017] for the Sapling *zk-SNARK* parameters.
- Write § A.4 ***'The Sapling Spend circuit'*** on p. 166 (draft).
- Add a reference to the ristretto_bulletproofs design notes [Dalek-notes] for the synthetic blinding factor technique.
- Ensure that the constraint costs in § A.3.3.1 ***'Checking that Affine-ctEdwards coordinates are on the curve'*** on p. 154 and § A.3.3.6 ***'Affine-ctEdwards nonsmall-order check'*** on p. 157 accurately reflect the implementation in sapling-crypto.
- Minor correction to the non-normative note in § A.3.2.2 ***'Range check'*** on p. 152.
- Clarify non-normative note in § 4.1.7 ***'Commitment'*** on p. 27 about the definitions of ValueCommit^{Sapling}.Output and NoteCommit^{Sapling}.Output.
- Clarify that the signer of a *spend authorization signature* is supposed to choose the *spend authorization randomizer*, α, itself. Only step 4 in § 4.13 ***'Spend Authorization Signature (Sapling)'*** on p. 44 may securely be delegated.
- Add a non-normative note to § 5.4.6 **'RedDSA *and* RedJubjub'** on p. 69 explaining that RedDSA key randomization may interact with other uses of additive properties of Schnorr keys.
- Add dates to Change History entries. (These are the dates of the git tags in local, i.e. UK, time.)

2018.0-beta-29 2018-08-15

- Finish § A.3.2.2 ***'Range check'*** on p. 152.
- Change § A.3.7 ***'BLAKE2s hashes'*** on p. 163 to correct the constraint count and to describe batched equality checks performed by the sapling-crypto implementation.

2018.0-beta-28 2018-08-14

- Finish § A.3.7 *'BLAKE2s hashes'* on p. 163.
- Minor corrections to § A.3.3.8 *'Variable-base Affine-ctEdwards scalar multiplication'* on p. 158.

2018.0-beta-27 2018-08-12

- Notational changes:
  - Use a superscript $^{(r)}$ to mark the subgroup order, instead of a subscript.
  - Use $\mathbb{G}^{(r)*}$ for the set of $r_{\mathbb{G}}$-order points in $\mathbb{G}$.
  - Mark the subgroup order in pairing groups, e.g. use $\mathbb{G}_1^{(r)}$ instead of $\mathbb{G}_1$.
  - Make the bit-representation indicator $\star$ an affix instead of a superscript.
- Clarify that when validating a Groth16 proof, it is necessary to perform a subgroup check for π_A and π_C as well as for π_B.
- Correct the description of Groth16 batch verification to explicitly take account of how verification depends on *primary inputs*.
- Add Charles Rackoff, Rafail Ostrovsky, and Amit Sahai to the acknowledgements section for their work on *zero-knowledge proofs*.

2018.0-beta-26 2018-08-05

- Add § B.2 *'Groth16 batch verification'* on p. 170.

2018.0-beta-25 2018-08-05

- Add the hashes of parameter files for Sapling.
- Add cross references for parameters and functions used in RedDSA batch validation.
- `Makefile` changes: name the PDF file for the Sprout version of the specification as `sprout.pdf`, and make `protocol.pdf` link to the **Sapling** version.

2018.0-beta-24 2018-07-31

- Add a missing consensus rule for version 4 *transactions*: if there are no *Sapling-pool* Spends or Outputs, then `valueBalanceSapling` **MUST** be 0.

2018.0-beta-23 2018-07-27

- Update RedDSA validation to use cofactor multiplication. This is necessary in order for the output of batch validation to match that of unbatched validation in all cases.
- Add § B.1 'RedDSA *batch validation*' on p. 169.

2018.0-beta-22 2018-07-18

- Update § 6 *'Network Upgrades'* on p. 88 to take account that **Overwinter** has activated.
- The recommendation for *transactions* without *JoinSplit descriptions* to be version 1 applies only before **Overwinter**, not before **Sapling**.
- Complete the proof of Theorem A.3.5 on p. 160.
- Add a note about redundancy in the nonsmall-order checking of rk.
- Clarify the use of $\mathsf{cv}^{\mathsf{new}}$ and $\mathsf{cm}^{\mathsf{new}}$, and the selection of *outgoing viewing key*, in sending *Sapling-pool notes*.
- Delete the description of optimizations for the affine twisted Edwards nonsmall-order check, since the Sapling circuit does not use them. Also clarify that some other optimizations are not used.

2018.0-beta-21 2018-06-22

- Remove the consensus rule "If `nJoinSplit` > 0, the *transaction* **MUST NOT** use *SIGHASH types* other than SIGHASH_ALL.", which was never implemented.
- Add section on signature hashing.
- Briefly describe the changes to computation of *SIGHASH transaction hashes* in Sprout.
- Clarify that interstitial *treestates* form a tree for each *transaction* containing *JoinSplit descriptions*.
- Correct the description of *P2PKH addresses* in § 5.6.1.1 *'Transparent Addresses'* on p. 82 — they use a hash of a compressed, not an uncompressed ECDSA key representation.
- Clarify the wording of the caveat[3] about the claimed security of shielded *transactions*.
- Correct the definition of set difference ($S \setminus T$).
- Add a note concerning malleability of *zk-SNARK proofs*.
- Clarify attribution of the **Zcash** protocol design.
- Acknowledge Alex Biryukov and Dmitry Khovratovich as the designers of *Equihash*.
- Acknowledge Shafi Goldwasser, Silvio Micali, Oded Goldreich, Rosario Gennaro, Bryan Parno, Jon Howell, Craig Gentry, Mariana Raykova, and Jens Groth for their work on *zero-knowledge proving systems*.
- Acknowledge Tomas Sander and Amnon Ta-Shma for [ST1999].
- Acknowledge Kudelski Security's audit.
- Use the more precise subgroup types $\mathbb{G}^{(r)}$ and $\mathbb{J}^{(r)}$ in preference to $\mathbb{G}$ and $\mathbb{J}$ where applicable.
- Change the types of *auxiliary inputs* to the *Spend statement* and *Output statement*, to be more faithful to the implementation.
- Rename the `cm` field of an *Output description* to `cmu`, reflecting the fact that it is a *Jubjub curve* u-coordinate.
- Add explicit consensus rules that the `anchorSapling` field of a *Spend description* and the `cmu` field of an *Output description* must be canonical encodings.
- Enforce that esk in `outCiphertext` is a canonical encoding.
- Add consensus rules that cv in a *Spend description*, and cv and epk in an *Output description*, are not of *small order*. Exclude 0 from the range of esk when encrypting *Sapling-pool notes*.
- Add a consensus rule that `valueBalanceSapling` is in the range $\{-\mathsf{MAX_MONEY} \,..\, \mathsf{MAX_MONEY}\}$.
- Enforce stronger constraints on the types of key components $\mathsf{pk_d}$, ak, and nk.
- Correct the conformance rule for `fOverwintered` (it must not be set before **Overwinter** has activated, not before **Sapling** has activated).
- Correct the argument that v^* is in range in § 4.12 *'Balance and Binding Signature (Sapling)'* on p. 42.

- Correct an error in the algorithm for RedDSA.Validate: the *validating key* vk is given directly to this algorithm and should not be computed from the unknown *signing key* sk.
- Correct or improve the types of $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}$, $\mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}$, $\mathsf{Extract}_{\mathbb{J}^{(r)}}$, $\mathsf{PRF}^{\mathsf{expand}}$, $\mathsf{PRF}^{\mathsf{ockSapling}}$, and $\mathsf{CRH}^{\mathsf{ivk}}$.
- Instantiate $\mathsf{PRF}^{\mathsf{ockSapling}}$ using BLAKE2b-256.
- Change the syntax of a *commitment scheme* to add COMM.GenTrapdoor. This is necessary because the intended distribution of *commitment trapdoors* may not be uniform on all values that are acceptable *trapdoor* inputs.
- Add notes on the purpose of *outgoing viewing keys*.
- Correct the encoding of a *full viewing key* (ovk was missing).
- Ensure that Sprout functions and values are given Sprout-specific types where appropriate.
- Improve cross-referencing.
- Clarify the use of BCTV14 vs Groth16 proofs in *JoinSplit statements*.
- Clarify that the $\sqrt[+]{a}$ notation refers to the positive square root. (This matters for the conversion in § A.3.3.3 ***'ctEdwards ↔ Montgomery conversion'*** on p. 154.)
- Model the group hash as a *random oracle*. This appears to be unavoidable in order to allow proving unlinkability of $\mathsf{DiversifyHash}^{\mathsf{Sapling}}$. Explain how this relates to the Discrete Logarithm Independence assumption used previously, and justify this modelling by showing that it follows from treating BLAKE2s-256 as a *random oracle* in the instantiation of $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}$.
- Rename CRS (Common Random String) to URS (*Uniform Random String*), to match the terminology adopted at the first ZKProof workshop held in Boston, Massachusetts on May 10–11, 2018.
- Generalize $\mathsf{PRF}^{\mathsf{expand}}$ to accept an arbitrary-length input. (This specification does not use that generalization, but [ZIP-32] does.)
- Change the notation for a multiplication constraint in Appendix § A ***'Circuit Design'*** on p. 149 to avoid potential confusion with cartesian product.
- Clarify the wording of the abstract.
- Correct statements about which algorithms are instantiated by BLAKE2s and BLAKE2b.
- Add a note explaining which conformance requirements of BIP 173 (defining *Bech32*) apply.
- Add the Jubjub bird image to the title page. This image has been edited from a scan of Peter Newell's original illustration (as it appeared in [Carroll1902]) to remove the background and Bandersnatch, and to restore the bird's clipped right wing.
- Change the light yellow background to white (indicating that this **Overwinter** and **Sapling** specification is no longer a draft).

2018.0-beta-20 2018-05-22

- Add Michael Dixon and Andrew Poelstra to acknowledgements.
- Minor improvements to cross-references.
- Correct the order of arguments to RedDSA.RandomizePrivate and RedDSA.RandomizePublic.
- Correct a reference to RedDSA.RandomizePrivate that was intended to be RedDSA.RandomizePublic.
- Fix the description of the *Sapling-pool balancing value* in § 4.12 ***'Balance and Binding Signature (Sapling)'*** on p. 42.
- Correct a type error in § 5.4.8.5 ***'Group Hash into Jubjub'*** on p. 79.
- Correct a type error in RedDSA.Sign in § 5.4.6 'RedDSA *and* RedJubjub' on p. 69.

- Ensure $\mathcal{G}^{\mathsf{Sapling}}$ is defined in § 5.4.6.1 *'Spend Authorization Signature (Sapling)'* on p. 71.
- Make the *validating key* prefix part of the input to the *hash function* in RedDSA, not part of the message.
- Correct the statement about FindGroupHash$^{\mathbb{J}^{(r)*}}$ never returning $\bot$.
- Correct an error in the computation of generators for *Pedersen hashes*.
- Change the order in which NoteCommit^{Sapling} commits to its inputs, to match the sapling-crypto implementation.
- Fail Sapling key generation if ivk $= 0$. (This has negligible probability.)
- Change the notation $\mathsf{H}^\star$ to $\mathsf{H}^{\circledast}$ in § 5.4.6 *'RedDSA and RedJubjub'* on p. 69, to avoid confusion with the * convention for representations of group elements.
- `cmu` encodes only the u-coordinate of the *note commitment*, not the full curve point.
- rk is checked to be not of *small order* outside the *Spend statement*, not in the *Spend statement*.
- Change terminology describing constraint systems.

2018.0-beta-19 2018-04-23

- Minor clarifications.

2018.0-beta-18 2018-04-23

- Clarify the security argument for balance in Sapling.
- Correct a subtle problem with the type of the value input to ValueCommit^{Sapling}: although it is only directly used to commit to values in $\{0 \,..\, 2^{\ell_{\mathsf{value}}}-1\}$, the security argument depends on a sum of commitments being *binding* on $\left\{-\frac{r_{\mathbb{J}}-1}{2} \,..\, \frac{r_{\mathbb{J}}-1}{2}\right\}$.
- Fix the loss of tightness in the use of PRF^{nfSapling} by specifying the keyspace more precisely.
- Correct type ambiguities for ρ.
- Specify the representation of i in group $\mathbb{G}_2$ of BLS12-381.

2018.0-beta-17 2018-04-21

- Correct an error in the definition of DefaultDiversifier.

2018.0-beta-16 2018-04-21

- Explicitly note that outputs from *coinbase transactions* include *Founders' Reward* outputs.
- The point represented by $\underline{R}$ in an Ed25519 signature is checked to not be of *small order*; this is not the same as checking that it is of *prime order* ℓ.
- Specify support for [BIP-111] (the `NODE_BLOOM` service bit) in *peer-to-peer protocol* version 170004.
- Give references [Vercauter2009] and [AKLGL2010] for the optimal ate pairing.
- Give references for BLS [BLS2002] and BN [BN2005] curves.
- Define KA^{Sprout}.DerivePublic for Curve25519.
- Caveat the claim about *note traceability set* in § 1.2 *'High-level Overview'* on p. 8 and link to [Peterson2017] and [Quesnelle2017].
- Do not require a generator as part of the specification of a *represented group*; instead, define it in the *represented pairing* or scheme using the group.

- Refactor the abstract definition of a *signature scheme* to allow derivation of *validating keys* independent of key pair generation.
- Correct the explanation in § 1.2 ***'High-level Overview'*** on p. 8 to apply to Sapling.
- Add the definition of a *signing key* to *validating key* homomorphism for *signature schemes*.
- Remove the output index as an input to $\mathsf{KDF}^{\mathsf{Sapling}}$.
- Allow dummy *Sapling-pool* input *notes*.
- Specify RedDSA and RedJubjub.
- Specify *Sapling binding signatures* and *spend authorization signatures*.
- Specify the randomness beacon.
- Add *outgoing ciphertexts* and ock.
- Define DefaultDiversifier.
- Change the *Spend circuit* and *Output circuit* specifications to remove unintended differences from sapling-crypto.
- Use $h_{\mathbb{J}}$ to refer to the *Jubjub curve* cofactor, rather than 8.
- Correct an error in the y-coordinate formula for addition in § A.3.3.4 ***'Affine-Montgomery arithmetic'*** on p. 155 (the constraints were correct).
- Add acknowledgements for Brian Warner, Mary Maller, and the Least Authority audit.
- `Makefile` improvements.

2018.0-beta-15 2018-03-19

- Clarify the bit ordering of SHA-256.
- Drop `_t` from the names of representation types.
- Remove functions from the Sprout specification that it does not use.
- Updates to transaction format and consensus rules for Overwinter and Sapling.
- Add specification of the *Output statement*.
- Change $\mathsf{MerkleDepth}^{\mathsf{Sapling}}$ from 29 to 32.
- Updates to Sapling construction, changing how the *nullifier* is computed and separating it from the *randomized Spend validating key* (rk).
- Clarify conversions between *bit sequences* and *byte sequences* for sk, $\mathsf{repr}_{\mathbb{J}}(\mathsf{ak})$, and $\mathsf{repr}_{\mathbb{J}}(\mathsf{nk})$.
- Change the `Makefile` to avoid multiple reloads in PDF readers while rebuilding the PDF.
- Spacing and pagination improvements.

2018.0-beta-14 2018-03-11

- Only cosmetic changes to Sprout.
- Simplify $\mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}$ to use a single-byte index.
- Changes to diversification for *Pedersen hashes* and *Pedersen commitments*.
- Improve security definitions for signatures.

2018.0-beta-13 2018-03-11

- Only cosmetic changes to Sprout.
- Change how $(\mathsf{ask}, \mathsf{nsk})$ are derived from the *spending key* sk to ensure they are on the full range of $\mathbb{F}_{r_{\mathbb{J}}}$.
- Change $\mathsf{PRF}^{\mathsf{nr}}$ to produce output computationally indistinguishable from uniform on $\mathbb{F}_{r_{\mathbb{J}}}$.
- Change $\mathsf{Uncommitted}^{\mathsf{Sapling}}$ to be a u-coordinate for which there is no point on the curve.
- Appendix A updates:
  - categorize components into larger sections
  - fill in the [de]compression and validation algorithm
  - more precisely state the assumptions for inputs and outputs
  - delete not-all-one component which is no longer needed
  - factor out xor into its own component
  - specify [un]packing more precisely; separate it from boolean constraints
  - optimize checking for non-*small order*
  - notation in variable-base multiplication algorithm.

2018.0-beta-12 2018-03-06

- Add references to **Overwinter** ZIPs and update the section on **Overwinter**/**Sapling** transitions.
- Add a section on re-randomizable signatures.
- Add definition of $\mathsf{PRF}^{\mathsf{nr}}$.
- Work-in-progress on Sapling *statements*.
- Rename *"raw"* to *"homomorphic" Pedersen commitments*.
- Add packing modulo the field size and range checks to Appendix A.
- Update the algorithm for variable-base scalar multiplication to what is implemented by sapling-crypto.

2018.0-beta-11 2018-02-26

- Add sections on *Spend descriptions* and *Output descriptions*.
- Swap order of cv and rt in a *Spend description* for consistency.
- Fix off-by-one error in the range of ivk.

2018.0-beta-10 2018-02-26

- Split the descriptions of SHA-256 and SHA256Compress, and of BLAKE2, into their own sections. Specify SHA256Compress more precisely.
- Add Kexin Hu to acknowledgements (for the idea of explicitly encoding the root of the *Sapling-pool note commitment tree* in *block headers*).
- Move bit/byte/integer conversion primitives into § 5.1 ***'Integers, Bit Sequences, and Endianness'*** on p. 56.
- Refer to **Overwinter** and **Sapling** just as "upgrades" in the abstract, not as the next "minor version" and "major version".
- $\mathsf{PRF}^{\mathsf{nr}}$ must be *collision-resistant*.
- Correct an error in the *Pedersen hash* specification.
- Use a named variable, c, for chunks per segment in the *Pedersen hash* specification, and change its value from 61 to 63. Add a proof justifying this value of c.

- Specify *Pedersen commitments*.
- Notation changes.
- Generalize the *distinct-x criterion* (Theorem A.3.4 on p. 155) to allow negative indices.

2018.0-beta-9 2018-02-10

- Specify the coinbase maturity rule, and the rule that *coinbase transactions* cannot contain *JoinSplit descriptions*, *Spend descriptions*, or *Output descriptions*.
- Delay lifting the 100000-byte *transaction* size limit from **Overwinter** to **Sapling**.
- Improve presentation of the proof of injectivity for $\mathsf{Extract}_{\mathbb{J}^{(r)}}$.
- Specify $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}$.
- Specify *Pedersen hashes*.

2018.0-beta-8 2018-02-08

- Add instantiation of $\mathsf{CRH}^{\mathsf{ivk}}$.
- Add instantiation of a hash extractor (later renamed to *coordinate extractor*) for Jubjub.
- Make the background lighter and the **Sapling** green darker, for contrast.

2018.0-beta-7 2018-02-07

- Specify the 100000-byte limit on *transaction* size. (The implementation in *zcashd* was as intended.)
- Specify that 0xF6 followed by 511 zero bytes encodes an empty *memo field*.
- Reference security definitions for *Pseudo Random Functions* and *Pseudo Random Generators*.
- Rename clamp to bound and ActualTimespanClamped to ActualTimespanBounded in the difficulty adjustment algorithm, to avoid a name collision with Curve25519 scalar "clamping".
- Change uses of the term *full node* to *full validator*. A *full node* by definition participates in the *peer-to-peer network*, whereas a *full validator* just needs a copy of the *block chain* from somewhere. The latter is what was meant.
- Add an explanation of how Sapling prevents Faerie Gold and roadblock attacks.
- Sapling work in progress.

2018.0-beta-6 2018-01-31

- Sapling work in progress, mainly on Appendix § A *'Circuit Design'* on p. 149.

2018.0-beta-5 2018-01-30

- Specify more precisely the requirements on Ed25519 *validating keys* and signatures.
- Sapling work in progress.

2018.0-beta-4 2018-01-25

- Update key components diagram for Sapling.

2018.0-beta-3 2018-01-22

- Explain how the chosen fix to Faerie Gold avoids a potential "roadblock" attack.
- Update some explanations of changes from **Zerocash** for Sapling.
- Add a description of the *Jubjub curve*.
- Add an acknowledgement to George Tankersley.
- Add an appendix on the design of the Sapling circuits at the *quadratic constraint program* level.

2017.0-beta-2.9 2017-12-17

- Refer to $\mathsf{sk}_{\mathsf{enc}}$ as a *receiving key* rather than as a viewing key.
- Updates for *incoming viewing key* support.
- Refer to Network Upgrade 0 as **Overwinter**.

2017.0-beta-2.8 2017-12-02

- Correct the non-normative note describing how to check the order of π_B.
- Initial version of draft Sapling protocol specification.

2017.0-beta-2.7 2017-07-10

- Fix an off-by-one error in the specification of the *Equihash* algorithm binding condition. (The implementation in *zcashd* was as intended.)
- Correct the types and consensus rules for *transaction version numbers* and *block version numbers*. (Again, the implementation in *zcashd* was as intended.)
- Clarify the computation of h_i in a *JoinSplit statement*.

2017.0-beta-2.6 2017-05-09

- Be more precise when talking about curve points and pairing groups.

2017.0-beta-2.5 2017-03-07

- Clarify the consensus rule preventing *double-spends*.
- Clarify what a *note commitment* opens to in § 8.8 *'Omission in **Zerocash** security proof'* on p. 109.
- Correct the order of arguments to COMM in § 5.4.7.1 *'Note Commitments (Sprout)'* on p. 72.
- Correct a statement about indistinguishability of *JoinSplit descriptions*.
- Change the *Founders' Reward* addresses, for *Testnet* only, to reflect the hard-fork upgrade described in [Zcash-Issue2113].

2017.0-beta-2.4 2017-02-25

- Explain a variation on the Faerie Gold attack and why it is prevented.
- Generalize the description of the InternalH attack to include finding collisions on $(\mathsf{a}_{\mathsf{pk}}, \rho)$ rather than just on ρ.
- Rename $\mathsf{enforce}_i$ to $\mathsf{enforceMerklePath}_i$.

2017.0-beta-2.3 2017-02-12

- Specify the security requirements on the SHA256Compress function, in order for the scheme in § 5.4.7.1 *'Note Commitments (Sprout)'* on p. 72 to be a secure commitment.
- Specify $\mathbb{G}_2$ more precisely.
- Explain the use of interstitial *treestates* in chained *JoinSplit transfers*.

2017.0-beta-2.2 2017-02-11

- Give definitions of computational *binding* and computational *hiding* for *commitment schemes*.
- Give a definition of statistical zero knowledge.
- Reference the white paper on MPC parameter generation [BGG2017].

2017.0-beta-2.1 2017-02-06

- ℓ_{Merkle} is a bit length, not a byte length.
- Specify the maximum *block* size.

2017.0-beta-2 2017-02-04

- Add abstract and keywords.
- Fix a typo in the definition of *nullifier* integrity.
- Make the description of *block chains* more consistent with upstream **Bitcoin** documentation (referring to "best" chains rather than using the concept of a *block chain view*).
- Define how nodes select a *best valid block chain*.

2016.0-beta-1.13 2017-01-20

- Specify the difficulty adjustment algorithm.
- Clarify some definitions of fields in a *block header*.
- Define $\mathsf{PRF}^{\mathsf{addr}}$ in § 4.2.1 *'Sprout Key Components'* on p. 32.

2016.0-beta-1.12 2017-01-09

- Update the hashes of proving and verifying keys for the final Sprout parameters.
- Add cross references from *shielded payment address* and *spending key* encoding sections to where the key components are specified.
- Add acknowledgements for Filippo Valsorda and Zaki Manian.

2016.0-beta-1.11 2016-12-19

- Specify a check on the order of π_B in a *zk-SNARK proof*.
- Note that due to an oversight, the **Zcash** *genesis block* does not follow [BIP-34].

2016.0-beta-1.10 2016-10-30

- Update reference to the *Equihash* paper [BK2016]. (The newer version has no algorithmic changes, but the section discussing potential ASIC implementations is substantially expanded.)
- Clarify the discussion of proof size in "Differences from the **Zerocash** paper".

2016.0-beta-1.9 2016-10-28

- Add *Founders' Reward* addresses for *Mainnet*.
- Change *"protected"* terminology to *"shielded"*.

2016.0-beta-1.8 2016-10-04

- Revise the lead bytes for *transparent P2SH* and *P2PKH* addresses, and reencode the *Testnet Founders' Reward* addresses.
- Add a section on which BIPs apply to **Zcash**.
- Specify that `OP_CODESEPARATOR` has been disabled, and no longer affects *SIGHASH transaction hashes*.
- Change the representation type of `vpub_old` and `vpub_new` to `uint64`. (This is not a consensus change because the type of v_{pub}^{old} and v_{pub}^{new} was already specified to be $\{0 \,..\, \mathsf{MAX_MONEY}\}$; it just better reflects the implementation.)
- Correct the representation type of the *block* `nVersion` field to `uint32`.

2016.0-beta-1.7 2016-10-02

- Clarify the consensus rule for payment of the *Founders' Reward*, in response to an issue raised by the NCC audit.

2016.0-beta-1.6 2016-09-26

- Fix an error in the definition of the sortedness condition for *Equihash*: it is the sequences of indices that are sorted, not the sequences of hashes.
- Correct the number of bytes in the encoding of `solutionSize`.
- Update the section on encoding of *transparent addresses*. (The precise prefixes are not decided yet.)
- Clarify why BLAKE2b-ℓ is different from truncated BLAKE2b-512.
- Clarify a note about SU-CMA security for signatures.
- Add a note about $\mathsf{PRF}^{\mathsf{nfSprout}}$ corresponding to $\mathsf{PRF}^{\mathsf{sn}}$ in **Zerocash**.
- Add a paragraph about key length in § 8.7 ***'In-band secret distribution'*** on p. 107.
- Add acknowledgements for John Tromp, Paige Peterson, Maureen Walsh, Jay Graber, and Jack Gavigan.

2016.0-beta-1.5 2016-09-22

- Update the *Founders' Reward* address list.
- Add some clarifications based on Eli Ben-Sasson's review.

2016.0-beta-1.4 2016-09-19

- Specify the *block subsidy*, *miner subsidy*, and the *Founders' Reward*.
- Specify *coinbase transaction* outputs to *Founders' Reward* addresses.
- Improve notation (for example "$\cdot$" for multiplication and "$T^{[\ell]}$" for sequence types) to avoid ambiguity.

2016.0-beta-1.3 2016-09-16

- Correct the omission of `solutionSize` from the *block header* format.
- Document that `compactSize` encodings must be canonical.
- Add a note about conformance language in the introduction.
- Add acknowledgements for Solar Designer, Ling Ren and Alison Stevenson, and for the NCC Group and Coinspect security audits.

2016.0-beta-1.2 2016-09-11

- Remove GeneralCRH in favour of specifying hSigCRH and EquihashGen directly in terms of BLAKE2b-ℓ.
- Correct the security requirement for EquihashGen.

2016.0-beta-1.1 2016-09-05

- Add a specification of abstract signatures.
- Clarify what is signed in the "Sending Notes" section.
- Specify ZK parameter generation as a randomized algorithm, rather than as a distribution of parameters.

2016.0-beta-1 2016-09-04

- Major reorganization to separate the abstract cryptographic protocol from the algorithm instantiations.
- Add type declarations.
- Add a "High-level Overview" section.
- Add a section specifying the *zero-knowledge proving system* and the encoding of proofs. Change the encoding of points in proofs to follow IEEE Std 1363[a].
- Add a section on consensus changes from **Bitcoin**, and the specification of *Equihash*.
- Complete the "Differences from the **Zerocash** paper" section.
- Correct the Merkle tree depth to 29.
- Change the length of *memo fields* to 512 bytes.
- Switch the *JoinSplit signature* scheme to Ed25519, with consequent changes to the computation of $\mathsf{h}_{\mathsf{Sig}}$.
- Fix the lead bytes in *shielded payment address* and *spending key* encodings to match the implemented protocol.
- Add a consensus rule about the ranges of $\mathsf{v}_{\mathsf{pub}}^{\mathsf{old}}$ and $\mathsf{v}_{\mathsf{pub}}^{\mathsf{new}}$.
- Clarify cryptographic security requirements and added definitions relating to the *in-band* secret distribution.
- Add various citations: the "Fixing Vulnerabilities in the Zcash Protocol" and "Why Equihash?" blog posts, several crypto papers for security definitions, the **Bitcoin** whitepaper, the CryptoNote whitepaper, and several references to **Bitcoin** documentation.
- Reference the extended version of the **Zerocash** paper rather than the Oakland proceedings version.
- Add *JoinSplit transfers* to the Concepts section.
- Add a section on Coinbase Transactions.
- Add acknowledgements for Jack Grigg, Simon Liu, Ariel Gabizon, jl777, Ben Blaxill, Alex Balducci, and Jake Tarren.
- Fix a `Makefile` compatibility problem with the escaping behaviour of `echo`.

- Switch to `biber` for the bibliography generation, and add backreferences.
- Make the date format in references more consistent.
- Add visited dates to all URLs in references.
- Terminology changes.

2016.0-alpha-3.1 2016-05-20

- Change main font to Quattrocento.

2016.0-alpha-3 2016-05-09

- Change version numbering convention (no other changes).

2.0-alpha-3 2016-05-06

- Allow anchoring to any previous output *treestate* in the same *transaction*, rather than just the immediately preceding output *treestate*.
- Add change history.

2.0-alpha-2 2016-04-21

- Change from truncated BLAKE2b-512 to BLAKE2b-256.
- Clarify endianness, and that uses of BLAKE2b are unkeyed.
- Minor correction to what *SIGHASH types* cover.
- Add "as intended for the **Zcash** release of summer 2016" to title page.
- Require $\mathsf{PRF}^{\mathsf{addr}}$ to be *collision-resistant* (see § 8.8 *'Omission in **Zerocash** security proof'* on p. 109).
- Add specification of path computation for the *incremental Merkle tree*.
- Add a note in § 4.16.1 *'JoinSplit Statement (Sprout)'* on p. 47 about how this condition corresponds to conditions in the **Zerocash** paper.
- Changes to terminology around keys.

2.0-alpha-1 2016-03-30

- First version intended for public review.

11 References

[ABR1999] Michel Abdalla, Mihir Bellare, and Phillip Rogaway. *DHAES: An Encryption Scheme Based on the Diffie–Hellman Problem*. Cryptology ePrint Archive: Report 1999/007. Received March 17, 1999. September 1998. URL: `https://eprint.iacr.org/1999/007` (visited on 2016-08-21) (↑ p24, 108).

[AGRRT2017] Martin Albrecht, Lorenzo Grassi, Christian Rechberger, Arnab Roy, and Tyge Tiessen. *MiMC: Efficient Encryption and Cryptographic Hashing with Minimal Multiplicative Complexity*. Cryptology ePrint Archive: Report 2016/492. Received May 21, 2016. January 5, 2017. URL: `https://eprint.iacr.org/2016/492` (visited on 2018-01-12) (↑ p165).

[AKLGL2010] Diego Aranha, Koray Karabina, Patrick Longa, Catherine Gebotys, and Julio López. *Faster Explicit Formulas for Computing Pairings over Ordinary Curves*. Cryptology ePrint Archive: Report 2010/526. Last revised September 12, 2011. URL: `https://eprint.iacr.org/2010/526` (visited on 2018-04-03) (↑ p74, 130).

[ANWW2013] Jean-Philippe Aumasson, Samuel Neves, Zooko Wilcox, and Christian Winnerlein. *BLAKE2: simpler, smaller, fast as MD5*. January 29, 2013. URL: `https://blake2.net/#sp` (visited on 2016-08-14) (↑ p59, 163).

[AR2017] Leo Alcock and Ling Ren. "A Note on the Security of Equihash". In: *CCSW '17. Proceedings of the 2017 Cloud Computing Security Workshop (Dallas, TX, USA, November 3, 2017); post-workshop of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. ACM. URL: `https://sci-hubtw.hkvisa.net/10.1145/3140649.3140652` (visited on 2021-04-05) (↑ p97, 124).

[BBDP2001] Mihir Bellare, Alexandra Boldyreva, Anand Desai, and David Pointcheval. *Key-Privacy in Public-Key Encryption*. September 2001. URL: `https://cseweb.ucsd.edu/~mihir/papers/anonenc.pdf` (visited on 2021-09-01). Full version. (↑ p24, 61, 108, 116, 126).

[BBJLP2008] Daniel Bernstein, Peter Birkner, Marc Joye, Tanja Lange, and Christiane Peters. *Twisted Edwards Curves*. Cryptology ePrint Archive: Report 2008/013. Received January 8, 2008. March 13, 2008. URL: `https://eprint.iacr.org/2008/013` (visited on 2018-01-12) (↑ p155, 156).

[BCCGLRT2014] Nir Bitansky, Ran Canetti, Alessandro Chiesa, Shafi Goldwasser, Huijia Lin, Aviad Rubinstein, and Eran Tromer. *The Hunting of the SNARK*. Cryptology ePrint Archive: Report 2014/580. Received July 24, 2014. URL: `https://eprint.iacr.org/2014/580` (visited on 2020-08-01) (↑ p30, 121).

[BCGGMTV2014] Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. *Zerocash: Decentralized Anonymous Payments from Bitcoin (extended version)*. Cryptology ePrint Archive: Report 2014/349. Received May 19, 2014. URL: `https://eprint.iacr.org/2014/349` (visited on 2021-04-05). A condensed version appeared in *Proceedings of the IEEE Symposium on Security and Privacy (Oakland) 2014*, pages 459–474; IEEE, 2014. (↑ p7, 8, 10, 21, 23, 42, 47, 51, 104, 106, 107, 109).

[BCGTV2013] Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, Eran Tromer, and Madars Virza. *SNARKs for C: Verifying Program Executions Succinctly and in Zero Knowledge*. Cryptology ePrint Archive: Report 2013/507. Last revised October 7, 2013. URL: `https://eprint.iacr.org/2013/507` (visited on 2016-08-31). An earlier version appeared in *Proceedings of the 33rd Annual International Cryptology Conference, CRYPTO 2013*, pages 90–108; IACR, 2013. (↑ p79).

[BCP1988] Jurgen Bos, David Chaum, and George Purdy. "A Voting Scheme". Unpublished. Presented at the rump session of CRYPTO '88 (Santa Barbara, California, USA, August 21–25, 1988); does not appear in the proceedings. (↑ p62).

[BCTV2014a] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. *Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture*. Cryptology ePrint Archive: Report 2013/879. Last revised February 5, 2019. URL: `https://eprint.iacr.org/2013/879` (visited on 2019-02-08) (↑ p79, 80, 125, 149).

[BCTV2014a-old] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. *Succinct Non-Interactive Zero Knowledge for a von Neumann Architecture (May 19, 2015 version)*. Cryptology ePrint Archive: Report 2013/879. Version: 20150519:172604. URL: https://eprint.iacr.org/2013/879/20150519:172604 (visited on 2019-02-08) (↑ p80).

[BCTV2014b] Eli Ben-Sasson, Alessandro Chiesa, Eran Tromer, and Madars Virza. "Scalable Zero Knowledge via Cycles of Elliptic Curves (extended version)". In: *Advances in Cryptology - CRYPTO 2014*. Vol. 8617. Lecture Notes in Computer Science. Springer, 2014, pages 276–294. URL: https://www.cs.tau.ac.il/~tromer/papers/scalablezk-20140803.pdf (visited on 2016-09-01) (↑ p31, 125).

[BDEHR2011] Johannes Buchmann, Erik Dahmen, Sarah Ereth, Andreas Hülsing, and Markus Rückert. *On the Security of the Winternitz One-Time Signature Scheme (full version)*. Cryptology ePrint Archive: Report 2011/191. Received April 13, 2011. URL: https://eprint.iacr.org/2011/191 (visited on 2016-09-05) (↑ p25).

[BDJR2000] Mihir Bellare, Anand Desai, Eric Jokipii, and Phillip Rogaway. *A Concrete Security Treatment of Symmetric Encryption: Analysis of the DES Modes of Operation*. September 2000. URL: https://cseweb.ucsd.edu/~mihir/papers/sym-enc.pdf (visited on 2021-09-01). An extended abstract appeared in *Proceedings of the 38th Annual Symposium on Foundations of Computer Science (Miami Beach, Florida, USA, October 20–22, 1997)*, pages 394–403; IEEE Computer Society Press, 1997; ISBN 0-8186-8197-7. (↑ p23, 116).

[BDLSY2012] Daniel Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. "High-speed high-security signatures". In: *Journal of Cryptographic Engineering* 2 (September 26, 2011), pages 77–89. URL: https://cr.yp.to/papers.html#ed25519 (visited on 2021-04-05). Document ID: a1a62a2f76d23f65d622484ddd09caf8. (↑ p68, 117, 170).

[Bernstein2001] Daniel Bernstein. *Pippenger's exponentiation algorithm*. December 18, 2001. URL: https://cr.yp.to/papers.html#pippenger (visited on 2018-07-27). Draft. Error pointed out by Sam Hocevar: the example in Figure 4 needs 2 and is thus of length 18. (↑ p170, 171).

[Bernstein2005] Daniel Bernstein. "Understanding brute force". In: *ECRYPT STVL Workshop on Symmetric Key Encryption, eSTREAM report 2005/036*. April 25, 2005. URL: https://cr.yp.to/papers.html#bruteforce (visited on 2016-09-24). Document ID: 73e92f5b71793b498288efe81fe55dee. (↑ p108).

[Bernstein2006] Daniel Bernstein. "Curve25519: new Diffie-Hellman speed records". In: *Public Key Cryptography – PKC 2006. Proceedings of the 9th International Conference on Theory and Practice in Public-Key Cryptography (New York, NY, USA, April 24–26, 2006)*. Springer, February 9, 2006. URL: https://cr.yp.to/papers.html#curve25519 (visited on 2021-04-05). Document ID: 4230efdfa673480fc079449d90f322c0. (↑ p24, 66, 83, 84, 107).

[BFIJSV2010] Olivier Blazy, Georg Fuchsbauer, Malika Izabachène, Amandine Jambert, Hervé Sibert, and Damien Vergnaud. *Batch Groth–Sahai*. Cryptology ePrint Archive: Report 2010/040. Last revised February 3, 2010. URL: https://eprint.iacr.org/2010/040 (visited on 2020-10-17) (↑ p111, 120, 170).

[BGG-mpc] Sean Bowe, Ariel Gabizon, and Matthew Green. *GitHub repository 'zcash/mpc': zk-SNARK parameter multi-party computation protocol*. URL: https://github.com/zcash/mpc (visited on 2017-01-06) (↑ p87).

[BGG1995] Mihir Bellare, Oded Goldreich, and Shafi Goldwasser. "Incremental Cryptography: The Case of Hashing and Signing". In: *Advances in Cryptology - CRYPTO '94. Proceedings of the 14th Annual International Cryptology Conference (Santa Barbara, California, USA, August 21–25, 1994)*. Ed. by Yvo Desmedt. Vol. 839. Lecture Notes in Computer Science. Springer, October 20, 1995, pages 216–233. ISBN: 978-3-540-48658-9. DOI: 10.1007/3-540-48658-5_22. URL: https://cseweb.ucsd.edu/~mihir/papers/inc1.pdf (visited on 2018-02-09) (↑ p62, 63, 159).

[BGG2017] Sean Bowe, Ariel Gabizon, and Matthew Green. *A multi-party protocol for constructing the public parameters of the Pinocchio zk-SNARK*. Cryptology ePrint Archive: Report 2017/602. Last revised June 25, 2017. URL: https://eprint.iacr.org/2017/602 (visited on 2019-02-10) (↑ p80, 87, 124, 135).

[BGM2017] Sean Bowe, Ariel Gabizon, and Ian Miers. *Scalable Multi-party Computation for zk-SNARK Parameters in the Random Beacon Model*. Cryptology ePrint Archive: Report 2017/1050. Last revised November 5, 2017. URL: https://eprint.iacr.org/2017/1050 (visited on 2018-08-31) (↑ p81, 87, 125, 126).

[BIP-11] Gavin Andresen. *M-of-N Standard Transactions*. Bitcoin Improvement Proposal 11. Created October 18, 2011. URL: https://github.com/bitcoin/bips/blob/master/bip-0011.mediawiki (visited on 2020-07-13) (↑ p103).

[BIP-13] Gavin Andresen. *Address Format for pay-to-script-hash*. Bitcoin Improvement Proposal 13. Created October 18, 2011. URL: https://github.com/bitcoin/bips/blob/master/bip-0013.mediawiki (visited on 2020-07-13) (↑ p82, 103).

[BIP-14] Amir Taaki and Patrick Strateman. *Protocol Version and User Agent*. Bitcoin Improvement Proposal 14. Created November 10, 2011. URL: https://github.com/bitcoin/bips/blob/master/bip-0014.mediawiki (visited on 2020-07-13) (↑ p103).

[BIP-16] Gavin Andresen. *Pay to Script Hash*. Bitcoin Improvement Proposal 16. Created January 3, 2012. URL: https://github.com/bitcoin/bips/blob/master/bip-0016.mediawiki (visited on 2020-07-13) (↑ p103).

[BIP-30] Pieter Wuille. *Duplicate transactions*. Bitcoin Improvement Proposal 30. Created February 22, 2012. URL: https://github.com/bitcoin/bips/blob/master/bip-0030.mediawiki (visited on 2020-07-13) (↑ p103).

[BIP-31] Mike Hearn. *Pong message*. Bitcoin Improvement Proposal 31. Created April 11, 2012. URL: https://github.com/bitcoin/bips/blob/master/bip-0031.mediawiki (visited on 2020-07-13) (↑ p103).

[BIP-32] Pieter Wuille. *Hierarchical Deterministic Wallets*. Bitcoin Improvement Proposal 32. Created February 11, 2012. Last updated January 15, 2014. URL: https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki (visited on 2020-07-13) (↑ p83, 122).

[BIP-34] Gavin Andresen. *Block v2, Height in Coinbase*. Bitcoin Improvement Proposal 34. Created July 6, 2012. URL: https://github.com/bitcoin/bips/blob/master/bip-0034.mediawiki (visited on 2020-07-13) (↑ p91, 103, 135).

[BIP-35] Jeff Garzik. *mempool message*. Bitcoin Improvement Proposal 35. Created August 16, 2012. URL: https://github.com/bitcoin/bips/blob/master/bip-0035.mediawiki (visited on 2020-07-13) (↑ p103).

[BIP-37] Mike Hearn and Matt Corallo. *Connection Bloom filtering*. Bitcoin Improvement Proposal 37. Created October 24, 2012. URL: https://github.com/bitcoin/bips/blob/master/bip-0037.mediawiki (visited on 2020-07-13) (↑ p103).

[BIP-61] Gavin Andresen. *Reject P2P message*. Bitcoin Improvement Proposal 61. Created June 18, 2014. URL: https://github.com/bitcoin/bips/blob/master/bip-0061.mediawiki (visited on 2020-07-13) (↑ p103).

[BIP-62] Pieter Wuille. *Dealing with malleability*. Bitcoin Improvement Proposal 62. Withdrawn November 17, 2015. URL: https://github.com/bitcoin/bips/blob/master/bip-0062.mediawiki (visited on 2020-07-13) (↑ p26).

[BIP-65] Peter Todd. *OP_CHECKLOCKTIMEVERIFY*. Bitcoin Improvement Proposal 65. Created October 10, 2014. URL: https://github.com/bitcoin/bips/blob/master/bip-0065.mediawiki (visited on 2020-07-13) (↑ p103).

[BIP-66] Pieter Wuille. *Strict DER signatures*. Bitcoin Improvement Proposal 66. Created January 10, 2015. URL: https://github.com/bitcoin/bips/blob/master/bip-0066.mediawiki (visited on 2020-07-13) (↑ p103).

[BIP-68] Mark Friedenbach, BtcDrak, Nicolas Dorier, and kinoshitajona. *Relative lock-time using consensus-enforced sequence numbers*. Bitcoin Improvement Proposal 68. Last revised November 21, 2015. URL: https://github.com/bitcoin/bips/blob/master/bip-0068.mediawiki (visited on 2020-07-13) (↑ p92).

[BIP-111] Matt Corallo and Peter Todd. *NODE_BLOOM service bit*. Bitcoin Improvement Proposal 111. Created August 20, 2015. URL: https://github.com/bitcoin/bips/blob/master/bip-0111.mediawiki (visited on 2020-07-13) (↑ p103, 130).

[Bitcoin-Base58] *Base58Check encoding — Bitcoin Wiki*. URL: https://en.bitcoin.it/wiki/Base58Check_encoding (visited on 2020-07-13) (↑ p82, 83).

[Bitcoin-Block] *Block Headers — Bitcoin Developer Reference*. URL: https://developer.bitcoin.org/reference/block_chain.html#block-headers (visited on 2020-07-13) (↑ p96, 97).

[Bitcoin-CbInput] *Coinbase Input — Bitcoin Developer Reference*. URL: https://developer.bitcoin.org/reference/transactions.html#coinbase-input-the-input-of-the-first-transaction-in-a-block (visited on 2022-03-17) (↑ p115).

[Bitcoin-CoinJoin] *CoinJoin — Bitcoin Wiki*. URL: https://en.bitcoin.it/wiki/CoinJoin (visited on 2020-07-13) (↑ p9).

[Bitcoin-Format] *Raw Transaction Format — Bitcoin Developer Reference*. URL: https://developer.bitcoin.org/reference/transactions.html#raw-transaction-format (visited on 2020-07-13) (↑ p92).

[Bitcoin-Multisig] *Transactions: Multisig — Bitcoin Developer Guide*. URL: https://developer.bitcoin.org/devguide/transactions.html#multisig (visited on 2020-07-13) (↑ p102).

[Bitcoin-nBits] *Target nBits — Bitcoin Developer Reference*. URL: https://developer.bitcoin.org/reference/block_chain.html#target-nbits (visited on 2020-07-13) (↑ p95, 99).

[Bitcoin-P2PKH] *Transactions: P2PKH Script Validation — Bitcoin Developer Guide*. URL: https://developer.bitcoin.org/devguide/transactions.html#p2pkh-script-validation (visited on 2020-07-13) (↑ p82).

[Bitcoin-P2SH] *Transactions: P2SH Scripts — Bitcoin Developer Guide*. URL: https://developer.bitcoin.org/devguide/transactions.html#pay-to-script-hash-p2sh (visited on 2020-07-13) (↑ p82).

[Bitcoin-Protocol] *Protocol documentation — Bitcoin Wiki*. URL: https://en.bitcoin.it/wiki/Protocol_documentation (visited on 2020-07-13) (↑ p8).

[Bitcoin-SigHash] *Signature Hash Types — Bitcoin Developer Guide*. URL: https://developer.bitcoin.org/devguide/transactions.html#signature-hash-types (visited on 2020-07-13) (↑ p40).

[BJLSY2015] Daniel Bernstein, Simon Josefsson, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. *EdDSA for more curves*. Technical Report. July 4, 2015. URL: https://cr.yp.to/papers.html#eddsa (visited on 2018-01-22) (↑ p69, 78).

[BK2016] Alex Biryukov and Dmitry Khovratovich. *Equihash: Asymmetric Proof-of-Work Based on the Generalized Birthday Problem (full version)*. Cryptology ePrint Archive: Report 2015/946. Last revised October 27, 2016. URL: https://eprint.iacr.org/2015/946 (visited on 2016-10-30) (↑ p10, 97, 135).

[BL-SafeCurves] Daniel Bernstein and Tanja Lange. *SafeCurves: choosing safe curves for elliptic-curve cryptography*. URL: https://safecurves.cr.yp.to (visited on 2018-01-29) (↑ p107, 145).

[BL2017] Daniel Bernstein and Tanja Lange. *Montgomery curves and the Montgomery ladder*. Cryptology ePrint Archive: Report 2017/293. Received March 30, 2017. URL: https://eprint.iacr.org/2017/293 (visited on 2017-11-26) (↑ p77, 149, 155, 156).

[BLS2002] Paulo Barreto, Ben Lynn, and Michael Scott. *Constructing Elliptic Curves with Prescribed Embedding Degrees*. Cryptology ePrint Archive: Report 2002/088. Last revised February 22, 2005. URL: https://eprint.iacr.org/2002/088 (visited on 2018-04-20) (↑ p75, 130).

[BN2005] Paulo Barreto and Michael Naehrig. *Pairing-Friendly Elliptic Curves of Prime Order*. Cryptology ePrint Archive: Report 2005/133. Last revised February 28, 2006. URL: https://eprint.iacr.org/2005/133 (visited on 2018-04-20) (↑ p74, 130).

[BN2007] Mihir Bellare and Chanathip Namprempre. *Authenticated Encryption: Relations among notions and analysis of the generic composition paradigm*. Cryptology ePrint Archive: Report 2000/025. Last revised July 14, 2007. URL: https://eprint.iacr.org/2000/025 (visited on 2016-09-02) (↑ p23).

[Bowe-bellman] Sean Bowe. *bellman: zk-SNARK library*. URL: https://github.com/ebfull/bellman (visited on 2018-04-03) (↑ p81, 87).

[Bowe2017] Sean Bowe. *ebfull/pairing source code, BLS12-381 – README.md as of commit e726600*. URL: https://github.com/ebfull/pairing/tree/e72660056e00c93d6b054dfb08ff34a1c67cb799/src/bls12_381 (visited on 2017-07-16) (↑ p75).

[Bowe2018] Sean Bowe. *Random Beacon*. March 22, 2018. URL: https://github.com/ZcashFoundation/powersoftau-attestations/tree/master/0088 (visited on 2018-04-08) (↑ p87).

[Carroll1876] Lewis Carroll. *The Hunting of the Snark*. With illustrations by Henry Holiday. MacMillan and Co. London. March 29, 1876. URL: https://www.gutenberg.org/files/29888/29888-h/29888-h.htm (visited on 2018-05-23) (↑ p77, 123).

[Carroll1902] Lewis Carroll. *Through the Looking-Glass, and What Alice Found There (1902 edition)*. Illustrated by Peter Newell and Robert Murray Wright. Harper and Brothers Publishers. New York. October 1902. URL: https://archive.org/details/throughlookinggl00carr4 (visited on 2018-06-20) (↑ p111, 129).

[CDvdG1987] David Chaum, Ivan Damgård, and Jeroen van de Graaf. "Multiparty computations ensuring privacy of each party's input and correctness of the result". In: *Advances in Cryptology - CRYPTO '87. Proceedings of the 14th Annual International Cryptology Conference (Santa Barbara, California, USA, August 16–20, 1987)*. Ed. by Carl Pomerance. Vol. 293. Lecture Notes in Computer Science. Springer, January 1988, pages 87–119. ISBN: 978-3-540-48184-3. DOI: 10.1007/3-540-48184-2_7. URL: https://link.springer.com/content/pdf/10.1007%2F3-540-48184-2_7.pdf (visited on 2022-08-31) (↑ p62, 114).

[CVE-2019-7167] Common Vulnerabilities and Exposures. *CVE-2019-7167*. URL: https://cve.mitre.org/cgi-bin/cvename.cgi?name=CVE-2019-7167 (visited on 2019-02-05) (↑ p80).

[CvHP1991] David Chaum, Eugène van Heijst, and Birgit Pfitzmann. *Cryptographically Strong Undeniable Signatures, Unconditionally Secure for the Signer*. February 1991. URL: https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.34.8570 (visited on 2021-04-05). An extended abstract appeared in *Advances in Cryptology - CRYPTO '91: Proceedings of the 11th Annual International Cryptology Conference (Santa Barbara, California, USA, August 11–15, 1991)*; Ed. by Joan Feigenbaum; Vol. 576, Lecture Notes in Computer Science, pages 470–484; Springer, 1992; ISBN 978-3-540-55188-1. (↑ p62, 159).

[Dalek-notes] Cathie Yun, Henry de Valence, Oleg Andreev, and Dimitris Apostolou. *Dalek bulletproofs notes, module r1cs_proof*. URL: https://doc-internal.dalek.rs/bulletproofs/notes/r1cs_proof/index.html (visited on 2021-04-07) (↑ p44, 126).

[Damgård1989] Ivan Damgård. "A Design Principle for Hash Functions". In: *Advances in Cryptology - CRYPTO '89. Proceedings of the 9th Annual International Cryptology Conference (Santa Barbara, California, USA, August 20–24, 1989)*. Ed. by Giles Brassard. Vol. 435. Lecture Notes in Computer Science. Springer, 1990, pages 416–427. ISBN: 978-0-387-34805-6. DOI: 10.1007/0-387-34805-0_39. URL: https://link.springer.com/chapter/10.1007/0-387-34805-0_39 (visited on 2022-01-19) (↑ p106).

[deRooij1995] Peter de Rooij. "Efficient exponentiation using precomputation and vector addition chains". In: *Advances in Cryptology - EUROCRYPT '94. Proceedings, Workshop on the Theory and Application of Cryptographic Techniques (Perugia, Italy, May 9–12, 1994)*. Ed. by Alfredo De Santis. Vol. 950. Lecture Notes in Computer Science. Springer, pages 389–399. ISBN: 978-3-540-60176-0. DOI: 10.1007/BFb0053453. URL: https://link.springer.com/chapter/10.1007/BFb0053453 (visited on 2018-07-27) (↑ p170, 171).

[DGKM2011] Dana Dachman-Soled, Rosario Gennaro, Hugo Krawczyk, and Tal Malkin. *Computational Extractors and Pseudorandomness*. Cryptology ePrint Archive: Report 2011/708. December 28, 2011. URL: https://eprint.iacr.org/2011/708 (visited on 2016-09-02) (↑ p108).

[DigiByte-PoW] DigiByte Core Developers. *DigiSpeed 4.0.0 source code, functions GetNextWorkRequiredV3/4 in src/main.cpp as of commit 178e134*. URL: https://github.com/digibyte/digibyte/blob/178e1348a67d9624db328062397fde0de03fe388/src/main.cpp#L1587 (visited on 2017-01-20) (↑ p98).

[DS2016] David Derler and Daniel Slamanig. *Key-Homomorphic Signatures and Applications to Multiparty Signatures and Non-Interactive Zero-Knowledge*. Cryptology ePrint Archive: Report 2016/792. Last revised February 6, 2017. URL: https://eprint.iacr.org/2016/792 (visited on 2018-04-09) (↑ p27).

[DSDCOPS2001] Alfredo De Santis, Giovanni Di Crescenzo, Rafail Ostrovsky, Guiseppe Persiano, and Amit Sahai. "Robust Non-Interactive Zero Knowledge". In: *Advances in Cryptology - CRYPTO 2001. Proceedings of the 21st Annual International Cryptology Conference (Santa Barbara, California, USA, August 19–23, 2001)*. Ed. by Joe Kilian. Vol. 2139. Lecture Notes in Computer Science. Springer, 2001, pages 566–598. ISBN: 978-3-540-42456-7. DOI: 10.1007/3-540-44647-8_33. URL: https://www.iacr.org/archive/crypto2001/21390566.pdf (visited on 2018-05-28) (↑ p31, 40).

[ECCZF2019] Electric Coin Company and Zcash Foundation. *Zcash Trademark Donation and License Agreement*. November 6, 2019. URL: https://electriccoin.co/wp-content/uploads/2019/11/Final-Consolidated-Version-ECC-Zcash-Trademark-Transfer-Documents-1.pdf (visited on 2022-06-22) (↑ p115).

[ElGamal1985] Taher ElGamal. "A public key cryptosystem and a signature scheme based on discrete logarithms". In: *IEEE Transactions on Information Theory* 31.4 (July 1985), pages 469–472. ISSN: 0018-9448. DOI: 10.1109/TIT.1985.1057074. URL: https://people.csail.mit.edu/alinush/6.857-spring-2015/papers/elgamal.pdf (visited on 2018-08-17) (↑ p61).

[EWD-340] Edsger W. Dijkstra. *The Humble Programmer*. ACM Turing Lecture. August 14, 1972. URL: https://www.cs.utexas.edu/users/EWD/transcriptions/EWD03xx/EWD340.html (visited on 2021-03-29) (↑ p21).

[EWD-831] Edsger W. Dijkstra. *Why numbering should start at zero*. Manuscript. August 11, 1982. URL: https://www.cs.utexas.edu/users/EWD/transcriptions/EWD08xx/EWD831.html (visited on 2016-08-09) (↑ p10).

[FKMSSS2016] Nils Fleischhacker, Johannes Krupp, Giulio Malavolta, Jonas Schneider, Dominique Schröder, and Mark Simkin. *Efficient Unlinkable Sanitizable Signatures from Signatures with Re-Randomizable Keys*. Cryptology ePrint Archive: Report 2012/159. Last revised February 11, 2016. URL: https://eprint.iacr.org/2015/395 (visited on 2018-03-03). An extended abstract appeared in *Public Key Cryptography – PKC 2016: 19th IACR International Conference on Practice and Theory in Public-Key Cryptography (Taipei, Taiwan, March 6–9, 2016), Proceedings, Part 1*; Ed. by Chen-Mou Cheng, Kai-Min Chung, Giuseppe Persiano, and Bo-Yin Yang; Vol. 9614, Lecture Notes in Computer Science, pages 301–330; Springer, 2016; ISBN 978-3-662-49384-7. (↑ p26, 27, 69).

[Gabizon2019] Ariel Gabizon. *On the security of the BCTV Pinocchio zk-SNARK variant*. Draft. February 5, 2019. URL: https://github.com/arielgabizon/bctv/blob/master/bctv.pdf (visited on 2019-02-07) (↑ p80, 110, 125).

[GGM2016] Christina Garman, Matthew Green, and Ian Miers. *Accountable Privacy for Decentralized Anonymous Payments*. Cryptology ePrint Archive: Report 2016/061. Last revised January 24, 2016. URL: https://eprint.iacr.org/2016/061 (visited on 2016-09-02) (↑ p105).

[Groth2016] Jens Groth. *On the Size of Pairing-based Non-interactive Arguments*. Cryptology ePrint Archive: Report 2016/260. Last revised May 31, 2016. URL: https://eprint.iacr.org/2016/260 (visited on 2017-08-03) (↑ p81, 124, 171).

[GWC2019] Ariel Gabizon, Zachary Williamson, and Oana Ciobotaru. *PLONK: Permutations over Lagrange-bases for Oecumenical Noninteractive arguments of Knowledge*. Cryptology ePrint Archive: Report 2019/953. Last revised September 3, 2020. URL: https://eprint.iacr.org/2019/953 (visited on 2021-01-28) (↑ p111).

[Hamdon2018] Elise Hamdon. *Sapling Activation Complete*. Electric Coin Company blog. June 28, 2018. URL: https://electriccoin.co/blog/sapling-activation-complete/ (visited on 2021-01-10) (↑ p88).

[Hopwood2018] Daira-Emma Hopwood. *GitHub repository 'daira/jubjub': Supporting evidence for security of the Jubjub curve to be used in Zcash*. URL: https://github.com/daira/jubjub (visited on 2018-02-18). Based on code written for SafeCurves [BL-SafeCurves] by Daniel Bernstein and Tanja Lange. (↑ p107).

[Hopwood2022] Daira-Emma Hopwood. *Explaining the Security of Zcash*. Presentation at Zcon3. Slides and a link to the video are available at: *GitHub repository 'daira/zcash-security': Code and documentation supporting security analysis of Zcash*. URL: https://github.com/daira/zcash-security (visited on 2023-10-30) (↑ p105, 114).

[HW2016] Taylor Hornby and Zooko Wilcox. *Fixing Vulnerabilities in the Zcash Protocol*. Electric Coin Company blog. April 26, 2016. URL: https://electriccoin.co/blog/fixing-zcash-vulns/ (visited on 2019-08-27). Updated December 26, 2017. (↑ p106).

[IEEE2000] IEEE Computer Society. *IEEE Std 1363-2000: Standard Specifications for Public-Key Cryptography*. IEEE, August 29, 2000. DOI: 10.1109/IEEESTD.2000.92292. URL: https://ieeexplore.ieee.org/document/891000 (visited on 2021-04-05) (↑ p75).

[IEEE2004] IEEE Computer Society. *IEEE Std 1363a-2004: Standard Specifications for Public-Key Cryptography – Amendment 1: Additional Techniques*. IEEE, September 2, 2004. DOI: 10.1109/IEEESTD.2004.94612. URL: https://ieeexplore.ieee.org/document/1335427 (visited on 2021-04-05) (↑ p75, 108, 110).

[Jedusor2016] Tom Elvis Jedusor. *Mimblewimble*. July 19, 2016. URL: https://diyhpl.us/~bryan/papers2/bitcoin/mimblewimble.txt (visited on 2021-04-05) (↑ p44).

[KvE2013] Kaa1el and Hagen von Eitzen. *If a group G has odd order, then the square function is injective (answer)*. Mathematics Stack Exchange. URL: https://math.stackexchange.com/a/522277/185422 (visited on 2018-02-08). Version: 2013-10-11. (↑ p78).

[KYMM2018] George Kappos, Haaroon Yousaf, Mary Maller, and Sarah Meiklejohn. *An Empirical Analysis of Anonymity in Zcash*. Preprint, to be presented at the 27th Usenix Security Syposium (Baltimore, Maryland, USA, August 15–17, 2018). May 8, 2018. URL: https://smeiklej.com/files/usenix18.pdf (visited on 2018-06-05) (↑ p9).

[LG2004] Eddie Lenihan and Carolyn Eve Green. *Meeting the Other Crowd: The Fairy Stories of Hidden Ireland*. TarcherPerigee, February 2004, pages 109–110. ISBN: 1-58542-206-1 (↑ p104).

[LGR2021] Julia Len, Paul Grubbs, and Thomas Ristenpart. "Partitioning Oracle Attacks". In: *Proceedings of the 30th USENIX Security Symposium (USENIX Security 21, August 11–13, 2021)*. USENIX Association, August 2021, pages 195–212. ISBN: 978-1-939133-24-3. URL: https://www.usenix.org/conference/usenixsecurity21/presentation/len (visited on 2021-10-12) (↑ p108, 109, 116).

[librustzcash-109] Jack Grigg. *librustzcash PR 109: PaymentAddress encapsulation*. URL: https://github.com/zcash/librustzcash/pull/109 (visited on 2023-08-25) (↑ p54, 120).

[libsodium] *libsodium documentation*. URL: https://libsodium.org/ (visited on 2020-03-02) (↑ p69).

[libsodium-Seal] *Sealed boxes — libsodium*. URL: https://download.libsodium.org/doc/public-key_cryptography/sealed_boxes.html (visited on 2016-02-01) (↑ p107).

[LM2017] Philip Lafrance and Alfred Menezes. *On the security of the WOTS-PRF signature scheme*. Cryptology ePrint Archive: Report 2017/938. Last revised February 5, 2018. URL: https://eprint.iacr.org/2017/938 (visited on 2018-04-16) (↑ p25).

[MÁEÁ2010] V. Gayoso Martínez, F. Hernández Álvarez, L. Hernández Encinas, and C. Sánchez Ávila. "A Comparison of the Standardized Versions of ECIES". In: *Proceedings of Sixth International Conference on Information Assurance and Security (Atlanta, Georgia, USA, August 23–25, 2010)*. IEEE, 2010, pages 1–4. ISBN: 978-1-4244-7407-3. DOI: 10.1109/ISIAS.2010.5604194. URL: https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.819.9345 (visited on 2021-04-08) (↑ p108).

[Maller2018] Mary Maller. *A Proof of Security for the Sapling Generation of zk-SNARK Parameters in the Generic Group Model*. November 16, 2018. URL: https://github.com/zcash/sapling-security-analysis/blob/master/MaryMallerUpdated.pdf (visited on 2018-02-10) (↑ p81, 125).

[ISO2015] ISO/IEC. *International Standard ISO/IEC 18004:2015(E): Information Technology – Automatic identification and data capture techniques – QR Code bar code symbology specification*. Third edition. February 1, 2015. URL: https://raw.githubusercontent.com/yansikeim/QR-Code/master/ISO%20IEC%2018004%202015%20Standard.pdf (visited on 2021-03-22) (↑ p82).

[Nakamoto2008] Satoshi Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System*. October 31, 2008. URL: https://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.221.9986 (visited on 2022-06-17) (↑ p7, 115).

[NIST2015] NIST. *FIPS 180-4: Secure Hash Standard (SHS)*. August 2015. DOI: 10.6028/NIST.FIPS.180-4. URL: https://csrc.nist.gov/publications/detail/fips/180/4/final (visited on 2021-03-08) (↑ p58, 59, 83).

[Parno2015] Bryan Parno. *A Note on the Unsoundness of vnTinyRAM's SNARK*. Cryptology ePrint Archive: Report 2015/437. Received May 6, 2015. URL: https://eprint.iacr.org/2015/437 (visited on 2019-02-08) (↑ p80, 110, 125).

[Peterson2017] Paige Peterson. *Transaction Linkability*. Electric Coin Company blog. January 25, 2017. URL: https://electriccoin.co/blog/transaction-linkability/ (visited on 2019-08-27) (↑ p9, 130).

[PHGR2013] Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. *Pinocchio: Nearly Practical Verifiable Computation*. Cryptology ePrint Archive: Report 2013/279. Last revised May 13, 2013. URL: https://eprint.iacr.org/2013/279 (visited on 2016-08-31) (↑ p79).

[Quesnelle2017] Jeffrey Quesnelle. *On the linkability of Zcash transactions*. arXiv:1712.01210 [cs.CR]. December 4, 2017. URL: https://arxiv.org/abs/1712.01210 (visited on 2018-04-15) (↑ p9, 130).

[RFC-2119] Scott Bradner. *Request for Comments 7693: Key words for use in RFCs to Indicate Requirement Levels*. Internet Engineering Task Force (IETF). March 1997. URL: https://www.rfc-editor.org/rfc/rfc2119.html (visited on 2016-09-14) (↑ p7).

[RFC-7539] Yoav Nir and Adam Langley. *Request for Comments 7539: ChaCha20 and Poly1305 for IETF Protocols*. Internet Research Task Force (IRTF). May 2015. URL: https://www.rfc-editor.org/rfc/rfc7539.html (visited on 2016-09-02). As modified by verified errata at https://www.rfc-editor.org/errata_search.php?rfc=7539 (visited on 2016-09-02). (↑ p66).

[RFC-8032] Simon Josefsson and Ilari Liusvaara. *Request for Comments 8032: Edwards-Curve Digital Signature Algorithm (EdDSA)*. Internet Engineering Task Force (IETF). January 2017. URL: https://www.rfc-editor.org/rfc/rfc8032.html (visited on 2020-07-06). As corrected by errata at https://www.rfc-editor.org/errata_search.php?rfc=8032 (visited on 2020-07-06). (↑ p68).

[RIPEMD160] Hans Dobbertin, Antoon Bosselaers, and Bart Preneel. *RIPEMD-160, a strengthened version of RIPEMD*. URL: `https://homes.esat.kuleuven.be/~bosselae/ripemd160.html` (visited on 2021-04-05) (↑ p83).

[ST1999] Tomas Sander and Amnon Ta-Shma. "Auditable, Anonymous Electronic Cash". In: *Advances in Cryptology - CRYPTO '99. Proceedings of the 19th Annual International Cryptology Conference (Santa Barbara, California, USA, August 15–19, 1999)*. Ed. by Michael Wiener. Vol. 1666. Lecture Notes in Computer Science. Springer, 1999, pages 555–572. ISBN: 978-3-540-66347-8. DOI: 10.1007/3-540-48405-1_35. URL: `https://link.springer.com/content/pdf/10.1007/3-540-48405-1_35.pdf` (visited on 2018-06-05) (↑ p111, 128).

[SVPBABW2012] Srinath Setty, Victor Vu, Nikhil Panpalia, Benjamin Braun, Muqeet Ali, Andrew J. Blumberg, and Michael Walfish. *Taking proof-based verified computation a few steps closer to practicality (extended version)*. Cryptology ePrint Archive: Report 2012/598. Last revised February 28, 2013. URL: `https://eprint.iacr.org/2012/598` (visited on 2018-04-25) (↑ p151).

[SWB2019] Josh Swihart, Benjamin Winston, and Sean Bowe. *Zcash Counterfeiting Vulnerability Successfully Remediated*. February 5, 2019. URL: `https://electriccoin.co/blog/zcash-counterfeiting-vulnerability-successfully-remediated/` (visited on 2019-08-27) (↑ p80, 125).

[Swihart2018] Josh Swihart. *Overwinter Activated Successfully*. Electric Coin Company blog. June 26, 2018. URL: `https://electriccoin.co/blog/overwinter-activated-successfully/` (visited on 2021-01-10) (↑ p88).

[vanSaberh2014] Nicolas van Saberhagen. *CryptoNote v 2.0*. Date disputed. URL: `https://bytecoin.org/old/whitepaper.pdf` (visited on 2021-04-07) (↑ p9).

[Vercauter2009] Frederik Vercauteren. *Optimal pairings*. Cryptology ePrint Archive: Report 2008/096. Last revised March 7, 2008. URL: `https://eprint.iacr.org/2008/096` (visited on 2018-04-06). A version of this paper appeared in *IEEE Transactions of Information Theory*, Vol. 56, pages 455–461; IEEE, 2009. (↑ p74, 130).

[WCBTV2015] Zooko Wilcox, Alessandro Chiesa, Eli Ben-Sasson, Eran Tromer, and Madars Virza. *A Bug in libsnark*. Least Authority blog. May 16, 2015. URL: `https://leastauthority.com/blog/a-bug-in-libsnark/` (visited on 2021-04-07) (↑ p80, 149).

[WG2016] Zooko Wilcox and Jack Grigg. *Why Equihash?* Electric Coin Company blog. April 15, 2016. URL: `https://electriccoin.co/blog/why-equihash/` (visited on 2019-08-27). Updated August 21, 2019. (↑ p97).

[Zaverucha2012] Gregory M. Zaverucha. *Hybrid Encryption in the Multi-User Setting*. Cryptology ePrint Archive: Report 2012/159. Received March 20, 2012. URL: `https://eprint.iacr.org/2012/159` (visited on 2016-09-24) (↑ p108).

[Zcash-Issue2113] Simon Liu. *GitHub repository 'zcash/zcash': Issue 2113 – Upgrade testnet to fix bug in test and update fr addresses*. URL: `https://github.com/zcash/zcash/issues/2113` (visited on 2017-02-20) (↑ p102, 134).

[Zcash-libsnark] *libsnark: C++ library for zkSNARK proofs (Zcash fork)*. URL: `https://github.com/zcash/zcash/tree/v2.0.7-3/src/snark` (visited on 2021-04-07) (↑ p79).

[zcashd-6459] Jack Grigg and Daira-Emma Hopwood. *zcashd PR 6459: Migrate to zcash_primitives 0.10*. URL: `https://github.com/zcash/zcash/pull/6459` (visited on 2023-08-25) (↑ p54, 120).

[zcashd-6725] Jack Grigg. *zcashd PR 6725: Retroactively use Rust to decrypt shielded coinbase before soft fork*. URL: `https://github.com/zcash/zcash/pull/6725` (visited on 2023-08-25) (↑ p54, 120).

[ZIP-32] Jack Grigg and Daira-Emma Hopwood. *Shielded Hierarchical Deterministic Wallets*. Zcash Improvement Proposal 32. URL: `https://zips.z.cash/zip-0032` (visited on 2019-08-28) (↑ p12, 23, 33, 37, 52, 62, 70, 113, 116, 118, 122, 123, 129).

[ZIP-76] Jack Grigg and Daira-Emma Hopwood. *Transaction Signature Validation before Overwinter*. Zcash Improvement Proposal 76 (in progress). (↑ p41, 103).

[ZIP-143] Jack Grigg and Daira-Emma Hopwood. *Transaction Signature Validation for Overwinter*. Zcash Improvement Proposal 143. Created December 27, 2017. URL: `https://zips.z.cash/zip-0143` (visited on 2019-08-28) (↑ p41, 59, 88).

[ZIP-173] Daira-Emma Hopwood. *Bech32 Format*. Zcash Improvement Proposal 173. Created June 13, 2018. URL: `https://zips.z.cash/zip-0173` (visited on 2020-06-01) (↑ p82, 85, 122).

[ZIP-200] Jack Grigg. *Network Upgrade Mechanism*. Zcash Improvement Proposal 200. Created January 8, 2018. URL: `https://zips.z.cash/zip-0200` (visited on 2019-08-28) (↑ p88, 92, 115).

[ZIP-201] Simon Liu and Daira-Emma Hopwood. *Network Peer Management for Overwinter*. Zcash Improvement Proposal 201. Created January 15, 2018. URL: `https://zips.z.cash/zip-0201` (visited on 2019-08-28) (↑ p41, 88).

[ZIP-202] Simon Liu and Daira-Emma Hopwood. *Version 3 Transaction Format for Overwinter*. Zcash Improvement Proposal 202. Created January 10, 2018. URL: `https://zips.z.cash/zip-0202` (visited on 2019-08-28) (↑ p88).

[ZIP-203] Jay Graber and Daira-Emma Hopwood. *Transaction Expiry*. Zcash Improvement Proposal 203. Created January 9, 2018. URL: `https://zips.z.cash/zip-0203` (visited on 2019-08-28) (↑ p88, 89).

[ZIP-205] Simon Liu and Daira-Emma Hopwood. *Deployment of the Sapling Network Upgrade*. Zcash Improvement Proposal 205. Created October 8, 2018. URL: `https://zips.z.cash/zip-0205` (visited on 2019-08-28) (↑ p41, 88, 99).

[ZIP-207] Jack Grigg and Daira-Emma Hopwood. *Funding Streams*. Zcash Improvement Proposal 207. Created January 4, 2019. URL: `https://zips.z.cash/zip-0207` (visited on 2019-08-28) (↑ p112).

[ZIP-209] Sean Bowe and Daira-Emma Hopwood. *Prohibit Out-of-Range Chain Value Pool Balances*. Zcash Improvement Proposal 209. Created February 25, 2019. URL: `https://zips.z.cash/zip-0209` (visited on 2020-07-06) (↑ p46, 120).

[ZIP-212] Sean Bowe. *Allow Recipient to Derive Sapling Ephemeral Secret from Note Plaintext*. Zcash Improvement Proposal 212. Created March 31, 2019. URL: `https://zips.z.cash/zip-0212` (visited on 2020-06-01) (↑ p117).

[ZIP-216] Jack Grigg and Daira-Emma Hopwood. *Require Canonical Point Encodings*. Zcash Improvement Proposal 216. Created February 11, 2021. URL: `https://zips.z.cash/zip-0216` (visited on 2021-02-25) (↑ p85, 120).

[ZIP-243] Jack Grigg and Daira-Emma Hopwood. *Transaction Signature Validation for Sapling*. Zcash Improvement Proposal 243. Created April 10, 2018. URL: `https://zips.z.cash/zip-0243` (visited on 2019-08-28) (↑ p41, 43, 45, 59, 88).

[ZIP-256] Daira-Emma Hopwood. *Deployment of Consensus Bug Fixes Between NU6.1 and NU6.2*. Zcash Improvement Proposal 256. Created June 9, 2026. URL: `https://zips.z.cash/zip-0256` (visited on 2026-06-16) (↑ p112).

[ZIP-302] Jay Graber and Jack Grigg. *Standardized Memo Field Format*. Zcash Improvement Proposal 302. Created February 8, 2017. URL: `https://zips.z.cash/zip-0302` (visited on 2022-06-22) (↑ p14, 115, 118, 122).

[ZIP-316] Daira-Emma Hopwood, Nathan Wilcox, Taylor Hornby, Jack Grigg, Sean Bowe, Kris Nuttycombe, Greg Pfeil, and Ying Tong Lai. *Unified Addresses and Unified Viewing Keys*. Zcash Improvement Proposal 316. Created April 7, 2021. URL: `https://zips.z.cash/zip-0316` (visited on 2021-04-29) (↑ p23, 86, 113, 116).

[Zips-Issue664] Daira-Emma Hopwood. *GitHub repository 'zcash/zips': Issue 664 – [protocol spec] [ZIP 216] Sapling pk_d should not allow the zero point*. URL: `https://github.com/zcash/zips/issues/664` (visited on 2025-09-07) (↑ p113, 120).

Appendices

A Circuit Design

A.1 Quadratic Constraint Programs

Sapling defines two circuits, Spend and Output, each implementing an abstract *statement* described in § 4.16.2 *'Spend Statement (Sapling)'* on p. 48 and § 4.16.3 *'Output Statement (Sapling)'* on p. 49 respectively. It also adds a Groth16 circuit for the *JoinSplit statement* described in § 4.16.1 *'JoinSplit Statement (Sprout)'* on p. 47.

At the next lower level, each circuit is defined in terms of a *quadratic constraint program* (specifying a *Rank 1 Constraint System*), as detailed in this section. In the BCTV14 or Groth16 proving systems, this program is translated to a *Quadratic Arithmetic Program* [BCTV2014a, section 2.3] [WCBTV2015]. The circuit descriptions given here are necessary to compute witness elements for each circuit, as well as the proving and verifying keys.

Let $\mathbb{F}_{r_\mathbb{S}}$ be the finite field over which Jubjub is defined, as given in § 5.4.8.3 *'Jubjub'* on p. 77.

A *quadratic constraint program* consists of a set of constraints over variables in $\mathbb{F}_{r_\mathbb{S}}$, each of the form:

$$(A) \times (B) = (C)$$

where (A), (B), and (C) are *linear combinations* of variables and constants in $\mathbb{F}_{r_\mathbb{S}}$.

Here $\times$ and $\cdot$ both represent multiplication in the field $\mathbb{F}_{r_\mathbb{S}}$, but we use $\times$ for multiplications corresponding to gates of the circuit, and $\cdot$ for multiplications by constants in the terms of a *linear combination*. $\times$ should not be confused with $\times$ which is defined as cartesian product in § 2 *'Notation'* on p. 9.

A.2 Elliptic curve background

The Sapling circuits make use of a *complete twisted Edwards elliptic curve* ("*ctEdwards curve*") Jubjub, defined in § 5.4.8.3 *'Jubjub'* on p. 77, and also a *Montgomery elliptic curve* $\mathbb{M}$ that is birationally equivalent to Jubjub. Following the notation in [BL2017] we use (u, v) for affine coordinates on the *ctEdwards curve*, and (x, y) for affine coordinates on the *Montgomery curve*.

A point P is normally represented by two $\mathbb{F}_{r_\mathbb{S}}$ variables, which we name as (P^u, P^v) for an *affine-ctEdwards* point, for instance.

The implementations of scalar multiplication require the scalar to be represented as a *bit sequence*. We therefore allow the notation $[k\star]\,P$ meaning $[\mathsf{LEBS2IP}_{\mathsf{length}(k\star)}(k\star)]\,P$. There will be no ambiguity because variables representing *bit sequences* are named with a $\star$ suffix.

The *Montgomery curve* $\mathbb{M}$ has parameters $A_\mathbb{M} = 40962$ and $B_\mathbb{M} = 1$. We use an affine representation of this curve with the formula:

$$B_\mathbb{M} \cdot y^2 = x^3 + A_\mathbb{M} \cdot x^2 + x$$

Usually, elliptic curve arithmetic over prime fields is implemented using some form of projective coordinates, in order to reduce the number of expensive inversions required. In the circuit, it turns out that a division can be implemented at the same cost as a multiplication, i.e. one constraint. Therefore it is beneficial to use affine coordinates for both curves.

We define the following types representing *affine-ctEdwards* and *affine-Montgomery* coordinates respectively:

$$\mathsf{AffineCtEdwardsJubjub} := (u : \mathbb{F}_{r_\mathbb{S}}) \times (v : \mathbb{F}_{r_\mathbb{S}}) : a_\mathbb{J} \cdot u^2 + v^2 = 1 + d_\mathbb{J} \cdot u^2 \cdot v^2$$

$$\mathsf{AffineMontJubjub} := (x : \mathbb{F}_{r_\mathbb{S}}) \times (y : \mathbb{F}_{r_\mathbb{S}}) : B_\mathbb{M} \cdot y^2 = x^3 + A_\mathbb{M} \cdot x^2 + x$$

We also define a type representing compressed, ***not necessarily valid***, ctEdwards coordinates:

$\mathsf{CompressedCtEdwardsJubjub} := (\tilde{u} : \mathbb{B}) \times (v : \mathbb{F}_{r_{\mathbb{S}}})$

See § 5.4.8.3 *'Jubjub'* on p. 77 for how this type is represented as a *byte sequence* in external encodings.

We use *affine-Montgomery* arithmetic in parts of the circuit because it is more efficient, in terms of the number of constraints, than *affine-ctEdwards* arithmetic.

An important consideration when using Montgomery arithmetic is that the addition formula is not complete, that is, there are cases where it produces the wrong answer. We must ensure that these cases do not arise.

We will need the theorem below about y-coordinates of points on *Montgomery curves*.

Fact: ${A_{\mathbb{M}}}^2 - 4$ is a nonsquare in $\mathbb{F}_{r_{\mathbb{S}}}$.

Theorem A.2.1. *$(0, 0)$ is the only point with $y = 0$ on certain Montgomery curves.*

Let $P = (x, y)$ be a point other than $(0, 0)$ on a Montgomery curve $E_{\mathsf{Mont}(A,B)}$ over $\mathbb{F}_r$, such that $A^2 - 4$ is a nonsquare in $\mathbb{F}_r$. Then $y \neq 0$.

Proof. Substituting $y = 0$ into the *Montgomery curve* equation gives $0 = x^3 + A \cdot x^2 + x = x \cdot (x^2 + A \cdot x + 1)$. So either $x = 0$ or $x^2 + A \cdot x + 1 = 0$. Since $P \neq (0, 0)$, the case $x = 0$ is excluded. In the other case, complete the square for $x^2 + A \cdot x + 1 = 0$ to give the equivalent $(2 \cdot x + A)^2 = A^2 - 4$. The left-hand side is a square, so if the right-hand side is a nonsquare, then there are no solutions for x. □

A.3 Circuit Components

Each of the following sections describes how to implement a particular component of the circuit, and counts the number of constraints required. Some components make use of others; the order of presentation is "bottom-up".

It is important for security to ensure that variables intended to be of boolean type are boolean-constrained; and for efficiency that they are boolean-constrained only once. We explicitly state for the boolean inputs and outputs of each component whether they are boolean-constrained by the component, or are assumed to have been boolean-constrained separately.

Affine coordinates for elliptic curve points are assumed to represent points on the relevant curve, unless otherwise specified.

In this section, variables have type $\mathbb{F}_{r_{\mathbb{S}}}$ unless otherwise specified. In contrast to most of this document, we use zero-based indexing in order to more closely match the implementation.

A.3.1 Operations on individual bits

A.3.1.1 Boolean constraints

A boolean constraint $b \in \mathbb{B}$ can be implemented as:

$(1 - b) \; \mathsf{X} \; (b) \; = \; (0)$

A.3.1.2 Conditional equality

The constraint "either $a = 0$ or $b = c$" can be implemented as:

$(a) \; \mathsf{X} \; (b - c) \; = \; (0)$

A.3.1.3 Selection constraints

A selection constraint $(b\ ?\ x : y) = z$, where $b : \mathbb{B}$ has been boolean-constrained, can be implemented as:

$$(b)\ \mathsf{X}\ (y - x) = (y - z)$$

A.3.1.4 Nonzero constraints

Since only nonzero elements of $\mathbb{F}_{r_\mathbb{S}}$ have a multiplicative inverse, the assertion $a \neq 0$ can be implemented by witnessing the inverse, $a_{\mathsf{inv}} = a^{-1} \pmod{r_\mathbb{S}}$:

$$(a_{\mathsf{inv}})\ \mathsf{X}\ (a) = (1)$$

This technique comes from [SVPBABW2012, Appendix D.1].

Non-normative note: A global optimization allows to use a single inverse computation outside the circuit for any number of nonzero constraints. Suppose that we have n variables (or *linear combinations*) that are supposed to be nonzero: $a_{0\,..\,n-1}$. Multiply these together (using $n-1$ constraints) to give $a^* = \prod_{i=0}^{n-1} a_i$; then, constrain a^* to be nonzero. This works because the product a^* is nonzero if and only if all of $a_{0\,..\,n-1}$ are nonzero. However, the Sapling circuit does not use this optimization.

A.3.1.5 Exclusive-or constraints

An exclusive-or operation $a \oplus b = c$, where $a, b : \mathbb{B}$ are already boolean-constrained, can be implemented in one constraint as:

$$(2 \cdot a)\ \mathsf{X}\ (b) = (a + b - c)$$

This automatically boolean-constrains c. Its correctness can be seen by checking the truth table of (a, b).

A.3.2 Operations on multiple bits

A.3.2.1 [Un]packing modulo $r_\mathbb{S}$

Let $n : \mathbb{N}^+$ be a constant. The operation of converting a field element, $a : \mathbb{F}_{r_\mathbb{S}}$, to a sequence of boolean variables $b_{0\,..\,n-1} : \mathbb{B}^{[n]}$ such that $a = \sum_{i=0}^{n-1} b_i \cdot 2^i \pmod{r_\mathbb{S}}$, is called *"unpacking"*. The inverse operation is called *"packing"*.

In the *quadratic constraint program* these are the same operation (but see the note about canonical representation below). We assume that the variables $b_{0\,..\,n-1}$ are boolean-constrained separately.

We have $a \bmod r_\mathbb{S} = \left(\sum_{i=0}^{n-1} b_i \cdot 2^i\right) \bmod r_\mathbb{S} = \left(\sum_{i=0}^{n-1} b_i \cdot (2^i \bmod r_\mathbb{S})\right) \bmod r_\mathbb{S}$.

This can be implemented in one constraint:

$$\left(\sum_{i=0}^{n-1} b_i \cdot (2^i \bmod r_\mathbb{S})\right)\ \mathsf{X}\ (1) = (a)$$

Notes:

- The bit length n is not limited by the field element size.
- Since the constraint has only a trivial multiplication, it is possible to eliminate it by merging it into the boolean constraint of one of the output bits, expressing that bit as a linear combination of the others and a. However, this optimization requires substitutions that would interfere with the modularity of the circuit implementation (for a saving of only one constraint per unpacking operation), and so we do not use it for the Sapling circuit.
- In the case $n = 255$, for $a < 2^{255} - r_{\mathbb{S}}$ there are two possible representations of $a : \mathbb{F}_{r_{\mathbb{S}}}$ as a sequence of 255 bits, corresponding to $\mathsf{I2LEBSP}_{255}(a)$ and $\mathsf{I2LEBSP}_{255}(a + r_{\mathbb{S}})$. This is a potential hazard, but it may or may not be necessary to force use of the canonical representation $\mathsf{I2LEBSP}_{255}(a)$, depending on the context in which the [un]packing operation is used. We therefore do not consider this to be part of the [un]packing operation itself.

A.3.2.2 Range check

Let $n : \mathbb{N}^+$ be a constant, and let $a = \sum_{i=0}^{n-1} a_i \cdot 2^i : \mathbb{N}$. Suppose we want to constrain $a \leq c$ for some ***constant*** $c = \sum_{i=0}^{n-1} c_i \cdot 2^i : \mathbb{N}$.

Without loss of generality we can assume that $c_{n-1} = 1$, because if it were not then we would decrease n accordingly.

Note that since a and c are provided in binary representation, their bit length n is not limited by the field element size. We ***do not*** assume that the bits $a_{0\,..\,n-1}$ are already boolean-constrained.

Define $\Pi_m = \prod_{i=m}^{n-1} (c_i = 0 \vee a_i = 1)$ for $m \in \{0\,..\,n-1\}$. Notice that for any $m < n - 1$ such that $c_m = 0$, we have $\Pi_m = \Pi_{m+1}$, and so it is only necessary to allocate separate variables for the Π_m such that $m < n - 1$ and $c_m = 1$. Furthermore if $c_{n-2\,..\,0}$ has $t > 0$ trailing 1 bits, then we do not need to allocate variables for $\Pi_{0\,..\,t-1}$ because those variables will not be used below.

More explicitly:

Let $\Pi_{n-1} = a_{n-1}$.

For i from $n - 2$ down to t,

- if $c_i = 0$, then let $\Pi_i = \Pi_{i+1}$;
- if $c_i = 1$, then constrain $(\Pi_{i+1})\ \mathsf{X}\ (a_i) = (\Pi_i)$.

Then we constrain the a_i as follows:

For i from $n - 1$ down to 0,

- if $c_i = 0$, constrain $(1 - \Pi_{i+1} - a_i)\ \mathsf{X}\ (a_i) = (0)$;
- if $c_i = 1$, boolean-constrain a_i as in § A.3.1.1 ***'Boolean constraints'*** on p. 150.

Note that the constraints corresponding to zero bits of c are ***in place of*** boolean constraints on bits of a_i.

This costs $n + k$ constraints, where k is the number of non-trailing 1 bits in $c_{n-2\,..\,0}$.

Theorem A.3.1. *Correctness of a constraint system for range checks.*

Assume $c_{0\,..\,n-1} : \mathbb{B}^{[n]}$ *and* $c_{n-1} = 1$. *Define* $A_m := \sum_{i=m}^{n-1} a_i \cdot 2^i$ *and* $C_m := \sum_{i=m}^{n-1} c_i \cdot 2^i$. *For any* $m \in \{0\,..\,n-1\}$, $A_m \leq C_m$ *if and only if the restriction of the above constraint system to* $i \in \{m\,..\,n-1\}$ *is satisfied. Furthermore the system at least boolean-constrains* $a_{0\,..\,n-1}$.

Proof. For $i \in \{0\,..\,n-1\}$ such that $c_i = 1$, the corresponding a_i are unconditionally boolean-constrained. This implies that the system constrains $\Pi_i \in \mathbb{B}$ for all $i \in \{0\,..\,n-1\}$. For $i \in \{0\,..\,n-1\}$ such that $c_i = 0$, the constraint $\left(1 - \Pi_{i+1} - a_i\right) \mathbin{\mathsf{X}} \left(a_i\right) = \left(0\right)$ constrains a_i to be 0 if $\Pi_{i+1} = 1$, otherwise it constrains $a_i \in \mathbb{B}$. So all of $a_{0\,..\,n-1}$ are at least boolean-constrained.

To prove the rest of the theorem we proceed by induction on decreasing m, i.e. taking successively longer prefixes of the *big-endian* binary representations of a and c.

Base case $m = n - 1$: since $c_{n-1} = 1$, the constraint system has just one boolean constraint on a_{n-1}, which fulfils the theorem since $A_{n-1} \leq C_{n-1}$ is always satisfied.

Inductive case $m < n - 1$:

- If $A_{m+1} > C_{m+1}$, then by the inductive hypothesis the constraint system must fail, which fulfils the theorem regardless of the value of a_m.
- If $A_{m+1} \leq C_{m+1}$, then by the inductive hypothesis the constraint system restricted to $i \in \{m+1\,..\,n-1\}$ succeeds. We have $\Pi_{m+1} = \prod_{i=m+1}^{n-1} (c_i = 0 \vee a_i = 1) = \prod_{i=m+1}^{n-1} (a_i \geq c_i)$.
  - If $A_{m+1} = C_{m+1}$, then $a_i = c_i$ for all $i \in \{m+1\,..\,n-1\}$ and so $\Pi_{m+1} = 1$. Also $A_m \leq C_m$ if and only if $a_m \leq c_m$.
    When $c_m = 1$, only a boolean constraint is added for a_m which fulfils the theorem.
    When $c_m = 0$, a_m is constrained to be 0 which fulfils the theorem.
  - If $A_{m+1} < C_{m+1}$, then it cannot be the case that $a_i \geq c_i$ for all $i \in \{m+1\,..\,n-1\}$, so $\Pi_{m+1} = 0$.
    This implies that the constraint on a_m is always equivalent to a boolean constraint, which fulfils the theorem because $A_m \leq C_m$ must be true regardless of the value of a_m.

This covers all cases. □

Correctness of the full constraint system follows by taking $m = 0$ in the above theorem.

The algorithm in § A.3.3.2 ***'ctEdwards [de]compression and validation'*** on p. 154 uses range checks with $c = r_{\mathbb{S}} - 1$ to validate *ctEdwards compressed encodings*. In that case $n = 255$ and $k = 132$, so the cost of each such range check is 387 constraints.

Non-normative note: It is possible to optimize the computation of $\Pi_{t\,..\,n-2}$ further. Notice that Π_m is only used when m is the index of the last bit of a run of 1 bits in c. So for each such run of 1 bits $c_{m\,..\,m+N-2}$ of length $N - 1$, it is sufficient to compute an N-ary AND of $a_{m\,..\,m+N-2}$ and Π_{m+N-1}: $R = \prod_{i=0}^{N-1} X_i$. This can be computed in 3 constraints for any N; boolean-constrain the output R, and then add constraints

$\left(N - \sum_{i=0}^{N-1} X_i\right) \mathbin{\mathsf{X}} \left(\mathsf{inv}\right) = \left(1 - R\right)$ to enforce that $\sum_{i=0}^{N-1} X_i \neq N$ when $R = 0$;

$\left(N - \sum_{i=0}^{N-1} X_i\right) \mathbin{\mathsf{X}} \left(R\right) = \left(0\right)$ to enforce that $\sum_{i=0}^{N-1} X_i = N$ when $R = 1$.

where inv is witnessed as $\left(N - \sum_{i=0}^{N-1} X_i\right)^{-1}$ if $R = 0$ or is unconstrained otherwise. (Since $N < r_{\mathbb{S}}$, the sums cannot overflow.)

In fact the last constraint is not needed in this context because it is sufficient to compute an upper bound on each Π_m (i.e. it does not benefit a malicious prover to witness $R = 1$ when the result of the AND should be 0). So the cost of computing Π variables for an arbitrarily long run of 1 bits can be reduced to 2 constraints. For example, for $c = r_{\mathbb{S}} - 1$ the overall cost would be reduced to $255 + 68 = 323$ constraints.

These optimizations are not used in Sapling.

A.3.3 Elliptic curve operations

A.3.3.1 Checking that Affine-ctEdwards coordinates are on the curve

To check that (u, v) is a point on the *ctEdwards curve*, the Sapling circuit uses 4 constraints:

$$(u) \mathbin{\mathsf{X}} (u) = (uu)$$
$$(v) \mathbin{\mathsf{X}} (v) = (vv)$$
$$(uu) \mathbin{\mathsf{X}} (vv) = (uuvv)$$
$$(a_{\mathbb{J}} \cdot uu + vv) \mathbin{\mathsf{X}} (1) = (1 + d_{\mathbb{J}} \cdot uuvv)$$

Non-normative note: The last two constraints can be combined into $(d_{\mathbb{J}} \cdot uu) \mathbin{\mathsf{X}} (vv) = (a_{\mathbb{J}} \cdot uu + vv - 1)$. The Sapling circuit does not use this optimization.

A.3.3.2 ctEdwards [de]compression and validation

Define DecompressValidate : CompressedCtEdwardsJubjub $\to$ AffineCtEdwardsJubjub as follows:

DecompressValidate($\tilde{u}, v$) :

// Prover supplies the u-coordinate.
Let $u : \mathbb{F}_{r_{\mathbb{S}}}$.

// § A.3.3.1 ***'Checking that Affine-ctEdwards coordinates are on the curve'*** on p. 154.
Check that (u, v) is a point on the *ctEdwards curve*.

// § A.3.2.1 ***'[Un]packing modulo $r_{\mathbb{S}}$'*** on p. 151.
Unpack u to $\sum_{i=0}^{254} u_i \cdot 2^i$, equating $\tilde{u}$ with u_0.

// § A.3.2.2 ***'Range check'*** on p. 152.
Check that $\sum_{i=0}^{254} u_i \cdot 2^i \leq r_{\mathbb{S}} - 1$.

Return (u, v).

This costs 4 constraints for the curve equation check, 1 constraint for the unpacking, and 387 constraints for the range check (as computed in § A.3.2.2 ***'Range check'*** on p. 152) for a total of 392 constraints. The cost of the range check includes boolean-constraining $u_{0\,..\,254}$.

The same *quadratic constraint program* is used for compression and decompression.

Non-normative note: The point-on-curve check could be omitted if (u, v) were already known to be on the curve. However, the Sapling circuit never omits it; this provides a consistency check on the elliptic curve arithmetic.

A.3.3.3 ctEdwards $\leftrightarrow$ Montgomery conversion

Define the notation $\sqrt[+]{\bullet}$ as in § 2 ***'Notation'*** on p. 9.

Define CtEdwardsToMont : AffineCtEdwardsJubjub $\to$ AffineMontJubjub as follows:

$$\mathsf{CtEdwardsToMont}(u, v) = \left(\frac{1+v}{1-v},\ \sqrt[+]{-40964} \cdot \frac{1+v}{(1-v) \cdot u}\right) \qquad [1 - v \neq 0 \text{ and } u \neq 0]$$

Define MontToCtEdwards : AffineMontJubjub $\to$ AffineCtEdwardsJubjub as follows:

$$\mathsf{MontToCtEdwards}(x, y) = \left(\sqrt[+]{-40964} \cdot \frac{x}{y},\ \frac{x-1}{x+1}\right) \qquad [x + 1 \neq 0 \text{ and } y \neq 0]$$

Either of these conversions can be implemented by the same *quadratic constraint program*:

$$(y) \mathbin{\mathsf{X}} (u) = \left(\sqrt[+]{-40964} \cdot x\right)$$
$$(x+1) \mathbin{\mathsf{X}} (v) = (x-1)$$

The above conversions should only be used if the input is guaranteed to be a point on the relevant curve. If that is the case, the theorems below enumerate all exceptional inputs that may violate the side-conditions.

Theorem A.3.2. *Exceptional points (ctEdwards → Montgomery).*

Let (u, v) be an affine point on a ctEdwards curve $E_{\mathsf{ctEdwards}(a,d)}$. Then the only points with $u = 0$ or $1 - v = 0$ are $(0, 1) = \mathcal{O}_{\mathbb{J}}$, and $(0, -1)$ of order 2.

Proof. The curve equation is $a \cdot u^2 + v^2 = 1 + d \cdot u^2 \cdot v^2$ with $a \neq d$ (see [BBJLP2008, Definition 2.1]). By substituting $u = 0$ we obtain $v = \pm 1$, and by substituting $v = 1$ and using $a \neq d$ we obtain $u = 0$. □

Theorem A.3.3. *Exceptional points (Montgomery → ctEdwards).*

Let (x, y) be an affine point on a Montgomery curve $E_{\mathsf{Mont}(A,B)}$ over $\mathbb{F}_r$ with parameters A and B such that $A^2 - 4$ is a nonsquare in $\mathbb{F}_r$, that is birationally equivalent to a ctEdwards curve. Then $x + 1 \neq 0$, and the only point (x, y) with $y = 0$ is $(0, 0)$ of order 2.

Proof. That the only point with $y = 0$ is $(0, 0)$ is proven by Theorem A.2.1 on p. 150.

If $x+1 = 0$, then subtituting $x = -1$ into the *Montgomery curve* equation gives $B \cdot y^2 = x^3 + A \cdot x^2 + x = A - 2$. So in that case $y^2 = (A-2)/B$. The right-hand-side is equal to the parameter d of a particular *ctEdwards curve* birationally equivalent to the *Montgomery curve* (see [BL2017, section 4.3.5]). For all *ctEdwards curves*, d is nonsquare, so this equation has no solutions for y, hence $x + 1 \neq 0$. □

(When the theorem is applied with $E_{\mathsf{Mont}(A,B)} = \mathbb{M}$ defined in § A.2 ***'Elliptic curve background'*** on p. 149, the *ctEdwards curve* referred to in the proof is an isomorphic rescaling of the *Jubjub curve*.)

A.3.3.4 Affine-Montgomery arithmetic

The incomplete *affine-Montgomery* addition formulae given in [BL2017, section 4.3.2] are:

$$x_3 = B_{\mathbb{M}} \cdot \lambda^2 - A_{\mathbb{M}} - x_1 - x_2$$
$$y_3 = (x_1 - x_3) \cdot \lambda - y_1$$
$$\text{where } \lambda = \begin{cases} \frac{3 \cdot x_1^2 + 2 \cdot A_{\mathbb{M}} \cdot x_1 + 1}{2 \cdot B_{\mathbb{M}} \cdot y_1}, & \text{if } x_1 = x_2 \\ \frac{y_2 - y_1}{x_2 - x_1}, & \text{otherwise.} \end{cases}$$

The following theorem helps to determine when these incomplete addition formulae can be safely used:

Theorem A.3.4. *Distinct-x theorem.*

Let Q be a point of odd-prime order s on a Montgomery curve $\mathbb{M} = E_{\mathsf{Mont}(A_{\mathbb{M}}, B_{\mathbb{M}})}$ over $\mathbb{F}_{r_{\mathbb{S}}}$. Let $k_{1\,..\,2}$ be integers in $\left\{-\frac{s-1}{2} \,..\, \frac{s-1}{2}\right\} \setminus \{0\}$. Let $P_i = [k_i]\,Q = (x_i, y_i)$ for $i \in \{1\,..\,2\}$, with $k_2 \neq \pm k_1$. Then the non-unified addition constraints

$$(x_2 - x_1) \mathbin{\mathsf{X}} (\lambda) = (y_2 - y_1)$$
$$(B_{\mathbb{M}} \cdot \lambda) \mathbin{\mathsf{X}} (\lambda) = (A_{\mathbb{M}} + x_1 + x_2 + x_3)$$
$$(x_1 - x_3) \mathbin{\mathsf{X}} (\lambda) = (y_3 + y_1)$$

implement the affine-Montgomery addition $P_1 + P_2 = (x_3, y_3)$ for all such $P_{1\,..\,2}$.

Proof. The given constraints are equivalent to the Montgomery addition formulae under the side condition that $x_1 \neq x_2$. (Note that neither P_i can be the zero point since $k_{1\,..\,2} \neq 0 \pmod{s}$.) Assume for a contradiction that $x_1 = x_2$. For any $P_1 = [k_1]\,Q$, there can be only one other point $-P_1$ with the same x-coordinate. (This follows from the fact that the curve equation determines $\pm y$ as a function of x.) But $-P_1 = [-1]\,[k_1]\,Q = [-k_1]\,Q$. Since $k : \left\{-\frac{s-1}{2} \,..\, \frac{s-1}{2}\right\} \mapsto [k]\,Q : \mathbb{M}$ is injective and $k_{1\,..\,2}$ are in $\left\{-\frac{s-1}{2} \,..\, \frac{s-1}{2}\right\}$, then $k_2 = \pm k_1$ (contradiction). □

The conditions of this theorem are called the *distinct-x criterion*.

In particular, if $k_{1\,..\,2}$ are integers in $\left\{1 \,..\, \frac{s-1}{2}\right\}$ then it is sufficient to require $k_2 \neq k_1$, since that implies $k_2 \neq \pm k_1$.

Affine-Montgomery doubling can be implemented as:

$$(x) \mathbin{\mathsf{X}} (x) = (xx)$$
$$(2{\cdot}B_{\mathbb{M}}{\cdot}y) \mathbin{\mathsf{X}} (\lambda) = (3{\cdot}xx + 2{\cdot}A_{\mathbb{M}}{\cdot}x + 1)$$
$$(B_{\mathbb{M}}{\cdot}\lambda) \mathbin{\mathsf{X}} (\lambda) = (A_{\mathbb{M}} + 2{\cdot}x + x_3)$$
$$(x - x_3) \mathbin{\mathsf{X}} (\lambda) = (y_3 + y)$$

This doubling formula is valid when $y \neq 0$, which is the case when (x, y) is not the point $(0, 0)$ (the only point of order 2), as proven in Theorem A.2.1 on p. 150.

A.3.3.5 Affine-ctEdwards arithmetic

Formulae for *affine-ctEdwards* addition are given in [BBJLP2008, section 6]. With a change of variable names to match our convention, the formulae for $(u_1, v_1) + (u_2, v_2) = (u_3, v_3)$ are:

$$u_3 = \frac{u_1{\cdot}v_2 + v_1{\cdot}u_2}{1 + d_{\mathbb{J}}{\cdot}u_1{\cdot}u_2{\cdot}v_1{\cdot}v_2}$$
$$v_3 = \frac{v_1{\cdot}v_2 - a_{\mathbb{J}}{\cdot}u_1{\cdot}u_2}{1 - d_{\mathbb{J}}{\cdot}u_1{\cdot}u_2{\cdot}v_1{\cdot}v_2}$$

We use an optimized implementation found by Daira-Emma Hopwood making use of an observation by Bernstein and Lange in [BL2017, last paragraph of section 4.5.2]:

$$(u_1 + v_1) \mathbin{\mathsf{X}} (v_2 - a_{\mathbb{J}}{\cdot}u_2) = (T)$$
$$(u_1) \mathbin{\mathsf{X}} (v_2) = (A)$$
$$(v_1) \mathbin{\mathsf{X}} (u_2) = (B)$$
$$(d_{\mathbb{J}}{\cdot}A) \mathbin{\mathsf{X}} (B) = (C)$$
$$(1 + C) \mathbin{\mathsf{X}} (u_3) = (A + B)$$
$$(1 - C) \mathbin{\mathsf{X}} (v_3) = (T - A + a_{\mathbb{J}}{\cdot}B)$$

The correctness of this implementation can be seen by expanding $T - A + a_{\mathbb{J}}{\cdot}B$:

$$\begin{aligned} T - A + a_{\mathbb{J}}{\cdot}B &= (u_1 + v_1) \cdot (v_2 - a_{\mathbb{J}}{\cdot}u_2) - u_1{\cdot}v_2 + a_{\mathbb{J}}{\cdot}v_1{\cdot}u_2 \\ &= v_1{\cdot}v_2 - a_{\mathbb{J}}{\cdot}u_1{\cdot}u_2 + u_1{\cdot}v_2 - a_{\mathbb{J}}{\cdot}v_1{\cdot}u_2 - u_1{\cdot}v_2 + a_{\mathbb{J}}{\cdot}v_1{\cdot}u_2 \\ &= v_1{\cdot}v_2 - a_{\mathbb{J}}{\cdot}u_1{\cdot}u_2 \end{aligned}$$

The above addition formulae are "unified", that is, they can also be used for doubling. *Affine-ctEdwards* doubling $[2]\,(u, v) = (u_3, v_3)$ can also be implemented slightly more efficiently as:

$$(u + v)\ \mathsf{X}\ (v - a_{\mathbb{J}} \cdot u) = (T)$$
$$(u)\ \mathsf{X}\ (v) = (A)$$
$$(d_{\mathbb{J}} \cdot A)\ \mathsf{X}\ (A) = (C)$$
$$(1 + C)\ \mathsf{X}\ (u_3) = (2 \cdot A)$$
$$(1 - C)\ \mathsf{X}\ (v_3) = (T + (a_{\mathbb{J}} - 1) \cdot A)$$

This implementation is obtained by specializing the addition formulae to $(u, v) = (u_1, v_1) = (u_2, v_2)$ and observing that $u \cdot v = A = B$.

A.3.3.6 Affine-ctEdwards nonsmall-order check

In order to avoid small-subgroup attacks, we check that certain points used in the circuit are not of *small order*. In practice the Sapling circuit uses this in combination with a check that the coordinates are on the curve (§ A.3.3.1 ***'Checking that Affine-ctEdwards coordinates are on the curve'*** on p. 154), so we combine the two operations.

The *Jubjub curve* has a large *prime-order subgroup* with a cofactor of 8. To check for a point P of order 8 or less, the Sapling circuit doubles three times (as in § A.3.3.5 ***'Affine-ctEdwards arithmetic'*** on p. 156) and checks that the resulting u-coordinate is not 0 (as in § A.3.1.4 ***'Nonzero constraints'*** on p. 151).

On a *ctEdwards curve*, only the zero point $\mathcal{O}_{\mathbb{J}}$, and the unique point of order 2 at $(0, -1)$ have zero u-coordinate. The point of order 2 cannot occur as the result of three doublings. So this u-coordinate check rejects only $\mathcal{O}_{\mathbb{J}}$.

The total cost, including the curve check, is $4 + 3 \cdot 5 + 1 = 20$ constraints.

Note: This ***does not*** ensure that the point is in the *prime-order subgroup*.

Non-normative notes:

- It would have been sufficient to do two doublings rather than three, because the check that the u-coordinate is nonzero would reject both $\mathcal{O}_{\mathbb{J}}$ and the point of order 2.
- It is possible to reduce the cost to 8 constraints by eliminating the redundant constraint in the curve point check (as mentioned in § A.3.3.1 ***'Checking that Affine-ctEdwards coordinates are on the curve'*** on p. 154); merging the first doubling with the curve point check; and then optimizing the second doubling based on the fact that we only need to check whether the resulting u-coordinate is zero. The Sapling circuit does not use these optimizations.

A.3.3.7 Fixed-base Affine-ctEdwards scalar multiplication

If the base point B is fixed for a given scalar multiplication $[k]\,B$, we can fully precompute window tables for each window position.

It is most efficient to use 3-bit fixed windows. Since the length of $r_{\mathbb{J}}$ is 252 bits, we need 84 windows.

Express k in base 8, i.e. $k = \sum_{i=0}^{83} k_i \cdot 8^i$.

Then $[k]\,B = \sum_{i=0}^{83} w_{(B,\,i,\,k_i)}$, where $w_{(B,\,i,\,k_i)} = [k_i \cdot 8^i]\,B$.

We precompute all of $w_{(B,\,i,\,s)}$ for $i \in \{0\,..\,83\}, s \in \{0\,..\,7\}$.

To look up a given window entry $w_{(B,\,i,\,s)} = (u_s, v_s)$, where $s = 4{\cdot}s_2 + 2{\cdot}s_1 + s_0$, we use:

$$
\begin{aligned}
&(s_1)\ \mathsf{X}\ (s_2) = (s_\&)\\
&(s_0)\ \mathsf{X}\ \big(-u_0{\cdot}s_\& + u_0{\cdot}s_2 + u_0{\cdot}s_1 - u_0 + u_2{\cdot}s_\& - u_2{\cdot}s_1 + u_4{\cdot}s_\& - u_4{\cdot}s_2 - u_6{\cdot}s_\&\\
&\qquad + u_1{\cdot}s_\& - u_1{\cdot}s_2 - u_1{\cdot}s_1 + u_1 - u_3{\cdot}s_\& + u_3{\cdot}s_1 - u_5{\cdot}s_\& + u_5{\cdot}s_2 + u_7{\cdot}s_\&\big) =\\
&\qquad (u_s - u_0{\cdot}s_\& + u_0{\cdot}s_2 + u_0{\cdot}s_1 - u_0 + u_2{\cdot}s_\& - u_2{\cdot}s_1 + u_4{\cdot}s_\& - u_4{\cdot}s_2 - u_6{\cdot}s_\&)\\
&(s_0)\ \mathsf{X}\ \big(-v_0{\cdot}s_\& + v_0{\cdot}s_2 + v_0{\cdot}s_1 - v_0 + v_2{\cdot}s_\& - v_2{\cdot}s_1 + v_4{\cdot}s_\& - v_4{\cdot}s_2 - v_6{\cdot}s_\&\\
&\qquad + v_1{\cdot}s_\& - v_1{\cdot}s_2 - v_1{\cdot}s_1 + v_1 - v_3{\cdot}s_\& + v_3{\cdot}s_1 - v_5{\cdot}s_\& + v_5{\cdot}s_2 + v_7{\cdot}s_\&\big) =\\
&\qquad (v_s - v_0{\cdot}s_\& + v_0{\cdot}s_2 + v_0{\cdot}s_1 - v_0 + v_2{\cdot}s_\& - v_2{\cdot}s_1 + v_4{\cdot}s_\& - v_4{\cdot}s_2 - v_6{\cdot}s_\&)
\end{aligned}
$$

For a full-length (252-bit) scalar this costs 3 constraints for each of 84 window lookups, plus 6 constraints for each of 83 *ctEdwards* additions (as in § A.3.3.5 ***'Affine-ctEdwards arithmetic'*** on p. 156), for a total of 750 constraints.

Fixed-base scalar multiplication is also used in two places with shorter scalars:

- § A.3.6 ***'Homomorphic Pedersen Commitment'*** on p. 162 uses 64 bits for the v input to $\mathsf{ValueCommit}^{\mathsf{Sapling}}$, requiring 22 windows at a cost of $3{\cdot}22 - 1 + 6{\cdot}21 = 191$ constraints;
- § A.3.3.10 ***'Mixing Pedersen hash'*** on p. 161 uses 32 bits for the pos input to MixingPedersenHash, requiring 11 windows at a cost of $3{\cdot}11 - 1 + 6{\cdot}10 = 92$ constraints.

None of these costs include the cost of boolean-constraining the scalar.

Non-normative notes:

- It would be more efficient to use arithmetic on the *Montgomery curve*, as in § A.3.3.9 ***'Pedersen hash'*** on p. 159. However since there are only three instances of fixed-base scalar multiplication in the *Spend circuit* and two in the *Output circuit*[11], the additional complexity was not considered justified for Sapling.
- For the multiplications with 64-bit and 32-bit scalars, the scalar is padded to a multiple of 3 bits with zeros. This causes the computation of $s_\&$ in the lookup for the most significant window to be optimized out, which is where the "$-\,1$" comes from in the above cost calculations. No further optimization is done for this lookup.

A.3.3.8 Variable-base Affine-ctEdwards scalar multiplication

When the base point B is not fixed, the method in the preceding section cannot be used. Instead we use a naïve double-and-add method.

Given $k = \sum_{i=0}^{250} k_i{\cdot}2^i$, we calculate $R = [k]\,B$ using:

// $\mathsf{Base}_i = [2^i]\,B$
let $\mathsf{Base}_0 = B$
let $\mathsf{Acc}_0^u = k_0\ ?\ \mathsf{Base}_0^u : 0$
let $\mathsf{Acc}_0^v = k_0\ ?\ \mathsf{Base}_0^v : 1$

for i from 1 up to 250:
  let $\mathsf{Base}_i = [2]\,\mathsf{Base}_{i-1}$
  // select Base_i or $\mathcal{O}_{\mathbb{J}}$ depending on the bit k_i
  let $\mathsf{Addend}_i^u = k_i\ ?\ \mathsf{Base}_i^u : 0$
  let $\mathsf{Addend}_i^v = k_i\ ?\ \mathsf{Base}_i^v : 1$
  let $\mathsf{Acc}_i = \mathsf{Acc}_{i-1} + \mathsf{Addend}_i$

let $R = \mathsf{Acc}_{250}$.

This costs 5 constraints for each of 250 *ctEdwards* doublings, 6 constraints for each of 250 *ctEdwards* additions, and 2 constraints for each of 251 point selections, for a total of 3252 constraints.

[11] A *Pedersen commitment* uses fixed-base scalar multiplication as a subcomponent.

Non-normative note: It would be more efficient to use 2-bit fixed windows, and/or to use arithmetic on the *Montgomery curve* in a similar way to § A.3.3.9 ***'Pedersen hash'*** on p. 159. However since there are only two instances of variable-base scalar multiplication in the *Spend circuit* and one in the *Output circuit*, the additional complexity was not considered justified for Sapling.

A.3.3.9 Pedersen hash

The specification of the *Pedersen hashes* used in Sapling is given in § 5.4.1.7 ***'Pedersen Hash Function'*** on p. 62. It is based on the scheme from [CvHP1991, section 5.2] –for which a tighter security reduction to the *Discrete Logarithm Problem* was given in [BGG1995]– but tailored to allow several optimizations in the circuit implementation.

Pedersen hashes are the single most commonly used primitive in the Sapling circuits. $\mathsf{MerkleDepth}^{\mathsf{Sapling}}$ *Pedersen hash* instances are used in the *Spend circuit* to check a *Merkle path* to the *note commitment* of the *note* being spent. We also reuse the *Pedersen hash* implementation to construct the *note commitment scheme* $\mathsf{NoteCommit}^{\mathsf{Sapling}}$.

This motivates considerable attention to optimizing this circuit implementation of this primitive, even at the cost of complexity.

First, we use a windowed scalar multiplication algorithm with signed digits. Each 3-bit message chunk corresponds to a window; the chunk is encoded as an integer from the set $\mathsf{Digits} = \{-4 \,..\, 4\} \setminus \{0\}$. This allows a more efficient lookup of the window entry for each chunk than if the set $\{1 \,..\, 8\}$ had been used, because a point can be conditionally negated using only a single constraint.

Next, we optimize the cost of point addition by allowing as many additions as possible to be performed on the *Montgomery curve*. An incomplete Montgomery addition costs 3 constraints, in comparison with a ctEdwards addition which costs 6 constraints.

However, we cannot do all additions on the *Montgomery curve* because the Montgomery addition is incomplete. In order to be able to prove that exceptional cases do not occur, we need to ensure that the *distinct-x criterion* from § A.3.3.4 ***'Affine-Montgomery arithmetic'*** on p. 155 is met. This requires splitting the input into segments (each using an independent generator), calculating an intermediate result for each segment, and then converting to the *ctEdwards curve* and summing the intermediate results using ctEdwards addition.

Abstracting away the changes of curve, this calculation can be written as:

$$\mathsf{PedersenHashToPoint}(D, M) = \sum_{j=1}^{N} [\langle M_j \rangle]\, \mathcal{I}(D, j)$$

where $\langle \bullet \rangle$ and $\mathcal{I}(D, j)$ are defined as in § 5.4.1.7 ***'Pedersen Hash Function'*** on p. 62.

We have to prove that:

- the Montgomery-to-*ctEdwards* conversions can be implemented without exceptional cases;
- the *distinct-x criterion* is met for all Montgomery additions within a segment.

The proof of Theorem 5.4.1 on p. 63 showed that all indices of addition inputs are in the range $\left\{-\frac{r_{\mathbb{J}}-1}{2} \,..\, \frac{r_{\mathbb{J}}-1}{2}\right\} \setminus \{0\}$.

Because the $\mathcal{I}(D, j)$ (which are outputs of $\mathsf{GroupHash}^{\mathbb{J}^{(r)*}}$) are all of *prime order*, and $\langle M_j \rangle \neq 0 \pmod{r_{\mathbb{J}}}$, it is guaranteed that all of the terms $[\langle M_j \rangle]\, \mathcal{I}(D, j)$ to be converted to *ctEdwards* form are of *prime order*. From Theorem A.3.3 on p. 155, we can infer that the conversions will not encounter exceptional cases.

We also need to show that the indices of addition inputs are all distinct disregarding sign.

Theorem A.3.5. *Concerning addition inputs in the Pedersen circuit.*

For all disjoint nonempty subsets S and S' of $\{1 .. c\}$, all $m \in \mathbb{B}^{[3][c]}$, and all $\Theta \in \{-1, 1\}$:

$$\sum_{j \in S} \mathsf{enc}(m_j) \cdot 2^{4 \cdot (j-1)} \neq \Theta \cdot \sum_{j' \in S'} \mathsf{enc}(m_{j'}) \cdot 2^{4 \cdot (j'-1)}.$$

Proof. Suppose for a contradiction that S, S', m, Θ is a counterexample. Taking the multiplication by Θ on the right hand side inside the summation, we have:

$$\sum_{j \in S} \mathsf{enc}(m_j) \cdot 2^{4 \cdot (j-1)} = \sum_{j' \in S'} \Theta \cdot \mathsf{enc}(m_{j'}) \cdot 2^{4 \cdot (j'-1)}.$$

Define $\mathsf{enc}' : \{-1, 1\} \times \mathbb{B}^{[3]} \to \{0 .. 8\} \setminus \{4\}$ as $\mathsf{enc}'_\theta(m_i) := 4 + \theta \cdot \mathsf{enc}(m_i)$.

Let $\Delta = 4 \cdot \sum_{i=1}^{c} 2^{4 \cdot (i-1)}$ as in the proof of Theorem 5.4.1 on p. 63. By adding Δ to both sides, we get

$$\sum_{j \in S} \mathsf{enc}'_1(m_j) \cdot 2^{4 \cdot (j-1)} + \sum_{j \in \{1 .. c\} \setminus S} 4 \cdot 2^{4 \cdot (j-1)} = \sum_{j' \in S'} \mathsf{enc}'_\Theta(m_{j'}) \cdot 2^{4 \cdot (j'-1)} + \sum_{j' \in \{1 .. c\} \setminus S'} 4 \cdot 2^{4 \cdot (j'-1)}$$

where all of the $\mathsf{enc}'_1(m_j)$ and $\mathsf{enc}'_\Theta(m_{j'})$ are in $\{0 .. 8\} \setminus \{4\}$.

Each term on the left and on the right affects the single hex digit indexed by j and j' respectively. Since S and S' are disjoint subsets of $\{1 .. c\}$ and S is nonempty, $S \cap (\{1 .. c\} \setminus S')$ is nonempty. Therefore the left hand side has at least one hex digit not equal to 4 such that the corresponding right hand side digit is 4; contradiction. □

This implies that the terms in the Montgomery addition –as well as any intermediate results formed from adding a distinct subset of terms– have distinct indices disregarding sign, hence distinct x-coordinates by Theorem A.3.4 on p. 155. (We make no assumption about the order of additions.)

We now describe the subcircuit used to process each chunk, which contributes most of the constraint cost of the hash. This subcircuit is used to perform a lookup of a Montgomery point in a 2-bit window table, conditionally negate the result, and add it to an accumulator holding another Montgomery point.

Suppose that the bits of the chunk, $[s_0, s_1, s_2]$, are already boolean-constrained.

We aim to compute $C = A + [(1 - 2 \cdot s_2) \cdot (1 + s_0 + 2 \cdot s_1)]\, P$ for some fixed base point P and accumulated sum A.

We first compute $s_\& = s_0 \,\&\, s_1$:

$$(s_0)\ \mathsf{X}\ (s_1) = (s_\&)$$

Let $(x_k, y_k) = [k]\, P$ for $k \in \{1 .. 4\}$. Define each coordinate of $(x_S, y_R) = [1 + s_0 + 2 \cdot s_1]\, P$ as a linear combination of s_0, s_1, and $s_\&$:

$$\text{let } x_S = x_1 + (x_2 - x_1) \cdot s_0 + (x_3 - x_1) \cdot s_1 + (x_4 + x_1 - x_2 - x_3) \cdot s_\&$$
$$\text{let } y_R = y_1 + (y_2 - y_1) \cdot s_0 + (y_3 - y_1) \cdot s_1 + (y_4 + y_1 - y_2 - y_3) \cdot s_\&$$

We implement the conditional negation as $(2 \cdot y_R)\ \mathsf{X}\ (s_2) = (y_R - y_S)$. After substitution of y_R this becomes:

$$\begin{aligned}&\big(2 \cdot (y_1 + (y_2 - y_1) \cdot s_0 + (y_3 - y_1) \cdot s_1 + (y_4 + y_1 - y_2 - y_3) \cdot s_\&)\big)\ \mathsf{X}\ (s_2) = \\ &\quad \big(y_1 + (y_2 - y_1) \cdot s_0 + (y_3 - y_1) \cdot s_1 + (y_4 + y_1 - y_2 - y_3) \cdot s_\& - y_S\big)\end{aligned}$$

Then we substitute x_S into the Montgomery addition constraints from § A.3.3.4 *'Affine-Montgomery arithmetic'* on p. 155, as follows:

$$(x_1 + (x_2 - x_1)\cdot s_0 + (x_3 - x_1)\cdot s_1 + (x_4 + x_1 - x_2 - x_3)\cdot s_{\&} - x_A) \mathbin{\mathsf{X}} (\lambda) = (y_S - y_A)$$
$$(B_{\mathbb{M}}\cdot\lambda) \mathbin{\mathsf{X}} (\lambda) = (A_{\mathbb{M}} + x_A + x_1 + (x_2 - x_1)\cdot s_0 + (x_3 - x_1)\cdot s_1 + (x_4 + x_1 - x_2 - x_3)\cdot s_{\&} + x_C)$$
$$(x_A - x_C) \mathbin{\mathsf{X}} (\lambda) = (y_C + y_A)$$

(In the sapling-crypto implementation, linear combinations are first-class values, so these substitutions do not need to be done "by hand".)

For the first addition in each segment, both sides are looked up and substituted into the Montgomery addition, so the first lookup takes only 2 constraints.

When these hashes are used in the circuit, the first 6 bits of the input are fixed. For example, in the Merkle tree hashes they represent the layer number. This would allow a precomputation for the first two windows, but that optimization is not done in Sapling.

The cost of a Pedersen hash over ℓ bits (where ℓ includes the fixed bits) is as follows. The number of chunks is $c = \mathsf{ceiling}\left(\frac{\ell}{3}\right)$ and the number of segments is $n = \mathsf{ceiling}\left(\frac{\ell}{3\cdot 63}\right)$.

The cost is then:

- $2{\cdot}c$ constraints for the lookups;
- $3{\cdot}(c - n)$ constraints for incomplete additions on the *Montgomery curve*;
- $2{\cdot}n$ constraints for Montgomery-to-*ctEdwards* conversions;
- $6{\cdot}(n - 1)$ constraints for *ctEdwards* additions;

for a total of $5{\cdot}c + 5{\cdot}n - 6$ constraints. This does not include the cost of boolean-constraining inputs.

In particular,

- for the Merkle tree hashes $\ell = 516$, so $c = 172$, $n = 3$, and the cost is 869 constraints;
- when a Pedersen hash is used to implement part of a Pedersen commitment for $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ (§ 5.4.7.2 *'Windowed Pedersen commitments'* on p. 72), $\ell = 6 + \ell_{\mathsf{value}} + 2{\cdot}\ell_{\mathbb{J}} = 582$, $c = 194$, and $n = 4$, so the cost of the hash alone is 984 constraints.

A.3.3.10 Mixing Pedersen hash

A mixing *Pedersen hash* is used to compute ρ from cm and pos in § 4.14 *'Computing ρ values and Nullifiers'* on p. 45. It takes as input a *Pedersen commitment* P, and hashes it with another input x.

Let $\mathcal{J}^{\mathsf{Sapling}}$ be as defined in § 5.4.1.8 *'Mixing Pedersen Hash Function'* on p. 63.

We define $\mathsf{MixingPedersenHash} : \{0\,..\,r_{\mathbb{J}} - 1\} \times \mathbb{J} \to \mathbb{J}$ by:

$$\mathsf{MixingPedersenHash}(P, x) := P + [x]\,\mathcal{J}^{\mathsf{Sapling}}.$$

This costs 92 constraints for a scalar multiplication (§ A.3.3.7 *'Fixed-base Affine-ctEdwards scalar multiplication'* on p. 157), and 6 constraints for a *ctEdwards* addition (§ A.3.3.5 *'Affine-ctEdwards arithmetic'* on p. 156), for a total of 98 constraints.

A.3.4 Merkle path check

Checking each layer of a Merkle authentication path, as described in § 4.8 ***'Merkle Path Validity'*** on p. 39, requires to:

- boolean-constrain the path bit specifying whether the previous node is a left or right child;
- conditionally swap the previous-layer and sibling hashes (as $\mathbb{F}_r$ elements) depending on the path bit;
- unpack the left and right hash inputs to two sequences of 255 bits;
- compute the Merkle hash for this node.

The unpacking need not be canonical in the sense discussed in § A.3.2.1 ***'[Un]packing modulo $r_{\mathbb{S}}$'*** on p. 151; that is, it is ***not*** necessary to ensure that the left or right inputs to the hash represent integers in the range $\{0 \,..\, r_{\mathbb{S}} - 1\}$. Since the root of the Merkle tree is calculated outside the circuit using the canonical representations, and since the *Pedersen hashes* are *collision-resistant* on arbitrary *bit-sequence* inputs, an attempt by an adversarial prover to use a *non-canonical* input would result in the wrong root being calculated, and the overall path check would fail.

For each layer, the cost is $1 + 2 \cdot 255$ boolean constraints, 2 constraints for the conditional swap (implemented as two selection constraints), and 869 constraints for the Merkle hash (§ A.3.3.9 ***'Pedersen hash'*** on p. 159), for a total of 1380 constraints.

Non-normative note: The conditional swap $(a_0, a_1) \mapsto (c_0, c_1)$ could be implemented in only one constraint by substituting $c_1 = a_0 + a_1 - c_0$ into the uses of c_1. The Sapling circuit does not use this optimization.

A.3.5 Windowed Pedersen Commitment

We construct *windowed Pedersen commitments* by reusing the Pedersen hash implementation described in § A.3.3.9 ***'Pedersen hash'*** on p. 159, and adding a randomized point:

$$\mathsf{WindowedPedersenCommit}_r(s) = \mathsf{PedersenHashToPoint}(\texttt{“Zcash_PH”}, s) + [r]\,\mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}(\texttt{“Zcash_PH”}, \texttt{“r”})$$

This can be implemented in:

- $5 \cdot c + 5 \cdot n - 6$ constraints for the Pedersen hash applied to $\ell = 6 + \mathsf{length}(s)$ bits, where $c = \mathsf{ceiling}\left(\frac{\ell}{3}\right)$ and $n = \mathsf{ceiling}\left(\frac{\ell}{3 \cdot 63}\right)$;
- 750 constraints for the fixed-base scalar multiplication;
- 6 constraints for the final *ctEdwards* addition.

When $\mathsf{WindowedPedersenCommit}$ is used to instantiate $\mathsf{NoteCommit}^{\mathsf{Sapling}}$, the cost of the Pedersen hash is 984 constraints as calculated in § A.3.3.9 ***'Pedersen hash'*** on p. 159, and so the total cost in that case is 1740 constraints. This does not include the cost of boolean-constraining the input s or the randomness r.

A.3.6 Homomorphic Pedersen Commitment

The *windowed Pedersen commitments* defined in the preceding section are highly efficient, but they do not support the homomorphic property we need when instantiating $\mathsf{ValueCommit}$.

In order to support this property, we also define *homomorphic Pedersen commitments* as follows:

$$\mathsf{HomomorphicPedersenCommit}^{\mathsf{Sapling}}_{\mathsf{rcv}}(D, \mathsf{v}) = [\mathsf{v}]\,\mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}(D, \texttt{“v”}) + [\mathsf{rcv}]\,\mathsf{FindGroupHash}^{\mathbb{J}^{(r)*}}(D, \texttt{“r”})$$

In the case that we need for ValueCommit, v has 64 bits[12]. This value is given as a bit representation, which does not need to be constrained equal to an integer.

ValueCommit can be implemented in:

- 750 constraints for the 252-bit fixed-base multiplication by rcv;
- 191 constraints for the 64-bit fixed-base multiplication by v;
- 6 constraints for the *ctEdwards* addition

for a total cost of 947 constraints. This does not include the cost to boolean-constrain the input v or randomness rcv.

A.3.7 BLAKE2s hashes

BLAKE2s is defined in [ANWW2013]. Its main subcomponent is a "G function", defined as follows:

$$G : \{0\,..\,9\} \times \{0\,..\,2^{32}-1\}^{[4]} \to \{0\,..\,2^{32}-1\}^{[4]}$$
$$G(a, b, c, d, x, y) = (a'', b'', c'', d'') \text{ where}$$
$$\begin{aligned}
a' &= (a + b + x) \bmod 2^{32} \\
d' &= (d \oplus a') \ggg 16 \\
c' &= (c + d') \bmod 2^{32} \\
b' &= (b \oplus c') \ggg 12 \\
a'' &= (a' + b' + y) \bmod 2^{32} \\
d'' &= (d' \oplus a'') \ggg 8 \\
c'' &= (c' + d'') \bmod 2^{32} \\
b'' &= (b' \oplus c'') \ggg 7
\end{aligned}$$

The following table is used to determine which message words the x and y arguments to G are selected from:

$$\begin{aligned}
\sigma_0 &= [\ 0,\ 1,\ 2,\ 3,\ 4,\ 5,\ 6,\ 7,\ 8,\ 9,\ 10,\ 11,\ 12,\ 13,\ 14,\ 15\ ] \\
\sigma_1 &= [\ 14,\ 10,\ 4,\ 8,\ 9,\ 15,\ 13,\ 6,\ 1,\ 12,\ 0,\ 2,\ 11,\ 7,\ 5,\ 3\ ] \\
\sigma_2 &= [\ 11,\ 8,\ 12,\ 0,\ 5,\ 2,\ 15,\ 13,\ 10,\ 14,\ 3,\ 6,\ 7,\ 1,\ 9,\ 4\ ] \\
\sigma_3 &= [\ 7,\ 9,\ 3,\ 1,\ 13,\ 12,\ 11,\ 14,\ 2,\ 6,\ 5,\ 10,\ 4,\ 0,\ 15,\ 8\ ] \\
\sigma_4 &= [\ 9,\ 0,\ 5,\ 7,\ 2,\ 4,\ 10,\ 15,\ 14,\ 1,\ 11,\ 12,\ 6,\ 8,\ 3,\ 13\ ] \\
\sigma_5 &= [\ 2,\ 12,\ 6,\ 10,\ 0,\ 11,\ 8,\ 3,\ 4,\ 13,\ 7,\ 5,\ 15,\ 14,\ 1,\ 9\ ] \\
\sigma_6 &= [\ 12,\ 5,\ 1,\ 15,\ 14,\ 13,\ 4,\ 10,\ 0,\ 7,\ 6,\ 3,\ 9,\ 2,\ 8,\ 11\ ] \\
\sigma_7 &= [\ 13,\ 11,\ 7,\ 14,\ 12,\ 1,\ 3,\ 9,\ 5,\ 0,\ 15,\ 4,\ 8,\ 6,\ 2,\ 10\ ] \\
\sigma_8 &= [\ 6,\ 15,\ 14,\ 9,\ 11,\ 3,\ 0,\ 8,\ 12,\ 2,\ 13,\ 7,\ 1,\ 4,\ 10,\ 5\ ] \\
\sigma_9 &= [\ 10,\ 2,\ 8,\ 4,\ 7,\ 6,\ 1,\ 5,\ 15,\ 11,\ 9,\ 14,\ 3,\ 12,\ 13,\ 0\ ]
\end{aligned}$$

The Initialization Vector is defined as:

$$\begin{aligned}
\mathsf{IV} : \{0\,..\,2^{32}-1\}^{[8]} := [\ &\texttt{0x6A09E667},\ \texttt{0xBB67AE85},\ \texttt{0x3C6EF372},\ \texttt{0xA54FF53A} \\
&\texttt{0x510E527F},\ \texttt{0x9B05688C},\ \texttt{0x1F83D9AB},\ \texttt{0x5BE0CD19}\ ]
\end{aligned}$$

[12] It would be sufficient to use 51 bits, which accomodates the range $\{0\,..\,\mathsf{MAX_MONEY}\}$, but the Sapling circuit uses 64.

The full hash function applied to an 8-byte personalization string and a single 64-byte block, in sequential mode with 32-byte output, can be expressed as follows.

Define BLAKE2s-256 : $(p : \mathbb{B}^{\mathbb{Y}[8]}) \times (x : \mathbb{B}^{\mathbb{Y}[64]}) \to \mathbb{B}^{\mathbb{Y}[32]}$ as:

let PB : $\mathbb{B}^{\mathbb{Y}[32]} = [32, 0, 1, 1] \,||\, [\texttt{0x00}]^{20} \,||\, p$

let $[\, t_0, t_1, f_0, f_1 \,] : \{0 \,..\, 2^{32}{-}1\}^{[4]} = [\, 0, 0, 0, \texttt{0xFFFFFFFF}, 0 \,]$

let $h : \{0 \,..\, 2^{32}{-}1\}^{[8]} = [\![\, \mathsf{LEOS2IP}_{32}(\mathsf{PB}_{4\cdot i \,..\, 4\cdot i+3}) \oplus \mathsf{IV}_i \text{ for } i \text{ from } 0 \text{ up to } 7 \,]\!]$

let $m : \{0 \,..\, 2^{32}{-}1\}^{[16]} = [\![\, \mathsf{LEOS2IP}_{32}(x_{4\cdot i \,..\, 4\cdot i+3}) \text{ for } i \text{ from } 0 \text{ up to } 15 \,]\!]$

let mutable $v : \{0 \,..\, 2^{32}{-}1\}^{[16]} \leftarrow h \,||\, [\, \mathsf{IV}_0, \mathsf{IV}_1, \mathsf{IV}_2, \mathsf{IV}_3, t_0 \oplus \mathsf{IV}_4, t_1 \oplus \mathsf{IV}_5, f_0 \oplus \mathsf{IV}_6, f_1 \oplus \mathsf{IV}_7 \,]$

for r from 0 up to 9:

set $(v_0, v_4, v_8, v_{12}) \leftarrow G(v_0, v_4, v_8, v_{12}, m_{\sigma_{r,0}}, m_{\sigma_{r,1}})$

set $(v_1, v_5, v_9, v_{13}) \leftarrow G(v_1, v_5, v_9, v_{13}, m_{\sigma_{r,2}}, m_{\sigma_{r,3}})$

set $(v_2, v_6, v_{10}, v_{14}) \leftarrow G(v_2, v_6, v_{10}, v_{14}, m_{\sigma_{r,4}}, m_{\sigma_{r,5}})$

set $(v_3, v_7, v_{11}, v_{15}) \leftarrow G(v_3, v_7, v_{11}, v_{15}, m_{\sigma_{r,6}}, m_{\sigma_{r,7}})$

set $(v_0, v_5, v_{10}, v_{15}) \leftarrow G(v_0, v_5, v_{10}, v_{15}, m_{\sigma_{r,8}}, m_{\sigma_{r,9}})$

set $(v_1, v_6, v_{11}, v_{12}) \leftarrow G(v_1, v_6, v_{11}, v_{12}, m_{\sigma_{r,10}}, m_{\sigma_{r,11}})$

set $(v_2, v_7, v_8, v_{13}) \leftarrow G(v_2, v_7, v_8, v_{13}, m_{\sigma_{r,12}}, m_{\sigma_{r,13}})$

set $(v_3, v_4, v_9, v_{14}) \leftarrow G(v_3, v_4, v_9, v_{14}, m_{\sigma_{r,14}}, m_{\sigma_{r,15}})$

return $\mathsf{LEBS2OSP}_{256}(\mathsf{concat}_{\mathbb{B}}([\![\, \mathsf{I2LEBSP}_{32}(h_i \oplus v_i \oplus v_{i+8}) \text{ for } i \text{ from } 0 \text{ up to } 7 \,]\!]))$

In practice the message and output will be expressed as *bit sequences*. In the Sapling circuit, the personalization string will be constant for each use.

Each 32-bit exclusive-or is implemented in 32 constraints, one for each bit position $a \oplus b = c$ as in § A.3.1.5 ***'Exclusive-or constraints'*** on p. 151.

Additions not involving a message word, i.e. $(a + b) \bmod 2^{32} = c$, are implemented using 33 constraints and a 33-bit equality check: constrain 33 boolean variables $c_{0\,..\,32}$, and then check $\sum_{i=0}^{i=31} (a_i + b_i) \cdot 2^i = \sum_{i=0}^{i=32} c_i \cdot 2^i$.

Additions involving a message word, i.e. $(a + b + m) \bmod 2^{32} = c$, are implemented using 34 constraints and a 34-bit equality check: constrain 34 boolean variables $c_{0\,..\,33}$, and then check $\sum_{i=0}^{i=31} (a_i + b_i + m_i) \cdot 2^i = \sum_{i=0}^{i=33} c_i \cdot 2^i$.

For each addition, only $c_{0\,..\,31}$ are used subsequently.

The equality checks are batched; as many sets of 33 or 34 boolean variables as will fit in a $\mathbb{F}_{r_{\mathbb{S}}}$ field element are equated together using one constraint. This allows 7 such checks per constraint.

Each G evaluation requires 262 constraints:

- $4 \cdot 32 = 128$ constraints for $\oplus$ operations;
- $2 \cdot 33 = 66$ constraints for 32-bit additions not involving message words (excluding equality checks);
- $2 \cdot 34 = 68$ constraints for 32-bit additions involving message words (excluding equality checks).

The overall cost is 21006 constraints:

- $10 \cdot 8 \cdot 262 - 4 \cdot 2 \cdot 32 = 20704$ constraints for 80 G evaluations, excluding equality checks (the deduction of $4 \cdot 2 \cdot 32$ is because v is constant at the start of the first round, so in the first four calls to G, the parameters b and d are constant, eliminating the constraints for the first two XORs in those four calls to G);
- ceiling $\left(\frac{10 \cdot 8 \cdot 4}{7}\right) = 46$ constraints for equality checks;
- $8 \cdot 32 = 256$ constraints for final $v_i \oplus v_{i+8}$ operations (the h_i words are constants so no additional constraints are required to exclusive-or with them).

This cost includes boolean-constraining the hash output bits (done implicitly by the final $\oplus$ operations), but not the message bits.

Non-normative notes:

- The equality checks could be eliminated entirely by substituting each check into a boolean constraint for c_0, for instance, but this optimization is not done in Sapling.
- It should be clear that BLAKE2s is very expensive in the circuit compared to elliptic curve operations. This is primarily because it is inefficient to use $\mathbb{F}_{r_{\mathbb{S}}}$ elements to represent single bits. However Pedersen hashes do not have the necessary cryptographic properties for the two cases where the *Spend circuit* uses BLAKE2s. While it might be possible to use variants of functions with low circuit cost such as MiMC [AGRRT2017], it was felt that they had not yet received sufficient cryptanalytic attention to confidently use them for Sapling.

A.4 The Sapling Spend circuit

The Sapling Spend *statement* is defined in § 4.16.2 *'Spend Statement (Sapling)'* on p. 48.

The primary input is

$$\begin{aligned}&\big(\mathsf{rt}^{\mathit{SaplingPool}} : \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]},\\ &\ \mathsf{cv}^{\mathsf{old}} : \mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Output},\\ &\ \mathsf{nf}^{\mathsf{old}} : \mathbb{B}^{[\ell_{\mathsf{PRFnfSapling}}/8]},\\ &\ \mathsf{rk} : \mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{Public}\big),\end{aligned}$$

which is encoded as $8\,\mathbb{F}_{r_{\mathbb{S}}}$ elements (starting with the fixed element 1 required by Groth16):

$$\big[1, \mathcal{U}(\mathsf{rk}), \mathcal{V}(\mathsf{rk}), \mathcal{U}(\mathsf{cv}^{\mathsf{old}}), \mathcal{V}(\mathsf{cv}^{\mathsf{old}}), \mathsf{LEBS2IP}_{\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}}\big(\mathsf{rt}^{\mathit{SaplingPool}}\big), \mathsf{LEBS2IP}_{254}\big(\mathsf{nf}^{\mathsf{old}}_{\star\ 0\,..\,253}\big), \mathsf{LEBS2IP}_{2}\big(\mathsf{nf}^{\mathsf{old}}_{\star\ 254\,..\,255}\big)\big]$$

where $\mathsf{nf}^{\mathsf{old}}_{\star} = \mathsf{LEOS2BSP}_{\ell_{\mathsf{PRFnfSapling}}}\big(\mathsf{nf}^{\mathsf{old}}\big)$.

The auxiliary input is

$$\begin{aligned}&\big(\mathsf{path} : \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}][\mathsf{MerkleDepth}^{\mathsf{Sapling}}]},\\ &\ \mathsf{pos} : \{0\,..\,2^{\mathsf{MerkleDepth}^{\mathsf{Sapling}}}-1\},\\ &\ \mathsf{g_d} : \mathbb{J},\\ &\ \mathsf{pk_d} : \mathbb{J},\\ &\ \mathsf{v}^{\mathsf{old}} : \{0\,..\,2^{\ell_{\mathsf{value}}}-1\},\\ &\ \mathsf{rcv}^{\mathsf{old}} : \{0\,..\,2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}}-1\},\\ &\ \mathsf{cm}^{\mathsf{old}} : \mathbb{J},\\ &\ \mathsf{rcm}^{\mathsf{old}} : \{0\,..\,2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}}-1\},\\ &\ \alpha : \{0\,..\,2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}}-1\},\\ &\ \mathsf{ak} : \mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{Public},\\ &\ \mathsf{nsk} : \{0\,..\,2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}}-1\}\big).\end{aligned}$$

$\mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Output}$ and $\mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{Public}$ are of type $\mathbb{J}$, so we have $\mathsf{cv}^{\mathsf{old}}$, $\mathsf{cm}^{\mathsf{old}}$, rk, $\mathsf{g_d}$, $\mathsf{pk_d}$, and ak that represent *Jubjub curve* points. However,

- $\mathsf{cv}^{\mathsf{old}}$ will be constrained to an output of $\mathsf{ValueCommit}^{\mathsf{Sapling}}$;
- $\mathsf{cm}^{\mathsf{old}}$ will be constrained to an output of $\mathsf{NoteCommit}^{\mathsf{Sapling}}$;
- rk will be constrained to $[\alpha]\,\mathcal{G}^{\mathsf{Sapling}} + \mathsf{ak}$;
- $\mathsf{pk_d}$ will be constrained to $[\mathsf{ivk}]\,\mathsf{g_d}$

so $\mathsf{cv}^{\mathsf{old}}$, $\mathsf{cm}^{\mathsf{old}}$, rk, and $\mathsf{pk_d}$ do not need to be explicitly checked to be on the curve.

In addition, $\mathsf{nk}\star$ and $\rho\star$ used in **Nullifier integrity** are compressed representations of *Jubjub curve* points. { **TODO: explain why these are implemented as § A.3.3.2 *'ctEdwards [de]compression and validation'* on p. 154 even though the statement spec doesn't explicitly say to do validation.** }

Therefore we have $\mathsf{g_d}$, ak, nk, and ρ that need to be constrained to valid *Jubjub curve* points as described in § A.3.3.2 *'ctEdwards [de]compression and validation'* on p. 154.

In order to aid in comparing the implementation with the specification, we present the checks needed in the order in which they are implemented in the sapling-crypto code:

Check	Implements	Cost	Reference
ak is on the curve { TODO: FIXME also decompressed below }	$\mathsf{ak} : \mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{Public}$	4	§ A.3.3.1 on p. 154
ak is not *small-order*	**Small order checks**	16	§ A.3.3.6 on p. 157
$\alpha\star : \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}]}$	$\alpha : \{0 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\}$	252	§ A.3.1.1 on p. 150
$\alpha' = [\alpha\star]\,\mathcal{G}^{\mathsf{Sapling}}$	**Spend authority**	750	§ A.3.3.7 on p. 157
$\mathsf{rk} = \alpha' + \mathsf{ak}$		6	§ A.3.3.5 on p. 156
inputize rk { TODO: not ccteddecompressvalidate => wrong count }	$\mathsf{rk} : \mathsf{SpendAuthSig}^{\mathsf{Sapling}}.\mathsf{Public}$	392?	§ A.3.3.2 on p. 154
$\mathsf{nsk}\star : \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}]}$	$\mathsf{nsk} : \{0 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\}$	252	§ A.3.1.1 on p. 150
$\mathsf{nk} = [\mathsf{nsk}\star]\,\mathcal{H}^{\mathsf{Sapling}}$	**Nullifier integrity**	750	§ A.3.3.7 on p. 157
$\mathsf{ak}\star = \mathsf{repr}_{\mathbb{J}}(\mathsf{ak} : \mathbb{J})$	**Diversified address integrity**	392	§ A.3.3.2 on p. 154
$\mathsf{nk}\star = \mathsf{repr}_{\mathbb{J}}(\mathsf{nk})$ { TODO: spec doesn't say to validate nk since it's calculated }	**Nullifier integrity**	392	§ A.3.3.2 on p. 154
$\mathsf{ivk}\star = \mathsf{I2LEBSP}_{251}\big(\mathsf{CRH}^{\mathsf{ivk}}(\mathsf{ak}, \mathsf{nk})\big)$ †	**Diversified address integrity**	21006	§ A.3.7 on p. 163
$\mathsf{g_d}$ is on the curve	$\mathsf{g_d} : \mathbb{J}$	4	§ A.3.3.1 on p. 154
$\mathsf{g_d}$ is not *small-order*	**Small order checks**	16	§ A.3.3.6 on p. 157
$\mathsf{pk_d} = [\mathsf{ivk}\star]\,\mathsf{g_d}$	**Diversified address integrity**	3252	§ A.3.3.8 on p. 158
$\mathsf{v}^{\mathsf{old}}_{\star} : \mathbb{B}^{[64]}$	$\mathsf{v}^{\mathsf{old}} : \{0 \,..\, 2^{64} - 1\}$	64	§ A.3.1.1 on p. 150
$\mathsf{rcv}\star : \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}]}$	$\mathsf{rcv} : \{0 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\}$	252	§ A.3.1.1 on p. 150
$\mathsf{cv} = \mathsf{ValueCommit}^{\mathsf{rcv}}_{()}(\mathsf{v}^{\mathsf{old}})$	**Value commitment integrity**	947	§ A.3.6 on p. 162
inputize cv		?	
$\mathsf{rcm}\star : \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}]}$	$\mathsf{rcm} : \{0 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\}$	252	§ A.3.1.1 on p. 150
$\mathsf{cm} = \mathsf{NoteCommit}^{\mathsf{Sapling}}_{\mathsf{rcm}}(\mathsf{g_d}, \mathsf{pk_d}, \mathsf{v}^{\mathsf{old}})$	**Note commitment integrity**	1740	§ A.3.5 on p. 162
$\mathsf{cm}_u = \mathsf{Extract}_{\mathbb{J}^{(r)}}(\mathsf{cm})$	**Merkle path validity**	0	
rt' is the root of a Merkle tree with leaf cm_u, and authentication path $(\mathsf{path}, \mathsf{pos}\star)$		$32 \cdot 1380$	§ A.3.4 on p. 162
$\mathsf{pos}\star = \mathsf{I2LEBSP}_{\mathsf{MerkleDepth}^{\mathsf{Sapling}}}(\mathsf{pos})$		1	§ A.3.2.1 on p. 151
if $\mathsf{v}^{\mathsf{old}} \neq 0$ then $\mathsf{rt}' = \mathsf{rt}^{\mathit{SaplingPool}}$		1	§ A.3.1.2 on p. 150
inputize $\mathsf{rt}^{\mathit{SaplingPool}}$		?	
$\rho = \mathsf{MixingPedersenHash}(\mathsf{cm}^{\mathsf{old}}, \mathsf{pos})$	**Nullifier integrity**	98	§ A.3.3.10 on p. 161
$\rho\star = \mathsf{repr}_{\mathbb{J}}(\rho)$ { TODO: spec doesn't say to validate ρ since it's calculated }		392	§ A.3.3.2 on p. 154
$\mathsf{nf}^{\mathsf{old}} = \mathsf{PRF}^{\mathsf{nfSapling}}_{\mathsf{nk}\star}(\rho\star)$		21006	§ A.3.7 on p. 163
pack $\mathsf{nf}^{\mathsf{old}}_{0\,..\,253}$ and $\mathsf{nf}^{\mathsf{old}}_{254\,..\,255}$ into two $\mathbb{F}_{r_{\mathbb{S}}}$ inputs	input encoding	2	§ A.3.2.1 on p. 151

† This is implemented by taking the output of BLAKE2s-256 as a *bit sequence* and dropping the most significant 5 bits, not by converting to an integer and back to a *bit sequence* as literally specified.

Note: The implementation represents $\alpha\star$, $\mathsf{nsk}\star$, $\mathsf{ivk}\star$, $\mathsf{rcm}\star$, $\mathsf{rcv}\star$, and $\mathsf{v}_\star^{\mathsf{old}}$ as *bit sequences* rather than integers. It represents nf as a *bit sequence* rather than a *byte sequence*.

A.5 The Sapling Output circuit

The Sapling Output *statement* is defined in § 4.16.3 *'Output Statement (Sapling)'* on p. 49.

The primary input is

$$\begin{aligned}&\big(\mathsf{cv}^{\mathsf{new}} : \mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Output},\\ &\ \mathsf{cm}_u : \mathbb{B}^{[\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}]},\\ &\ \mathsf{epk} : \mathbb{J}\big),\end{aligned}$$

which is encoded as $6\ \mathbb{F}_{r_{\mathbb{S}}}$ elements (starting with the fixed element 1 required by Groth16):

$$\big[1, \mathcal{U}(\mathsf{cv}^{\mathsf{new}}), \mathcal{V}(\mathsf{cv}^{\mathsf{new}}), \mathcal{U}(\mathsf{epk}), \mathcal{V}(\mathsf{epk}), \mathsf{LEBS2IP}_{\ell^{\mathsf{Sapling}}_{\mathsf{Merkle}}}(\mathsf{cm}_u)\big]$$

The auxiliary input is

$$\begin{aligned}&(\mathsf{g_d} : \mathbb{J},\\ &\ \mathsf{pk}\star_{\mathsf{d}} : \mathbb{B}^{[\ell_{\mathbb{J}}]},\\ &\ \mathsf{v}^{\mathsf{new}} : \{0 \,..\, 2^{\ell_{\mathsf{value}}} - 1\},\\ &\ \mathsf{rcv}^{\mathsf{new}} : \{0 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\},\\ &\ \mathsf{rcm}^{\mathsf{new}} : \{0 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\},\\ &\ \mathsf{esk} : \{0 \,..\, 2^{\ell^{\mathsf{Sapling}}_{\mathsf{scalar}}} - 1\})\end{aligned}$$

$\mathsf{ValueCommit}^{\mathsf{Sapling}}.\mathsf{Output}$ is of type $\mathbb{J}$, so we have $\mathsf{cv}^{\mathsf{new}}$, epk, and $\mathsf{g_d}$ that represent *Jubjub curve* points. However,

- $\mathsf{cv}^{\mathsf{new}}$ will be constrained to an output of $\mathsf{ValueCommit}^{\mathsf{Sapling}}$;
- epk will be constrained to $[\mathsf{esk}]\ \mathsf{g_d}$

so $\mathsf{cv}^{\mathsf{new}}$ and epk do not need to be explicitly checked to be on the curve.

Therefore we have only $\mathsf{g_d}$ that needs to be constrained to a valid *Jubjub curve* point as described in § A.3.3.2 *'ctEdwards [de]compression and validation'* on p. 154.

Note: $\mathsf{pk}\star_{\mathsf{d}}$ is ***not*** checked to be a valid compressed representation of a *Jubjub curve* point.

In order to aid in comparing the implementation with the specification, we present the checks needed in the order in which they are implemented in the sapling-crypto code:

Check	Implements	Cost	Reference
$\mathsf{v}_\star^{\mathsf{old}} : \mathbb{B}^{[64]}$	$\mathsf{v}^{\mathsf{old}} : \{0 .. 2^{64}-1\}$	64	§ A.3.1.1 on p. 150
$\mathsf{rcv}\star : \mathbb{B}^{[\ell_{\mathsf{scalar}}^{\mathsf{Sapling}}]}$	$\mathsf{rcv} : \{0 .. 2^{\ell_{\mathsf{scalar}}^{\mathsf{Sapling}}}-1\}$	252	§ A.3.1.1 on p. 150
$\mathsf{cv} = \mathsf{ValueCommit}_{\mathsf{rcv}}^{\mathsf{Sapling}}(\mathsf{v}^{\mathsf{old}})$	**Value commitment integrity**	947	§ A.3.6 on p. 162
inputize cv		?	
$\mathsf{g}\star_{\mathsf{d}} = \mathsf{repr}_{\mathbb{J}}(\mathsf{g}_{\mathsf{d}} : \mathbb{J})$	**Note commitment integrity**	392	§ A.3.3.2 on p. 154
g_{d} is not *small-order*	**Small order checks**	16	§ A.3.3.6 on p. 157
$\mathsf{esk}\star : \mathbb{B}^{[\ell_{\mathsf{scalar}}^{\mathsf{Sapling}}]}$	$\mathsf{esk} : \{0 .. 2^{\ell_{\mathsf{scalar}}^{\mathsf{Sapling}}}-1\}$	252	§ A.3.1.1 on p. 150
$\mathsf{epk} = [\mathsf{esk}\star]\,\mathsf{g}_{\mathsf{d}}$	**Ephemeral public key integrity**	3252	§ A.3.3.8 on p. 158
inputize epk		?	
$\mathsf{pk}\star_{\mathsf{d}} : \mathbb{B}^{[\ell_{\mathbb{J}}]}$	$\mathsf{pk}\star_{\mathsf{d}} : \mathbb{B}^{[\ell_{\mathbb{J}}]}$	256	§ A.3.1.1 on p. 150
$\mathsf{rcm}\star : \mathbb{B}^{[\ell_{\mathsf{scalar}}^{\mathsf{Sapling}}]}$	$\mathsf{rcm} : \{0 .. 2^{\ell_{\mathsf{scalar}}^{\mathsf{Sapling}}}-1\}$	252	§ A.3.1.1 on p. 150
$\mathsf{cm} = \mathsf{NoteCommit}_{\mathsf{rcm}}^{\mathsf{Sapling}}(\mathsf{g}_{\mathsf{d}}, \mathsf{pk}_{\mathsf{d}}, \mathsf{v}^{\mathsf{old}})$	**Note commitment integrity**	1740	§ A.3.5 on p. 162
pack inputs		?	

Note: The implementation represents $\mathsf{esk}\star$, $\mathsf{pk}\star_{\mathsf{d}}$, $\mathsf{rcm}\star$, $\mathsf{rcv}\star$, and $\mathsf{v}_\star^{\mathsf{old}}$ as *bit sequences* rather than integers.

B Batching Optimizations

B.1 RedDSA batch validation

The reference validation algorithm for RedDSA signatures is defined in § 5.4.6 *'RedDSA and RedJubjub'* on p. 69.

Let the RedDSA parameters $\mathbb{G}$ (defining a subgroup $\mathbb{G}^{(r)}$ of order $r_{\mathbb{G}}$, a cofactor $h_{\mathbb{G}}$, a group operation $+$, an additive identity $\mathcal{O}_{\mathbb{G}}$, a bit-length $\ell_{\mathbb{G}}$, a representation function $\mathsf{repr}_{\mathbb{G}}$, and an abstraction function $\mathsf{abst}_{\mathbb{G}}$); $\mathcal{P}_{\mathbb{G}} : \mathbb{G}$; $\ell_{\mathsf{H}} : \mathbb{N}$; $\mathsf{H} : \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}} \to \mathbb{B}^{\mathbb{Y}^{[\ell_{\mathsf{H}}/8]}}$; and the derived hash function $\mathsf{H}^{\circledast} : \mathbb{B}^{\mathbb{Y}^{[\mathbb{N}]}} \to \mathbb{F}_{r_{\mathbb{G}}}$ be as defined in that section.

Implementations **MAY** alternatively use the optimized procedure described in this section to perform faster validation of a batch of signatures, i.e. to determine whether all signatures in a batch are valid. Its input is a sequence of N *signature batch entries*, each of which is a (*validating key*, message, signature) triple.

Let LEOS2BSP, LEOS2IP, and LEBS2OSP be as defined in § 5.1 *'Integers, Bit Sequences, and Endianness'* on p. 56.

Define RedDSA.BatchEntry := RedDSA.Public × RedDSA.Message × RedDSA.Signature.

Define $\mathsf{RedDSA.BatchValidate} : (\mathsf{entry}_{0\,..\,N-1} : \mathsf{RedDSA.BatchEntry}^{[N]}) \to \mathbb{B}$ as:

For each $j \in \{0\,..\,N-1\}$:

Let $(\mathsf{vk}_j, M_j, \sigma_j) = \mathsf{entry}_j$.

Let $\underline{R_j}$ be the first $\mathsf{ceiling}\left(\ell_{\mathbb{G}}/8\right)$ bytes of σ_j, and let $\underline{S_j}$ be the remaining $\mathsf{ceiling}\left(\mathsf{bitlength}(r_{\mathbb{G}})/8\right)$ bytes.

Let $R_j = \mathsf{abst}_{\mathbb{G}}\left(\mathsf{LEOS2BSP}_{\ell_{\mathbb{G}}}(\underline{R_j})\right)$, and let $S_j = \mathsf{LEOS2IP}_{8 \cdot \mathsf{length}(\underline{S_j})}(\underline{S_j})$.

Let $\underline{\mathsf{vk}_j} = \mathsf{LEBS2OSP}_{\ell_{\mathbb{G}}}\left(\mathsf{repr}_{\mathbb{G}}(\mathsf{vk}_j)\right)$.

Let $c_j = \mathsf{H}^{\circledast}(\underline{R_j} \,||\, \underline{\mathsf{vk}_j} \,||\, M_j)$.

Choose random $z_j : \mathbb{F}^*_{r_{\mathbb{G}}} \xleftarrow{R} \{1\,..\,2^{128}-1\}$.

Return 1 if

- for all $j \in \{0\,..\,N-1\}$, $R_j \neq \perp$ and $S_j < r_{\mathbb{G}}$; and
- $[h_{\mathbb{G}}]\left(-\left[\sum_{j=0}^{N-1}(z_j \cdot S_j) \pmod{r_{\mathbb{G}}}\right]\mathcal{P}_{\mathbb{G}} + \sum_{j=0}^{N-1}[z_j]\,R_j + \sum_{j=0}^{N-1}[z_j \cdot c_j \pmod{r_{\mathbb{G}}}]\,\mathsf{vk}_j\right) = \mathcal{O}_{\mathbb{G}}$,

otherwise 0.

The z_j values **MUST** be chosen independently of the *signature batch entries*.

Non-normative note: It is also acceptable to sample each z_j from $\{0\,..\,2^{128}-1\}$, since the probability of obtaining zero for any z_j is negligible.

The performance benefit of this approach arises partly from replacing the per-signature scalar multiplication of the base $\mathcal{P}_{\mathbb{G}}$ with one such multiplication per batch, and partly from using an efficient algorithm for multiscalar multiplication such as Pippinger's method [Bernstein2001] or the Bos–Coster method [deRooij1995], as explained in [BDLSY2012, section 5].

Note: Spend authorization signatures (§ 5.4.6.1 ***'Spend Authorization Signature (Sapling)'*** on p. 71) and binding signatures (§ 5.4.6.2 ***'Binding Signature (Sapling)'*** on p. 71) use different bases $\mathcal{P}_{\mathbb{G}}$. It is straightforward to adapt the above procedure to handle multiple bases; there will be one $-\left[\sum_j (z_j \cdot S_j) \pmod{r_{\mathbb{G}}}\right]\mathcal{P}$ term for each base $\mathcal{P}$. The benefit of this relative to using separate batches is that the multiscalar multiplication can be extended across a larger batch.

B.2 Groth16 batch verification

The reference verification algorithm for Groth16 proofs is defined in § 5.4.9.2 ***'Groth16'*** on p. 81. The batch verification algorithm in this section applies techniques from [BFIJSV2010, section 4].

Let $q_{\mathbb{S}}$, $r_{\mathbb{S}}$, $\mathbb{S}^{(r)}_{1,2,T}$, $\mathbb{S}^{(r)*}_{1,2,T}$, $\mathcal{P}_{\mathbb{S}_{1,2,T}}$, $\mathbf{1}_{\mathbb{S}}$, and $\hat{e}_{\mathbb{S}}$ be as defined in § 5.4.8.2 ***'BLS12-381'*** on p. 75.

Define $\mathsf{MillerLoop}_{\mathbb{S}} : \mathbb{S}^{(r)}_1 \times \mathbb{S}^{(r)}_2 \to \mathbb{S}^{(r)}_T$ and $\mathsf{FinalExp}_{\mathbb{S}} : \mathbb{S}^{(r)}_T \to \mathbb{S}^{(r)}_T$ to be the Miller loop and final exponentiation respectively of the $\hat{e}_{\mathbb{S}}$ pairing computation, so that:

$$\hat{e}_{\mathbb{S}}(P, Q) = \mathsf{FinalExp}_{\mathbb{S}}(\mathsf{MillerLoop}_{\mathbb{S}}(P, Q))$$

where $\mathsf{FinalExp}_{\mathbb{S}}(R) = R^t$ for some fixed t.

Define $\mathsf{Groth16}_{\mathbb{S}}.\mathsf{Proof} := \mathbb{S}^{(r)*}_1 \times \mathbb{S}^{(r)*}_2 \times \mathbb{S}^{(r)*}_1$.

A $\mathsf{Groth16}_{\mathbb{S}}$ proof comprises a tuple $(\pi_A, \pi_B, \pi_C) : \mathsf{Groth16}_{\mathbb{S}}.\mathsf{Proof}$.

Verification of a single $\mathsf{Groth16}_{\mathbb{S}}$ proof against an instance encoded as $a_{0\,..\,\ell} : \mathbb{F}_{r_{\mathbb{S}}}^{[\ell+1]}$ requires checking the equation

$$\hat{e}_{\mathbb{S}}(\pi_A, \pi_B) = \hat{e}_{\mathbb{S}}(\pi_C, \Delta) \cdot \hat{e}_{\mathbb{S}}\Big(\sum_{i=0}^{\ell} [a_i]\, \Psi_i, \Gamma\Big) \cdot Y$$

where $\Delta = [\delta]\, \mathcal{P}_{\mathbb{S}_2}$, $\Gamma = [\gamma]\, \mathcal{P}_{\mathbb{S}_2}$, $Y = [\alpha \cdot \beta]\, \mathcal{P}_{\mathbb{S}_T}$, and $\Psi_i = \left[\frac{\beta \cdot u_i(x) + \alpha \cdot v_i(x) + w_i(x)}{\gamma}\right] \mathcal{P}_{\mathbb{S}_1}$ for $i \in \{0\,..\,\ell\}$ are elements of the verification key, as described (with slightly different notation) in [Groth2016, section 3.2].

This can be written as:

$$\hat{e}_{\mathbb{S}}(\pi_A, -\pi_B) \cdot \hat{e}_{\mathbb{S}}(\pi_C, \Delta) \cdot \hat{e}_{\mathbb{S}}\Big(\sum_{i=0}^{\ell} [a_i]\, \Psi_i, \Gamma\Big) \cdot Y = \mathbf{1}_{\mathbb{S}}.$$

Raising to the power of random $z \neq 0$ gives:

$$\hat{e}_{\mathbb{S}}([z]\, \pi_A, -\pi_B) \cdot \hat{e}_{\mathbb{S}}([z]\, \pi_C, \Delta) \cdot \hat{e}_{\mathbb{S}}\Big(\sum_{i=0}^{\ell} [z \cdot a_i]\, \Psi_i, \Gamma\Big) \cdot Y^z = \mathbf{1}_{\mathbb{S}}.$$

This justifies the following optimized procedure for performing faster verification of a batch of $\mathsf{Groth16}_{\mathbb{S}}$ proofs. Implementations **MAY** use this procedure to determine whether all proofs in a batch are valid.

Define a type $\mathsf{Groth16}_{\mathbb{S}}.\mathsf{BatchEntry} := \mathsf{Groth16}_{\mathbb{S}}.\mathsf{Proof} \times \mathsf{Groth16}_{\mathbb{S}}.\mathsf{PrimaryInput}$ representing *proof batch entries*.

Define $\mathsf{Groth16}_{\mathbb{S}}.\mathsf{BatchVerify} : (\mathsf{entry}_{0\,..\,N-1} : \mathsf{Groth16}_{\mathbb{S}}.\mathsf{BatchEntry}^{[N]}) \to \mathbb{B}$ as:

For each $j \in \{0\,..\,N-1\}$:

Let $((\pi_{j,A},\ \pi_{j,B},\ \pi_{j,C}),\ a_{j,\,0\,..\,\ell}) = \mathsf{entry}_j$.

Choose random $z_j : \mathbb{F}_{r_{\mathbb{S}}}^{*} \xleftarrow{R} \{1\,..\,2^{128} - 1\}$.

Let $\mathsf{Accum}_{AB} = \prod_{j=0}^{N-1} \mathsf{MillerLoop}_{\mathbb{S}}\big([z_j]\, \pi_{j,A}, -\pi_{j,B}\big)$.

Let $\mathsf{Accum}_{\Delta} = \sum_{j=0}^{N-1} [z_j]\, \pi_{j,C}$.

Let $\mathsf{Accum}_{\Gamma,i} = \sum_{j=0}^{N-1} (z_j \cdot a_{j,i}) \pmod{r_{\mathbb{S}}}$ for $i \in \{0\,..\,\ell\}$.

Let $\mathsf{Accum}_{Y} = \sum_{j=0}^{N-1} z_j \pmod{r_{\mathbb{S}}}$.

Return 1 if

$$\mathsf{FinalExp}_{\mathbb{S}}\Big(\mathsf{Accum}_{AB} \cdot \mathsf{MillerLoop}_{\mathbb{S}}\big(\mathsf{Accum}_{\Delta}, \Delta\big) \cdot \mathsf{MillerLoop}_{\mathbb{S}}\Big(\sum_{i=0}^{\ell} [\mathsf{Accum}_{\Gamma,i}]\, \Psi_i, \Gamma\Big)\Big) \cdot Y^{\mathsf{Accum}_Y} = \mathbf{1}_{\mathbb{S}},$$

otherwise 0.

The z_j values **MUST** be chosen independently of the *proof batch entries*.

Non-normative note: It is also acceptable to sample each z_j from $\{0\,..\,2^{128} - 1\}$, since the probability of obtaining zero for any z_j is negligible.

The performance benefit of this approach arises from computing two of the three Miller loops, and the final exponentiation, per batch instead of per proof. For the multiplications by z_j, an efficient algorithm for multiscalar multiplication such as Pippinger's method [Bernstein2001] or the Bos–Coster method [deRooij1995] may be used.

Note: Spend proofs (of the *statement* in § 4.16.2 ***'Spend Statement (Sapling)'*** on p. 48) and output proofs (of the *statement* in § 4.16.3 ***'Output Statement (Sapling)'*** on p. 49) use different verification keys, with different parameters Δ, Γ, Y, and $\Psi_{0\,..\,\ell}$. It is straightforward to adapt the above procedure to handle multiple verification keys; the accumulator variables Accum_{Δ}, $\mathsf{Accum}_{\Gamma,i}$, and Accum_{Y} are duplicated, with one term in the verification equation for each variable, while Accum_{AB} is shared.

Neglecting multiplications in $\mathbb{S}_T^{(r)}$ and $\mathbb{F}_{r_{\mathbb{S}}}$, and other trivial operations, the cost of batched verification is therefore

- for each proof: the cost of decoding the proof representation to the form $\mathsf{Groth16}_{\mathbb{S}}.\mathsf{Proof}$, which requires three point decompressions and three subgroup checks (two for $\mathbb{S}_1^{(r)*}$ and one for $\mathbb{S}_2^{(r)*}$);
- for each successfully decoded proof: a Miller loop; and a 128-bit scalar multiplication by z_j in $\mathbb{S}_1^{(r)}$;
- for each verification key: two Miller loops; an exponentiation in $\mathbb{S}_T^{(r)}$; a multiscalar multiplication in $\mathbb{S}_1^{(r)}$ with N 128-bit scalars to compute Accum_{Δ}; and a multiscalar multiplication in $\mathbb{S}_1^{(r)}$ with $\ell + 1$ 255-bit scalars to compute $\sum_{i=0}^{\ell} [\mathsf{Accum}_{\Gamma,i}]\, \Psi_i$;
- one final exponentiation.

List of Theorems and Lemmata

Theorem 5.4.1 The encoding function $\langle \bullet \rangle$ is injective . . . 63

Theorem 5.4.2 $\mathsf{Uncommitted}^{\mathsf{Sapling}}$ is not in the range of $\mathsf{NoteCommit}^{\mathsf{Sapling}}$ . . . 73

Lemma 5.4.3 Let $P = (u, v) \in \mathbb{J}^{(r)}$. Then $(u, -v) \notin \mathbb{J}^{(r)}$ . . . 78

Theorem 5.4.4 $\mathcal{U}$ is injective on $\mathbb{J}^{(r)}$ . . . 78

Theorem A.2.1 $(0, 0)$ is the only point with $y = 0$ on certain *Montgomery curves* . . . 150

Theorem A.3.1 Correctness of a constraint system for range checks . . . 153

Theorem A.3.2 Exceptional points (ctEdwards $\to$ Montgomery) . . . 155

Theorem A.3.3 Exceptional points (Montgomery $\to$ ctEdwards) . . . 155

Theorem A.3.4 Distinct-x theorem . . . 155

Theorem A.3.5 Concerning addition inputs in the Pedersen circuit . . . 160

Index

Heartwood, 88, 119, 122, 123

account, ***37***
Action description, 43
Action transfer, 19
activation block, 16, 20, 80
activation block height, ***88***, 88
anchor, ***17***, 17–19, 34, 35, 47, 48, 114
authenticated one-time symmetric encryption, ***23***, 24, 66
auxiliary input, ***30***, 31, 38–40, 45, 47–50, 128

balancing value (Sapling pool), ***42***, 42, 43, 129
Base58Check, 82–84, 102
BCTV14, 31, 32, 34, 40, ***79***, 80, 89, 93, 110, 111, 115, 119, 125, 129, 149

Bech32, 33, 82, 129
bellman, ***81***, 81, 87
best valid block chain, ***16***, 16, 51, 53, 135
big-endian, 75, 76, 80, 81, 87, 98, 153
bilateral consensus rule change, ***88***
binding (commitment scheme), 21, ***27***, 28, 43, 44, 72, 73, 106, 109, 116, 124, 130, 135
binding signature, 70, 108
binding signature (Sapling), 18, 25, 26, 40, ***42***, 43, 44, 89, 131
binding signature scheme, 69, ***71***
bit sequence, 10, 11, 32, 33, 39, 40, 49, 50, 52, 56–58, 64, 71, 112, 120, 121, 125, 131, 149, 162, 164, 168, 169
Bitcoin, 1, ***7***, 7, 8, 17, 19, 20, 25, 40, 41, 82, 83, 87, 89, 91, 92, 95–98, 103, 110, 115, 118, 121, 135, 137
Bitcoin Core, 92, 110, 115
block, 14, 16–20, 32, 34, 35, 41, 46, 53, 80, 87, 88, 90, 91, ***95***, 95–100, 103, 115–117, 122, 124, 135, 136
block chain, 8, 9, 13–15, ***16***, 16, 17, 20, 46, 51, 53, 55, 56, 80, 81, 91, 92, 96, 99, 100, 103, 105, 122, 124, 133, 135
block chain reorganization, 51, 53, 88
block hash, ***20***, 80
block header, 16, 58, ***95***, 95–98, 100, 132, 135, 137
block height, 14, ***16***, 16, 20, 46, 54, 88–91, 93, 94, 96, 98–102, 117, 119, 125
block subsidy, ***20***, 20, 90, 100, 113, 116, 120, 136
block target spacing, ***98***, 98
block timestamp, ***95***
block version number, ***95***, 95–97, 124, 134
Blossom, 88, 123–125
BLS12-381 pairing, 31, 32, ***75***, 81, 110, 130
BN-254 pairing, 31, 32, ***74***, 76, 80
Bulletproofs, 44
byte sequence, 10, 57, 59, 64, 66, 70, 71, 82, 117, 118, 120, 131, 150, 168

Canopy, 88, 119, 121, 122
chain value pool, ***8***, 8, 14, 17, 113, 115
chunk (of a Pedersen hash input), ***62***
coinbase transaction, 17, ***20***, 20, 41, 90–92, 96, 101, 102, 115, 116, 118, 120, 130, 133, 136
coins (in Zerocash), ***8***
collision resistance, 21–23, 30, 59, 60, 62–65, 72, 105–107, 109, 110, 132, 138, 162
commitment scheme, 21, ***27***, 27, 28, 30, 72, 73, 106, 109, 116, 124, 129, 135
commitment trapdoor, 13, 14, ***27***, 27, 28, 36, 38, 44, 73, 104, 114, 129
complete twisted Edwards affine coordinates, 53, 73, ***149***, 149, 150, 156, 157
complete twisted Edwards compressed encoding, ***78***, 85, 86, 153
complete twisted Edwards elliptic curve, 11, 38, 68, ***77***, 77, 78, 124, 149, 154, 155, 157, 159
consensus branch, ***92***, 115
consensus branch ID, 41
consensus rule change, 88
coordinate extractor, ***29***, 119, 133
CryptoNote, 9, 137
ctEdwards, ***11***, 77, 158, 159, 161–163

Decentralized Anonymous Payment scheme, 1, ***7***
Decisional Diffie–Hellman Problem, 30, 61, 106, 116
default diversified payment address, ***33***, 79
discrete logarithm, 30, 43, 44, 71, 112
Discrete Logarithm Independence, 30
Discrete Logarithm Problem, 30, 62, 159
distinct-x criterion, 133, ***156***, 159
diversified base, 22, ***33***, 38, 52, 61
diversified payment address, ***12***, 13, 14, 33, 39, 61, 62, 106, 111, 124, 126
diversified transmission key, 14, ***33***, 36, 38, 51, 52, 94
diversifier, 14, 22, ***33***, 33, 61, 62, 85, 86, 113
diversifier index, 33, 86, 113
diversifier key, 33, 86, 113
double-spend, 9, 15–19, 106, 109, 111, 134
dummy note, ***38***, 38, 39, 104, 105, 119

ECDSA, 25, 83, 128
Ed25519, 25, 59, 67–69, 90, 117, 119, 121, 122, 130, 133, 137
ephemeral private key, 36, 38, 52, 94
ephemeral public key, 24, 50, 51, 108
Equihash, 1, 22, 64, 95, 96, ***97***, 97, 98, 110, 124, 128, 134–137
expanded spending key, ***12***, 12
extended spending key, 12, 13, 123

family of group hashes into a subgroup, ***29***
Founders' Reward, ***20***, 20, 90, 91, 100–103, 124, 130, 134, 136
full node, ***133***, 133
full validator, ***16***, 16, 19, 20, 80, 96, 133

full viewing key, 9, ***12***, 13, 33, 36, 39, 45, 55, ***86***, 86, 116, 129

genesis block, ***16***, 16, 20, 91, ***95***, 96, 102, 103, 120, 135
Groth16, 31, 32, 34, 40, 43, 44, ***81***, 81, 89, 93, 110, 124, 125, 127, 129, 149, 166, 168, 170
group hash, ***29***, 79

Halo 2, 110, 111, 115
halving, 102
hash function, 17, 22, 32, 58, 62–65, 68, 69, 108, 130
hash value (of a Merkle tree node), ***19***, 19, 39, 40, 59
hiding (commitment scheme), ***27***, 28, 72, 73, 106, 116, 135
Hierarchical Deterministic Wallet, 12
homomorphic Pedersen commitment, ***73***, 162
Human-Readable Part, ***85***, 85, 86

in-band, 107, 137
incoming viewing key, ***12***, 13, 22, 33, 45, 50, 51, 55, 60, 62, 82, ***84***, 84, ***85***, 86, 109, 113, 116, 134
incremental Merkle tree, ***19***, 19, 39, 40, 59, 62, 122, 138
index (of a Merkle tree node), ***19***, 19, 39
internal node (of a Merkle tree), ***40***, 40
IVK Encoding (in a Unified Incoming Viewing Key), 86, 113

JoinSplit circuit, 87
JoinSplit description, 8, 14, 15, ***17***, 17–19, 25, ***34***, 34, 35, 37–39, 41, 42, 50, 55, 81, 89, 91, 93, 104, 105, 108, 120, 128, 133, 134
JoinSplit proof, 38
JoinSplit signature, 26, 40, ***41***, 137
JoinSplit signing key, ***37***
JoinSplit statement, 8, 17, 18, 31, 32, 34, 38, 41, 44, 45, ***47***, 80, 105, 106, 109, 110, 116, 129, 134, 149
JoinSplit transfer, ***17***, 17–19, 34, 38, 40–42, 93, 103, 104, 106, 108, 110, 116, 135, 137
Jubjub elliptic curve, 15, 25, 29, 30, 38, 42, 44, 49, 50, 53, 54, 61, 62, 65, 67, 69, 72, 73, ***77***, 78, 85, 86, 94, 106–108, 113, 119, 121, 124, 128, 131, 133, 134, 149, 155, 157, 166, 168

key agreement scheme, ***23***, 23, 24, 32, 50, 51, 66, 67, 107
Key Derivation Function, ***24***, 24, 50, 51, 66, 67
key privacy, ***9***, 9, ***24***, 24, 61, 108, 111, 124, 126

layer (of a Merkle tree), ***19***, 19, 39, 40, 49
lead byte (of a note plaintext), 14, 38, 39, 113, 121
leaf node (of a Merkle tree), 16, 19, ***40***, 40
librustzcash, 54, 120
libsnark (Zcash fork), 79, ***80***, 80, 87, 110
linear combination, ***149***, 149, 151
little-endian, 72, 86, 91, 98

Mainnet, 16, ***20***, 20, 54, 58, 80, 82–86, 88, 91, 96, 97, 99, 101, 103, 121, 124, 136
MAY, ***7***, 16, 33, 36, 44, 45, 53, 70, 73, 80, 82, 114, 169, 171
median-time-past, ***95***, 96, 122
memo field, ***14***, 14, 50, 51, 55, 56, 104, 118, 122, 133, 137
mempool, 53, 54, 120
Merkle path, 38, 39, ***40***, 40, 47, 48, 159
Mimblewimble, 44
miner subsidy, ***20***, 20, 41, 100, 116, 136
monomorphism, ***27***, 122
Montgomery affine coordinates, ***149***, 150, 155, 156
Montgomery elliptic curve, 126, ***149***, 149, 150, 155, 158, 159, 161, 172
multi-party computation, 87
MUST, ***7***, 14, 16–20, 35, 36, 38, 41, 42, 46, 49, 50, 52–54, 68, 70, 80, 81, 84–86, 88, 90, 91, 94, 96, 98, 102, 113, 115, 121, 127, 170, 171
MUST NOT, ***7***, 19, 35, 36, 53, 68, 75, 76, 78, 80, 90, 91, 96, 112, 115, 116, 118, 128

network, 16, ***20***, 20
network upgrade, 7, 16, 20, 80, ***88***, 88, 112, 114, 115, 125
node (of a Merkle tree), ***19***, 19, 39, 40
non-canonical (compressed encoding of a point), ***29***, 29, 53, 54, 85, 113, 119, 120
non-canonical (encoding of a field element), 36, 40, 118, 162
nonmalleability (of proofs), ***31***
nonmalleability (of signatures), ***26***
note, 8, 9, ***13***, 13–18, 22, 23, 26, 34–40, 42, 44, 45, 50–56, 65, 81, 93, 94, 103–109, 112, 114, 116, 119, 120, 128, 131, 159
note commitment, 8, 14, ***15***, 15, 18, 19, 21, 34, 36, 40, 45, 51–53, 93, 94, 104–109, 114, 116, 130, 134, 159
note commitment scheme, 28, 72, 109, 159
note commitment tree, 15, 17, ***19***, 19, 39, 60, 93–96, 105, 116, 132
note plaintext, ***14***, 14, 36, 38, 39, 50, 52, 56, 81, 82, 94, 108, 113, 114, 120, 121
note position, 8, 15, ***19***, 45, 105
note traceability set, ***9***, 9, 130

NU5, 88, 112, 114, 117–119
NU6, 88, 113
NU6.1, 88, 112, 113
NU6.2, 88
NU6.3, 16, 20, 88
NU7, 20, 88
nullifier, 8, 9, 13, ***15***, 15–18, ***19***, 19, 22, 34, 35, 45, 53, 55, 56, 65, 93, 94, 104–107, 109, 111, 114, 120, 131, 135
nullifier deriving key, 8, 16, 45, 53
nullifier private key, ***12***
nullifier set, 15, 17, ***19***, 19, 51, 53

one-time (authenticated symmetric encryption), ***23***
one-time (signature scheme), ***25***
open (a commitment), ***27***
OPTIONAL, ***7***, 37
out-of-band, 50–52, 56
outgoing cipher key, 36, ***52***, 65, 94
outgoing ciphertext, ***52***, 54, 65, 109, 116, 131
outgoing viewing key, ***12***, 33, 37, 38, 52, 53, 86, 128, 129
Output circuit, 87, 131, 158, 159
Output description, 8, 9, 14, 15, ***18***, 18, 25, 28, ***36***, 36–38, 42–44, 52, 53, 56, 73, 81, 89–92, 94, 108, 114, 120, 128, 132, 133
Output statement, 18, 31, 32, 36, 38, 44, ***49***, 49, 81, 94, 112, 128, 131
Output transfer, 17, ***18***, 18, 25, 36, 42, 44, 94, 103, 104, 106, 112
Overwinter, 1, 7, 41, 59, ***88***, 88, 90–92, 96, 112, 124, 128, 129, 132–134

packing, ***151***
partitioning oracle attack, ***108***, 109, 116
paying key, ***13***, 38
payment address, 82, 118
Pedersen commitment, 18, 43, 44, 62, 63, ***72***, 106, 131–133, 158, 161
Pedersen hash, 30, ***62***, 63, 72, 105, 107, 130–133, 159, 161, 162
Pedersen value commitment, ***18***, 18, 104
peer-to-peer protocol, 37, 38, 88, 95, 103, 130, 133
PLONK, 111
positioned note, ***15***, 15, 45, 53, 105
prevout (previous output), 91, 115
primary input, ***30***, 31, 34–36, 40, 45, 47–50, 118, 127
prime order (of a group element), 54, 114, 130, 159
prime-order curve, 76
prime-order group, 61
prime-order subgroup, 42, 61, 67, 114, 120, 157
private key, 8, 9, ***23***, 23, 24, ***26***, 44, 51, 53, 66, 84, 108
proof authorizing key, 8, ***12***, 33, 45, 65
proof batch entry, ***171***, 171
proof-of-work, 1, 97, 110
proving key (for a zk-SNARK), ***31***, 32, 87
proving system (preprocessing zk-SNARK), 1, 7–9, 18, ***30***, 30, 31, 79–81, 104, 111, 113, 125, 126, 128, 137
Pseudo Random Function, 16, 21, ***22, 23***, 23, 32, 34, 59, 64, 65, 72, 106, 107, 109, 133
Pseudo Random Permutation, ***62***, 62
public key, ***13***, 13, 23, 24, 34, 36, 50, 52, 61, 66, 83, 85, 93, 94, 107, 108

Quadratic Arithmetic Program, ***80***, 81, ***149***
quadratic constraint program, 7, 80, 81, 134, ***149***, 149, 151, 154, 155

random oracle, 29, 30, 61, 64, 79, 121, 129
randomized Spend validating key, 131
randomizer, ***26***, 26, 27, 44
Rank 1 Constraint System, ***149***
raw encoding, 50, 52, ***82***, 82, ***83***, 83–86, 120
re-randomized, ***26***, 26, 27, 44, 69, 71
receiving key, 9, ***12***, 134
RECOMMENDED, ***7***, 14, 33, 70, 82
represented group, ***29***, 29, 69, 77, 119, 130
represented pairing, ***30***, 74, 75, 130
represented subgroup, ***29***, 29, 30
root (of a Merkle tree), ***19***, 19, 39, 40, 93–96
RPC byte order, ***20***, 87

Sapling, 1, 7, 8, 12–14, 19, 25, 31, 32, 34, 41, 59, 79–81, 87, ***88***, 88, 90–93, 95, 96, 104–106, 119, 124, 125, 127–129, 132, 133
Sapling pool, 7, ***8***, 8, 13, ***14***, 14–19, 22, 23, 37, 39, 42, 44, 45, ***46***, 46, 51–53, 65, 89, 94–96, 105, 106, 119–121, 127, 128, 131, 132
Sapling shielded protocol, 7
secp256k1 elliptic curve, 25
segment (of a Pedersen hash input), ***62***
serial numbers (in Zerocash), ***8***
settled, ***16***, 16, 80, 115
SHA-256, ***58***, 58, 59, 65, 72, 87, 106, 116, 132

SHA-256d, ***58***, 90, 95, 97, 98
SHA-512, ***59***, 59, 68, 121
SHA256Compress, ***58***, 59, 64, 65, 72, 83, 84, 106, 107, 109, 132, 135
shielded, 8, 17, 37, 54, 103, 104, 120
shielded input, ***8***, 8, 17, ***18***, 18, 39, 92
shielded output, ***8***, 8, ***18***, 18, 50
shielded payment address, 8, 9, ***12***, 13, 14, 22, 38, 39, 44, 52, 55, 56, 60, 61, 82, ***83***, 83, ***85***, 85, 107, 112, 120, 135, 137
shielded pool, ***13***, 13, 14, 19, 111, 112
shielded protocol, 13, 16, 19, 45, 108, 112, 125
shielded transfer, ***8***, 8
SHOULD, ***7***, 14, 16, 37, 38, 43, 62, 80, 91, 96
SHOULD NOT, ***7***, 124
side-channel, 21, 107, 108
SIGHASH algorithm, ***41***, 41
SIGHASH transaction hash, 36, ***40***, 40, 43–45, 59, 89, 103, 128, 136
SIGHASH type, ***40***, 41, 43, 45, 128, 138
signature batch entry, ***169***, 170
signature scheme, 21, ***24***, 25–27, 59, 67, 69, 71, 131
signature scheme with key monomorphism, ***27***, 71, 122
signature scheme with re-randomizable keys, ***26***, 32, 44, 71
signing key, ***24***, 25–27, 41, 43, 129, 131
slanted text, ***7***
small order (of a group element), 35, 36, 48, 49, 54, 114, 128, 130, 132, 157, 167, 169
spend authorization randomizer, ***45***, 126
spend authorization signature, ***35***, 35, 36, 40, 42, ***44***, 45, 93, 108, 126, 131
spend authorization signature scheme, 44, 69, ***71***
Spend authorizing key, ***12***, 33, 45, 65, 112
Spend circuit, 40, 87, 114, 118, 131, 158, 159, 165
Spend description, 8, 15, ***18***, 18, 19, 25, 28, ***35***, 35, 39, 42–45, 56, 73, 81, 89–94, 114, 128, 132, 133
Spend proof, 40
Spend statement, 18, 22, 31, 32, 35, 39, 44, 45, ***48***, 48, 49, 60, 81, 93, 128, 130
Spend transfer, 17, ***18***, 18, 19, 25, 35, 40, 42, 44, 93, 104, 106
Spend validating key, 44, 112
spending authority, 9, 13, 33, 45, 61
spending key, 8, 9, ***12***, 12, 13, 16, 22, 32, 33, 38, 44, 45, 51, 55, 82, 83, ***84***, 84, ***86***, 86, 104, 105, 109, 120, 132, 135, 137
Sprout pool, ***8***, 13, ***14***, 14–19, 37–39, 45, ***46***, 46, 50, 93, 106, 109, 121
Sprout shielded protocol, 7
standard P2SH script, 102
standard redeem script, 102
statement, 18, ***30***, 31, 44, 47, 109, 132, 149, 166, 168, 171
synthetic blinding factor, ***44***

target threshold, 95, 98, ***99***, 99
TAZ, 20
tazoshi, ***20***
Testnet, 16, ***20***, 20, 54, 58, 82–86, 91, 96, 97, 99, 102, 103, 121, 125, 134, 136
total input value, 90, 91
total issued supply, ***46***, 46, 112, 113
total output value, 90, 91
transaction, 8, 9, 14–20, 25, 34–38, 40–44, 51, 53–56, 80, ***89***, 89–94, 96, 103–105, 112, 115, 117, 120, 124, 128, 133, 138
transaction binding validating key, ***43***, 90
transaction fee, ***17***, ***20***, 20, 90, 120
transaction ID, ***16***, 16, 90, 117
transaction version number, 14, 40, 81, 89, ***90***, 90–93, 120, 134
transactions, 9, 13, ***16***, 16, 17, 19, 20, 25, 26, 34–38, 41, 42, 44–46, 51, 53, 54, 88, 90–92, 95, 96, 105, 115, 120, 127, 128
transmission key, 9, ***13***, 14, 24, 37, 50, 51, 55, 120
transmitted note ciphertext, 53, 108, 109
transmitted note ciphertext (Sapling), 36, 38, ***52***, 53, 54, 56, 94
transmitted notes ciphertext (Sprout), 34, 35, 37, ***50***, 50, 55, 93
transparent, 8, 17, 18, 41, 83, 89, 103, 124, 136
transparent address, ***82***, 101, 136
transparent address (P2PKH), ***82, 83***, 83, 128, 136
transparent address (P2SH multisig), 102
transparent address (P2SH), ***82***, 82, 83, 136
transparent input, ***16***, 17, 20, 40, 41, 89–92, 115, 118, 122
transparent output, ***16***, 17, 41, 89–91, 101, 120, 122
transparent pool, ***8***, ***46***, 46, 112
transparent transaction value pool, ***16***, 16, 17, 34, 41, 42, 93

treestate, 13, 15, ***17***, 17–19, 34, 35, 95, 96, 128, 135, 138

unified incoming viewing key, 86, 113
Uniform Random String, 29, ***30***, 129
unpacking, ***151***
US-ASCII, 10, 87
UTXO (unspent transaction output), 46, 91, 103, 104, 115
UTXO (unspent transaction output) set, 17, ***19***, 19, 46

valid block chain, ***16***, 16, 19, 105
valid Equihash solution, 96, ***97***, 97, 112
validating key (for a signature scheme), ***24***, 25–27, 34–36, 41, 44, 68, 71, 78, 83, 89, 90, 94, 122, 129–131, 133, 169
value commitment, 8, ***18***, 18, 25, 35, 36, 42–44, 52, 94
value commitment scheme, 44
verifying key (for a zk-SNARK), ***31***, 32, 87
version group ID, 90, 91, ***92***, 120

weak PRF, ***108***
windowed, ***72***
windowed Pedersen commitment, ***162***, 162

zatoshi, ***13***, 13, 14, 20, 42, 46, 58, 90, 100, 102, 112
Zcash, ***1***, 1, 7–9, 12, 16, 17, 20, 21, 25, 30, 31, 40–42, 56, 58, 59, 70, 76, 79–84, 87–89, 91, 92, 95, 97, 98, 103–111, 114, 120, 121, 128, 135, 136, 138
zcashd, 16, 20, 33, 54, 69, 90, 97, 99, 106, 115, 119, 120, 122–124, 133, 134
zebra, 16, 20, 54
ZEC, ***13***, 20
Zerocash, 1, ***7***, 7, 8, 21, 23, 42, 103–110, 114, 118, 134–138
zk-SNARK circuit, 62, 77, 87
zk-SNARK proof, 15, 17, 18, 26, ***31***, 31, 34–36, 44, 45, 81, 93, 94, 106, 108, 111, 115, 116, 127, 128, 135